

Noise and Uncertainty in Molecular Property Prediction: Robust Models and Adaptive Representations

Adelaide Punt

St Catherine's College

University of Oxford

A thesis submitted for the degree of

Doctor of Philosophy

October 2026

Declaration of AI Use

During this doctorate I used Claude, a large language model developed by Anthropic (https://claude.ai), [model versions and dates of use]. I used it for three purposes. First, I used it to help write, test and debug software, including parts of the code that ran the experiments reported in this thesis. Second, I used it to draft and edit parts of the text and to typeset it in LaTeX. Third, I used it to search for literature and to check citations against their sources. I reviewed, edited and verified all of its output before using it. [No data, results or figures were generated by the tool; every reported number comes from code that I ran and checked.] The research questions, experimental design, analysis and conclusions are my own. [My supervisors were aware of this use.] I take full responsibility for the content of this thesis, including any errors.

Acknowledgements

I would like to thank my supervisors, Garrett Morris at the University of Oxford and Thierry Hanser and Stéphane Werner at Lhasa Limited, for their guidance, their time and their patience throughout this project.

I also thank the EPSRC (EP/L015803/1) as well as my industrial partner Lhasa for their financial support.

Thank you to my parents and grandparents for their support, and to my cat, [name].

Abstract

Contents

Declaration of AI Use ii

Acknowledgements iii

Abstract iv

List of Abbreviations x

1 Introduction 1
- 1.1 Outline of the Thesis and Contributions . . . 4
- 1.2 Collaborations . . . 6

2 Molecular Representations, Models and Label Noise 7
- 2.1 Overview . . . 7
- 2.2 Molecular Representations in QSAR . . . 7
- 2.3 Models for Molecular Property Prediction . . . 13
  - 2.3.1 Tree Ensembles . . . 15
  - 2.3.2 Support Vector Machines . . . 16
  - 2.3.3 Gaussian Processes . . . 16
  - 2.3.4 Bayesian Neural Networks . . . 17
- 2.4 Molecular Similarity and Activity Cliffs . . . 18
- 2.5 Noise in Molecular Data . . . 20
  - 2.5.1 Artificial Noise Injection . . . 22
  - 2.5.2 Learning with Noisy Labels . . . 25
- 2.6 NoiseInject: A Benchmarking Framework . . . 26
  - 2.6.1 NoiseInject . . . 26
- 2.7 Evaluation Methodology . . . 30
- 2.8 Conclusions . . . 30

3 Noise Robustness Across Molecular Representations and Model Architectures 31
- 3.1 Introduction . . . 31
- 3.2 Methods . . . 32
  - 3.2.1 Datasets . . . 32
  - 3.2.2 Representations . . . 33
  - 3.2.3 Models . . . 34
  - 3.2.4 Noise Conditions and Levels . . . 35
  - 3.2.5 QM9 Data Pipeline . . . 36
  - 3.2.6 Uncertainty . . . 37

3.2.7 Evaluation . . . 39
3.3 Variance Decomposition by Representation and Architecture . . . 41
3.4 Probabilistic and Deterministic Model Variants . . . 46
3.5 Model Robustness Rankings across Noise Strategies . . . 47
3.6 Validation on Assay Datasets . . . 49
3.7 Uncertainty under Label Noise . . . 52
3.7.1 Per-Sample Uncertainty and Label Noise . . . 52
3.7.2 Treatment of Large Residuals . . . 53
3.7.3 Aleatoric and Epistemic Attribution . . . 53
3.7.4 Representation and Uncertainty Tracking . . . 56
3.8 Conclusions . . . 56

4 The Flaws of the Similarity Assumption 59
4.1 Introduction . . . 59
4.2 Flaws in Standard Base Representations . . . 61
4.2.1 Constant and Near-Constant Features . . . 61
4.2.2 Molecules with Identical Feature Vectors . . . 64
4.3 Flaws in Similarity and Other Pairwise Computations . . . 69
4.4 Consequences for Similarity-Based Methods . . . 76
4.4.1 Exploratory Study of Learning with Noisy Labels . . . 76
4.4.2 Noise Detection by Neighbors . . . 78
4.5 Conclusions . . . 81

5 Building Adaptive Hybrid Representations 84
5.1 Introduction . . . 84
5.1.1 Combining Representations and the Alternatives . . . 85
5.1.2 The Adaptive Hybrid Representation . . . 91
5.1.3 Blueprints and Their Chemical Interpretation . . . 94
5.1.4 Beyond Small Molecules . . . 95
5.2 Methods: The Hybrid Search . . . 97
5.2.1 From Base Representations to a Hybrid . . . 97
5.2.2 The Search Space . . . 104
5.2.3 Augmentation . . . 118
5.2.4 Fast Tier and Adaptive Pruning . . . 122
5.3 Methods: Experimental Setup . . . 127
5.3.1 Datasets and Splitting . . . 128
5.3.2 Representations by Domain . . . 133
5.3.3 Models and Metrics . . . 144
5.3.4 Comparators, Controls and Ablations . . . 146
5.3.5 Transfer, Mutational Scans and Separate Sites . . . 153
5.4 Results: Comparison with Single Representations and Ensembles . . . 158
5.5 Results: Sources of the Gain . . . 158
5.6 Results: Blueprints on Specific Datasets . . . 158
5.7 Results: Transfer across Datasets and Antibody Mutational Scans . . . 158
5.8 Results: Data Held at Separate Sites . . . 158
5.9 Discussion . . . 158
5.10 Limitations . . . 159
5.11 Conclusions . . . 160

6 Hybrid Representations for Similarity and Label Noise **161**
6.1 Introduction . . . 161
6.2 Methods . . . 162
6.2.1 Similarity in a Hybrid . . . 163
6.2.2 Noise Conditions . . . 165
6.2.3 Noise Mitigation Methods . . . 167
6.3 Results: Similarity . . . 170
6.4 Results: Noise Detection and Estimation . . . 170
6.5 Results: Noise Mitigation by Noise Type . . . 170
6.5.1 Censoring: Correction of Clipped Labels . . . 170
6.5.2 Gaussian Noise: Methods for Learning with Noisy Labels . . . 170
6.5.3 Real Labels: Cleaning without Injected Noise . . . 170
6.5.4 Comparison across Noise Types . . . 170
6.6 Conclusions . . . 170

7 Discussion and Conclusions **171**
7.1 Summary . . . 171
7.2 Dataset Dependence of the Best Method . . . 173
7.2.1 Simple Models and Small Feature Sets . . . 174
7.3 Limitations . . . 176
7.4 Future Directions . . . 177
7.5 Conclusions . . . 179

Summary of Research Contributions **181**

A Datasets and curation **182**
A.1 Small Molecules . . . 182
A.2 The Chapter 3 versions of LogD, Caco-2, hERG K_i and QM9 . . . 186
A.3 Chemical Reactions . . . 187
A.4 Antibodies . . . 188
A.5 Nucleotide Sequences . . . 190
A.6 Cancer Genomics (TCGA) . . . 191

B Data and code availability **192**

List of Figures

2.1 Three representations of one molecule. 8
2.2 Nodes of a deterministic and a Bayesian neural network. 17
2.3 Label distributions under the seven noise conditions. 23
2.4 Evaluation of a noise mitigation method. 25

3.1 In-memory and memory-mapped data loading. 38
3.2 ANOVA variance decomposition of performance and robustness on QM9. 43
3.3 Model–representation interaction in noise robustness on QM9. 44
3.4 Noise robustness of all models on PDV for QM9. 45
3.5 Neural network families and random forests under Gaussian noise on QM9. 47
3.6 Noise degradation slope on LogD, Caco-2 and hERG K_i. 50
3.7 Noise robustness on QM9 and three assay datasets. 51
3.8 Mean predicted uncertainty under label noise on QM9. 54

4.1 Shares of constant, near-constant and varying features per representation. 63
4.2 Two hERG K_i stereoisomers with identical ECFP4 vectors. 68
4.3 Absolute pK_i difference against pair distance on hERG K_i. 74

5.1 Schematic overview of the hybrid search. 97

List of Tables

2.1 Sources of label noise in bioactivity data. 21
2.2 The seven noise conditions and their parameters. 24

3.1 ANOVA variance decomposition by noise strategy on QM9. 42
3.2 Wilcoxon signed-rank tests of probabilistic variants on QM9. 46
3.3 Noise degradation slope by model on PDV for QM9. 48
3.4 Strongest and weakest uncertainty–noise correlations on QM9. 52

4.1 Constant and near-constant features per representation, part 1. . . . 65
4.2 Constant and near-constant features per representation, part 2. . . . 65
4.3 Molecules that share a feature vector with another molecule. 69
4.4 All-pairs correlation, five-nearest-neighbor prediction and activity cliffs per dataset. 74
4.5 Agreement between representations per dataset. 75
4.6 Recovery with neighbor-based sample weights by representation. . . . 80
4.7 Flaws of standard representations and similarity, and the requirement each sets. 83

5.1 Methods for combining or reducing representations. 92
5.2 Settings of the three importance methods searched. 108
5.3 Importance method of the best configuration in a preliminary run. . . 110
5.4 The five allocation methods searched. 112
5.5 Worked example of the four fixed-share allocation methods. 113
5.6 The three augmentation conditions. 122
5.7 Values searched for each setting at the two search tiers. 124
5.8 The adaptive pruning rules. 126
5.9 Datasets used in the hybrid experiments. 129
5.10 Datasets and their best single representations. 130
5.11 Representations used for small molecules. 136
5.12 Representations used for chemical reactions. 137
5.13 Representations used for antibodies. 138
5.14 Representations used for nucleotide sequences. 139
5.15 Representations used for cancer patients. 140
5.16 Base and augmentation representations per dataset. 141

6.1 Noise magnitude estimators. 168
6.2 Label smoothing and sample weighting rules. 169

A.1 Sample counts of each dataset before and after curation. 183
A.2 Curation of the three assay datasets in two chapters. 187

List of Abbreviations

AC Activity cliff
ADME Absorption, distribution, metabolism, and excretion
AI Artificial intelligence
ANOVA Analysis of variance
ASCII American Standard Code for Information Interchange
AT Adenine–thymine
AUC_{norm} Normalized area under the retention curve
AUROC Area under the receiver operating characteristic curve
BLOSUM Blocks substitution matrix
BNN Bayesian neural network
BRCA Breast invasive carcinoma
CASH Combined algorithm selection and hyperparameter optimization
CDR Complementarity-determining region
CGR Condensed graph of reaction
CIMP CpG island methylator phenotype
CLS Classification token
CpG Cytosine–phosphate–guanine
CPU Central processing unit
CSV Comma-separated values
DeepLIFT Deep Learning Important FeaTures
DNA Deoxyribonucleic acid
DNN Deep neural network
$\mathbf{EC_{50}}$ Half-maximal effective concentration
ECE Expected calibration error
ECFP Extended-connectivity fingerprint
ECFP4 Extended-connectivity fingerprint, diameter 4
ESM Evolutionary Scale Modeling
FcRn Neonatal Fc receptor
FLuID Federated learning using information distillation
GC Guanine–cytosine
GCN Graph convolutional network
GIN Graph isomorphism network
GNN Graph neural network
GP Gaussian process
GPL GNU General Public License
GPU Graphics processing unit
H3K4me3 Trimethylation of lysine 4 on histone H3

HER2 Human epidermal growth factor receptor 2
hERG Human ether-à-go-go-related gene
HLM Human liver microsomes
HOMO Highest occupied molecular orbital
HRD Homologous recombination deficiency
IC_{50} Half-maximal inhibitory concentration
ICC Intraclass correlation coefficient
IMGT International ImMunoGeneTics information system
InChIKey Hashed International Chemical Identifier
JSON JavaScript Object Notation
KL Kullback–Leibler
kNN k-nearest neighbors
LightGBM Light Gradient Boosting Machine
LM Language model
LNL Learning with noisy labels
LOF Local outlier factor
LogD Logarithm of the distribution coefficient
LOH Loss of heterozygosity
LST Large-scale state transitions
LUAD Lung adenocarcinoma
LUMO Lowest unoccupied molecular orbital
LUSC Lung squamous cell carcinoma
MACCS Molecular ACCess System
MCC Matthews correlation coefficient
MCS Maximum common substructure
MDR1–MDCK Multidrug resistance protein 1 in Madin–Darby canine kidney cells
MHG-GNN Molecular Hypergraph Grammar Graph Neural Network
miRNA MicroRNA
ML Machine learning
MLP Multi-layer perceptron
mRMR Minimum redundancy, maximum relevance
NDS Noise degradation slope
NGBoost Natural gradient boosting
NLP Natural language processing
NN Neural network
OPIG Oxford Protein Informatics Group
PAM50 Prediction Analysis of Microarray 50
PCA Principal component analysis
PDV Physicochemical descriptor vector
PLS Partial least squares
QRF Quantile regression forest
QSAR Quantitative structure–activity relationship
RAM Random-access memory
RBF Radial basis function
ReLU Rectified linear unit
RF Random forest
RLM Rat liver microsomes

RMSE Root mean squared error
RNA Ribonucleic acid
RNA-seq RNA sequencing
RPPA Reverse-phase protein array
SELFIES Self-referencing embedded strings
SHAP SHapley Additive exPlanations
SMARTS SMILES arbitrary target specification
SMILES Simplified molecular-input line-entry system
SNS Sort & Slice
SVM Support vector machine
t-SNE t-distributed stochastic neighbor embedding
TAI Telomeric allelic imbalance
TCGA The Cancer Genome Atlas
Th1 T helper type 1
Th2 T helper type 2
TNF Tetranucleotide frequency
UCSC University of California, Santa Cruz
UMAP Uniform manifold approximation and projection
USR Ultrafast shape recognition
USRCAT Ultrafast shape recognition with CREDO atom types
UTF-8 8-bit Unicode Transformation Format
VBLL Variational Bayesian Last Layer
VEGF Vascular endothelial growth factor
WL Weisfeiler–Lehman
XGBoost eXtreme Gradient Boosting

Chapter 1

Introduction

New avenues in the drug discovery pipeline have opened up thanks to advances in large dataset collection, analysis, and computation. Therapeutic targets are identified, potential drug candidates are screened and optimized, and the most promising of these go on to pre-clinical and clinical trials [1]. There are various methods by which potential drug candidates can be screened. Virtual screening allows for the rapid and large-scale analysis of libraries of compounds, and is a particularly useful approach used to speed up the drug discovery process [2]. One standard screening method is the prediction of bioactivities and pharmacological properties of such candidates. There is a long history of using Quantitative Structure-Activity Relationship (QSAR) models to make these predictions, to act as a filter for compounds with undesirable properties, and thus reduce the time and cost associated with experimental screening [3]. QSAR models aim to predict the relationship between a molecule's molecular features as encoded within a particular molecular representation and a desired molecular property [3] of interest. A QSAR model typically consists of a molecular representation paired with a machine learning (ML) model. An ideal molecular representation will allow a model to learn only the relevant information about each molecule with respect to the target property. Representations are either 'static' hand-crafted descriptors or circular fingerprints that encode predefined structural and physicochemical features [4–7]; or learned representations, where the encoding is derived from a predictive model, often taken from the activations of the final layer of a neural network, or a latent space embedding [3, 8].

QSAR models are of course, sensitive to the quality of the input data on which they are trained. Experimental noise can arise from measurement errors, variability under experimental conditions, varying provenance of the data, and intrinsic biolog-

ical and chemical variability. Even the very measures of model performance may be flawed if test and validation sets are laden with noise. Kolmar and Grulke [9] show that the variance associated with experimental data can contribute more to prediction error than the model's own error. There is also a limit to how much noise the model can tolerate; Jorner et al. [10] found that once the model error approaches the magnitude of the experimental measurement error on labels, adding more training data yields diminishing returns in predictive performance. In the context of activity prediction, the inherent variability within experimental setups and assays, batch effects, measurement errors, and biological variability introduce additional layers of complexity [9, 11]. All of this increases the likelihood of encountering noise in the data [12]. Even the molecular representations in the training data themselves may be incorrect or misleading [13], and chemical datasets contain too few repeat measurements of each compound to characterize the distribution the label is drawn from. Thus, models treat a single label as the population mean [9]. Repeat measurements of the same compound-target pair disagree by about 0.68 log units for pIC_{50}, and by about 0.54 log units for $\mathrm{p}K_i$ [14, 15]. Measurements from a single assay lowers the mean disagreement from 0.50 to 0.27 log units [16].

This thesis addresses the challenge of conducting molecular property prediction when the underlying experimental data contain noisy labels. Most existing work on noise in QSAR modeling assumes that label noise can be modeled as random error drawn from a Gaussian distribution, and evaluates either molecular representations or predictive model architectures in isolation. Cortes-Ciriano et al. [17] examined 12 algorithms across multiple noise levels but used fixed representations. Studies that compare representations are typically done on clean data. No work has systematically compared the relative importance of representation versus architecture choice for noise robustness, assessed whether probabilistic methods justify their computational overhead in noisy settings, or tested whether findings generalize across different corruption patterns and molecular property types. The first half of this thesis therefore asks four questions about the behavior of QSAR models under label noise. First, how much of the variance in predictive performance is explained by the choice of molecular representation and by the choice of model architecture, and does that division change when we instead consider how much performance a QSAR model retains under increasing label noise? Second, what does the choice of model provide under label noise, and how do deterministic and probabilistic models compare in this respect? Third, does the type of label noise matter, or only its

quantity? To answer this, we generate and add label noise in several ways, both to clean data and to data with inherent experimental noise. Fourth, what is the impact of artificial label noise on a model's uncertainty estimates? In particular, does the estimated uncertainty rise with increasing label noise, and can it identify which labels are corrupted? Together, these questions aim to identify what, if anything, makes a QSAR model robust to noise. To answer them we introduce NoiseInject, an open-source Python package for benchmarking noise robustness and uncertainty quantification in machine learning models.

Learning with noisy labels (LNL) is the broad family of methods for training models when some labels are wrong, and noise mitigation, which removes, reweights or corrects labels, is one part of it.

We hypothesized that the fault lay in the distance between molecules on which such methods rely. On a given dataset, most features of a general-purpose representation either do not vary or do not relate to the property, yet each of these features adds a term to the distance. Together, these irrelevant features dominate the distance, and their contribution grows linearly with the number of features. Chapter 4 measures this effect on base representations and on the distances computed from them. We propose instead to select a fixed number of task-relevant features from several representations and to compute distances in the resulting compact space, which we call a hybrid representation. A first version of this approach showed that the selected feature sets also work as ordinary model inputs, without any distance involved, so the question became how to choose the features, which is the subject of Chapter 5. Chapter 6 then returns to distances, noise detection and LNL, including noise mitigation, using the hybrid representation as the feature space.

The second half of this thesis asks whether the representation itself can be adapted to the task, through five questions:

1. How can the best set of features be selected for a given prediction task?

2. Do hybrid representations achieve higher noise robustness, measured as the percentage of performance retained, than their base representations?

3. Do the features selected for a hybrid representation help stabilize distance-based noise mitigation?

4. Does weighting features by their importance improve the construction of hybrid representations, model training and noise mitigation?

5. When sample removal is suboptimal, can sample weighting or label smoothing handle Gaussian label noise better?

1.1 Outline of the Thesis and Contributions

Chapter 2 introduces the concepts on which the later chapters rely, namely molecular representations, model families, molecular similarity and activity cliffs, noise in molecular data, and the evaluation protocol. It also defines the seven noise conditions used in Chapter 3 and closes by introducing NoiseInject, the open-source benchmarking framework released with this work.

Chapter 3 studies the noise robustness of QSAR models. Its main task is the prediction of the gap between the highest occupied and the lowest unoccupied molecular orbital (HOMO–LUMO gap) on QM9, whose labels are computed and therefore carry negligible measurement noise, and the experiments are repeated on three assay datasets with experimental labels, namely LogD, Caco-2 and hERG K_i. The chapter divides the variance in predictive performance, and in the performance retained under noise, between the choice of representation and the choice of model architecture. It also compares seven noise conditions at a matched amount of noise, compares each model with its probabilistic counterpart, and examines the effect of label noise on predicted uncertainty. Model architecture matters more than representation for noise robustness, whereas the two matter about equally for R^2 under noise. The shape of the noise distribution makes little difference, while noise shared within a scaffold family and labels clipped at an assay limit (censoring) do the most damage. Natural Gradient Boosting (NGBoost) and random forests were the most robust models on QM9, and random forests were the most consistent across all four datasets. Predicted uncertainty rises with noise on average, but it does not identify which labels were corrupted.

Chapter 4 motivates the methods of the later chapters. It introduces no method of its own, and its measurements are on a smaller scale than those of Chapter 3. The chapter develops its argument in three steps. First, on a given dataset, standard representations carry many features that do not vary, and some map different molecules to the same vector. Second, similarity computed over these representations inherits both problems, and third, methods that rely on this similarity fail as a result. One such method, a noise mitigation method tested in an exploratory study of LNL carried out earlier in this project, cleans labels by comparing each molecule's

label with the labels of its nearest neighbors. The chapter ends by stating what a representation would need in order to fix these problems, which is what Chapter 5 builds.

Chapter 5, planned as the first of two method papers for submission to a venue still to be confirmed, presents the adaptive hybrid representation. From each of several base representations, the method selects the features most important for the task, divides a fixed budget of features between the representations, and combines the selected features into a single representation. The chapter also describes the datasets and the experimental setup. It will test when the hybrid representation helps, by comparing it with the best single representation and with an ensemble of one model per representation. It will also test why it helps, by comparing it with principal component analysis, minimum-redundancy maximum-relevance selection, recursive feature elimination and random features at the same number of features. We will further examine whether the selected features can be interpreted in terms of the underlying chemistry, how stable the selection is across folds, and whether search settings found on one dataset carry over to another. It will also test whether the method extends beyond small molecules, covering antibodies in the most depth and reactions, nucleotide sequences and cancer genomics as proofs of concept. Finally, it will test whether the method extends to data held at separate sites that cannot be pooled. **[RESULT NEEDED: one sentence of findings for Chapter 5.]**

Chapter 6, planned as the second method paper and also intended for a venue still to be confirmed, returns to the failures described in Chapter 4 and asks whether a hybrid representation repairs them. For similarity, it compares standard representations with hybrid representations selected for predicting each molecule's activity from its five nearest neighbors. For noise detection and estimation, it asks whether neighbors in a hybrid representation identify corrupted labels better than neighbors in a standard representation. Finally, it tests LNL methods, including noise mitigation, on one noise type at a time. **[RESULT NEEDED: one sentence of findings for Chapter 6.]**

Chapter 7 summarizes each chapter and argues that the noise in a dataset should be modeled before a method is chosen, since no single method fits every dataset and simple methods are often as good as complex ones. It then sets out the limitations of this work and directions for future research, and closes with the conclusions of the thesis.

The software contributions of this work are NoiseInject, an open-source package,

and the KIRBy software repository, which implements the search for hybrid representations and the associated experiments. The repositories, licenses and data are described in Appendix B.

1.2 Collaborations

This work was supervised by Garrett Morris and co-supervised by Thierry Hanser and Stephane Werner at Lhasa Limited, which runs the federated learning platform Effiris [18]. Lhasa also supplied the hERG data used in Chapter 5, comprising 7,772 training molecules and 4,383 test molecules, which we treat as data held at separate sites, the setting that federated learning addresses [18, 19].

The Oxford Protein Informatics Group (OPIG) pointed us to the antibody and protein datasets used in Chapters 5 and 6, and the full list of datasets is given in Appendix A. The collaboration also produced five posts on OPIG's blog, Blopig, which are listed under Blogposts in the Summary of Research Contributions.

The adaptive hybrid representation, its software and the experiments in Chapters 4 to 6 are my own work.

Chapter 2

Molecular Representations, Models and Label Noise

2.1 Overview

This chapter introduces the concepts to which every later chapter refers. It covers molecular representations, which turn a molecule into a fixed-length vector of features, the model families that learn from them, and the assumption that similar molecules have similar properties, together with the cases in which this assumption breaks down. It then turns to label noise, describing its sources in real assay data, the seven artificial noise conditions used in this thesis and the methods for LNL. The chapter closes with NoiseInject, the package that injects the noise and reports robustness and uncertainty metrics, and with the group-aware splits and metrics used throughout.

2.2 Molecular Representations in QSAR

The success of machine learning hinges on the quality of the input data. Although quality can refer to noise, on which most of this work centers, here we mean the type of input that the model receives. Molecular representations provide different features from which QSAR models and noise detection techniques can learn, and Figure 2.1 shows three of them for a single molecule. A potential bottleneck on larger datasets should nonetheless be noted, since the more complex the data structure, the greater the computational cost. On smaller datasets, the number of parameters is limited by the size of the training set in order to prevent overfitting.

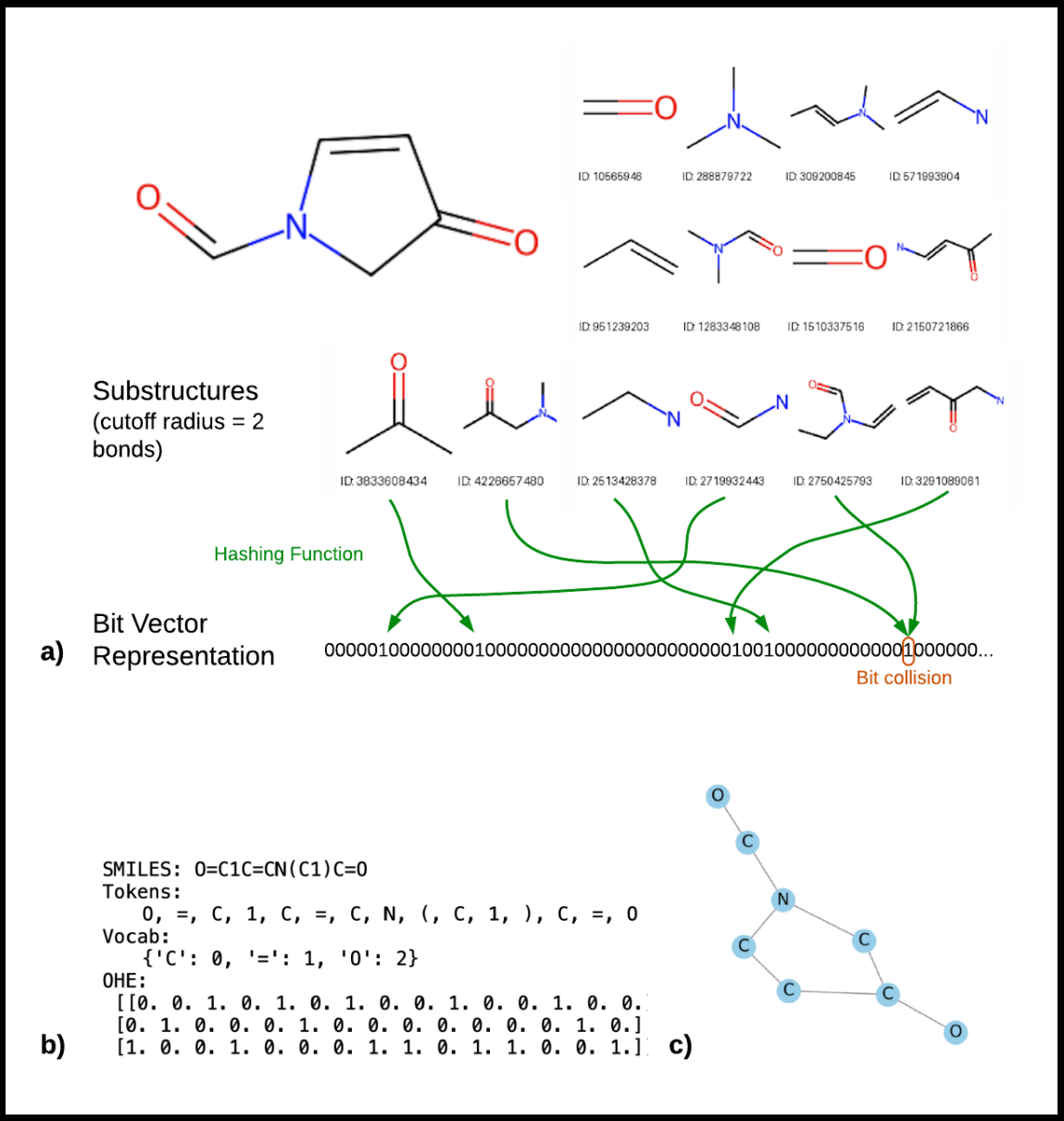


Figure 2.1: Three representations of the molecule O=C1C=CN(C1)C=O. a) ECFP4, in which circular substructures up to a radius of two bonds are hashed (green arrows) to positions in a bit vector, and two substructures can share a position (bit collision, orange). b) Tokenization and one-hot encoding of the SMILES string. c) Molecular graph, with atoms as nodes and bonds as edges.

One of the most common and intuitive molecular representations are Simplified Molecular-Input Line-Entry System (SMILES) strings which consist of a sequence of ASCII characters generated by a fixed set of rules to specify a given chemical identity [20]. A common way to use SMILES strings as inputs to QSAR models is to tokenize and one-hot-encode them rather than to use the strings themselves [21]. The strings are tokenized into atomic branches and symbols and mapped to indices of a vocabulary, and finally converted into binary vectors using one-hot encodings [21]. This representation, referred to as one-hot-encoded SMILES, allows models

to process molecular structures as bit-vector-based fingerprints or as sequences of vectors. Sequence-based models such as recurrent neural networks and transformers have proven to be particularly successful in capturing text semantics within a sequence. When applied to raw or one hot encoded SMILES strings, these models can effectively capture the structure of the molecule in its textual representation [21].

Another common representation is the molecular fingerprint, a vector containing information about the properties of the molecule in question. In a typical bit vector-based fingerprint, each bit accounts for the presence (1) or absence (0) of a given chemical feature or substructure [22]. One common fingerprint is the Molecular ACCess System (MACCS) key, a substructure key-based fingerprint that encodes predefined atom symbols and properties, as well as bond types [23]. Another is the Extended Connectivity Fingerprint (ECFP), which encodes local neighborhoods around each atom and bond connectivity in molecules [23]. ECFPs are typically constructed with a radius of 2, a length of 2048 bits, and active chirality flags. These existing molecular representations are well established and have been quite successful when coupled with QSAR models [3, 24]. They are relatively simple, and as the bits can be mapped back to one or more substructures, they are human-interpretable, and furthermore easy-to-compute. However, they are subject to information loss due to bit collisions, which happens when two different substructures are encoded at the same bit position [22]. They are also based on a fixed procedure and are not trainable or adaptable to a particular dataset. Alternative representations that capture different structural, chemical, or physical features are able to overcome these limitations.

Recent advances have led to the development of new molecular fingerprints. One such method is the Sort & Slice (SNS) technique, an alternative to hash-based folding for pooling ECFP substructures [6]. Sort & Slice begins by sorting all circular substructures in the training set according to their frequency of occurrence. It then "slices" away all but the N most frequent substructures, which are subsequently used to generate an N-dimensional, bit-collision-free binary fingerprint. This approach solves the ambiguity problem of hashed fingerprints, where multiple substructures might be encoded at the same bit position, making it very interpretable [6]. Sort & Slice fingerprints outperform ECFP of diameter four (ECFP4) with standard hash-based folding methods across various molecular property prediction tasks. These experiments consisted of different data splitting techniques, machine learning models, fingerprint diameters, fingerprint dimensions, and atomic invari-

ants [6]. Furthermore, Sort & Slice outperformed advanced substructure selection schemes such as filtered fingerprints and mutual information maximization [6].

Neural fingerprints are a category of learned representations that overcome the fixed, hard-coded nature of traditional fingerprints. Unlike ECFP or MACCS keys, which rely on predefined hashing schemes or substructure catalogs, neural fingerprints train on molecular data to capture chemical information [25, 26]. These representations are the result of deep learning architectures that have learned dense, continuous embeddings which can be used for downstream tasks.

Mol2vec, a common neural fingerprint, adapts the Word2vec algorithm from natural language processing (NLP) to molecular substructures [25]. It treats molecules as "sentences" and the circular substructures from which ECFPs are built as "words", and it learns an embedding for each substructure through unsupervised training, such that chemically similar substructures are mapped to nearby points in the embedding space [25]. The embeddings of all substructures present in a molecule are summed to give a 300-dimensional dense vector that represents the whole molecule [25]. Mol2vec has performed strongly on various QSAR benchmarks, matching or exceeding ECFPs on solubility and bioactivity prediction tasks [25].

Transformer-based architectures have also been applied to molecular representation learning, similarly stemming from NLP ideas. MolBERT applies a bidirectional transformer to learn molecular representations from SMILES strings through self-supervised pre-training [27]. The model has domain-relevant auxiliary adaptations, such as the prediction of calculated physicochemical properties alongside the masked token prediction, to improve the chemical fidelity of the learned embeddings [27]. MolBERT demonstrated improvements over traditional fingerprints and graph-based methods on Virtual Screening and QSAR benchmarks [27]. ChemBERTa is another transformer-based approach which pre-trains masked language models on SMILES data from PubChem [26]. ChemBERTa uses a tokenization strategy adapted for the SMILES notation, which allows the model to learn representations of atoms and functional groups through self-supervised learning [26]. Attention visualization reveals that ChemBERTa learns chemically meaningful patterns, with different attention heads specializing in functional groups, ring structures, or SMILES syntax [26]. ChemBERTa-2, an optimized variant pre-trained in 77 million compounds, achieved even stronger performance than the original ChemBERTa in MoleculeNet benchmarks [28].

Neural fingerprints provide several advantages over traditional fingerprints. They

produce dense continuous vectors that avoid sparseness and information loss from bit collisions [25]. They can also be pre-trained once on large unlabeled datasets and then fine-tuned for specific downstream tasks, opening the door to transfer learning [26, 27]. However, these methods are often computationally expensive [25] and due to their black-box nature, they are not as interpretable. [26]. However, attention mechanisms can have some degree of interpretability through visualization [26].

Beyond fingerprints, physicochemical-descriptor vectors (PDVs) are a class of continuous-value molecular descriptors that encode structural and chemical properties of molecules [29]. Unlike binary fingerprints or graph-based representations, PDVs aggregate atom and bond-level quantities into fixed-length feature vectors that capture a wide variety of information about a molecule and its structure [30]. PDVs typically consist of hundreds of properties including constitutional descriptors such as molecular weight, atom and bond counts, and heteroatom ratios; topological indices encoding molecular connectivity patterns; physicochemical properties such as logP (lipophilicity), molar refractivity, and topological polar surface area; and electronic descriptors derived from partial charges and polarizabilities [29, 31]. The `RDKit` cheminformatics toolkit provides a comprehensive suite of more than 200 molecular descriptors that cover these categories [32]. PDVs are very interpretable; each descriptor has a clear physicochemical meaning [31]. They are also less computationally expensive than some 3D methods or neural fingerprints, with many descriptors simply requiring a 2D molecular structure without conformer generation or extensive pre-training. This efficiency makes PDVs particularly appropriate for high-throughput applications. However, it is also possible to use descriptors based on 3D components [30].

However, PDVs have limitations. The dimensionality of PDV feature spaces can lead to problems with multicollinearity [29]. This can be mitigated by feature selection or dimensionality reduction techniques [31]. Additionally, PDVs encode molecular properties as aggregated global features, which may not capture fine-grained structural details such as specific substructure arrangements or local chemical environments, information important for particular endpoints [30].

One of the more intuitive representations are molecular graphs. These graphs can be categorized into 2D and 3D graphs. 2D graphs rely on adjacency matrices that encode topological atom- and bond-level features but leave out geometric properties such as bond angles or torsion, meaning they may fail to capture stereochemistry. 3D graphs incorporate these spatial features, but are often limited by remaining in

a single conformation [33]. A standard 3D molecular graph is constructed such that each molecule is represented as $G = (V, E)$ with node features $\mathbf{x}_i$ and edge features $\mathbf{e}_{ij}$. Nodes encode atom type, degree, hybridization, aromaticity, and formal charge; edges encode bond order and conjugation [33].

Another common molecular graph is the standard distance compatibility graph, in which atoms and bonds are mapped to sets of nodes and edges [5, 34]. These graphs are typically 2D objects that encode 3D information such as atomic coordinates, bond angles and chirality [5]. Alternative graph representations that incorporate 3D structure and physical interactions include graphs with quantum 3D embeddings, weight alignment graphs, association graphs containing pairs of molecules, and geometric interaction graph neural networks [34–36]. Du et al. [37] combined 2D and 3D graphs in a zipper-like arrangement, in which the 3D data guarantee precision and completeness by providing unabridged information and the 2D data provide chemical intuition for guidance. These representations pose various challenges, for example atoms localized at single voxels leave the points too sparse, which leads to poor performance. Kuzminykh et al. [38] addressed this problem by extending each atom to fill nearby voxels with a wave transformation.

These graph representations do not highlight specific substructures or specific arrangements of atoms and bonds within each molecule. The substructures contain functional groups that are related to a range of chemical properties and are potentially involved in downstream tasks related to the function of molecules. Bouritsas et al. [39] developed graph substructure networks, a topologically aware message passing scheme based on substructure encoding. The method involves encoding structural roles based on the appearance of specific substructures and using them in message passing updates. Since message-passing graph neural networks (GNNs) are at most as powerful as the Weisfeiler–Lehman test, they cannot isolate sufficient features from the graph structure [39]. Graph substructure networks, on the other hand, capture these substructure-related features. And recent research has highlighted the abilities of pretrained GNN encoders produce embeddings on unlabeled datasets that capture structural features [8, 40].

Kolmar and Grulke [9] examined the predictive limits of QSAR in the presence of noise and showed that, by learning the underlying trends rather than fitting the noise, models can make predictions that are more accurate than their own training labels. They assumed that experimental noise is Gaussian, arguing from the central limit theorem that measurements aggregated over multiple independent error

sources follow a Gaussian distribution. Heid et al. [41], however, used mean-variance estimation and bias-variance decomposition to show that noise can be tied to specific properties of the molecules, such as their structure. For example, nitrogen-containing functional groups introduce measurement deviations, which implies the presence of systematic noise.

2.3 Models for Molecular Property Prediction

One key issue with QSAR models is the standard assumption that labels represent true values. Most ML models rely on the assumption that the training data is, in fact, an accurate reflection of the population. Therefore, they treat these potentially noisy labels as discrete quantities rather than distributions, which opens the door to overfitting to noise and giving unduly optimistic assessments of the models.

There are a wide variety of model architectures that can be used for QSAR modeling. Starting off with simpler and more explainable models, we have tree-based methods. Decision trees are non-parametric supervised learning algorithms that perform recursive binary partitioning of the feature space [42]. A decision tree builds a predictive model by learning a series of if-then rules that recursively split the data based on given feature values, creating a tree where internal nodes represent decision rules, branches represent the results of those decisions, and leaf nodes contain predicted values or class labels [42]. Each recursive split is chosen to maximize homogeneity within the resulting partitions. For classification, Gini impurity is commonly used and for regression, mean squared error reduction [42].

There are several properties that make decision trees a good foundation for QSAR modeling. First, they handle non-linear relationships between molecular descriptors and target properties without requiring explicit feature engineering or transformation [42]. They also do not make assumptions about the underlying distribution of the data, making them robust to violations of parametric assumptions that can limit linear regression or other classical methods [43].

Decision trees handle data in a "contributory" rather than "consensus" manner. In consensus methods such as linear regression or neural networks, every data point influences the prediction at every other point in the feature space. Tree-based methods make predictions based on the subset of training samples that fall into the same leaf node as the test point [42]. This mechanism essentially quarantines outliers in their respective leaf nodes, limiting their impact on predictions for samples in other

regions or chemical domains.

Individual decision trees tend to have high variance and often overfit, particularly at large depths [42]. Small modifications made to the training data can result in large differences in the structure of the tree, resulting in unstable predictions [44]. Ensemble methods are used to counteract this instability, aggregating predictions from many trees. Random forests (RFs) aggregate predictions from trees trained on bootstrap samples of data with random subsets of features at each split [44]. Gradient boosting methods such as eXtreme Gradient Boosting (XGBoost) sequentially fit trees to the residuals of previous iterations, progressively reducing the prediction error [45]. Bootstrapping ensures that each tree is trained on a slightly different subset of data, which effectively reduce the influence of any individual sample [44]. This is especially beneficial with noisy samples. The selection of random features at every node decorrelates the trees, preventing systematic biases from propagating through the ensemble [44]. The aggregation of trees further prevents overfitting to noise by smoothing out prediction errors from individual noisy labels.

Neural networks (NNs), a very popular family of models, can also be used for QSAR. A NN is a system of interconnected nodes which learn by passing information through it's layers. There are weights and biases attached to the nodes and various other parameters which the model optimizes to minimize the difference between the predicted output and the actual target value. During training, the network parameters are updated using backpropagation, in which gradients of the loss function with respect to each weight are propagated backward through the network [46]. The structure of these networks and the hyperparameters used depend on the type of NN. In QSAR, one of the most commonly used architectures is the multilayer perceptron (MLP), a NN containing fully-connected layers with non-linear activation functions which transforms molecular feature vectors (i.e., fingerprints, descriptors) into a property or activity prediction [47].

For molecular property prediction, it is recommended that architectures use 2-3 hidden layers with 128-512 units each, rectified linear unit (ReLU) activations, and dropout regularization to prevent overfitting. Training is performed using backpropagation with gradient descent optimizers such as Adam, to minimize a loss function (mean squared error for regression, cross-entropy for classification) on batches of training data [48].

However, NN performance on QSAR-related tasks has been very mixed. They are often outperformed by more traditional methods: decision trees and related en-

semble models tend to perform stronger on smaller datasets, overfit less, and don't rely on a static 3D molecular structure like NNs do [49, 50]. Their complexity and use of gradients make them prone to label memorization [51], which in the context of noise is problematic, as it means that models may memorize the noise themselves. Although models that are memorizing labels may initially appear to have strong predictive performance, they cannot generalize out of distribution samples properly and often provide inaccurate uncertainty estimates. Various approaches have been proposed to improve neural network robustness to label noise, including noise-robust loss functions [51], confident learning to identify mislabeled examples [52], and regularization techniques such as mixup or label smoothing [51].

All the models up to this point have assumed a vector-based input. Molecular graphs, on the other hand, require different types of predict modeling. Two standard ML methods on molecular graphs include Graph Convolutional Networks (GCNs) and Graph Isomorphism Networks (GINs). GCNs apply the concept of convolution to graph-structured data. Instead of operating on grid-based data, such as images, GCNs aggregate information from neighbors of a node to update its representation [21, 53]. In contrast, GINs address the limitations in GCNs' ability to differentiate between non-isomorphic graphs. Unlike GCNs, which normalize and weight the sum of neighboring node features, GINs use a simple sum followed by a trainable MLP, giving the model flexibility to learn complex transformations of node features. The GIN model's design is informed by the Weisfeiler–Lehman test for graph isomorphism, enabling it to distinguish between molecular graphs that are structurally similar but differ in subtle ways [21, 53]. This makes GINs particularly suited to activity cliffs (ACs), where small variations in molecular structure can lead to significant changes in chemical or biological activity [21].

2.3.1 Tree Ensembles

RFs are among the most common choices for QSAR modeling because of their robustness and interpretability [43]. Quantile regression forests (QRFs) keep the distribution of training labels in each leaf and compute quantiles over all trees [54].

Gradient-boosted tree ensembles include XGBoost [55, 56], Light Gradient-Boosting Machine (LightGBM) [57], and NGBoost [58]. NGBoost extends deterministic gradient boosting by treating the parameters of a chosen parametric distribution as regression targets, which it learns through boosting with a natural

gradient update rule [58].

2.3.2 Support Vector Machines

Support vector machines (SVMs) are a well-established baseline in QSAR modeling [43, 59].

Support vector regression fits a function that is as flat as possible while keeping training errors small [59, 60]. It uses the ϵ-insensitive loss, under which errors smaller than ϵ cost nothing and larger errors cost their size minus ϵ. A constant C trades flatness against tolerance of large errors, and classification uses the same machinery with a maximum-margin boundary [61]. The features enter the model only through the kernel. With the radial basis function (RBF) kernel, the kernel is a function of the Euclidean distance, so every feature enters every distance, and we therefore standardize the features before fitting. Three properties of the method suggest robustness to label noise. Errors inside the ϵ tube change nothing, the loss outside the tube grows linearly rather than quadratically, and the weight of each molecule is capped at C [60]. Under simulated experimental error, the RBF-kernel SVM degraded smoothly and linearly [17], and Chapter 3 reports how it holds up under each of our noise conditions. Its main limitation is computational cost, because training operates on a kernel matrix that grows with the square of the number of training molecules [62].

2.3.3 Gaussian Processes

Gaussian processes (GPs) a non-parametric class of models that use a kernel to produce a Gaussian predictive distribution over every data point [63, 64]. One of the major limitations of using GPs in practice is computational scaling; inference has $O(N^3)$ time complexity and $O(N^2)$ memory, where N is the number of training points [64]. This can be mitigated with sparse GP approximations, but these introduce approximation error and can be more difficult to tune [65].

The `Gauche` framework [66] defines kernels for strings, fingerprints, and graphs that operate on a range of widely used molecular representations. The Tanimoto kernel [67, 68] is defined for binary vectors $\mathbf{x}, \mathbf{x}' \in \{0,1\}^d$ for $d \geq 1$ as:

$$k_{\text{Tanimoto}}(\mathbf{x}, \mathbf{x}') := \sigma_f^2 \cdot \frac{\langle \mathbf{x}, \mathbf{x}' \rangle}{\|\mathbf{x}\|^2 + \|\mathbf{x}'\|^2 - \langle \mathbf{x}, \mathbf{x}' \rangle}, \tag{2.1}$$

where $\|\cdot\|$ represents the Euclidean norm [66].

The RBF kernel is defined for $\mathbf{x}, \mathbf{x}' \in \mathbb{R}^d$ as:

$$k_{\text{RBF}}(\mathbf{x}, \mathbf{x}') := \sigma_f^2 \cdot \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\ell^2}\right), \tag{2.2}$$

where ℓ is the lengthscale.

2.3.4 Bayesian Neural Networks

NNs are often deterministic; however, they can be transformed into probabilistic models (Figure 2.2).

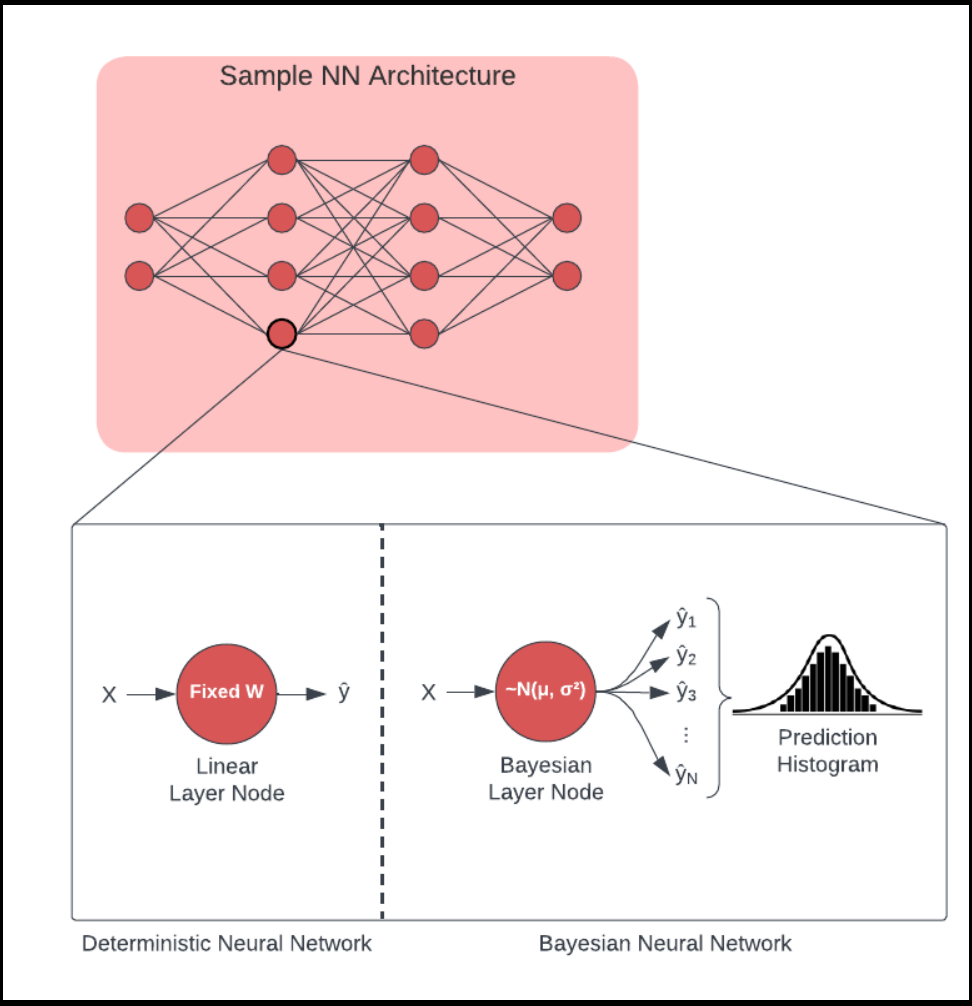


Figure 2.2: One node of a deterministic and of a Bayesian neural network. The deterministic node (bottom left) applies a fixed weight W and returns one prediction $\hat{y}$. The Bayesian node (bottom right) draws its weight from $\mathcal{N}(\mu, \sigma^2)$, so repeated predictions $\hat{y}_1, \ldots, \hat{y}_N$ form a histogram.

Unlike standard deterministic ML methods, Bayesian methods model distributions over predictions using Bayes' rule, allowing for the quantification of uncertainty

within parameters and labels. Bayesian methods treat the model parameters as random variables with distributions. Uncertainty can also be estimated from other non-deterministic methods, including deep ensembles [69], bootstrap modeling [70], and federated learning [19].

Epistemic uncertainty is model-driven and can be reduced by collecting either more or higher quality training data. Aleatoric is data-driven and reflects the intrinsic noise in the labels.

2.4 Molecular Similarity and Activity Cliffs

Distance-based reasoning rely on the assumption that the data embeddings contain quality information. When the labels are corrupted or the important features are scarce, similarity scores over-rely on compounding false or useless information, and the assumption that similar molecules have similar properties breaks down. Bajusz et al. [71] found that while the Tanimoto coefficient can be effective with binary fingerprints, changing the similarity formula yields minimal improvement if the underlying representation is weak. This problem is further complicated by ACs, which are defined as structurally similar molecules with large property differences, often due to a subtle chemical change [72, 73]. These molecules, which potentially contain the most important information for model prediction, can be confused with noise. Another barrier to distance-based reasoning is persistent noise across labels. The task of trying to determine if a molecule's value matches that of its neighbors becomes a lot harder when all of the neighbors have some form of noise as well.

Much of cheminformatics rests on the similar property principle, which states that structurally similar molecules tend to have similar properties [74, 75]. One of its clearest applications is nearest-neighbor prediction, which appears across cheminformatics workflows and, in this thesis, in the kernels used by GPs and at the basis of many noise identification algorithms.

The difficulty with this principle is that similarity is defined only relative to a similarity measure, which is fixed by the choice of representation and coefficient [76]. There is no standard definition of similarity between molecules, especially when different molecular embeddings are handled in a computational setting. Two molecules that are near-identical under a substructure fingerprint may be far apart under a PDV, and Maggiora et al. [76] caution that quantified similarity can be over-interpreted in this way.

The most common measure for binary fingerprints is the Tanimoto coefficient [71]. For two molecules with fingerprints $\mathbf{x}, \mathbf{x}' \in \{0,1\}^d$,

$$T(\mathbf{x}, \mathbf{x}') = \frac{\sum_{j=1}^{d} x_j x'_j}{\sum_{j=1}^{d} x_j + \sum_{j=1}^{d} x'_j - \sum_{j=1}^{d} x_j x'_j}, \tag{2.3}$$

which is the number of bits set in both molecules divided by the number of bits set in either. Continuous representations are instead compared with a distance, most often the Euclidean distance $\sqrt{\sum_{j=1}^{d}(x_j - x'_j)^2}$. Both measures are sums over the features, and neither weighs a feature by its relation to the property. In the Tanimoto coefficient, a bit that is off in both molecules contributes nothing, whereas every bit that is set in either molecule counts. In the Euclidean distance, every feature that differs between the two molecules counts. Chapter 4 returns to this point (Equation 4.3).

There are many downstream tasks which rely on the concept of similarity. The second half of this thesis relies on many such tasks, and as such focuses on identifying representations that agree with this principle.

The sharpest failure of the similar property principle is the activity cliff, a pair of structurally similar molecules with a large difference in potency that is produced by a small chemical change [72, 77]. Activity cliffs inherit the measure dependence of the principle they violate, and there is no widely accepted definition of one [75]. The MoleculeACE benchmark used in this thesis makes this choice explicit. A pair is considered highly similar if its similarity exceeds 90% under at least one of three measures, namely the Tanimoto coefficient on ECFPs, the Tanimoto coefficient on scaffold fingerprints and a scaled Levenshtein distance on SMILES strings. Such a pair forms a cliff if the two molecules also differ tenfold or more in K_i or EC_{50} [75]. This soft consensus over three measures is used because each measure captures only part of the underlying chemistry. Across the thirty MoleculeACE targets, cliff compounds account for between 7% and 52% of a dataset.

Dimensionality itself poses a further difficulty. Aggarwal et al. [78] show that as the number of dimensions grows, the distances to the nearest and to the furthest point become increasingly similar, so that pairwise distances become progressively less discriminative. Molecular representations are routinely high-dimensional and sparse, and when a large proportion of their dimensions carry no information relevant to the property being modeled, distances computed over them are dominated by

those uninformative dimensions.

2.5 Noise in Molecular Data

Real-world cheminformatics datasets contain label noise arising from multiple sources across the drug discovery pipeline, and modeling experimental noise properly requires first understanding where it comes from.

Studies involving noise, and specifically the quantification of noise robustness, typically approach label noise by adding independent, zero-mean, Gaussian error to the labels. However, there's evidence to suggest that noise from experimental assays takes on different forms. Krüger and Overington [79] rejected normality of bioactivity differences by Anderson-Darling test and fitted a Laplace distribution instead. Kalliokoski et al. [14] examined the widest disagreements among 16,844 pairs of repeat pIC_{50} measurements and found that in almost all cases, the wide gap between repeat values was due to annotation error rather than to an imprecise assay.

Beyond just the shape, the assumptions of independence and zero mean may fail as well. Bentz et al. [80] examined P-glycoprotein transport measurements across 23 laboratories and found that 62% of the variance in log efflux ratio was due to differences between laboratories. When you pool data from different sources, each source has the potential to carry a shared offset. These sources usually contain families of related compounds, breaking the independence assumption. The zero-mean assumption is not assured either, especially when true values fall above or below an assay's detection limit, called censoring. Svensson et al. [81] count censored labels in fifteen industrial assays, of which thirteen carry censored labels and eight sit between a quarter and two thirds censored. In this study, we tested noise conditions that vary the shape, the correlation, and the bias of injected error separately at a matched quantity.

Label noise enters bioactivity data at every step from the assay plate to the database. Table 2.1 lists these sources, with examples of each and the kind of error each tends to produce. The kind of error describes how the error is distributed across molecules, which is the property that the noise conditions in Section 2.5.1 vary.

These sources of noise can interact with one another and produce errors that a simple Gaussian does not describe. Unlike artificially injected noise, real-world

Source	Example	Kind of Error
Assay	Edge effects at plate boundaries from evaporation or temperature gradients [82]; carryover between wells, incomplete liquid transfers and air bubbles [83]; reagents or compounds degrading during a run [84]	Error that varies with plate position or read time
Instrument	Higher relative error near a detection limit [85]; fluorescence quenching by the test compound [86]; background subtraction in radiometric assays [85]	Larger error for weak binders; a systematic bias; labels clipped at the limit
Biology	Seeding density, growth rate, cell cycle and serum properties [87]; passage speed [88]	Variation between repeats of the same experiment
Aggregation across sources	ChEMBL, BindingDB and PubChem pool hundreds of laboratories [89, 90]; K_i or IC_{50} values for one compound–target pair spanning two orders of magnitude [15]	An offset shared by each laboratory or source, hard to separate from the true value without metadata [91]
Federated learning	Teacher models label a shared public set, and a student model learns from those labels [18]	The teachers' prediction errors on top of the measurement error
Annotation	Mis-registered structures, wrong stereochemistry and salt forms [92]; 1–5% structure errors in public databases [93]; ambiguous protonation states and counterions [94]	A label attached to the wrong molecule; distinct molecules merged

Table 2.1: Sources of label noise in bioactivity data, with reported examples and the kind of error each source produces.

noise is neither independent, identically distributed, nor symmetric [41]. We address this complexity by examining several ways of both modeling noise and adding it to datasets.

Although the general assumption in many of the noise analysis and mitigation literature is that noise, specifically label noise, will have a negative effect on the model, this is not necessarily the case. There are multiple instances of using noise as a method of regularization. Although not the optimal approach to counteracting overfitting in linear regression, adding artificial noise to labels is considered a form of regularization [95]. Furthermore, adding artificial noise drawn from a Gaussian distribution to the activation layers of a neural network is considered another form of regularization [96]. These studies indicate that the presence of noise may act as a form of regularization in some contexts. It is important to address this potential of regularization from label noise in QSAR and quantitative structure–property

relationship models.

2.5.1 Artificial Noise Injection

Our objective is to evaluate the robustness of QSAR models against label noise. To do so, we "inject" artificial noise into the labels during training while validation and test data remain free of noise. Validation labels are given noise of the same type and quantity as the training labels, drawn independently. This artificial noise simulates real-world experimental noise. The clean target label y_i is replaced by a noisy label $\tilde{y}_i = y_i + \epsilon_i$, where ϵ_i is the artificial noise injected at the index i, the value of which is determined by a specified noise strategy. As we increased the quantity of noise, we measured how much predictive performance fell.

We explored seven ways of adding noise, which we refer to as noise conditions and which are illustrated in Figure 2.3 and summarized in Table 2.2. Three conditions vary the shape of the distribution from which the error is drawn, and three concentrate the error on some molecules or share it within Bemis–Murcko scaffold families. The seventh condition, censoring, clips labels at a limit.

For a given label, every condition except censoring delivers the same expected quantity of noise at a set noise level. These levels are defined as a fraction of the standard deviation of clean training labels. The noise added to the label of molecule i is

$$\epsilon_i = \frac{\tau}{G}\, s_i z_i, \qquad G = \sqrt{\frac{1}{n}\sum_{j=1}^{n} s_j^2}\;\; \sigma_z, \tag{2.4}$$

Here τ is the target standard deviation of the noise in the units of the label, and z_i is drawn from the distribution of the condition at unit scale. The multiplier s_i equals 1 for most molecules and $\lambda = 3$ for the molecules that the condition affects, which are a random 10% of molecules under the Outlier condition and whole scaffold families covering 20% of the split under the Grouped-wider condition. The constant σ_z is the standard deviation of z, which equals 1 for the Gaussian, $\sqrt{\nu/(\nu-2)}$ for the Student-t and $\sqrt{2}$ for the Laplace distribution. The factor G divides out both of these quantities, so that ϵ has a root-mean-square value of τ in expectation, whatever the distribution and whichever molecules are affected. This matching is the new element of our design. Most noise-robustness studies vary only the amount of Gaussian noise, whereas here every condition except censoring delivers the same root-mean-square noise at each level, so that the conditions differ only in how the

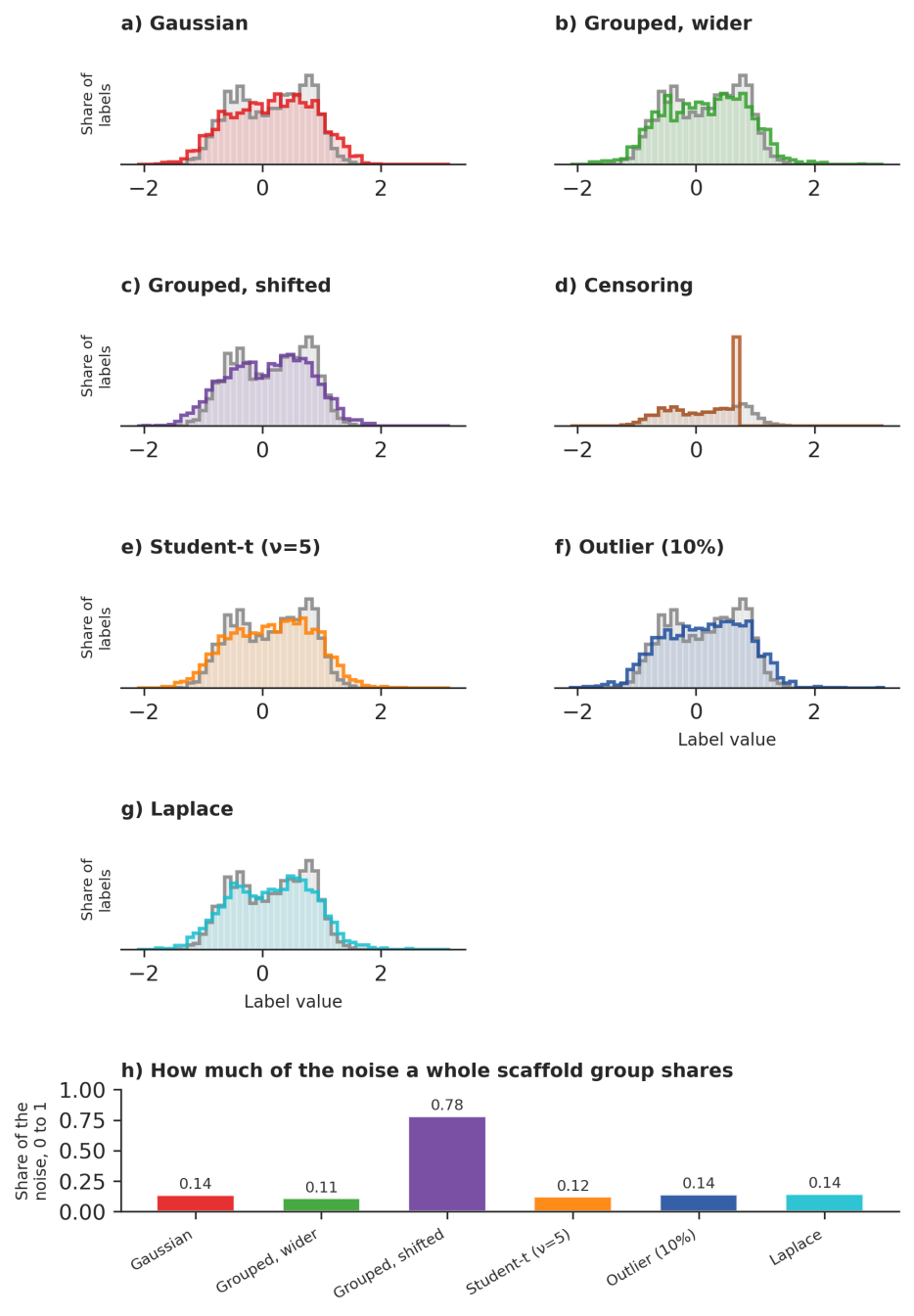


Figure 2.3: Label distributions under the seven noise conditions, for 2,000 synthetic labels in 40 random groups that stand in for scaffold families. a)–g) Clean (gray) and noised (color) labels at a noise level of 0.5 times the standard deviation of the clean labels, with 25% of labels clipped under censoring. h) Standard deviation of the mean error within a group divided by that of all errors. Noise that ignores the groups gives about 0.14, and censoring is omitted.

noise is spread across molecules.

The Grouped-shifted condition widens no molecule's error and instead gives every scaffold family g an offset of its own,

$$\epsilon_i = \frac{\tau}{G}\left(\sqrt{\rho}\, b_{g(i)} + \sqrt{1-\rho}\, w_i\right), \tag{2.5}$$

with one draw b per family and one draw w per molecule, where the two variances sum to τ^2 and the family offset carries a share ρ of that total.

We selected the parameter values from published measurement errors wherever possible, and Table 2.2 notes where no published estimate exists.

Condition	**What It Changes**	**Value**	**Source for the Value**
Gaussian	Shape: every label's error is Gaussian	–	The standard assumption in noise-robustness studies
Student-t	Shape: heavier tails than Gaussian	$\nu = 5$	Distributions fitted to repeated bioactivity measurements [79]
Laplace	Shape: sharper peak and heavier tails than Gaussian	–	As for Student-t [79]
Outlier	Error λ times wider on a random share of molecules (Huber's contamination model [97])	10% of molecules; $\lambda = 3$	1–10% of routine data contain gross errors, upper bound taken [98]; inter-laboratory error is about three times intra-laboratory error [99]
Grouped-wider	Error λ times wider on whole scaffold families	Families covering 20% of molecules; $\lambda = 3$	No published estimate of the share; λ as for outlier [99]
Grouped-shifted	Every scaffold family gets its own offset; no error is widened	$\rho = 0.62$	Laboratory identity explains 62% of the variance in Caco-2 across 23 laboratories [80]
Censoring	Labels past a limit are set to the limit; the level is the fraction clipped	Set by the noise level	Censored labels in industrial assays [81]

Table 2.2: The seven noise conditions and their parameter values. ν is the number of degrees of freedom of the Student-t distribution, λ the widening factor (Equation 2.4) and ρ the share of the noise variance carried by the scaffold-family offset (Equation 2.5). For censoring, the noise level is the fraction of labels clipped.

2.5.2 Learning with Noisy Labels

LNL covers every method for training a model when some of its labels are wrong, and we divide it into two parts. Noise mitigation changes, drops or reweights the training labels once, before any model is trained, so that any model can then be fitted to the result. Its methods include removal, down-weighting, smoothing, relabeling and correction, and Figure 2.4 shows how such a method is evaluated. Noise-robust training instead leaves the labels unchanged and alters how the model trains, through robust losses, noise-modeling layers, regularization, or reweighting inside the training loop, as in co-teaching and Meta-Weight-Net. A sample-selection step that is taken out of its training loop and used only to delete molecules before a refit counts as noise mitigation. Noise detection and noise estimation, which find noisy labels or estimate how many there are, feed into noise mitigation but are not LNL methods themselves. Song et al. [100] sort 62 deep-learning methods into five groups, namely robust architecture, robust regularization, robust loss function, loss adjustment and sample selection.

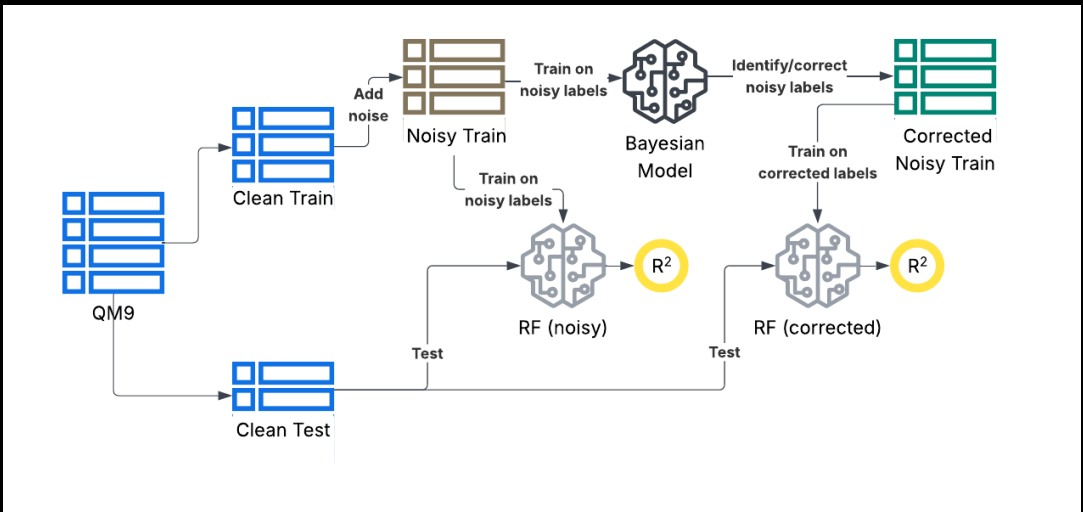


Figure 2.4: Evaluation of a noise mitigation method, with QM9 as the example. Noise is added to the training labels only. A random forest trained on the noisy labels is compared with one trained on labels corrected by a Bayesian model, and both are scored by R^2 on the clean test set.

No single method is expected to win, since averaged over all problems no learning algorithm outperforms another [101, 102]. The analog in chemistry is a study of twelve algorithms on twelve bioactivity datasets with simulated experimental errors [17]. Neighbor-based cleaning dates back to editing a training set by its nearest neighbors [103], and it relies on similar molecules having similar labels, an assumption that activity cliffs break. QSAR models also frequently fail to predict activity

cliffs [73], and Chapter 6 tests this assumption.

2.6 NoiseInject: A Benchmarking Framework

The noise conditions described above are implemented twice. The three experimental datasets and every uncertainty result use NoiseInject, an open-source Python package, whereas QM9 uses a separate pipeline, written in Python and Rust, that carries its own Rust implementation of the same conditions. An automated check compares the two implementations before any experiment is run on the computing cluster. Because the random number generators of Rust and NumPy produce different streams, the check compares statistics rather than individual draws. On the same labels, groups and seeds, it compares the delivered noise, the factor G, the median error, the number of badly corrupted labels and the number of molecules affected.

2.6.1 NoiseInject

NoiseInject (version 1.0.0) evaluates model robustness and uncertainty calibration under artificial label noise, for both regression and classification. It operates on arrays of labels and on the predictions of any externally trained model, which makes it model-agnostic, and its core depends only on NumPy, pandas, SciPy and scikit-learn. Results are returned as `pandas` data frames with one row per noise level and one aggregate summary row. The software is available under an MIT license and archived on Zenodo [104].

Regression noise. The regression injector specifies a noise shape (Gaussian, Student-t or Laplace) and a targeting rule (Uniform, Grouped-wider, Grouped-shifted, Outlier or Censoring) separately. Each named condition in the package is one combination of the two, together with its parameters, and this name is recorded with every result. The package defines more settings than this study uses, namely Student-t distributions with $\nu \in \{3, 5, 10\}$ and outlier contamination of 1%, 5% and 10% of molecules. It also defines an outlier variant in which the unselected molecules receive no noise at all, rather than the base-scale noise of Equation 2.4.

The requested noise level is the root-mean-square noise τ to be delivered, in the units of the label. The injector computes the per-molecule multipliers s_i and solves

for the scale τ/G in closed form (Equation 2.4), so no calibration search is needed. A level of zero returns the labels exactly unchanged. The affected molecules are selected with a second random number generator, which is seeded separately from the noise draws and re-seeded on every call. As a result, the affected molecules are the same at every noise level, including zero, while the noise draws themselves change. Under the Grouped-wider condition, whole scaffold families are added in random order until the fraction of affected molecules is as close as possible to the target of 20%, because a fixed fraction of families gives a very different fraction of molecules when family sizes are uneven.

The Censoring condition is parameterized by the fraction f of labels clipped, and its limit is the $(1 - f)$ quantile of the clean training labels. On the training split, the censored set is therefore exactly the $k = \mathrm{round}(fn)$ largest labels, whereas on any other split it consists of whichever molecules lie past the training limit.

Alongside the noisy labels, each injection returns a record of how they were produced. This record contains the target amount of noise, the factor G, the solved scale, the delivered root-mean-square noise (in label units and as a fraction of the clean label standard deviation), the mean of ϵ, the fraction of molecules affected, the number of groups and the largest share held by one group, and the random seeds. The injector also checks the delivered amount of noise. Because the delivered amount is an estimate from a finite sample, it is compared against a band of three standard errors,

$$\left|\frac{\hat{\tau}}{\tau} - 1\right| \le \max\left(3\sqrt{\frac{\kappa - 1}{4\, n_{\mathrm{eff}}}},\ 0.005\right), \qquad n_{\mathrm{eff}} = \frac{\left(\sum_i s_i^2\right)^2}{\sum_i s_i^4}, \tag{2.6}$$

where $\hat{\tau}$ is the delivered root-mean-square noise and κ is the sample kurtosis of the unscaled draws z, bounded between 3 and 60. The effective count n_{eff} is the number of independent contributions over which the delivered amount of noise averages, and it falls below n when the noise is concentrated on a few molecules. Under the Grouped-shifted condition it is $n_{\mathrm{eff}} = \left(\rho^2/g_{\mathrm{eff}} + (1-\rho)^2/n\right)^{-1}$, with $g_{\mathrm{eff}} = \left(\sum_g n_g\right)^2 / \sum_g n_g^2$ for family sizes n_g. At $\nu \le 4$ the Student-t distribution has no fourth moment, so the standard error is instead $1.3\, n_{\mathrm{eff}}^{2/\nu - 1}$, with the constant calibrated on the measured spread. A draw outside the band raises a warning rather than an error. At three standard errors, a correct injector falls outside the band on about 1% of draws, and the delivered amount is recorded with the result in either case.

A second function of the package returns the noise scale that each molecule's region would receive, without drawing any noise. We use it to score held-out molecules against the pattern of noise to which the training labels were exposed. Under the Uniform and Grouped-shifted targeting rules this scale is the same for every molecule, so the package flags the result as degenerate, because a rank correlation against a constant is undefined and must not be reported as zero.

A separate injector sizes the noise against a unit supplied by the user rather than against the label standard deviation. In the experiments implemented in the KIRBy repository, this unit is the standard deviation of a model's out-of-fold residuals on the clean training labels, so that the noise is sized against the error the model already makes. Its conditions either add Gaussian noise with variance equal to the level times the square of the unit, or move 2%, 5% or 10% of labels by the level times the unit, with a random sign or always upward. They can also add Gaussian noise whose standard deviation follows either a per-molecule covariate or a hidden log-normal draw, and both of these are scaled to the same mean variance as the first condition.

Classification noise. The package provides six label-flipping strategies, namely uniform flips, flip rates set by class frequency, asymmetric flip rates for binary tasks, a random flip rate for each sample, a separate flip rate for each class, and flips directed by a confusion matrix. Because the realized flip rate differs from the nominal flip probability under most of these strategies, a binary search finds the flip probability that produces a target fraction of changed labels.

Robustness metrics. For regression, the package computes R^2, the root mean squared error (RMSE) and the mean absolute error at every noise level. For classification, it computes accuracy, together with macro- and weighted-averaged precision, recall and F1 score and a per-class breakdown. Robustness is read from the retention curve $\mathrm{ret}(x) = m(x)/m(0)$, where m is a bounded metric for which higher values are better (R^2 for regression; accuracy, F1, precision or recall for classification) and x is the noise level. The primary summary is the normalized area under this curve,

$$\mathrm{AUC}_{\mathrm{norm}} = \frac{1}{x_{\max} - x_{\min}} \int_{x_{\min}}^{x_{\max}} \mathrm{ret}(x)\,\mathrm{d}x, \tag{2.7}$$

evaluated by the trapezoid rule. It is the mean fraction of clean performance retained across the noise levels, lies approximately within $[0, 1]$, and does not depend on how well the model performed on clean labels. A secondary summary fits $\mathrm{ret}(x) \approx \exp\bigl(-(x/\tau_W)^\beta\bigr)$ to the curve. The shape parameter β separates curves that hold and then collapse ($\beta > 1$) from curves that fall early and then plateau ($\beta < 1$), while τ_W sets the scale of the decay, and the fit requires at least four noise levels. These two summaries replace the linear slope of performance against noise used in earlier versions, which misdescribed a nonlinear curve and was coupled to clean performance. The package also reports the clean baseline and the retention at the highest noise level. A flag marks any curve that falls below zero retention, and an optional threshold withholds the curve summaries for models whose clean baseline is too low for a meaningful degradation.

Uncertainty metrics. For a model that returns a predicted standard deviation u_i for each sample, the package computes several metrics at each noise level. These are the Spearman correlation between u_i and the absolute error $|y_i - \hat{y}_i|$, the empirical coverage $\frac{1}{n}\sum_i \mathbb{1}[|y_i - \hat{y}_i| \le k u_i]$ at $k = 1$ and $k = 2$ against Gaussian targets of 68.3% and 95.4%, the mean interval width $\frac{1}{n}\sum_i 2u_i$, and the expected calibration error (ECE),

$$\mathrm{ECE} = \sum_{b=1}^{10} \frac{|B_b|}{n} \left| \bar{u}_b - \overline{|y - \hat{y}|}_b \right|, \tag{2.8}$$

computed over ten equal-count bins B_b of u. All of these metrics are computed on the clean test labels. The Spearman correlation between u_i and the injected noise $|\epsilon_i|$ can only be computed where the noise is known, which is on the corrupted molecules, so it requires a separate pair of inputs. It is reported within each noise level, and a correlation pooled across all levels is returned separately. The pooled value mostly reflects the fact that uncertainty and noise both rise with the level, so it does not measure whether a model can identify which molecules are corrupted.

Three optional wrappers produce u_i from a point predictor. The first applies split-conformal prediction to any scikit-learn-compatible model, optionally scaled by the label spread of each molecule's nearest training neighbors. The second fits an exact Gaussian process with a Tanimoto or RBF kernel through `Gauche` [66], and the third applies Monte Carlo dropout to any `PyTorch` network [105]. Example notebooks and scripts cover regression and classification from start to finish.

2.7 Evaluation Methodology

RMSE and the coefficient of determination (R^2) were primarily used to compare QSAR predictive performance.

When evaluating uncertainty, we assessed the relationship between predicted uncertainty and prediction quality. Spearman's rank correlation coefficient (ρ) was used to quantify the relationship between predicted uncertainty (u_i) and absolute error ($|y_i - \hat{y}_i|$), as well as between predicted uncertainty and injected noise magnitude. Higher correlations indicate that the model's uncertainty estimates reliably identify predictions that are likely to be incorrect.

For probabilistic models, the empirical coverage at 1σ and 2σ and the ECE (Equation 2.8) are defined in Section 2.6.1, together with the other NoiseInject uncertainty metrics.

2.8 Conclusions

This chapter has introduced both halves of a predictor, namely the representation and the model. Because real assay noise is not independent, identically distributed or symmetric, our artificial noise model comprises seven conditions. We have also set out the methods for LNL, of which noise mitigation is the subset that acts on the labels, and noted that no single method wins on every dataset. NoiseInject injects these noise conditions and reports the robustness and uncertainty metrics, and the section on evaluation methodology fixes the splits and metrics. Chapter 3 asks whether the representation or the model architecture governs how strongly a predictor degrades under label noise, through the four questions set out in Chapter 1. The representations, similarity definitions and noise conditions introduced here are reused in Chapters 4 to 6.

Chapter 3

Noise Robustness Across Molecular Representations and Model Architectures

3.1 Introduction

While there have been many investigations into the effect of noise on molecular property prediction, direction comparisons between how different molecular representations responds to label noise remains limited. The choice of molecular representation certainly affects predictive performance. Deng et al. [106] evaluated 62,820 models and found that ECFPs frequently outperform learned graph embeddings across models. Whether these conclusions translate to noise robustness is our question. Between different types of label noise, Heid et al. [41] found no difference in model performance between error distributions with the same mean and standard deviation. We expand on this by considering alternative patterns of noise, namely noise concentrated in structural groups or on a sparse subset of samples.

This chapter addresses the first four questions set out in Chapter 1, namely how representation and model architecture divide the variance in predictive performance and in noise robustness, what probabilistic models add, whether the type of label noise matters, and whether uncertainty estimates track label noise.

3.2 Methods

We used four datasets, namely QM9 with the HOMO–LUMO gap as the prediction target and three assay datasets for LogD, Caco-2 and hERG K_i. On each dataset we combined six representations with 20 model configurations, which comprise thirteen base models and seven variant models. Every pairing of representation and model was exposed to seven noise conditions (Section 2.5.1) at seven noise levels, and the hyperparameters of every model are given in Additional file 1.

3.2.1 Datasets

Although we are investigating the effects of label noise stemming from experimental settings, for the purposes of testing we inject controlled amounts of artificial noise into clean labels. Data were extracted from the QM9 molecular property dataset distributed by PyTorch Geometric [107], which contains 130,831 small organic molecules with pre-computed quantum-mechanical properties obtained from density functional theory calculations at the B3LYP/6-31G(2df,p) level of theory. We selected the HOMO–LUMO gap as the prediction target in QM9, as it captures a molecule's electronic excitability, charge transfer capability, and chemical stability [108]. We assumed that QM9 labels were free of noise, notwithstanding approximations in the level of theory. Each experiment was replicated ten times with a distinct random seed and a randomly selected subset of $N = 10{,}000$ molecules, split 80/10/10 into 8,000 training, 1,000 validation and 1,000 test molecules. All models were fitted on those 8,000 training molecules, except the Gaussian processes, which were fitted on a random subset of 5,000 of them; the validation and test molecules were unchanged.

We repeated these experiments on three molecular property datasets with experimentally determined endpoints. From the OpenADMET initiative [109], we used LogD and Caco-2 efflux permeability, with $N = 5{,}039$ and $N = 2{,}161$ respectively. We also selected hERG K_i data from ChEMBL [110], following a protocol inspired by Landrum and Riniker [16]: filter for binding assays, deduplicate by median pChEMBL value, and remove compounds with an inter-assay standard deviation greater than 1.0 log unit, resulting in $N = 1{,}415$ compounds. Each of the three was partitioned into five folds by grouped k-fold cross-validation, and each fold's training block gave up a further grouped fifth as a validation set for early stopping. A model therefore fits between 3,194 and 3,364 of the 5,039 LogD molecules, de-

pending on the fold. Unlike QM9's ten replicates, which are ten independent draws from one pool, each assay dataset is a single fixed set in which every molecule is tested exactly once and nothing is redrawn.

All splits and cross-validation folds were grouped by the chirality-blind Bemis–Murcko scaffold, implemented such that each distinct acyclic molecule forms its own group, and groups are filled in random order. Prior to modeling, all molecular structures were sanitized and canonicalized using `RDKit` to remove invalid valence states and standardize atom and bond typing [32]. Prediction targets were standardized to zero mean and unit variance. Experiments that track uncertainty were run once rather than in replicates.

3.2.2 Representations

In this study, we evaluated six molecular representations: ECFP4, Avalon, Sort & Slice, PDV, ChemBERTa and Molecular Hypergraph Grammar GNNs (MHG-GNNs). The same six were used on QM9 and on the three assay datasets. ECFP4 is an ECFP with radius $r = 2$ with $d = 2048$ bits [4, 32]. Sort & Slice takes the same substructures as ECFP4, but it ranks them by prevalence and selects the top L ($L = 1024$) [6]. We fitted the Sort & Slice vocabulary on the training molecules alone, refitting it for each QM9 replicate and for each cross-validation fold. Avalon is a 2,048-bit fingerprint, generated with the Avalon toolkit through RDKit [7, 32]. The PDV representation is a vector of 200 physicochemical descriptors computed with RDKit's `MolecularDescriptorCalculator` [31, 32]; the 200 are listed in Additional file 1, and the list is fixed in our code rather than taken from RDKit's default descriptor set, which grows between releases. ChemBERTa embeddings are the mean over the non-padding token embeddings of the `DeepChem/ChemBERTa-77M-MTR` checkpoint, producing 384 features [28]. MHG-GNNs are a GIN-based autoencoder pre-trained on 1.34 million PubChem molecules using a β-variational autoencoder loss, producing 1024-dimensional embeddings through iterative message passing and found in the published `mhggnn_pretrained_model_0724_2023` checkpoint [8]. We z-score normalized PDV, ChemBERTa and MHG-GNN per feature, using the training molecules' mean and standard deviation; ECFP4, Avalon and Sort & Slice reach the model as built.

3.2.3 Models

To understand model behavior in the presence of label noise, we fitted models from several families, namely tree ensembles (random forests, quantile regression forests, XGBoost, LightGBM and NGBoost), a support vector machine, Gaussian processes and neural networks. Several of these models are variants of one another that differ only in the probabilistic components they add, and the hyperparameters of all models described below are listed in Additional file 1.

While we used 100 trees for RF, we observed a noisy quantile estimate with 100 trees for QRF, prompting us to use 300 instead. To observe the effect of the quantile mechanism, we then fit a second RF at 300 trees so the two models were idential in all other settings.

NGBoost relies on scoring off validation data, which differs by dataset. On QM9 the validation split is already held out of every fit. However, on the three assay datasets the validation set is a scaffold-grouped fifth of the rows NGBoost is handed, which already exclude the validation molecules.

We used an RBF with $C = 1.0$ and $\gamma =$ `scale` as the kernel, as it accomodates continuous vectors.

We fitted three variants of GPs, each with a constant mean and a scaled kernel. The first uses an RBF kernel and runs on all six representations. The second uses the Tanimoto kernel [67, 68] and was run on ECFP4 alone. The third is a heteroscedastic variant in which the RBF process is given a second network that predicts the observation noise of each molecule. As such, we used a standard GP and only fit on at most 5,000 training molecules, drawn by a seeded sample. Due to the size of the datasets, this cap only affected QM9. We implemented these GP models in `GPyTorch` [111], with the Tanimoto kernel taken from the `Gauche` framework [66].

While preliminary tests indicated that the Tanimoto kernel provides GP with substantial benefits over other kernels, for SVM, the RBF kernel was sufficient for all types of data.

We tested two deterministic, fully connected feed-forward architectures, both implemented in PyTorch [112]. NN-α has two hidden layers of 128 and 64 units with dropout after each hidden layer, and NN-β has two hidden layers of 128 units with dropout applied before the output layer. Both were trained with the Adam optimizer and early stopping on the validation loss. Bayesian transformations converted these networks into Bayesian neural networks (BNNs) with ReLU activations and dropout

regularization ($p = 0.2$), and Additional file 1 gives the full widths, dropout fractions, learning rates and batch size of every network.

To get these networks producing uncertainty estimates, we applied four probabilistic transformations to each architecture, producing eight probabilistic networks. The first replaces every linear layer with a Bayesian layer carrying a Gaussian prior $\mathcal{N}(0, 0.1^2)$ on its weights, producing BNN-α and BNN-β. They are trained on an evidence lower bound whose Kullback–Leibler (KL) divergence term is divided by the number of training molecules. While this is a straightforward approach, it is computationally expensive. We explored alternative techniques like our second variant, where we add an output head to the standard NN and fit the negative log likelihood of a Gaussian with that variance [113]. The third, Variational Bayesian Last Layer (VBLL), replaces every linear layer with a layer that produces mean-field variational posteriors $q(\mathbf{W}) = \mathcal{N}(\boldsymbol{\mu}_W, \mathrm{diag}(\boldsymbol{\sigma}_W^2))$ over the last-layer weights and trains by maximizing the evidence lower bound. This can be thought of as minimizing the reconstruction loss plus KL divergence $D_{\mathrm{KL}}(q(\mathbf{W})\|p(\mathbf{W}))$ from a standard normal prior, scaled by $1/N$ [114]. Because only its output layer keeps a learned observation-noise parameter, VBLL networks report only epistemic uncertainty. The fourth makes observation noise a function of the input. All eight probabilistic networks estimate their predictive distributions from 100 stochastic forward passes. Each network and its Bayesian counterpart were tuned to the same set of parameters so that NN-α against BNN-α and NN-β against BNN-β differ by the Bayesian layers alone.

3.2.4 Noise Conditions and Levels

The seven noise conditions used in this chapter are defined in Section 2.5.1.

We ran experiments across seven noise levels, 0, 0.2, 0.3, 0.5, 0.75, 1.0 and 1.5. We added the noise to the raw label and standardized the target afterward, using the clean training mean and spread. For censoring, the level is the fraction of labels clipped, at 0, 0.1, 0.2, 0.25, 0.3, 0.4 and 0.5 [81].

We ran Gaussian, grouped-wider and grouped-shifted noise on every model and representation pairing. The remaining four conditions ran on selected pairs.

3.2.5 QM9 Data Pipeline

QM9 [107], loaded through PyTorch Geometric, was processed by a separate pipeline in which Python code loads and splits the data, computes every representation, and trains and scores the models. Between featurization and training, a program written in Rust injects the noise, standardizes the labels and writes the files from which the models are trained.

Each replicate used a new scaffold split, drawn as described in the Datasets subsection above. Each acyclic molecule forms its own group, identified by its canonical SMILES, and no group is ever divided between two splits. The same grouping defines the scaffold families targeted by the grouped noise conditions.

The six representations described above are computed in Python. Binary fingerprints are stored in bit-packed form, Sort & Slice counts as 16-bit integers, and the descriptors and learned embeddings as 32-bit floating-point numbers. The run halts if any molecule has an ECFP4 or Avalon fingerprint that is entirely zero, so that a molecule that failed featurization cannot enter training. The pipeline can also build one-hot encoded SMILES, although these were not used in this thesis.

Conventional approaches load entire datasets into memory from comma-separated values (CSV) or Parquet files, which creates memory bottlenecks at scale and repeats the cost of deserialization in every experimental replicate. Our pipeline instead serializes the preprocessed data into custom memory-mapped binary files that are read on demand, so that memory use stays low and constant regardless of dataset size (Figure 3.1). Python writes one file per split, the Rust program reads each file and writes a new one in which the processed labels replace the raw ones, and Python then reads the new files back for training.

Each memory-mapped file stores a sequence of fixed-format entries. An entry holds the isomeric and canonical SMILES strings in UTF-8 with a length prefix, the raw target value as a 4-byte float, the requested representations as fixed-width blocks, and the processed target value, which includes the noise and normalization, as a 4-byte float. The blocks occupy 2,048 bytes for Sort & Slice (1,024 16-bit integers), 800 bytes for PDV (200 32-bit floats), 1,536 bytes for ChemBERTa (384 32-bit floats), 4,096 bytes for MHG-GNN (1,024 32-bit floats), and 256 bytes each for Avalon and ECFP4. Only the requested representations are written, so the byte offset of every field depends on the list of representations. This format permits zero-copy memory mapping, in which Python and Rust access the same disk-backed

memory pages without deserialization and memory use stays constant regardless of dataset size.

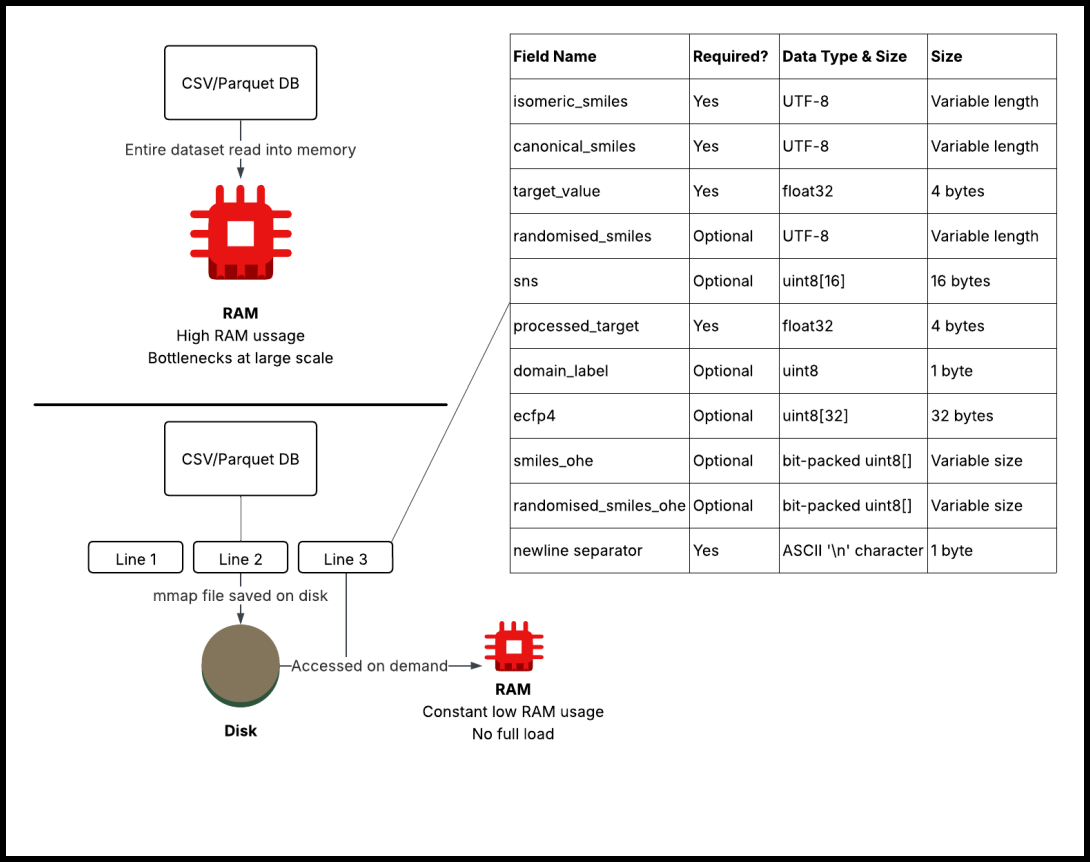
The Rust program draws the noise for each split separately, following the order in which that split's records are stored. The noise parameters are set on the training split, since the amount of noise, the censoring limit and the affected scaffold families are all computed from the clean training labels. Validation labels receive an independent draw, from a seed derived from the training seed, with the same amount of noise in label units, the same limit and the same affected families, and the program verifies that validation received as much noise as training. Test labels are never corrupted, although every split records the per-molecule noise scale its region would have received, so that held-out molecules can be scored against the pattern applied in training.

For every molecule in every split, the Rust program records its canonical SMILES, clean label, injected noise, noise scale, the noise pattern applied to its region, noisy label and the standardized value written to file. It also writes a summary of each run that lists the parameters of the noise condition and the amount of noise delivered. The run halts if the record at any row index holds a different molecule when it is written than when its noise was drawn. A separate self-test mode applies the same checks on the amount and targeting of noise to a label file, without running the rest of the pipeline.

Model training. The pipeline trains the models described in the Models subsection above, with hyperparameters taken from package defaults, tuned with Optuna [115], or read from a JSON configuration file. For models that return uncertainty, the pipeline can also score the training molecules out of fold, so that the uncertainty of each training molecule comes from a model that never saw its corrupted label.

3.2.6 Uncertainty

Up to this point we have been concerned with how noise affects the predictive accuracy of QSAR models. However, label noise should also affect the uncertainty these models report. Some models return only a point prediction, while others also return an estimate of uncertainty, either as a predicted distribution or as the spread across an ensemble. Furthermore, some report a single value of uncertainty while others report one that can be decomposed into an aleatoric and an epistemic compo-



Field Name	Required?	Data Type & Size	Size
isomeric_smiles	Yes	UTF-8	Variable length
canonical_smiles	Yes	UTF-8	Variable length
target_value	Yes	float32	4 bytes
randomised_smiles	Optional	UTF-8	Variable length
sns	Optional	uint8[16]	16 bytes
processed_target	Yes	float32	4 bytes
domain_label	Optional	uint8	1 byte
ecfp4	Optional	uint8[32]	32 bytes
smiles_ohe	Optional	bit-packed uint8[]	Variable size
randomised_smiles_ohe	Optional	bit-packed uint8[]	Variable size
newline separator	Yes	ASCII '\n' character	1 byte

Figure 3.1: In-memory (top left) and memory-mapped (bottom left) data loading. Memory-mapped loading keeps each molecule as one line of a binary file on disk and reads lines into memory on demand. The table on the right lists the fields of one line.

nent [113]. The aleatoric component describes underlying noise the model associates with the data, while the epistemic component represents the uncertainty within the model's fit. We sought to determine if, as expected, the aleatoric component rises with injected label noise while the epistemic component does not.

For the BNN and VBLL transformations, we took the two components over $T = 100$ stochastic forward passes [113]:

$$\hat{u}^2_{\text{ale}}(x) = \frac{1}{T}\sum_{t=1}^{T}\hat{v}_t(x), \qquad \hat{u}^2_{\text{epi}}(x) = \frac{1}{T}\sum_{t=1}^{T}\left(\hat{\mu}_t(x) - \bar{\mu}(x)\right)^2, \tag{3.1}$$

where $\hat{\mu}_t(x)$ and $\hat{v}_t(x)$ are the mean and the variance returned for molecule x on pass t, and $\bar{\mu}(x)$ is the mean over passes. Here $\hat{v}_t(x)$ is the predicted observation noise variance. It varies per molecule in the variance-head and heteroscedastic variants, and is one number for the whole fit in the VBLL transformation. A full-BNN without a variance head returns no variance, so its uncertainty is epistemic alone.

The other models follow the same split. For GPs, the epistemic component is

the posterior variance of the latent function and the aleatoric component is the learned observation noise [64]. All variants of the GP produce one number for the whole fit except heteroscedastic. For QRF models, each tree holds multiple training molecules. So for a given molecule, the aleatoric component is the variance the labels the molecule's given leaf. The total aleatoric uncertainty are those variances averaged over the trees. The epistemic component is the variance of the leaf means across the trees. NGBoost predicts one variance for each molecule from a single fit, which is aleatoric alone [58]. In every model, the total uncertainty is the sum of the the aleatoric and epistemic components.

No held-out label carries injected noise, so every comparison against the injected noise was made on training molecules. Each training set was divided into five folds grouped on Murcko scaffolds, and each molecule was scored by a refit that excluded its own fold. For NGBoost on QM9, three of the five folds were scored to limit its cost. No uncertainty was calibrated after fitting.

3.2.7 Evaluation

We compute statistics on the held-out molecules, for QM9 the test split and for three assay datasets the held-out fold of the scaffold cross-validation. Different combinations of dataset, model, representation, noise condition, and noise level are examined in each scenario. We used the coefficient of determination (R^2) throughout. For molecule i, let y_i be the clean label, $\hat{y}_i$ the prediction, ϵ_i the noise injected into its label and u_i the predicted uncertainty.

To evaluate the effect of label noise, we tracked predictive performance retention under increasing artificial noise. For each QSAR pairing we recorded $R^2(\tau)$ at each noise level τ, and normalized it to obtain the retention ratio $R^2(\tau)/R^2(0)$. To quantify robustness, we used the normalized area under this retention curve, $\mathrm{AUC}_{\mathrm{norm}}$(Equation 2.7), evaluated using the trapezoidal rule over the seven noise levels. While we tested other metrics of robustness, we found $\mathrm{AUC}_{\mathrm{norm}}$to be less directly correlated to clean-label R^2 predictions. For censoring, we sweep over a set of fractions to clip the labels prior to normalization.

An $\mathrm{AUC}_{\mathrm{norm}}$value near 1 indicates that the QSAR model in question has retained predictive performance with artificial noise, meaning it is more robust to noise. Since we normalize the curve by its own clean-label predictive performance, a model with a low clean R^2 has less to lose, so $\mathrm{AUC}_{\mathrm{norm}}$is reported beside clean R^2. We excluded

replicates with a clean $R^2 < 0.3$ from the robustness analysis (Additional file 2), as performance retention ratios become unstable when the clean-label denominator approaches zero. On QM9 one $\mathrm{AUC}_{\mathrm{norm}}$is computed per replicate, and we report the median over the ten replicates. On the three assay datasets one value is computed per scaffold fold, and the median over the five folds is reported.

To compare the relative effect of the thirteen base models across datasets and noise conditions, we compared their rankings using Spearman's rank correlation coefficient (ρ) between clean R^2 and $\mathrm{AUC}_{\mathrm{norm}}$. We compared probabilistic models with its deterministic counterpart using a two-sided Wilcoxon signed-rank test paired on the replicate, at $\alpha = 0.05$, separately for each representation and noise condition. This test was run only on QM9, as with five scaffold folds the smallest attainable p is 0.0625.

We conducted a separate two-way analysis of variance (ANOVA) for each noise condition to identify the relative contributions of molecular representation and model architecture to both predictive performance and noise robustness. For a given metric y, either R^2 at a noise level of 1.0 or $\mathrm{AUC}_{\mathrm{norm}}$:

$$y_{ijr} = \mu + \alpha_i + \beta_j + (\alpha\beta)_{ij} + e_{ijr}, \tag{3.2}$$

where α_i represents the effect of model architecture i, β_j represents the effect of molecular representation j, $(\alpha\beta)_{ij}$ is the interaction term, and e_{ijr} is the residual for replicate r. The proportion of variance explained by each term was calculated as the effect size

$$\eta^2_{\mathrm{term}} = \frac{SS_{\mathrm{term}}}{SS_{\mathrm{total}}}, \tag{3.3}$$

using Type I (sequential) sums of squares, entered in the order model architecture, molecular representation, interaction. The ten QM9 replicates provide the residual. The uncertainty on each share is the range of η^2 when the decomposition is repeated with each replicate left out in turn.

ANOVA relies on the assumption that observations both within and across groups are independence. Across the whole set of models and representations in this study, some pairs produced similar predictions that are not independent. This causes the residual degrees of freedom to increase, while the residual sum of squares barely moves. To handle this issue, we computed pairwise Spearman rank correlations between all models and between all representations across noise strategies.

Models whose profiles correlated at $\rho > 0.99$ with another model were excluded, including QRFs, which were very similar to RF, as well as different BNN variants.

Across representations, Sort & Slice correlated at $\rho > 0.90$ with the ECFP4 fingerprint and was excluded to avoid inflating representation degrees of freedom, as both encode overlapping circular substructure information. We also computed intraclass correlation coefficients ICC(1,1) for all model pairs. ICC(1,1) measures the proportion of total variance attributable to between-subject differences, with values near 1.0 indicating that two models rank configurations almost identically. The full redundancy and ICC tables are reported in Additional files 3–5.

To assess the interaction between uncertainty estimation and label noise, we posed three questions. First, whether uncertainty rise with label noise, as measured by the mean u_i over held-out molecules at the highest noise level divided by its value with no added noise, in each replicate or fold. The second pertains to models who report separate aleatoric and epistemic components. We divided the increase of mean epistemic uncertainty between the highest level of noise and clean labels by the mean of the aleatoric component to determine how each responds to label noise. A ratio near 0 means the added noise was attributed to the labels while a ratio of 1 or above means the epistemic component grew as much or more. We also compared the mean aleatoric component with the standard deviation of the injected noise. The third question we had was whether per-sample uncertainty values track injected label noise, as measured by the Spearman correlation between u_i and $|\epsilon_i|$ over out-of-fold training molecules, per fold and per noise level. We also report the Spearman correlation between u_i and the absolute error of the prediction. Finally, we asked whether dividing a molecule's out-of-fold absolute error by u_i ranks the corrupted labels better than the error alone, and report the difference between the two Spearman correlations with $|\epsilon_i|$.

3.3 Variance Decomposition by Representation and Architecture

Considering the two main components of a QSAR model, the molecular representation and the model architecture, we sought to understand how each contributes to predictive performance and noise robustness. ANOVA decomposition across six noise strategies on models to predict HOMO–LUMO gap from QM9 demonstrates

that for predictive performance (R^2 at $\sigma = 0.3$), the model–representation interaction term is the largest source of variance across all six strategies, followed by model architecture and representation (Table 3.1; Figure 3.2a). However, for noise robustness as measured by the noise degradation slope (NDS), this ordering shifts. The choice of model architecture is instead the largest source of variance, while the interaction term drops to second and representation third. This observed inversion makes sense because the molecular representation dictates the chemical information the model receives, while the architecture determines how efficiently that information is used. The interaction term reflects the idea that these two are not independent. In the context of label noise robustness, representation has less of an impact since the features are not corrupted by noise. Meanwhile, the model's regularization, ensembling, and uncertainty handling have a stronger impact on how it handles noise.

The impact varied by type of noise, represented by noise strategy (Figure 3.2b). The strategies which degraded in performance the most, threshold and value-proportional, both corrupted label regions, forcing models to rely on their inductive biases. This translates to the model architecture explaining a significant portion of the variance. On the other hand, outlier noise only affects statistical outliers. For outlier noise, model and interaction effects were both smaller.

	Performance (η^2, %)				**Robustness (η^2, %)**			
Strategy	**Model**	**Rep**	**Inter.**	**Resid.**	**Model**	**Rep**	**Inter.**	**Resid.**
Gaussian	24.6	22.8	**48.1**	4.5	**48.7**	8.2	11.9	31.2
Quantile	25.1	22.4	**46.9**	5.6	**47.9**	6.8	14.9	30.4
Threshold	25.0	21.2	**48.0**	5.8	**48.0**	10.1	28.0	13.9
Heteroscedastic	27.9	23.5	**46.2**	2.5	**37.0**	4.5	17.5	41.0
Value-prop.	27.8	22.0	**46.5**	3.8	**52.8**	6.9	22.2	18.2
Outlier	25.3	22.0	**49.2**	3.5	12.7	3.2	4.7	**79.3**

Table 3.1: ANOVA variance decomposition by noise strategy on QM9. Entries are the share of variance η^2 (%) explained by the model (Model), the representation (Rep), their interaction (Inter.) and the residual (Resid.), for R^2 at $\sigma = 0.3$ (performance) and for the NDS (robustness). Bold marks the largest share in each block of each row.

The interaction between model architecture and representation is seen in Figure 3.3. While some model architectures, like SVM and full BNNs maintain consistent noise robustness scores across all representations, others like NN-β display greater variation. Model robustness on one representation strongly predicts robustness on another (Spearman $\rho = 0.73$, $p = 0.007$ for ECFP4 vs PDV; 11 models).

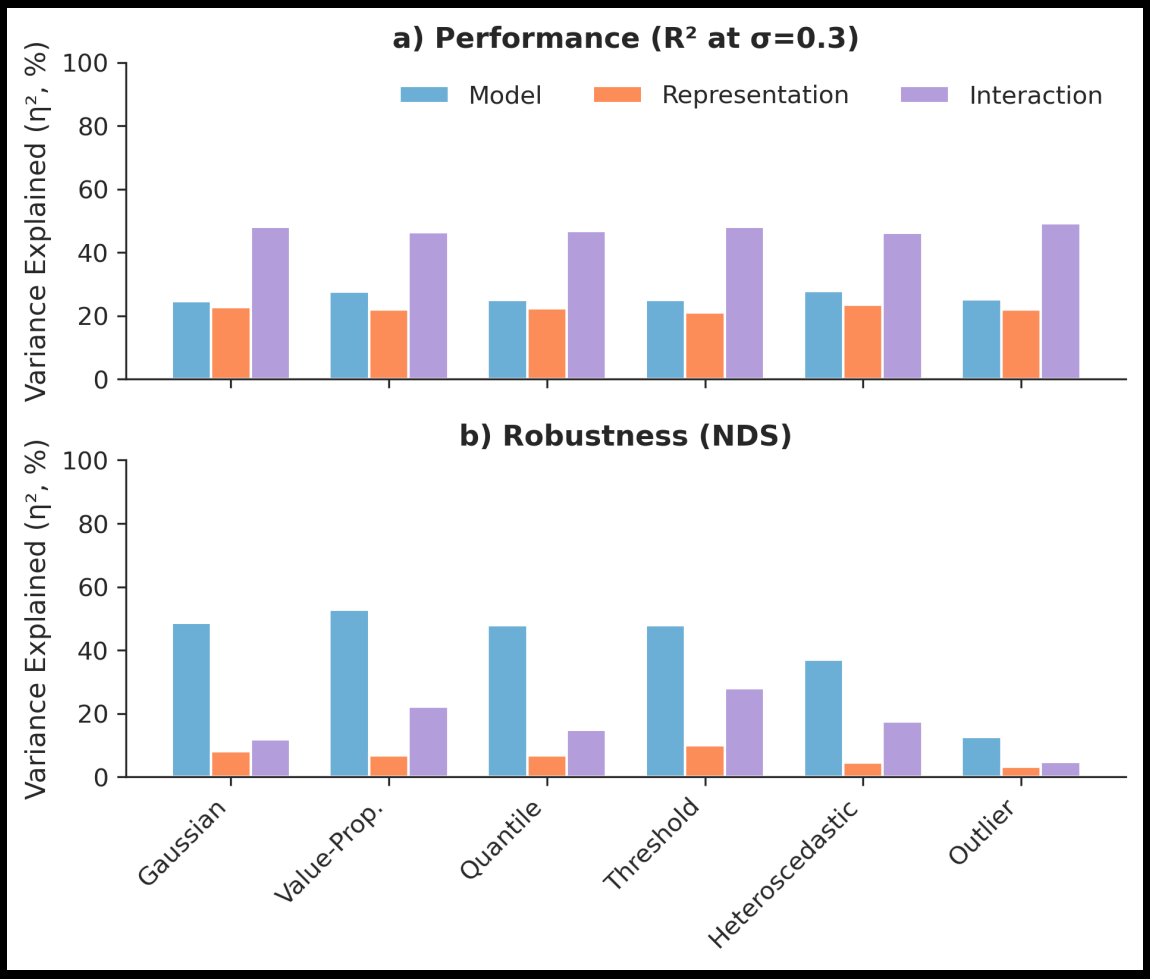


Figure 3.2: ANOVA variance decomposition of a) performance, as R^2 at $\sigma = 0.3$, and b) robustness, as the NDS, by noise strategy on QM9. Bars give the share of variance η^2 (%) explained by model, representation and their interaction, and the residual shares are in Table 3.1.

However, the scatter around the diagonal reflects the interaction term identified by the ANOVA.

We filtered out model–representation configurations with baseline predictive performance $R^2 < 0.6$, excluding 66 configurations where poor clean-data performance would produce misleadingly shallow degradation slopes (Additional file 2). We evaluated all model architectures under increasing label noise on the PDV representation across six strategies, considering both predictive performance as measured by R^2 as well as the NDS. We observe that NDS varies substantially across architectures and noise strategies, as seen in Figure 3.4.

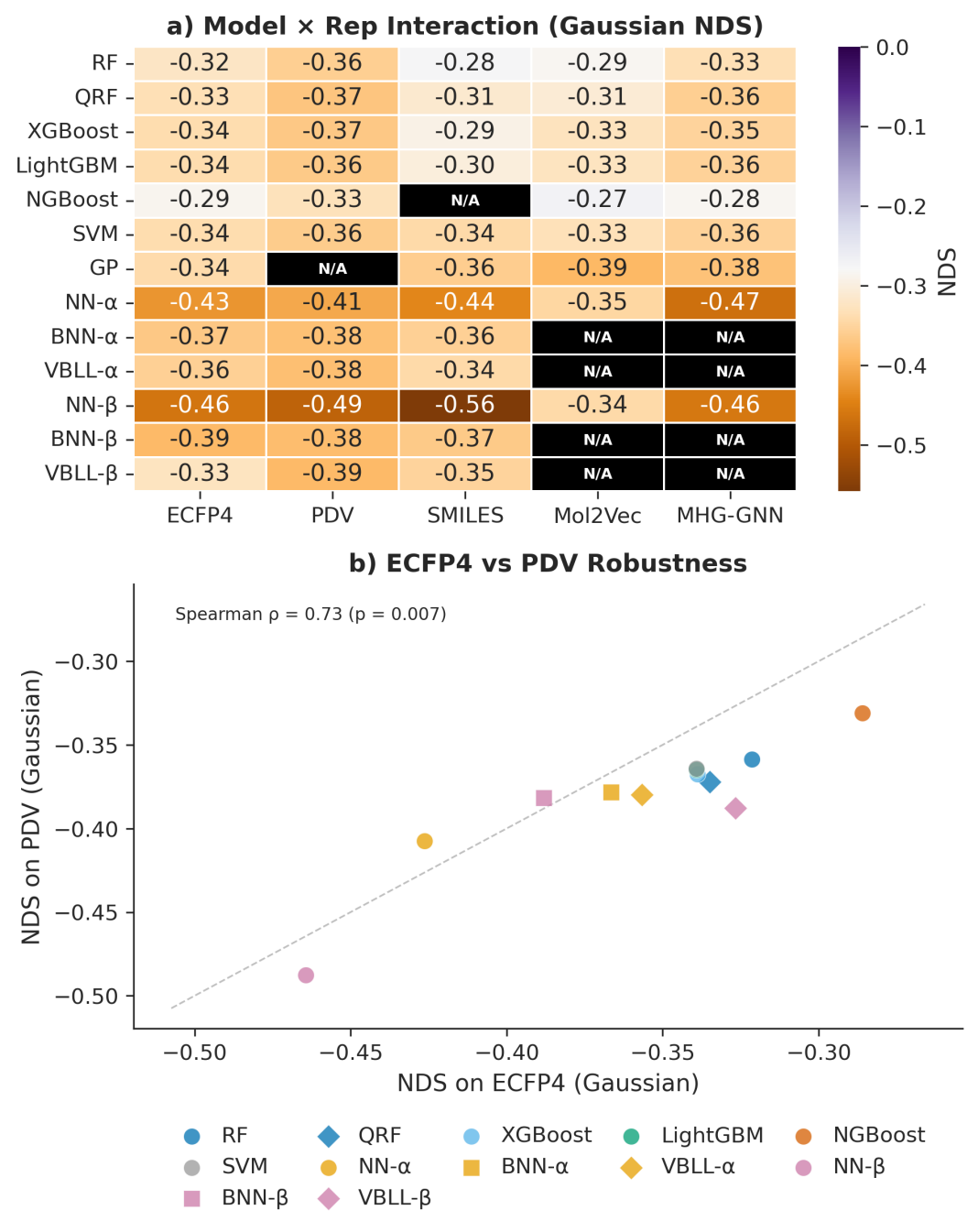


Figure 3.3: Model–representation interaction in noise robustness on QM9 under Gaussian noise. a) NDS of each model on each representation, where darker orange means faster degradation and N/A marks excluded configurations. b) NDS on PDV against NDS on ECFP4 for each model, with the line of equality (dashed) and the Spearman correlation ρ.

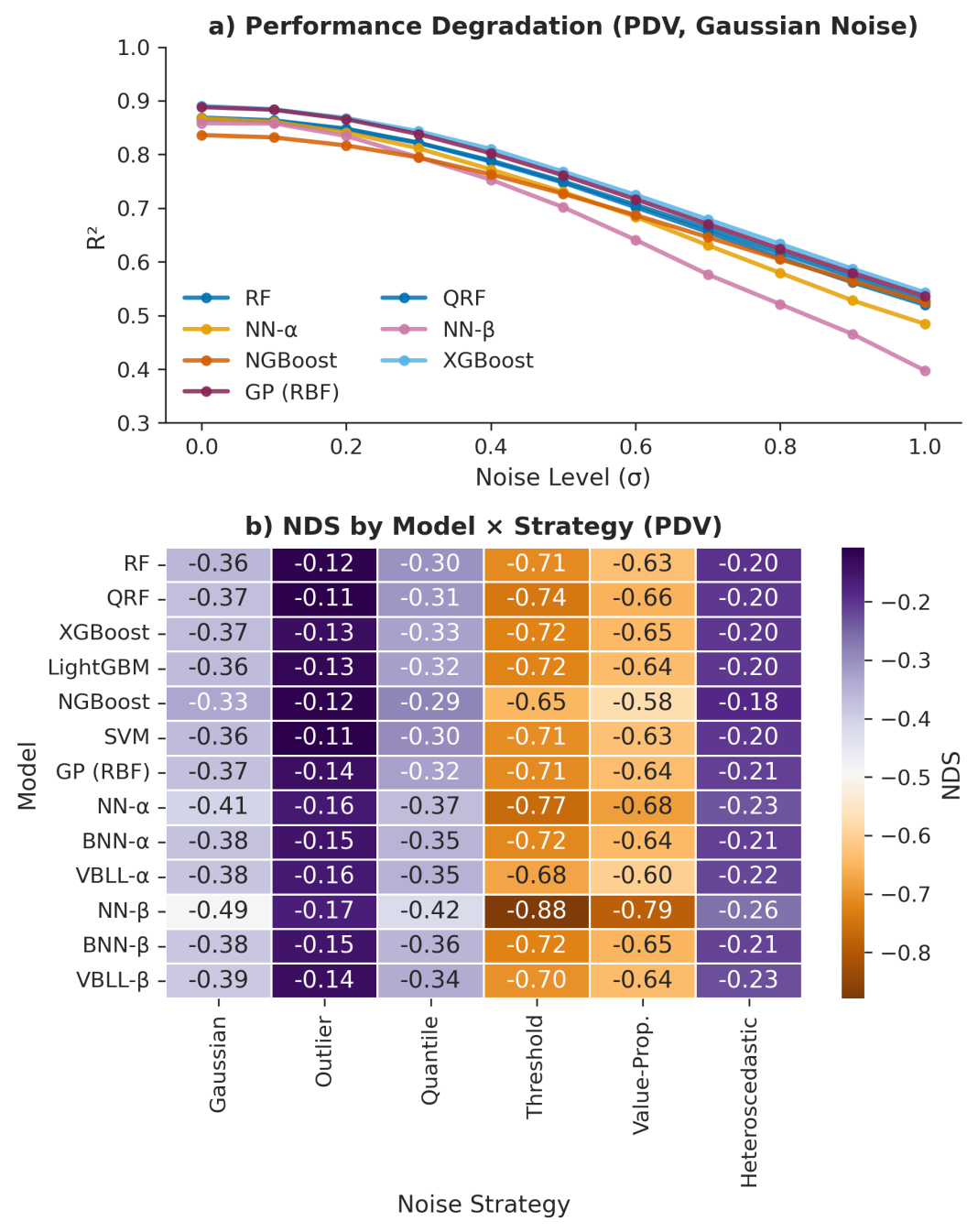


Figure 3.4: Noise robustness on QM9 with PDV. a) R^2 against the noise level σ under Gaussian noise for seven models. b) NDS of every model under six noise strategies, where orange means faster and purple slower degradation. The GP with an RBF kernel replaces the GP with a Tanimoto kernel, which cannot be applied to PDV.

3.4 Probabilistic and Deterministic Model Variants

To further explore the impact of model architecture on noise robustness, we examined variants of the same core architecture. Specifically, Bayesian transformations were used to explore the impact of the Bayesian framework on noise. By comparing models with the same core architecture, we observe that with full Bayesian transformations, representation explains less than 11% of NDS variance ($p > 0.25$) while VBLL on the same NNs is representation-dependent. On MHG-GNN and Mol2vec representations, VBLL models were unstable, such that all 24 VBLL $\times$ {MHG-GNN, Mol2vec} configurations were excluded due to baseline $R^2 < 0.6$. Further work could involve more thoroughly-tuning these models and exploring different core architectures to adapt to these embedding-style inputs.

In general, the full Bayesian transformations reduced the degradation in performance with Gaussian noise the most, as seen in Figure 3.5. Both full BNN and VBLL transformations significantly improved robustness for both NN-α and NN-β (Table 3.2). Full transformations are more computationally expensive, making VBLL a potentially stronger approach to handling noise with NNs when working with larger data sets or modeling architectures. The choice of transformation varies by representation, as seen in Additional file 8, with SMILES receiving the largest benefits from Bayesian transformations. QRF was significantly less robust than RF (Table 3.2), suggesting that quantile regression adds complexity and flexibility which may result in overfitting.

Base	**Variant**	**Δ NDS**	**p-value**	**Sig.**
NN-α	BNN-α	+0.056	2.9×10^{-10}	*
NN-α	VBLL-α	+0.061	1.2×10^{-6}	*
NN-β	BNN-β	+0.096	2.9×10^{-11}	*
NN-β	VBLL-β	**+0.124**	1.2×10^{-7}	*
RF	QRF	**−0.022**	1.6×10^{-10}	*

Table 3.2: Wilcoxon signed-rank tests of each deterministic model against its probabilistic variant on QM9. ΔNDS is the mean change in NDS from the base model to the variant, and positive values mean that the variant is more robust. An asterisk marks $p < 0.05$, and bold marks the largest positive and the only negative change.

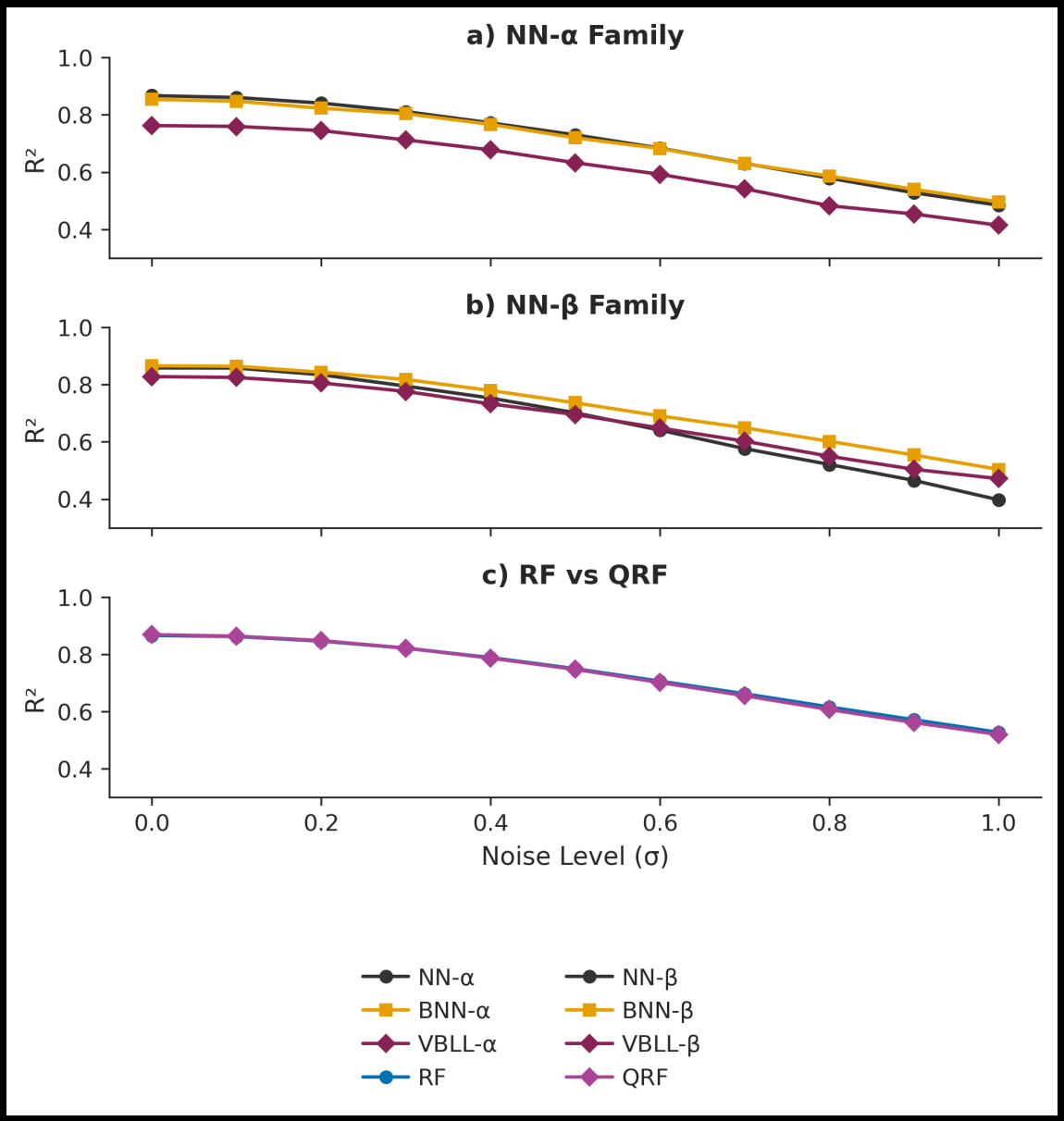


Figure 3.5: Performance of neural network families and random forests under Gaussian noise on QM9 with PDV. Each panel shows R^2 against the noise level σ for a) NN-α, BNN-α and VBLL-α, b) the corresponding NN-β family and c) RF and QRF.

3.5 Model Robustness Rankings across Noise Strategies

NGBoost and SVM showed the smallest degradation in predictive performance across all strategies, while NN-β showed the steepest (Table 3.3). When sorting models by NDS, their ranks across noise strategies were consistent: Kendall's $W = 0.92$ ($p = 2.7 \times 10^{-8}$; 11 models, 6 strategies), indicating that a model's relative robustness is largely strategy-independent. The same ranking pattern holds on the ECFP4 fingerprint (Additional file 6). Robustness is also largely decoupled from baseline

predictive performance: across models on PDV under Gaussian noise, NDS clusters near -0.38 regardless of baseline R^2 (Additional file 7), so a high-performing model is not automatically a more robust one.

Model	Gauss.	Outlier	Quant.	Thresh.	Hetero.	Val.-P.	Mean
NGBoost	**−0.33**	−0.12	**−0.29**	**−0.65**	**−0.18**	**−0.58**	**−0.36**
SVM	−0.36	**−0.11**	−0.30	−0.71	−0.20	−0.63	−0.38
RF	−0.36	−0.12	−0.30	−0.71	−0.20	−0.63	−0.39
LightGBM	−0.36	−0.13	−0.32	−0.72	−0.20	−0.64	−0.40
VBLL-α	−0.38	−0.16	−0.35	−0.68	−0.22	−0.60	−0.40
XGBoost	−0.37	−0.13	−0.33	−0.72	−0.20	−0.65	−0.40
VBLL-β	−0.39	−0.14	−0.34	−0.70	−0.23	−0.64	−0.41
BNN-α	−0.38	−0.15	−0.35	−0.72	−0.21	−0.64	−0.41
BNN-β	−0.38	−0.15	−0.36	−0.72	−0.21	−0.65	−0.41
NN-α	−0.41	−0.16	−0.37	−0.77	−0.23	−0.68	−0.44
NN-β	−0.49	−0.17	−0.42	−0.88	−0.26	−0.79	−0.50

Table 3.3: NDS of each model on PDV for QM9 under six noise strategies and their mean, with models ordered by the mean. Values closer to zero mean greater robustness, and bold marks the best value in each column and the two best models. Gauss. = Gaussian, Quant. = quantile, Thresh. = threshold, Hetero. = heteroscedastic, Val.-P. = value-proportional.

The strategies differed not only in predictive performance severity, but also in their ability to discriminate between architectures. The spread of NDS across models differed by noise strategy, with outlier noise having a spread of 0.06 and threshold noise having 0.23, presumably exposing architectural differences that are masked by other types of noise. As seen in Table 3.3, NGBoost was the most resistant to noise across strategies. With respect to the VBLL models, VBLL-α showed a similar pattern, outperforming all tree-based models except NGBoost under threshold and value-proportional noise despite ranking below them under Gaussian noise. This suggests that its learned noise variance is most beneficial when label corruption is systematic rather than uniform. This observation with VBLL is not alone; predictive performance on clean data is not well-correlated with noise robustness. In fact, some of the models with the lowest NDS across strategies such as NGBoost and SVM did not perform particularly well on clean data. These models may be more prone to picking up on noise rather than underlying signals in the data.

The ANOVA showed that model architecture and the model–representation interaction term dominate the variance in NDS. SVM and full BNNs stood out as noise-robust regardless of representation, and both rely on strong inductive biases, margin maximization in the case of SVM and weight priors in the case of BNNs. RF and NN-β showed the opposite pattern, since they can be robust to noise but

only when paired with particular representations, and these representations differ between the two models.

Although representation is not a dominant determinant of NDS variance, it has an effect through the model–representation interaction term. On SMILES, model choice explains over 91% of the variance in robustness, compared with 72% on PDV. Across all six noise strategies, PDV configurations produced the shallowest mean NDS of all representations, with the best values coming from models with strong inductive biases such as SVM and full BNNs. This suggests that PDVs are preferable when dealing with noisy data, and it appears that the compact feature space of PDV complements margin-based and Bayesian regularization. Embeddings performed poorly, particularly when paired with NNs. These patterns are consistent with the importance of the interaction term and indicate that the choice of representation modulates noise robustness within a given architecture.

Taking a closer look at model choice for noise robustness, we ranked all model–representation configurations on QM9 by their Gaussian NDS. Tree-based ensembles dominate the top 10: NGBoost appears five times, XGBoost and RF twice each, and LightGBM once. No neural-network configuration appears the top 10. Rankings are consistent across strategies and the top QM9 configurations maintain their advantage on absorption, distribution, metabolism and excretion (ADME) datasets, though SVM configurations rank higher on these ADME datasets than NGBoost.

3.6 Validation on Assay Datasets

In addition to working with "clean" quantum molecular properties, we tested our findings on experimentally-obtained data, replicating our process of adding artificial noise. Unlike QM9, where $\sigma = 0$ corresponds to a noise-free baseline, the experimental endpoints carry their own measurement noise in both training and test labels; the injected σ therefore stacks on top of an unknown noise floor. Per-dataset NDS heatmaps across all model architectures and noise strategies are shown in Figure 3.6. The ANOVA on these additional datasets confirms that model architecture dominates robustness variance on all three datasets, as seen in Additional file 10, and that the trends with respect to noise robustness generalize. NGBoost ranks first on both QM9 and external datasets, SVM also transfer quite well. Some models' predictive capabilities are limited on these external datasets, likely due to a combination of smaller training sets, narrower chemical coverage, and the unknown experimental

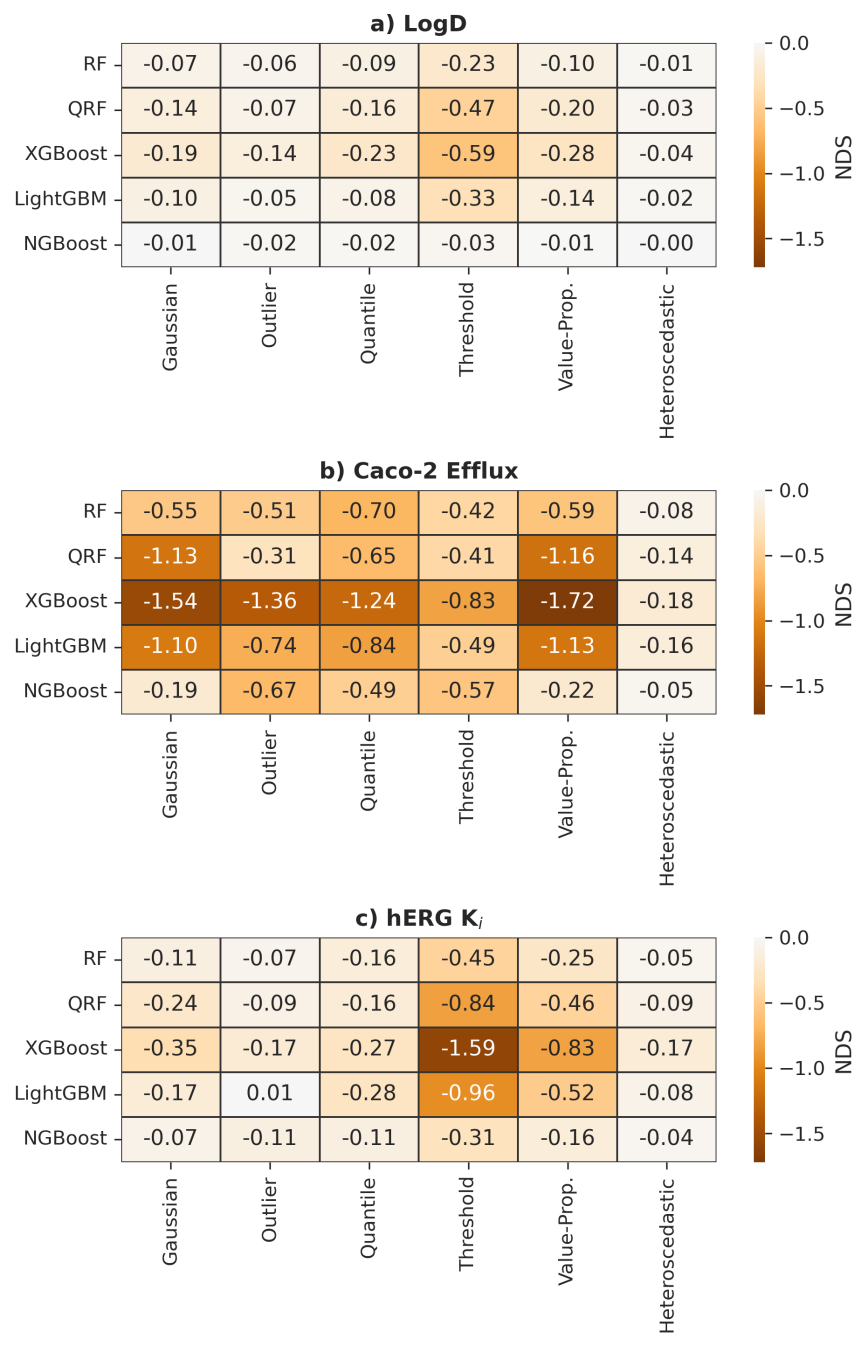


Figure 3.6: NDS of five tree-based models under six noise strategies on a) LogD, b) Caco-2 and c) hERG K_i. Darker orange means faster degradation, and the three panels share one color scale.

noise in the labels. XGBoost and BNN variants suffer the most here, as seen in Figure 3.7a.

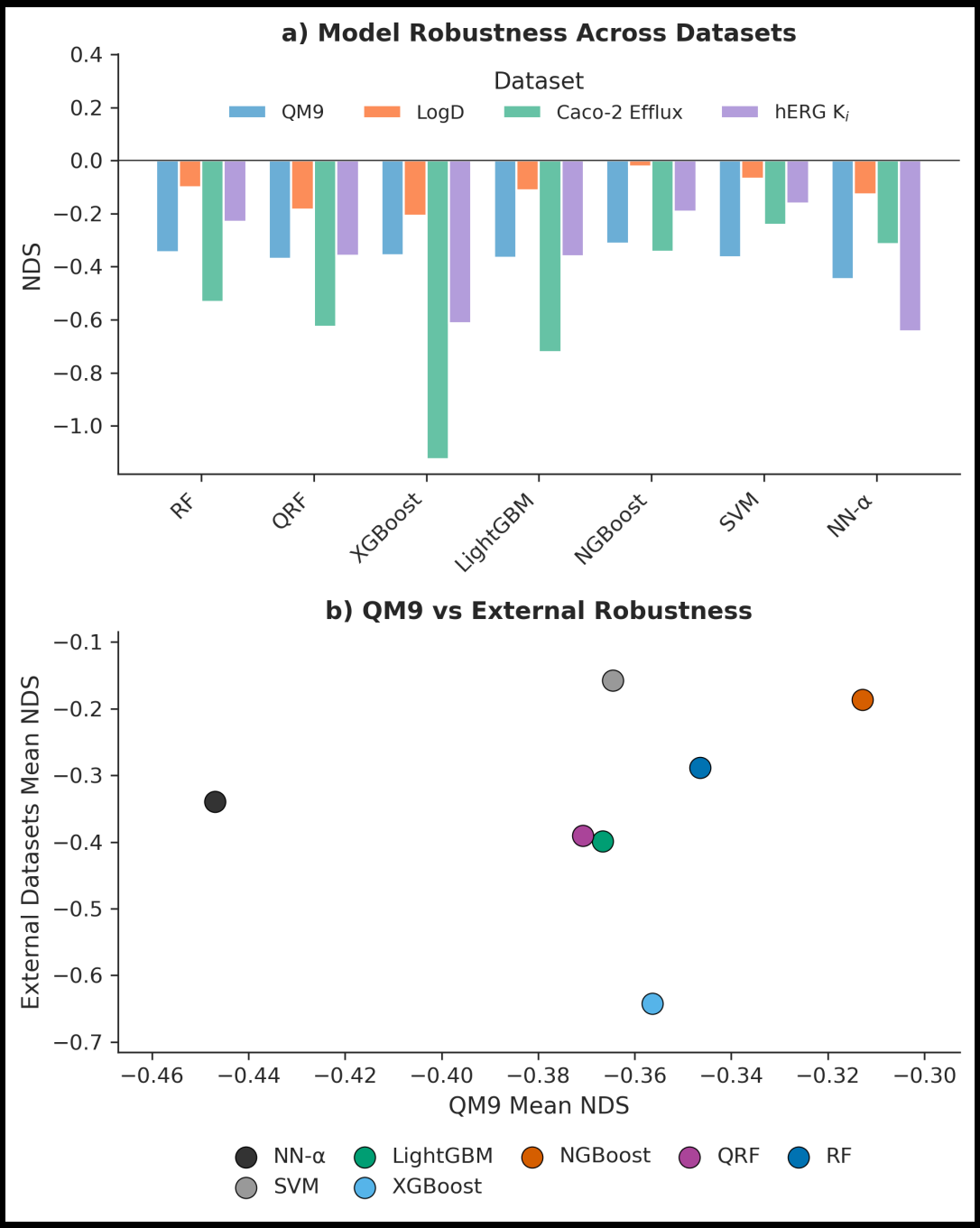


Figure 3.7: NDS on QM9 and the three assay datasets. a) NDS of seven models on each dataset. b) Mean NDS over the three assay datasets against the NDS on QM9, for each model.

QRF was consistently less robust than RF on every external data set (Additional file 11). As seen with QM9, choice of molecular representation has a minimal effect on noise robustness across all three external datasets, reinforcing the finding that model architecture, not representation, is the main driver of noise robustness.

3.7 Uncertainty under Label Noise

The sections above showcased the importance of model selection when working with noisy labels. We take this one step further and ask, for the models which produce uncertainty estimates, how those estimates are affected by noise and if they can be used to track noise.

3.7.1 Per-Sample Uncertainty and Label Noise

Kolmar and Grulke [9] found that the mean predicted uncertainty derived from the GPs increases with the amount of label noise in the training data. We go beyond the population level and instead ask if a model's per-sample uncertainty tracks label noise. One key sign that a model is able to handle noise is its ability to track it, represented by the per-sample Spearman correlation between predicted uncertainty and noise magnitude (Table 3.4). While GPs and NGBoost gave the strongest correlations, BNNs were moderate. Learned embeddings (MHG-GNN and Mol2vec) struggled, so much so that their correlations dropped to near zero or below on every model tested.

Model	Rep.	Unc-Noise ρ	Unc-Error ρ	ECE	Cov. 1σ	Cov. 2σ
GP	**SNS**	**0.56**	0.23	0.14	71.2%	94.6%
GP	ECFP4	0.53	0.22	0.14	71.4%	95.4%
NGBoost	**SNS**	**0.47**	0.16	0.13	69.3%	95.4%
NGBoost	**PDV**	**0.47**	**0.29**	**0.11**	69.0%	95.2%
NGBoost	PDV (binary)	0.44	0.25	0.13	69.8%	95.4%
NGBoost	ECFP4	0.44	0.20	0.14	69.8%	95.3%
BNN-α	SNS	0.42	0.21	0.17	73.4%	95.1%
BNN-β	SNS	0.40	0.21	0.18	73.5%	95.1%
BNN-β	ECFP4	0.38	0.20	0.18	72.1%	95.0%
BNN-β	PDV	0.38	0.24	0.19	72.1%	94.0%
Contrast: lowest correlations (graph/embedding representations)						
BNN-β	Mol2vec	−0.05	0.02	1.33	50.6%	76.4%
VBLL-α	MHG-GNN	−0.05	0.09	13.41	26.6%	38.6%
VBLL-β	MHG-GNN	−0.03	0.10	15.34	31.1%	47.1%
VBLL-β	Mol2vec	−0.03	0.15	23.86	44.8%	58.0%

Table 3.4: Model–representation combinations on QM9 under Gaussian noise with the highest (upper block) and lowest (lower block) Spearman correlation between predicted uncertainty and the injected noise magnitude $|\epsilon|$ (Unc-Noise ρ). Unc-Error ρ is the correlation with the absolute error, and Cov. 1σ and Cov. 2σ are empirical coverages with nominal values of 68% and 95%. Bold marks the highest Unc-Noise ρ for the GP and for NGBoost, and the best Unc-Error ρ and ECE.

3.7.2 Treatment of Large Residuals

Samples containing label noise tend to produce large residuals in training, and every model has its own way of handling them. NGBoost contains predicted scales that increase to absorb residual [58]. The GP contains has a single global observation noise term. It does not absorb individual residuals, so its per-sample uncertainty derives primarily from the posterior variance rather than label noise [63, 64]. BNNs treat these large residuals differently; they broaden their weight posteriors, increasing predicted uncertainty without explicitly modeling observation noise [105, 113]. VBLL adds a learned noise variance to the BNN loss [114], but this value is a global scalar, independent of the input. As seen in Table 3.4 and Additional file 9, both BNNs and VBLLs achieve moderate correlations which improve when paired with fingerprint or descriptor-based representations. Among models that learn separate distributional parameters during training, GP and NGBoost produced the strongest correlations between uncertainty and label noise, though QRF, which separates parameters using quantile distributions, produced both poor label predictions and uncertainty-noise correlations. At the population level, mean predicted uncertainty increases with artificial noise. Although we expect to see aleatoric uncertainty increase with injected noise, for VBLL both the aleatoric and epistemic components increased (Figure 3.8b).

3.7.3 Aleatoric and Epistemic Attribution

Mean predicted uncertainty rises with label noise, but this alone does not show to which component a model attributes the rise. We expected the aleatoric component to rise with injected label noise and the epistemic component to stay roughly flat. Three kinds of model split their uncertainty per molecule, namely the full BNNs with a variance head, the heteroscedastic GP and QRF [54, 64, 113]. We excluded the other GPs because their observation noise is a single number for the whole fit and therefore cannot follow individual molecules [64]. For each model, we divided the rise in mean epistemic uncertainty on held-out molecules, from no added noise to a noise level of 1.5, by the rise in mean aleatoric uncertainty over the same range. A ratio near 0 means that the added noise went to the aleatoric component, while a ratio of 1 or more means that the epistemic component grew as much or more (**[FIGURE NEEDED: table of the ratio of epistemic rise to aleatoric rise, one row per model (the two full BNNs with a variance head, het-**

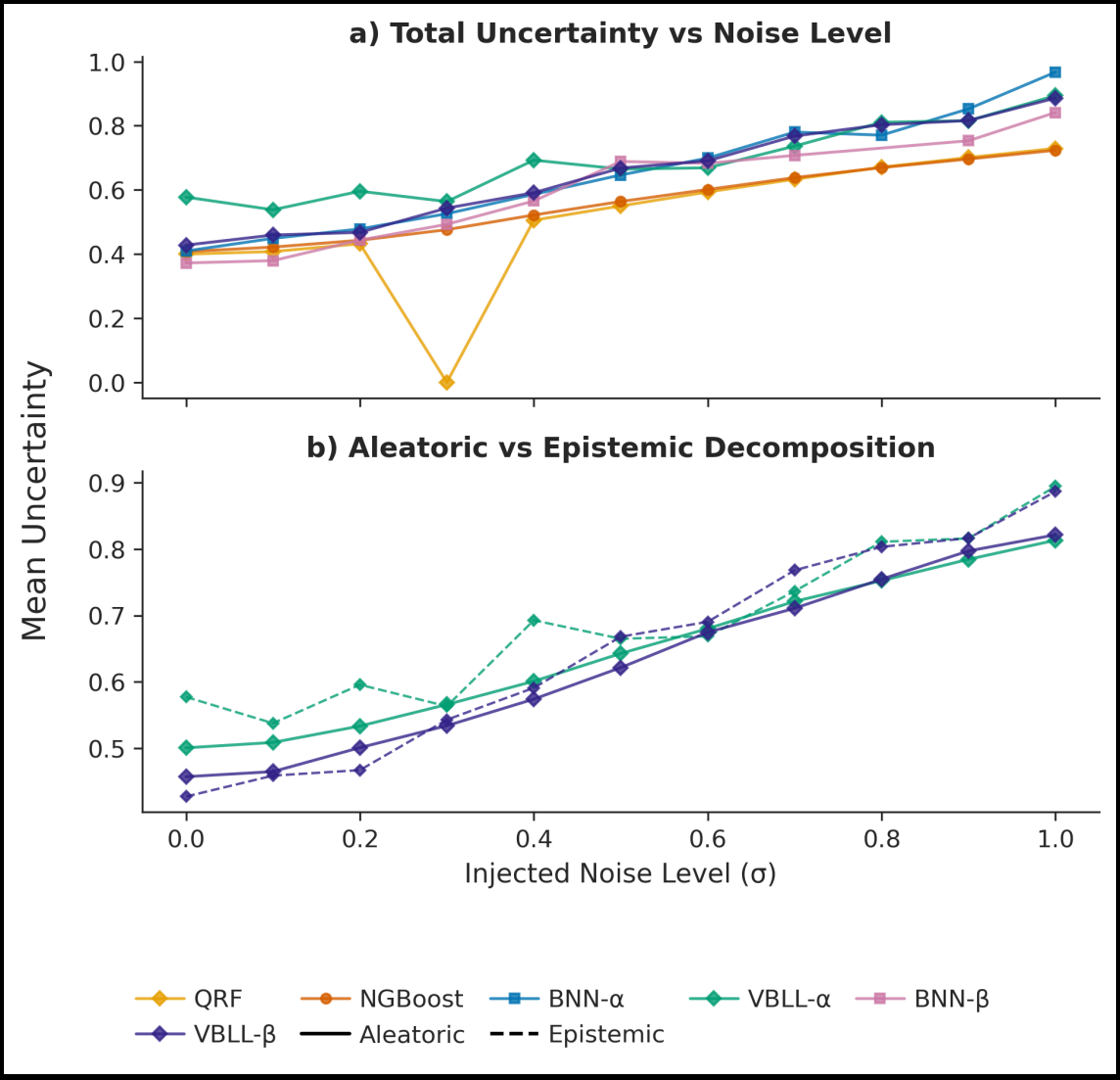


Figure 3.8: Mean predicted uncertainty against the noise level σ on QM9. a) Total uncertainty of QRF, NGBoost and the BNN and VBLL variants of NN-α and NN-β. b) Aleatoric (solid) and epistemic (dashed) uncertainty of VBLL-α and VBLL-β.

eroscedastic GP, QRF) and noise condition (Gaussian, grouped-shifted), one column per representation, each entry the range over datasets on held-out molecules]).

The full BNNs with a variance head split the added noise as expected, with a ratio of at most 0.43 for every model, dataset, representation and noise condition. QRF behaved differently, since both of its components rise with noise and on some representations the epistemic component rises faster (**[FIGURE NEEDED: aleatoric and epistemic parts against noise level, one panel per model with a per-molecule split, QM9 HOMO–LUMO gap, ECFP4, Gaussian noise]**). This holds although QRF is among the more noise-robust models, ranking 4th of the 19 models and variants on $\mathrm{AUC}_{\mathrm{norm}}$on four of the six representations on

QM9 under Gaussian noise. Its clean R^2 on the same data ranks between 6th and 13th of the 19, depending on the representation.

Representation explained little of the variance in noise robustness, but for QRF it determines how the added noise is split. Over all datasets and every noise condition except censoring, the QRF ratio on held-out molecules ranges from 0.63 to 1.09 on the three fingerprints (ECFP4, Avalon and Sort & Slice), from 1.03 to 1.42 on PDV, and from 1.28 to 2.05 on ChemBERTa and MHG-GNN. Under Gaussian noise alone, the held-out ranges are 0.65–0.75 on ECFP4, 0.69–0.76 on Sort & Slice and 0.81–0.89 on Avalon, compared with 1.17–1.32 on PDV, 1.54–1.72 on MHG-GNN and 1.60–1.88 on ChemBERTa. On the learned embeddings, QRF therefore attributes more of the added noise to its epistemic component than to its aleatoric component.

The type of noise also shifts the split, most clearly for the heteroscedastic GP. Under Gaussian noise, its ratio on held-out molecules lies between -0.05 and 0.36 on every representation. Under grouped-shifted noise, the range over datasets is 0.73–1.00 on ECFP4, 0.44–0.61 on Sort & Slice and 0.32–0.46 on Avalon, compared with 0.13–0.22, 0.14–0.23 and 0.21–0.25 on PDV, ChemBERTa and MHG-GNN, respectively. Laplace, Student-t and outlier noise give splits similar to those under Gaussian noise. Grouped-shifted noise raises the epistemic share of both the heteroscedastic GP and QRF for every dataset and representation. One possible explanation is that a shared offset within each scaffold family appears to the model as structure in the data rather than as label noise.

We next asked how closely the aleatoric component follows the amount of added noise. Because held-out labels carry no injected noise, this comparison uses out-of-fold training molecules. We fitted the slope of mean aleatoric uncertainty against the standard deviation of the added noise, both in the units of the label, under Gaussian noise, and we report the median over folds (**[FIGURE NEEDED: aleatoric part against the standard deviation of the added noise, per model, ECFP4 and PDV, out-of-fold training molecules, dashed line where reported noise equals added noise]**). On ECFP4 and PDV, the slope ranges from 0.36, for QRF on PDV for Caco-2, to 1.09, for a full BNN with a variance head on PDV for hERG K_i. Including ChemBERTa widens the range to between 0.28, for QRF on Caco-2, and 1.62, for a full BNN with a variance head on hERG K_i. The aleatoric component can therefore be used to compare noise between datasets or between noise levels, but it does not estimate the measurement error of a dataset.

Censoring behaves differently from the other noise conditions, because clipping

labels at a limit narrows their spread. NGBoost and QRF shrink their total uncertainty with that spread on every dataset, and the GPs do the same on hERG K_i and Caco-2. None of these models treats a clipped label as a noisy one, so they become more confident while the labels lose information.

3.7.4 Representation and Uncertainty Tracking

The choice of representation has a much stronger impact on per-sample uncertainty tracking than on noise robustness. Fingerprints and physicochemical descriptors provided a clear signal with which models such as GP and NGBoost could separate noise-induced residuals from structural variation. However, learned embeddings such as MHG-GNN and Mol2vec did not produce viable uncertainty-noise correlations (Table 3.4). Although these learned embeddings can be effective representations for many tasks, this research suggests that they provide challenges for per-sample noise tracking. However, there is room for further investigation, as only two embeddings were tested.

3.8 Conclusions

These results lead to two main conclusions. The first is that noise robustness, defined as the rate at which predictive performance degrades with increasing label noise, is primarily determined by the model's training mechanism. The margin maximization of SVM, the weight priors of BNNs, and the ensembling in tree methods all appear to keep those models from fitting to noise. The choice of representation has a stronger influence on predictive performance than noise robustness. The interaction term between representation and model has a stronger impact; some models like SVM and full BNNs remain robust across all representations, while RF and NNs are robust only with certain ones. While the pattern of noise has a strong impact on predictive performance, it does not influence noise robustness rankings between models or representations much.

We investigated how a wide variety of molecular representations and model architectures perform under increasing label noise in QSAR modeling. We used ANOVA variance decomposition across three noise conditions on the QM9 HOMO–LUMO gap task to show that with regards to predictive performance, model architecture and molecular representation account for comparable shares of the variance. How-

ever, for robustness to label noise, measured as $\mathrm{AUC}_{\mathrm{norm}}$, the normalized area under the R^2 retention curve, model architecture is the dominant factor, while representation explains less than 10% of the variance.

Threshold and value-proportional noise produce the widest spread of NDS across models; outlier noise barely separates them. Model rankings are highly concordant across strategies (Kendall's $W = 0.92$), suggesting that a model's robustness to noise is independent of type of noise. Predictive performance on clean data is not correlated with noise robustness; NGBoost and SVM, the most noise-robust models, did not perform particularly well on clean data. Embeddings (MHG-GNN, Mol2vec) struggled with regards to noise robustness, especially with neural network architectures. On the flip side, QRF was significantly less noise-robust than RF across datasets, suggesting that quantile regression may be overfitting to noisy labels rather than absorbing them.

The second conclusion concerns whether the uncertainty estimates of a model track per-sample label noise. Some models struggle to do so, particularly BNNs, whose uncertainty comes from a posterior over weights, and QRF, which produces a full quantile distribution but does not track the noise successfully. Models that fit a separate per-sample scale or observation-noise parameter during training track label noise more effectively. NGBoost identifies noisy labels through a per-sample predicted scale that absorbs large residuals. The GP does not, because it has a single global observation-noise term. Its per-sample uncertainty therefore comes from the posterior variance, which depends on the position of a molecule relative to the training molecules and not on its label. Among representations, uncertainty tracking collapses with the learned embeddings MHG-GNN and Mol2vec, while fingerprints and descriptors produce a usable signal. Neither strong predictive performance nor noise robustness implies noise tracking, which again depends on the architecture of the model and on how it handles the loss.

Although the choice of representation carries less weight than that of models, we observed an interesting phenomenon. PDVs produce the strongest predictive performance on clean data, but are the least noise-resistant. Embeddings, particularly Mol2vec, are the most robust to noise yet perform the worst noise tracking. At the end of the day, the choice of both model and representation comes down to your problem, including the amount of noise in your data, your objectives, and your compute limits.

QM9 served as a relatively clean baseline; it is computationally-derived with

negligible measurement noise. The validation datasets, LogD, Caco-2 [109], and hERG K_i [110], tested are smaller and cover a narrow range of the chemical and biological space, and have an unknown inherent amount of noise from experimentally-determined values, on top of the added artificial noise. These validation datasets supported the QM9 findings, with XGBoost the notable exception, degrading on the external datasets. Future work should extend this type of noise robustness benchmarking to classification tasks and larger experimental datasets with better-known uncertainty estimates.

When working with noisy experimental data in QSAR settings, it is important to keep in mind that noise robustness and noise-aware uncertainty estimation are primarily driven by the model's mechanism of training, and to a certain extent the choice of molecular representation, usually with respect to the model. While Bayesian transformations on neural networks improve noise robustness, they do not produce uncertainty estimates that correlate particularly strongly with noise. Models like NGBoost and GPs, which include a learned noise or scale parameter in the training objective, are the most robust to noise across both various types of noise and datasets and often produce uncertainty estimates which track per-sample label noise. Pairing these models with fingerprint-based representations provides additional benefits in restricting, and potentially detecting and mitigating label noise.

Chapter 4

The Flaws of the Similarity Assumption

This chapter asks whether the standard ways of comparing molecules suit the dataset and task at hand. Because general-purpose representations are built for all of chemistry, we first count how many of their features vary on a single dataset. On hERG K_i, 1,815 of the 2,048 ECFP4 features take a single value in at least 95% of the 1,412 molecules. Some representations also give different molecules the same vector, mostly stereoisomers, so that no model can tell those molecules apart. Because a distance compares every feature, features irrelevant to the task dilute it, and different representations disagree about which molecules are close. In an exploratory study of learning with noisy labels, all 20 configurations with the lowest retention under noise used a distance, and noise detection by neighbors usually made the model worse.

4.1 Introduction

One of the fundamental assumptions in cheminformatics is that two structurally similar compounds display similar chemical properties [74]. While there are important exceptions, namely activity cliffs [72, 75], this is a well-established concept. However, there is no single way to define similarity between compounds, and this is where the assumption begins to break down in practice. When defining similarity between two molecules, we typically rely on some pairwise mathematical function, or coefficient, that takes in two molecular representations as input. There's two varying factors here, the representation and the coefficient. Bajusz et al. [71] found that

among standard coefficient, the choice between them matters little. This question questions the underlying representation.

General-purpose molecular representations are designed to work across chemical domains. Consequently, when considering data from a specific domain, many of their features show little to no variance. Some representations assign separate molecules identical feature vectors. Since common fingerprints do not encode stereochemistry [116], they are often unable to differentiate stereoisomers, which can have substantially different measured activity [117]. As these coefficients compare every feature, irrelevant features dilute the result. Different representations also disagree about which molecules are close based on the information they carry.

The problem of whether neighboring molecules can predict chemical activity has been measure before. The modelability index of Golbraikh et al. [118] scores a binary dataset by the proportion of nearest-neighbor pairs that share an activity class, producing a single value for how strongly the similarity assumption holds on that particular combination of representation and data. It is not clear whether a low value is due to the data being intrinsically hard to model or to the representation not capturing adequate features for the task. Other work in this area relates to activity cliffs. Sheridan et al. [119] compared experimental error, activity distribution and activity-cliff metrics as predictors of how well a dataset can be modeled, and found the cliff metrics the strongest of the three. However, they found that these metrics cannot separate real cliffs from those produced by uncertain activity measurements. Medina-Franco [120] point out that some of these activity cliffs are artifacts of the chosen representation, similarity threshold, or noise in the data. Activity cliffs are distrinct from noise, as they carry some of the most valuable information about the interaction between structure and activity. None of these measures ask specifically about whether the specific features within these representations matter for the property at hand.

There has been work that looks into how much the choice of representation changes similarity measure. Riniker and Landrum [121] benchmarked 12 fingerprints for similarity across 3 public datasets and found high correlations between the, only two simple baseline fingerprints differed signficantly from the rest. The same was found for learned embeddings as neural embeddings gave negligible or no improvement over ECFP across 25 pretrained models and 25 datasets [122]. Huber and Pollmann [123] did find a difference between the structure of the fingerprints. Measured using a graph-based structural reference, fingerprints that store raw counts

improve agreement with the reference over folded fingerprints. The process of folding introduces bit collisions that distort similarity for molecules with many bit sets [123]. Activity cliffs follow the same pattern, across 15 combinations of representation and similarity meausre on 3 datasets, the representation disagreed about which pairs form cliffs [124]. This supports the point Medina-Franco [120] make about cliffs as artifacts of their representation. Any method built on similarity like activity cliffs inherits this choice Wilson [103]. For the purposes of this thesis, these similarity measures are used across noise estimation and mitigation tasks.

The purpose of this chapter is to establish the problem that Chapter 5 sets out to solve. Section 4.2 asks for a given dataset, how much the features of standard representations vary and how many molecules map to identical feature vectors.

Section 4.3 looks at what happens when you used these features to quantify similarity, while Section 4.4 further follows the consequences of using these similarity measures in downstream methods. This is supported by a sweep of LNL methods reported in Section 4.4.1.

4.2 Flaws in Standard Base Representations

To quantify similarity between two compounds, we must first choose a representation to encode each compound. If the features within these representations are flawed, the similarity measure inherits them. Many standard representations are built to represent compounds across a wide array of chemical domains and be task-independent. However, chemical datasets often only cover a small region of the chemical space, leaving these general-purpose representations to describe chemistry which is absent from the dataset at hand.

4.2.1 Constant and Near-Constant Features

ECFP4 folded to 2,048 bits, defined in Chapter 2, is built to describe any molecule before any dataset or task is known [4]. A single dataset, by contrast, often comes from a few chemical series, and on such a dataset most bits are off in almost every molecule. The features that matter for a task form a small subset of those that vary, and which subset matters depends on the task. Here we count only how many features vary at all, while the question of whether the varying features relate to the task is taken up in Chapter 5.

For each representation and dataset we count two quantities. A constant feature takes one value in every molecule of the dataset, and a near-constant feature takes one value in at least 95% of the molecules, so that the near-constant count includes the constant features. The counts use all molecules of each dataset and the same feature matrices as the main experiments. The datasets are hERG K_i, Caco-2, LogD and QM9, together with MDR1–MDCK efflux, human liver microsome (HLM) clearance, rat liver microsome (RLM) clearance and the Lhasa hERG blocker training set. On each dataset we count its base representations, together with the MACCS keys wherever they were computed.

On these datasets the hashed substructure fingerprints have the most near-constant features, while the learned embeddings have none, with one exception described below. ECFP4 on hERG K_i (1,412 molecules) has 1,815 near-constant features out of 2,048. Across the seven datasets where ECFP4 is a base representation, this count runs from 1,815 (hERG K_i) to 1,997 (QM9, 9,978 molecules). SMI-TED (768 features) and ChemBERT (256 features) have no near-constant features on hERG K_i. Neither do GROVER, GraphMVP, MoLFormer, ChemBERTa-2, SELFormer and Mol2vec on any dataset where they are base representations. The exception is MHG-GNN, for which 292 of its 1,024 features are zero in every molecule. These are the same 292 features on all seven datasets that use MHG-GNN, so they arise from the saved MHG-GNN output rather than from the data. We have not checked why the encoder returns zero for them. The Weisfeiler–Lehman (WL) graph kernel also has no near-constant features, because it keeps only labels present in at least 5% of a dataset's molecules and is therefore filtered on each dataset before counting. All counts are given in Tables 4.1 and 4.2, and Figure 4.1 shows them as shares of each representation's features.

Which of the two counts matters depends on the distance. A bit that is off in every molecule does not change the Tanimoto similarity, because bits that are off in both molecules are ignored. A bit that is on in every molecule, however, adds one to both the shared and the total bit counts and so raises every Tanimoto similarity slightly. On hERG K_i all 105 constant ECFP4 bits are off, whereas 1 of the 194 constant ECFP4 bits is on for Caco-2 and 1 of 39 for LogD. On hERG K_i, 2 of the 24 constant MACCS features are on. After standardization, a constant feature adds zero to a Euclidean or Manhattan distance. Near-constant features, in contrast, distort these distances, because after standardization they take large values on the few molecules that carry the minority value. A bit that is on in 5% of molecules

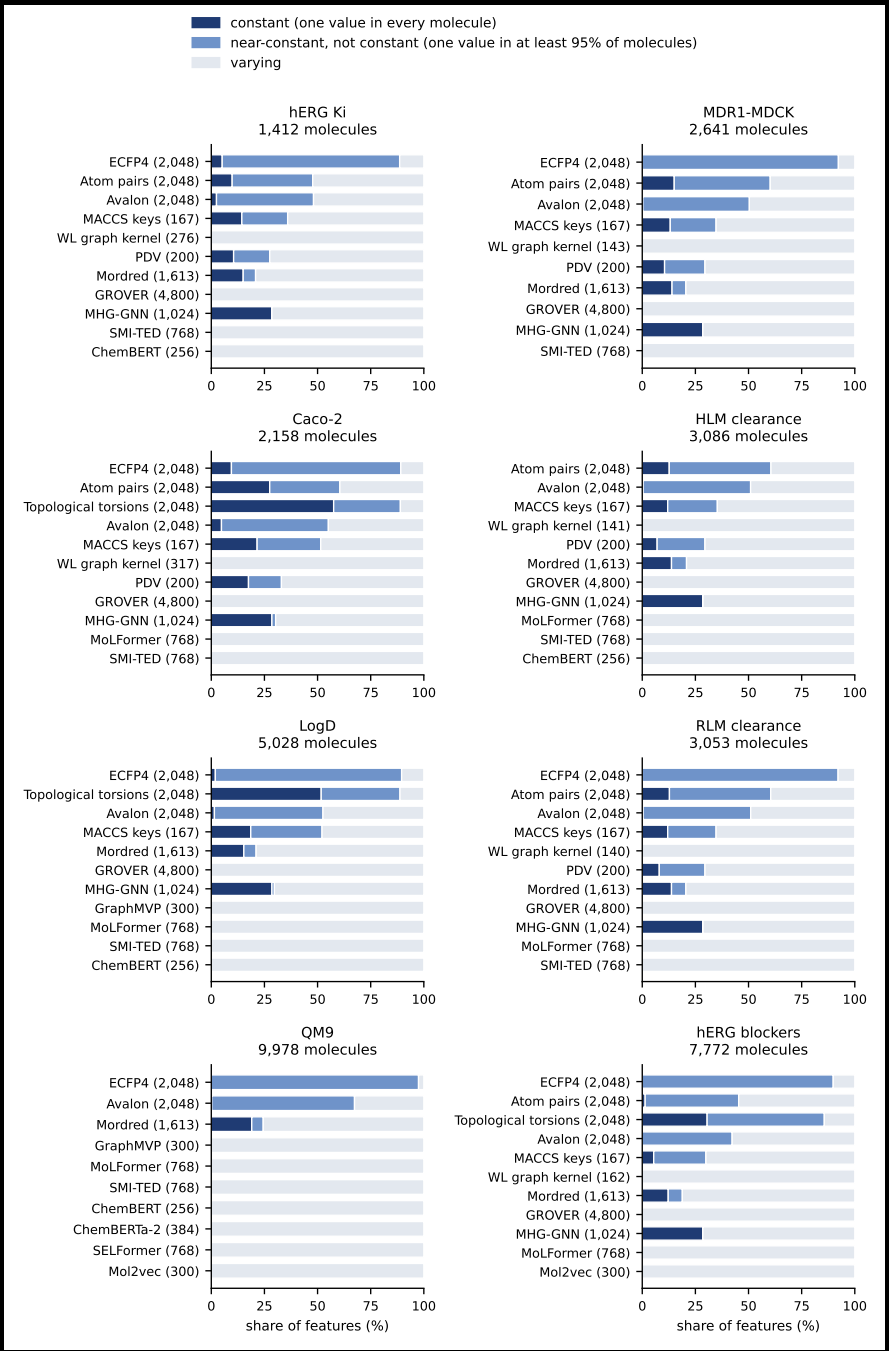


Figure 4.1: Shares of constant (dark blue), near-constant (mid blue) and varying (gray) features of each representation on eight datasets. The number of features of each representation is in parentheses.

becomes 4.36 on those molecules, and a bit that is on in one of the 1,412 hERG K_i molecules becomes 37.56 on that molecule. The argument of this chapter therefore rests on the near-constant count rather than the constant count, and Section 4.3 shows the effect on distances.

Dablander [125, p. 112] states, without giving a count, that the least frequent substructures usually appear in only a few compounds, and the same point recurs on p. 121. His statement concerns substructures before hashing, whereas our count is of bits after folding to 2,048. Bit collisions (Chapter 2) add a further difficulty, because one bit can stand for several substructures, which makes a selected fingerprint bit harder for a chemist to read than a named descriptor. This point returns in Chapter 5, in the discussion of blueprints and what they tell a chemist.

When too few bits vary, different molecules can receive identical feature vectors.

Tables 4.1 and 4.2 give the numbers of constant and near-constant features for each representation and dataset.

TODO: every cell of this table must hold a count. The dashes are representations outside that dataset's base representations; handoff in HANDOFF_fill_constant_features_tables.md.

TODO: every cell of this table must hold a count. The dashes are representations outside that dataset's base representations; handoff in HANDOFF_fill_constant_features_tables.md.

The MACCS keys have 167 features here, because RDKit stores the 166 MACCS keys together with an unused bit 0 that is off in every molecule. That bit is one of the constant MACCS features counted above. On every dataset except QM9, all constant Mordred features are zero in every molecule, and on QM9 one of the 307 constant features is 1 in every molecule while the rest are zero. Missing or infinite descriptor values are set to 0 (Chapter 5, "Representations by domain"), so some of these constant features may be descriptors that failed to compute, and we have not checked which.

4.2.2 Molecules with Identical Feature Vectors

A standard fingerprint- or descriptor-based representation can produce the same feature vector for two separate molecules. In turn, models using this representation will produce a single prediction for the molecules with identical features. It should be noted that identical feature vectors within a dataset do not necessarily mean that

Representation	Features	hERG K_i (1,412)	Caco-2 (2,158)	LogD (5,028)	QM9 (9,978)
ECFP4	2,048	105 / 1,815	194 / 1,826	39 / 1,834	0 / 1,997
Atom pairs	2,048	201 / 977	563 / 1,239	–	–
Topological torsions	2,048	–	1,179 / 1,822	1,059 / 1,818	–
Avalon	2,048	50 / 985	98 / 1,128	30 / 1,076	4 / 1,380
MACCS keys	167	24 / 60	36 / 86	31 / 87	n/a
WL graph kernel	140–317	0 / 0 / 276	0 / 0 / 317	–	–
PDV	200	21 / 55	35 / 66	–	–
Mordred	1,613	242 / 336	–	247 / 340	307 / 394
GROVER	4,800	0 / 0	0 / 0	0 / 0	–
MHG-GNN	1,024	292 / 292	292 / 310	292 / 304	–
GraphMVP	300	–	–	0 / 0	0 / 0
MoLFormer	768	–	0 / 0	0 / 0	0 / 0
SMI-TED	768	0 / 0	0 / 0	0 / 0	0 / 0
ChemBERT	256	0 / 0	–	0 / 0	0 / 0
ChemBERTa-2	384	–	–	–	0 / 0
SELFormer	768	–	–	–	0 / 0
Mol2vec	300	–	–	–	0 / 0

Table 4.1: Constant and near-constant features of each representation on hERG K_i, Caco-2, LogD and QM9. Each entry gives the number of constant features and the number of near-constant features, which take one value in at least 95% of molecules and include the constant ones. The number of molecules is under each dataset name. The width of the WL graph kernel varies, so its entries add its number of features. A dash marks a representation outside the dataset's base representations, and no MACCS keys were saved for QM9.

Representation	Features	MDR1–MDCK (2,641)	HLM clearance (3,086)	RLM clearance (3,053)	hERG blockers (7,772)
ECFP4	2,048	1 / 1,890	–	1 / 1,887	0 / 1,840
Atom pairs	2,048	308 / 1,233	260 / 1,240	262 / 1,238	28 / 929
Topological torsions	2,048	–	–	–	625 / 1,754
Avalon	2,048	5 / 1,032	8 / 1,044	8 / 1,045	0 / 866
MACCS keys	167	22 / 58	20 / 59	20 / 58	9 / 50
WL graph kernel	140–317	0 / 0 / 143	0 / 0 / 141	0 / 0 / 140	0 / 0 / 162
PDV	200	21 / 59	14 / 59	16 / 59	–
Mordred	1,613	226 / 331	222 / 336	222 / 331	197 / 304
GROVER	4,800	0 / 0	0 / 0	0 / 0	0 / 0
MHG-GNN	1,024	292 / 292	292 / 293	292 / 293	292 / 292
MoLFormer	768	–	0 / 0	0 / 0	0 / 0
SMI-TED	768	0 / 0	0 / 0	0 / 0	–
ChemBERT	256	–	0 / 0	–	–
Mol2vec	300	–	–	–	0 / 0

Table 4.2: Constant and near-constant features of each representation on MDR1–MDCK efflux, HLM clearance, RLM clearance and the Lhasa hERG blocker training set of 7,772 molecules. Entries read as in Table 4.1.

they derive from the same underlying molecule. The dataset could just hold two different measurements of the same molecule twice, as is possible with data pooled across different sources.

Many fingerprints are computed without stereochemistry [116]. In this research, we build ECFP4 fingerprints with stereochemistry off while the 2D descriptor sets (PDV, Mordred) also ignore it such that for every dataset, each molecule they label a tie is a stereoisomer of another. Any representation read from the 2D graph merges stereoisomers, since the two share an identical graph. Separately, methods using the Weisfeiler–Lehman relabeling scheme are bounded by the 1-Weisfeiler–Lehman test explained in Chapter 2, so some non-isomorphic graphs also receive the same vector; the graph-kernel features tie 105 of 1,412 hERG K_i molecules, 56 of which differ only in stereochemistry and 49 in more than that. Whether a learned embedding differentiates stereoisomers is up to its individual implementation. SMI-TED, a pretrained SMILES encoder, ties 52 hERG K_i, 633 LogD and 177 Caco-2 molecules, all stereoisomers. A tokenizer can drop stereochemistry before the model even sees the input string. The ChemBERTa-77M-MTR tokenizer used in Chapter 3 gives 72 of 1,415 hERG K_i molecules the same token sequence as another molecule. If the difference between stereoisomers was negligible, this wouldn't matter. However, stereoisomers can differ in measured activity [117], meaning these representations are all lacking potentially important chemical information.

For each dataset and representation, we grouped the molecules whose stored feature vectors are exactly identical and assigned every molecule in such a group to one of three causes. A molecule counts as a repeat if another molecule in its group has the same InChIKey, as a featurization failure if its feature vector is all zeros, and as a distinct molecule otherwise. We call a distinct molecule stereo-only when every molecule sharing its vector has the same InChIKey connectivity block, so that the molecules differ only in stereochemistry. This count requires no model to be fitted.

Every molecule in a tied group must receive the same prediction, and the prediction with the lowest squared error is the group's mean label. The lowest error any model can reach is therefore

$$\mathrm{RMSE}_{\min} = \sqrt{\frac{1}{n}\sum_{G}\sum_{i\in G}(y_i - \bar{y}_G)^2}, \tag{4.1}$$

where n is the number of molecules, G runs over the tied groups and $\bar{y}_G$ is the mean

label of group G. A molecule that belongs to no tied group adds nothing to the sum, and $\mathrm{RMSE}_{\min}$ is expressed in the dataset's label units. The corresponding best R^2 is $1 - \mathrm{RMSE}^2_{\min}/\mathrm{Var}(y)$. We also report the label spread inside each tied group, defined as its highest label minus its lowest. For scale, repeat pK_i measurements of one compound disagree by about 0.54 log units [15], so a tied group whose pK_i values differ by more than that is not explained by measurement noise alone. We counted ties on hERG K_i (1,412 molecules after deduplication by InChIKey), LogD (5,028 molecules) and Caco-2 (2,158 molecules), using each dataset's base representations and its augmentation features.

No tie on any of the three datasets came from a repeated molecule or from a row of zeros, so every tie is a merge made by the representation. Table 4.3 lists every representation that ties at least one molecule. Atom pairs, PDV, Mordred and SMI-TED tie only stereoisomers, on every dataset whose representations include them. ECFP4, Avalon and topological torsions tie mostly stereoisomers, plus a few molecules that are not stereo-only. ECFP4 on hERG K_i ties none of the latter, and topological torsions on LogD tie the most, with 44 such molecules. On hERG K_i, ECFP4, atom pairs, PDV, Mordred and SMI-TED all split the 1,412 molecules into the same 26 groups holding the same 56 molecules, so concatenating any of these five leaves all 56 tied. The WL graph kernel ties 49 hERG K_i and 26 Caco-2 molecules that are not stereo-only. The MACCS keys tie the most molecules on every dataset, and most of their tied molecules are not stereo-only, with only 30 of 377 stereo-only on hERG K_i. The 3D representations tie no molecules, and neither do the learned embeddings ChemBERT, GROVER, MHG-GNN, MoLFormer and GraphMVP on any dataset that includes them. This includes ChemBERT on hERG K_i and on LogD. Concatenating all of a dataset's representations leaves no molecule tied on any of the three datasets.

The lowest reachable error stays small relative to the spread of the labels. On hERG K_i it is at most 0.134 pK_i (MACCS keys), against a label standard deviation of 0.915 pK_i. The spread inside a single tied group can nevertheless be large. The largest ECFP4 group spread on hERG K_i is 1.14 pK_i, more than twice the repeat-measurement scale, and Figure 4.2 shows this pair. Any model trained on ECFP4 must give the two molecules the same prediction. The prediction with the lowest squared error is their mean, 5.59 pK_i, which leaves each molecule 0.57 pK_i from its measured value.

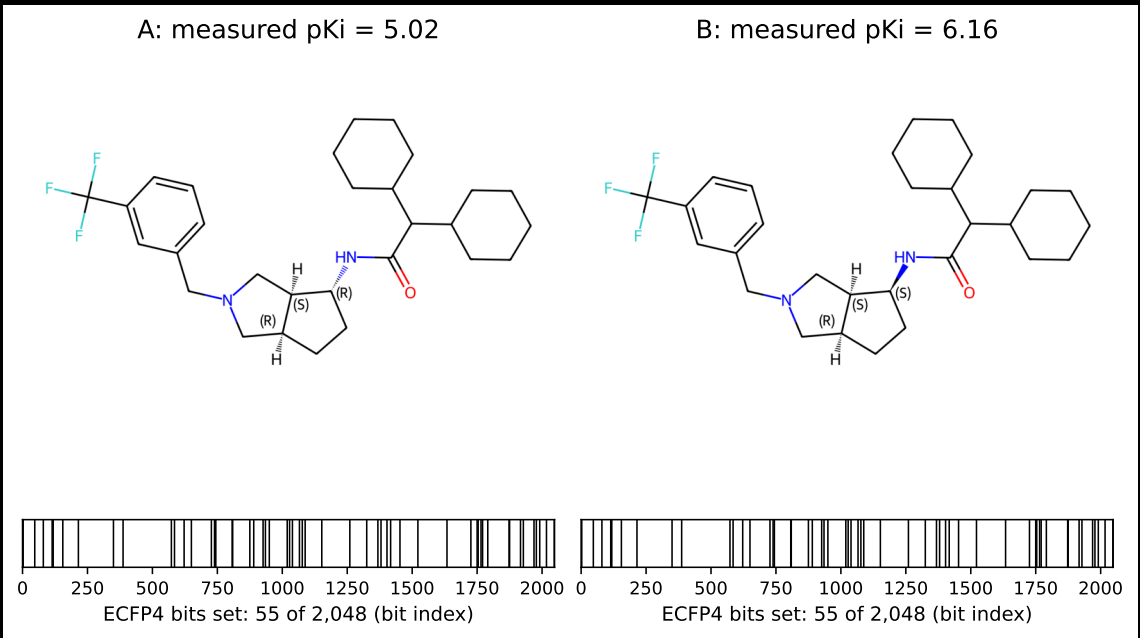


Figure 4.2: Two hERG K_i stereoisomers with identical ECFP4 vectors, which differ at the carbon bearing the amide nitrogen (R in A, S in B). The top row shows the structures with their measured pK_i values, and the bottom row shows the 55 of 2,048 ECFP4 bits set in both.

The structure–activity landscape index of a pair of molecules i and j is

$$\mathrm{SALI}_{ij} = \frac{|y_i - y_j|}{1 - \mathrm{sim}(i, j)} \tag{4.2}$$

[126]. A tied pair has similarity 1, so its index is infinite whenever the two labels differ, and such a tie is therefore the most extreme activity cliff possible. Some cliffs are thus artifacts of the representation. A method that uses nearest neighbors sees a tied neighbor at distance 0 with a different label. Such a method may then flag a correct label as noise (Section 4.4.2).

On hERG K_i, the mean label spread inside a tied group is 0.28 pK_i under ECFP4 and 0.39 pK_i under the MACCS keys. Of the 26 ECFP4 groups, 24 contain molecules with different labels and 6 differ by more than 0.54 pK_i. Of the 153 MACCS groups, 138 contain different labels and 38 differ by more than 0.54 pK_i.

It is worth noting that some of these representations are more straightforward than others in terms of the underlying chemistry they reference. For example, the PDV carries very clear physicochemical properties such that if you examined one of it's features, you'd know exactly the type of chemical information it contains. Other representations like frozen embeddings, representing the last layer of a network, do not tell us what type of chemical information, if any, they relate to.

Dataset	Representation	Molecules	Tied	Stereo-only	Best RMSE	Best R^2	Largest spread
hERG K_i (pK_i)	ECFP4	1,412	56 (4.0%)	56	0.041	0.998	1.14
	Atom pairs	1,412	56 (4.0%)	56	0.041	0.998	1.14
	PDV	1,412	56 (4.0%)	56	0.041	0.998	1.14
	Mordred	1,412	56 (4.0%)	56	0.041	0.998	1.14
	SMI-TED	1,412	56 (4.0%)	56	0.041	0.998	1.14
	Avalon	1,412	66 (4.7%)	56	0.049	0.997	1.20
	WL graph kernel	1,412	105 (7.4%)	56	0.071	0.994	1.49
	MACCS keys (aug.)	1,412	377 (26.7%)	30	0.134	0.978	1.80
LogD (log units)	ECFP4	5,028	641 (12.7%)	633	0.044	0.999	1.00
	Mordred	5,028	633 (12.6%)	633	0.043	0.999	1.00
	SMI-TED	5,028	633 (12.6%)	633	0.043	0.999	1.00
	Avalon	5,028	647 (12.9%)	629	0.047	0.998	1.40
	Topological torsions	5,028	675 (13.4%)	631	0.047	0.998	1.10
	MACCS keys (aug.)	5,028	2,343 (46.6%)	437	0.220	0.965	2.80
Caco-2 (log units)	ECFP4	2,158	179 (8.3%)	177	0.027	0.996	0.74
	Atom pairs	2,158	177 (8.2%)	177	0.027	0.996	0.74
	PDV	2,158	177 (8.2%)	177	0.027	0.996	0.74
	SMI-TED	2,158	177 (8.2%)	177	0.027	0.996	0.74
	Avalon	2,158	181 (8.4%)	177	0.027	0.996	0.74
	Topological torsions	2,158	186 (8.6%)	177	0.029	0.996	0.74
	WL graph kernel	2,158	203 (9.4%)	177	0.028	0.996	0.74
	MACCS keys (aug.)	2,158	804 (37.3%)	139	0.111	0.935	1.72

Table 4.3: Molecules that share their feature vector with at least one other molecule, for each representation that ties at least two molecules. Tied gives the number and share of such molecules, and Stereo-only counts those tied only with their own stereoisomers. Best RMSE and best R^2 are reached by predicting the mean label of each tied group, the lowest error any model can reach. Largest spread is the largest label range within one tied group, in the dataset's label units, and (aug.) marks an augmentation representation.

4.3 Flaws in Similarity and Other Pairwise Computations

We've already discussed some of the flaws of individual representations, but what happens when you compare two instanes of one of these representations? These pairwise comparisons, as described in Chapter 2, have the potential to violate the similarty assumption. When considering similarity and other pairwise comparisons, we discuss both the representation and the coefficient.

The standard way of comparing two small molecules is the Tanimoto coefficient, which operates on any bit vector, applied to ECFPs [127]. Independently, the Tanimoto coefficient is described as the "coefficient of choice" for molecular similarity [71], while Morgan fingerprint, represented as ECFP4 in this research, is likewise one of the strongest fingerprints for the task, ranking among the strongest fingerprint for virtual screening and target prediction [128]. ECFP4 and ECFP6 specifically,

alongside topological torsion, rank among the best of 28 fingerprints tested for ordering diverse structures by similarity [129]. However, it's not a one-size-fits-all solution. Across 96,456 human metabolites, 72.9% had an ECFP4 nearest neighbor at Tanimoto distance 0 [128], again running into the issue of the representation lacking stereochemical information and more generally being unable to distinguish molecules.

Chapter 2 writes out the Tanimoto coefficient T as the ratio inside the Tanimoto kernel of Equation 2.1, that is, the number of bits set in both molecules divided by the number set in either. The corresponding distance is $1-T$ [71], and the bits that are always off or always on across a dataset are counted in Section 4.2.1.

All a similarity coefficient can do is compare every feature, yet it cannot tell which features matter to the prediction task at hand. And the similarity assumption holds only relative to such a measure, which is fixed by the choice of representation. To estimate what this costs, we categorize a representation's features into those relevant to the task, R, and those that are irrelevant, I. We take the squared Euclidean distance as a sum over both:

$$d^2(\mathbf{x}, \mathbf{y}) = \sum_{j \in R} (x_j - y_j)^2 + \sum_{j \in I} (x_j - y_j)^2. \tag{4.3}$$

Every irrelevant feature that varies adds one further term, so the irrelevant part grows linearly with $|I|$. After standardization each feature contributes roughly the same amount, so the share of the distance carrying signal is about $|R|/(|R| + |I|)$. When we use a general-purpose representation like ECFP4 on a dataset spanning a small portion of the chemiccal space, $|I|$ can be a lot larger than $|R|$. Features that remain constant across the dataset contribute nothing. Rather, the damage comes from features that vary but bear no relation to property in question. The worst case scenario involves a single rare bit set in only a handful of molecules. After standarization, uncommon bits take on larger values, outweighing common features that may be more relevant. This is why the near-constant count in Section 4.2.1 matters more than the constant count.

Larger representations, or higher dimensionality, compounds the problem. As the number of dimensions grow, the nearest and farthest molecules end up at nearly the same distance, meaning that these pairwise distances are less able to discriminate, as explained in Chapter 2. We can measure this effect by measuring the relative contrast, defined as the difference between the largest and smallest mea-

sured distance divided by the smallest, and hubness, a measure of how unevenly the molecules appear in other molecule's nearest neighbors [130]. These errors propogate along larger data pipelines, starting from the representation into similarity, from similarity into thes set of neighbors a molecule is compared against, and from there into a prediction of some kind.

We count activity cliffs with the rule of Chapter 2. A pair counts as similar at an ECFP4 Tanimoto similarity of at least 0.55, and the activity gap is set per dataset so that 15% of molecules are cliffs. Because the share moves in steps as the gap changes, LogD reaches only 14.5%. The number of cliff molecules is therefore close to fixed by the rule, and it is the gap that varies between datasets (Table 4.4). On hERG K_i, 211 of the 1,412 molecules are cliffs at a gap of 1.76 pK_i. On QM9, only 1,093 pairs among 9,978 molecules reach a Tanimoto similarity of 0.55. The gap that flags 15% of QM9 molecules is then 0.05 eV, or 0.06 times the median absolute deviation of the labels. Three datasets come from the Biogen set, namely RLM clearance, MDR1–MDCK efflux and HLM clearance, and on these the gap is 0.20 to 0.34 log units, or 0.50 to 0.60 times the median absolute deviation. On QM9 and these three datasets, the gap is therefore smaller than the median absolute deviation of the labels. A gap of twice the median absolute deviation, the rule used in an earlier version of this analysis, flags 31 of 9,978 QM9 molecules and 3,529 of 5,028 LogD molecules.

The count also depends on the representation. Keeping each dataset's gap, we replaced ECFP4 with each of the dataset's other base representations and took as similar pairs that representation's own closest pairs, as many as ECFP4 calls similar. On hERG K_i the number of cliff molecules then ranges from 138 (atom pairs) to 267 (WL graph kernel). Only 57% to 74% of the pairs that ECFP4 calls similar are among another representation's closest pairs, and on QM9 that share falls to 1% to 24%. Every count is given in Table S.11.

Two different questions can be asked of a distance. The first concerns all pairs and asks whether distance tracks the difference in property across the whole dataset. Plotting descriptor distance against activity difference for every pair gives the neighborhood-behavior plot of Patterson et al. [131], which we summarize by a single number, the Spearman correlation between pair distance and absolute label difference. The second question concerns nearest neighbors only and asks whether each molecule's label can be predicted as the mean label of its five nearest neighbors. The neighbors can carry signal even when the all-pairs correlation is near zero, and

each claim below states which of the two questions it addresses.

We computed both quantities for every base representation of each of the seven regression datasets, that is, the six datasets of Table 5.16 and HLM clearance. The six comprise the small-molecule datasets and QM9, and HLM clearance is not in that table. Representations with only binary features were compared by Tanimoto distance, $1 - T$, and all others by Manhattan distance on standardized features, leaving out features that are constant across the dataset. Manhattan distance is defined in the next subsection. For the all-pairs correlation, each dataset with more than 2,000 molecules was reduced to a fixed random sample of 2,000 molecules (seed 0), giving 1,999,000 pairs per dataset. On hERG K_i, all 996,166 pairs of the 1,412 molecules were used. For the neighbor prediction all molecules were used, and each molecule's five neighbors were drawn from other scaffold groups, so that no molecule is predicted from its own scaffold.

Across the 70 combinations of dataset and representation, the all-pairs correlation ranged from -0.04 (MACCS keys on MDR1–MDCK efflux) to 0.23 (PDV on the same dataset) and was below 0.10 in 44 of the 70. The Spearman correlation of the five-neighbor prediction ranged from 0.29 (Avalon on RLM clearance) to 0.89 (ChemBERTa-2 on QM9), and it was the larger of the two numbers in all 70 combinations. On hERG K_i, ECFP4 gives an all-pairs correlation of 0.06, while its five-neighbor prediction reaches 0.72, with an RMSE of 0.63 pK_i against a label standard deviation of 0.92 pK_i. Table 4.4 gives both numbers for three representations per dataset.

Figure 4.3 shows why the two numbers differ on hERG K_i. Of the 996,166 pairs, 93% have an ECFP4 Tanimoto similarity below 0.20 and 1.1% reach 0.55. The median label difference is 0.81 pK_i across all pairs but falls to between 0.20 and 0.30 pK_i above a Tanimoto similarity of about 0.4. Even among the 774 pairs at 0.8 or above, however, 8.3% still differ by more than 1 pK_i. Under Mordred, the closest 5% of pairs have a median difference of 0.44 pK_i, whereas every other 5% of pairs has a median between 0.78 and 1.11 pK_i. The signal therefore sits in a thin tail of close pairs. The all-pairs correlation is set mostly by the other 95% of pairs, whereas a five-nearest-neighbor prediction looks only at the closest few pairs of each molecule.

A third view asks how often two representations agree on each molecule's five nearest neighbors. We compared ECFP4, one descriptor set and SMI-TED, a learned embedding that is a base representation of every one of these datasets. The descrip-

tor set is Mordred, except on Caco-2, where Mordred is not a base representation and PDV is used instead. HLM clearance does not include ECFP4 among its base representations, so its stored ECFP4 features were used for this comparison only. The neighbor sets are those used for prediction. On hERG K_i any two of the three representations share on average 0.47 to 0.52 of each molecule's five neighbors. On the three Biogen datasets and QM9 they share 0.13 to 0.21, fewer than one neighbor in five (Table 4.5).

A fourth view asks whether fingerprint distance agrees with a structural distance. We sampled 500 random pairs per dataset (seed 0) and computed their maximum common substructure (MCS) with RDKit [32], matching ring atoms only to ring atoms, keeping only complete rings and allowing each pair 2 seconds. The MCS distance is $1 - n/(a + b - n)$, where n is the number of atoms in the common substructure and a and b are the numbers of heavy atoms of the two molecules. Pairs that reached the time limit were left out, namely 5 on hERG K_i, 8 on Caco-2 and 6 on LogD. The Spearman correlation between ECFP4 Tanimoto distance and MCS distance ranged from 0.20 (hERG K_i and HLM clearance) to 0.51 (Caco-2 and LogD).

A similarity can also be learned. Coupry and Pogány [132] learn a molecular similarity from graph embeddings, but independently of any endpoint, so that it is still not chosen for the property being predicted. Earlier work did choose the similarity for the task. Zheng and Tropsha [133] predict each molecule's activity as the average over its k most similar molecules, using Euclidean distance over descriptors, and select the descriptor subset by simulated annealing to maximize the leave-one-out q^2. Itskowitz and Tropsha [134] extend this approach to optimize the descriptor subset, the number of neighbors and the neighbor weighting function jointly. Both methods select features for a nearest-neighbor model of one endpoint, whereas the method of this thesis selects features across several representations. Every method in Section 4.4 takes a similarity as input and inherits the dilution described here. Chapter 6 ("Similarity in a hybrid") tests whether a feature set chosen for the task gives a better similarity.

Distance Metrics for Continuous Molecular Descriptors

We now consider 8 commonly used distance metrics and how their formulation impacts down-the-line applications to distance-based noise detection. While binary

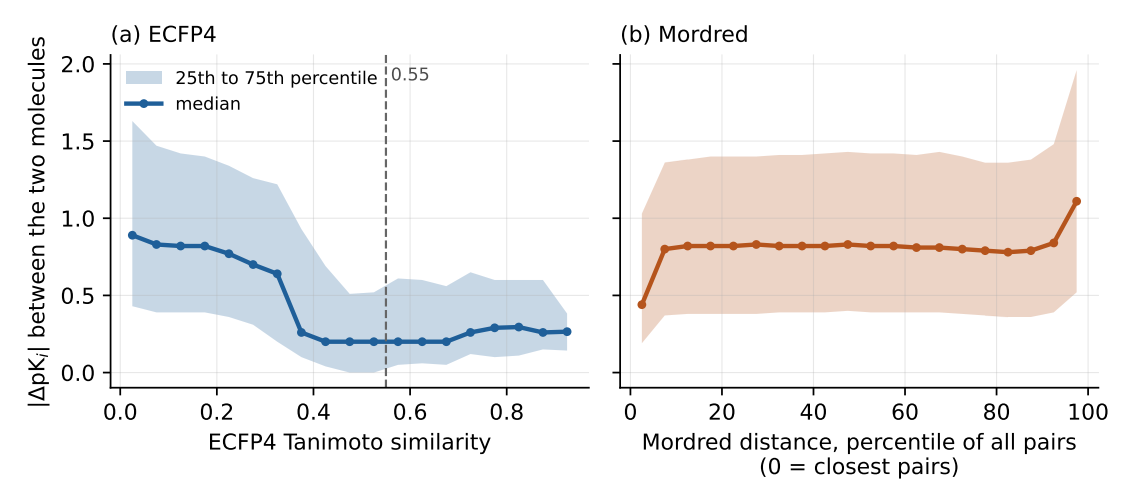


Figure 4.3: Absolute pK_i difference against distance for all 996,166 pairs of the 1,412 hERG K_i molecules, with the median in each bin (markers) and the interquartile range (shaded). a) ECFP4 Tanimoto similarity in bins of 0.05, omitting bins with fewer than 20 pairs, with the activity-cliff threshold of 0.55 dashed. b) Manhattan distance on standardized Mordred features, as a percentile of all pairs.

		All pairs			Five neighbors			Cliffs	
Dataset	Molecules	ECFP4	Desc.	SMI-TED	ECFP4	Desc.	SMI-TED	Molecules	Gap
hERG K_i	1,412	0.06	0.07	0.09	0.72	0.72	0.69	211	1.76 pK_i
LogD	5,028	0.12	0.06	0.10	0.76	0.80	0.76	731	2.90 log
Caco-2 efflux	2,158	0.08	0.18	0.14	0.59	0.62	0.57	324	1.21 log
RLM clearance	3,053	0.02	0.07	0.04	0.36	0.49	0.46	458	0.34 log
MDR1–MDCK efflux	2,641	0.02	0.16	0.10	0.51	0.58	0.59	397	0.20 log
HLM clearance	3,086	−0.00†	0.07	0.03	0.37†	0.49	0.45	462	0.27 log
QM9	9,978	0.19	0.18	0.13	0.88	0.84	0.85	1,495	0.05 eV

Table 4.4: All-pairs correlation, five-nearest-neighbor prediction and activity cliffs for each dataset. All pairs gives the Spearman correlation between pair distance and absolute label difference, on a sample of 2,000 molecules for larger datasets. Five neighbors gives the Spearman correlation between each label and the mean label of its five nearest neighbors from other scaffold groups. Desc. is Mordred, or PDV on Caco-2 efflux. Cliffs gives the number of cliff molecules and the label gap, where a cliff pair has an ECFP4 Tanimoto similarity of at least 0.55 and a label difference above the gap, which is set so that 15% of molecules are cliffs. †ECFP4 is not a base representation of this dataset. Table S.11 gives every representation.

fingerprints are compared with the Tanimoto coefficient, as described in Chapter 2, continuous features need a different measure. The performance of these 8 metrics in noise detection algorithms is compared in Section 4.4.2.

- Euclidean (L_2): standard geometric distance, $d(x, y) = \sqrt{\sum_i (x_i - y_i)^2}$
- Manhattan (L_1): sum of absolute differences, $d(x, y) = \sum_i |x_i - y_i|$, more robust to outlier features

Dataset	Desc.	Five-neighbor overlap			ECFP4 against MCS		
		ECFP4–Desc.	ECFP4–SMI-TED	Desc.–SMI-TED	Spearman	Pairs	Timed out
hERG K_i	Mordred	0.52	0.50	0.47	0.20	495	5
LogD	Mordred	0.38	0.41	0.41	0.51	494	6
Caco-2 efflux	PDV	0.38	0.41	0.38	0.51	492	8
RLM clearance	Mordred	0.15	0.21	0.18	0.28	500	0
MDR1–MDCK efflux	Mordred	0.15	0.20	0.17	0.26	500	0
HLM clearance	Mordred	0.15	0.21	0.18	0.20	500	0
QM9	Mordred	0.13	0.18	0.21	0.46	500	0

Table 4.5: Agreement between representations for each dataset. Five-neighbor overlap is the mean fraction of a molecule's five nearest neighbors, drawn from other scaffold groups, that two representations share. ECFP4 against MCS gives the Spearman correlation between ECFP4 Tanimoto distance and MCS distance on 500 random pairs, with the numbers of pairs used and left out at the 2-second limit.

- Chebyshev (L_∞): maximum absolute difference, $d(x, y) = \max_i |x_i - y_i|$
- Cosine: angle-based, $d(x, y) = 1 - \frac{x \cdot y}{\|x\|\|y\|}$, invariant to magnitude
- Correlation: $d(x, y) = 1 - \rho(x, y)$, where ρ is the Pearson correlation taken across the features of one pair of molecules, not across molecules
- Canberra: weighted L_1, $d(x, y) = \sum_i \frac{|x_i - y_i|}{|x_i| + |y_i|}$
- Bray–Curtis: ecological dissimilarity, $d(x, y) = \frac{\sum_i |x_i - y_i|}{\sum_i |x_i + y_i|}$ [135]
- Mahalanobis: covariance-adjusted distance, $d(x, y) = \sqrt{(x - y)^\top S^{-1} (x - y)}$, where S is the feature covariance estimated on the training molecules [136]

All features are standardized to zero mean and unit variance before distances are computed, so that features on different scales contribute equally. Bray–Curtis requires non-negative input, so for it the standardized matrix is shifted once to make its minimum zero, whereas Canberra is computed on the standardized values directly.

Chapters 5 and 6 use four of these: Manhattan by default, Euclidean, cosine, and Minkowski with $p = 0.5$, $d(x, y) = \left(\sum_i |x_i - y_i|^{0.5}\right)^2$, which breaks the triangle inequality and is therefore a dissimilarity rather than a metric. Feature weights from the hybrid are applied after standardization, as defined in Chapter 6.

4.4 Consequences for Similarity-Based Methods

We have identified flaws in the representations themselves and in the similarities computed over them. For a given molecule, its set of neighbors, built from one representation and one distance, is the input to several other processes such as predicting a molecule's activity from its nearest neighbors, detecting noisy labels by comparing a label against its neighbors' labels, defining an applicability domain by distance to the training set [137], grouping molecules into clusters for data splits [138], identifying activity cliffs, and, in federated learning, weighting each site's vote on a molecule by that molecule's distance to the site's own data [139]. If the distance is flawed, then every application of distance will suffer the consequences. When I was exploring LNL methods, I noticed that the ones that relied on the concept of molecular distance were unsuccessful, prompting me to look into the distance itself, as desribed in Section 4.4.1.

Section 4.4.1 reports the exploratory LNL study, in which every configuration that used a distance ranked among the worst. Section 4.4.2 turns to noise detection by neighbors, the most direct use of similarity in this thesis.

4.4.1 Exploratory Study of Learning with Noisy Labels

This section reuses an exploratory study of LNL carried out earlier in this project. In Chapter 3, the choice of representation explained more of the variance in clean performance than the choice of model, whereas for the performance lost under noise the model had the stronger effect. We therefore asked whether training methods built for noisy labels improve noise robustness, and whether the representation still matters when they are used.

LNL methods and the five groups of Song et al. [100] are described in Section 2.5.2, and such methods have been studied across machine learning, including QSAR [17]. Cortes-Ciriano et al. [17] examined 12 algorithms on 12 bioactivity datasets, adding Gaussian noise ranging from zero up to each dataset's range of pIC50 values. Gradient boosting tolerated noise worst, SVM and GP models with polynomial kernels were sensitive at high noise in some cases, and the other algorithms degraded smoothly and linearly with the noise level. Most LNL methods were built for image classification with flipped labels [100], and because the labels here are continuous, these methods do not transfer automatically. Cheminformatics data add further challenges in the form of high-dimensional molecular representations

and a wide variety of chemical domains.

The exploratory study tested 12 LNL methods on the HOMO–LUMO gap from QM9, using 10,000 molecules and scaffold splits. Gaussian noise with standard deviation σ from 0 to 1.0, in steps of 0.1, was added to the training labels only, and the test labels stayed clean. The methods were paired with five representations, ECFP4 (2,048 bits), Sort & Slice [125], PDV, canonical SMILES and randomized SMILES. Depending on the method, a configuration could include up to four optional components, namely the model's own uncertainty, a distance between molecules, an alternative GP kernel and an alternative loss function. The distances were Tanimoto on ECFP4, Euclidean on PDV and Mahalanobis, and the GP kernels were Tanimoto, RBF and Matérn. In total we tested 221 configurations over method, loss function, distance and representation. The study grouped the 12 methods into five categories of its own, namely meta-learning, sample selection, curriculum learning, iterative refinement and GP extensions, which differ from the five groups of Song et al. [100]. Four of the 12 methods, small-loss selection, confident learning, distance-based selection and multistage cleaning, remove suspected molecules and then retrain. These four are noise mitigation methods, and the other eight are noise-robust training methods.

We measured noise robustness by retention, defined as R^2 under noise divided by R^2 with clean labels, as a percentage. Methods that used a distance did worse than those that did not, and all 20 configurations with the lowest retention used a distance. None of the configurations with a distance reached both a clean R^2 above 0.6 and a retention above 80%. The failure lies in the distance step, since the same methods without the distance component did not fail in this way. Not every use of similarity failed, however, because the heteroscedastic GP compares molecules through its kernel and still held up. The failure is specific to judging one label by the labels of its neighbors.

The methods that held up estimate noise from the model's own loss or uncertainty. DivideMix [140] retained the most, 86.7% (standard deviation 0.4%), followed by small-loss selection [141] with 84.2% (standard deviation 2.1%) and the uncertainty curriculum [142] with 82.7% (standard deviation 3.5%). The heteroscedastic GP [143] retained 79.0%, averaged over 15 configurations (3 kernels and 5 representations). Its R^2 was 0.71 with clean labels and 0.56 at $\sigma = 0.6$, the noise level at which its retention was measured. The original study does not state the noise level at which the other methods' retention was measured. Regularization

alone retained less, with 73.0% for Mixup and 74.0% for sharpness-aware minimization. None of the methods that held up compares a label with its neighbors' labels. Under these methods, the choice of representation explained even less variance than in Chapter 3. Chapter 6 compares twelve LNL methods on the current datasets.

The study used one dataset, one property and one noise type, namely Gaussian noise on every training label, and Chapter 6 repeats the question on more datasets and noise types. The distance step was also added to methods designed without one, so a poor adaptation is an alternative explanation. The recommendation of noise-robust training over noise mitigation (Chapter 7, "Limitations") originates in this study.

There are two possible reasons for the poor performance of the distances. First, a single molecular representation may lack sufficient information for distance calculations, with and possibly even without noise. PDVs encode global molecular properties, ECFP4 contains substructure information and MHG-GNNs encode the molecular graph [8], and any one of these sources alone may not be sufficient for robust similarity estimation. Second, some of these representations are sparse and contain many features that are not relevant to the molecules of a given dataset. When a large proportion of features is uninformative, similarity measures become dominated by them. Each uninformative feature that varies adds one more term to the distance, so their share of it grows linearly with their number. The evidence here does not separate the two reasons. We sought to address the second with a representation of informative features and low dimensionality, the budget-constrained hybrid representation, which concatenates the top-ranked features by importance from a selection of base representations.

4.4.2 Noise Detection by Neighbors

Similar molecules should have similar labels, so a label far from the labels of its neighbors is treated as probably wrong. The idea goes back to editing a nearest-neighbor training set by removing points whose label disagrees with their neighbors [103]. The wider literature on filtering noisy labels concerns classification [144, 145]. Martín et al. [146] adapt fourteen classification noise filters to regression, inject noise into the labels of 20 datasets and find that the filters improve the regression methods. Seven of the fourteen are nearest-neighbor filters (ENN, AENN, GE, BBNR, CNDC, CNN and RNN), and for regression each replaces the neighbors' majority class with

the mean of the neighbors' labels [146]. The other seven build ensembles of models and remove a sample by vote, and four of these (DF, EF, FMF and HRRF) include regression k-NN among their models [146]. Their 20 datasets are general regression benchmarks rather than QSAR datasets. Noise detection by neighbors is the most direct use of the similarity assumption in this thesis. The neighbor-based noise scores, sample removal, sample weighting, label smoothing and the recovery rate are defined in Chapter 6, "Noise mitigation methods".

The method requires both that the distance picks the right neighbors and that the labels of those neighbors are clean. The earlier sections of this chapter argue that the first condition fails, and the second fails whenever noise is spread over the whole training set. A large gap between a label and the labels of its neighbors can indicate a noisy label, a noisy neighbor or a wrong set of neighbors, and the score alone cannot tell which. Activity cliffs also look like noise to such a detector (Chapter 2, "Molecular similarity and activity cliffs").

The exploratory study also measured what happens when neighbors are used to clean labels. It used 5,000 randomly chosen QM9 molecules with the HOMO–LUMO gap as the target and a scaffold split, and added Gaussian noise to the training labels at $\sigma = 0.3$ and $\sigma = 0.6$. Each cleaning method was judged by retraining one random forest on ECFP4 with clean, noisy and cleaned labels, and from these three fits we computed the recovery rate in R^2 (Chapter 6, "Recovery rate"). Neighbor-based scores compared each molecule's label with the labels of its k nearest training molecules under Euclidean, Manhattan or cosine distance. Table 4.6 holds only the configurations built on a single representation, because those built on a hybrid used an allocation method that is no longer part of the thesis. This study is separate from the one in Chapter 6 ("Sample removal methods"), which removed molecules and used 10,000 QM9 molecules and eight distances. The present study weighted and smoothed labels and used 5,000 molecules and three distances.

Table 4.6 reports the recovery when the neighbor-based scores were used as sample weights at strength $\alpha = 0.1$. With these weights, mean recovery was negative for every representation at both noise levels. Across all 860 cleaning configurations, 8 had positive recovery at both noise levels. Of these 8 configurations, 6 used a single representation, and 4 of those 6 were neighbor-based. Label smoothing toward neighbor or model estimates gave positive recovery in 5.0% of configurations at $\sigma = 0.3$ and 0.8% at $\sigma = 0.6$. Mean recovery was negative at every weighting and smoothing strength tried, from 0.1 to 0.7.

	Mean recovery (%)		Positive (% of configurations)	
Representation	$\sigma = 0.3$	$\sigma = 0.6$	$\sigma = 0.3$	$\sigma = 0.6$
ECFP4	−3.0	−7.3	33.3	16.7
PDV	−2.4	−3.4	50.0	27.8
MHG-GNN	−0.1	−7.2	38.9	5.6

Table 4.6: Recovery rate in R^2 when neighbor-based noise scores are used as sample weights ($\alpha = 0.1$), for each representation used to find the neighbors. The task is the QM9 HOMO–LUMO gap on 5,000 molecules with Gaussian noise on the training labels, and the downstream model is a random forest on ECFP4. Positive is the share of configurations with recovery above zero.

Judging a label by its neighbors usually made the model worse and rarely helped at both noise levels, although it did not always fail. The best single configuration at $\sigma = 0.6$ was neighbor-based, using PDV, Euclidean distance and the local variance ratio score, and it recovered +17.2%. Because noise was added to every training label, however, a molecule's neighbors are noisy too, and the neighbor mean is itself a noisy target. The original study gave this as the reason that label smoothing failed.

Overall, and in agreement with Section 4.4.1, using a sample's neighbors to determine its ground truth is flawed in this setting, most likely because all labels are too uniformly noisy for such a method. Uncertainty-based methods, which use no neighbors, show potential at lower noise, but their estimation degrades at higher noise. Sample weighting did better than label smoothing but still usually hurt, giving positive recovery in 14.2% of configurations at $\sigma = 0.3$ and 4.7% at $\sigma = 0.6$, with mean recoveries of −22.9% and −20.9%. The best configuration described above, which used exponential sample weighting, recovered +11.4% at $\sigma = 0.3$ and +17.2% at $\sigma = 0.6$.

The hybrid in that study combined PDV and MHG-GNN at 100 features and was built by greedy forward selection, which is no longer part of the method. The current hybrid is built as described in Chapter 5, and its search sweeps budgets from 10 to 2,000 features. For neighbor-based scores, the mean recovery of the earlier hybrid was −0.9% at $\sigma = 0.3$ and −3.0% at $\sigma = 0.6$, with 44.4% and 27.8% of configurations positive. Distance-based methods remain challenging under Gaussian noise regardless of representation, which suggests that the core issue is the noise distribution rather than representation quality alone. Chapter 6 repeats the measurement with neighbors found in a hybrid and under noise types other than

Gaussian.

4.5 Conclusions

This chapter asked three questions about the representations used to compare molecules and answered each with a direct measurement.

The first question was how much of a standard representation varies on a single dataset, and the answer is often very little. On hERG K_i, 1,815 of the 2,048 ECFP4 features take one value in at least 95% of the 1,412 molecules (Section 4.2.1). Several representations also give different molecules the same feature vector. ECFP4 gives 56 of the 1,412 hERG K_i molecules a vector shared with another molecule, all of them stereoisomers ("Molecules with identical feature vectors"). No model can separate those molecules, whatever their labels.

The second question was whether distance follows the property. Because a distance sums over every feature, the remaining features dominate the sum when only a few features matter for the property (Section 4.3). Across 70 combinations of dataset and representation, the Spearman correlation between the distance of a pair and its label difference was below 0.10 in 44. We also predicted each molecule's label as the mean of its five nearest neighbors' labels, and the Spearman correlation of this prediction with the label was higher in all 70 combinations. The useful information therefore sits in a few close pairs rather than in the distance as a whole.

The third question was what happens to methods that rely on distance. In the exploratory LNL study, the configurations that used a distance between molecules did worst under label noise (Section 4.4.1). Judging a label by its neighbors' labels usually made the model worse and rarely helped at both noise levels (Section 4.4.2). It did not always fail, since one neighbor-based configuration gave the best recovery at the higher noise level. The noise was Gaussian and added to every training label, so each molecule's neighbors were noisy too, and Chapter 6 returns to other noise types.

There are two possible reasons for the failure of the distance. A single representation may carry too little information to judge similarity, or a sparse representation may carry many features irrelevant to the molecules of the dataset. Our evidence does not separate the two, and the requirements below address the second.

A representation that avoids these flaws would need five properties. Its features would be chosen for the task, using only the training labels of each fold. It would

be compact, with features that do not vary or do not matter removed before any distance is taken or any model is fitted, and the number of features kept would be searched rather than fixed. It would draw on several representations, since each misses information that the others capture, and plain concatenation meets this requirement but not the previous one (Chapter 5, "Combining representations and the alternatives"). It would work both as a model input and as a space in which to compute similarity. Finally, it would be traceable, so that each chosen feature leads back to its representation and can be read chemically wherever the base feature can.

Feature selection cannot fix every flaw. Molecules that are identical in every base representation stay identical in the hybrid, and a selected hashed bit still merges every substructure that folds into it. Selection also needs enough labeled molecules to estimate which features matter.

These flaws are where the thesis began. Our hypothesis was that similarity as usually computed, with ECFP4 and the Tanimoto coefficient or their variants, carries many features irrelevant to the task. We further expected that methods built on it, such as noise detection and noise mitigation, would suffer as a result. Here a task means a supervised model fitted on a training set. The first version of our method selected the task-relevant features of several representations, with the number of features set by the user, and was built only to compute similarity. The selected feature sets then turned out to work as ordinary model inputs without any similarity step, and the selection itself became the method, the adaptive hybrid representation.

Building this method raises several questions, which Chapter 5 addresses. They concern how a feature is scored (the importance method), how the budget is split across representations (the allocation method) and how a candidate hybrid is scored (the internal evaluator). They also concern how many features are kept (the budget) and which downstream model the selected features feed. Features are chosen per representation rather than from one pooled ranking, because some datasets prefer features from only the top few representations, and Chapter 5 develops this argument.

Table 4.7 lists each flaw found in this chapter, the requirement it sets for a representation and where the thesis addresses it.

Flaw (where shown)	Requirement	Where handled
Features that do not vary ("Constant and near-constant features")	Features chosen for the task, within a budget	"Feature importance scoring"
Molecules merged into one vector ("Molecules with identical feature vectors")	Features drawn from several representations	"Combining representations and the alternatives"
Distance dominated by irrelevant features ("Flaws in similarity and other pairwise computations")	Compact, with the number of features searched	"The search space and why each axis is swept"
Poor performance of neighbor-based methods ("Consequences for similarity-based methods")	Usable as a space for similarity	"Similarity in a hybrid" (Chapter 6)

Table 4.7: Flaws of standard representations and similarity, the requirement each sets for a representation, and where the requirement is addressed, in Chapter 5 unless stated.

Chapter 5

Building Adaptive Hybrid Representations

This chapter asks how several representations can be joined without also joining their irrelevant features. We answer with the adaptive hybrid representation, a set of at most a fixed number of features, the budget, selected from several base representations by a search for one dataset and one downstream model. The hybrid is therefore an input to a model rather than a model itself. The search covers the importance method, the allocation method, the number of representations, the internal evaluator, the budget and the downstream model. Because every selected feature traces back to its representation, each finished search yields a blueprint that a chemist can read. The method needs only a supervised label and several vector representations of the same samples, and we test it on small molecules, antibodies, reactions, nucleotide sequences and tumor data. **[RESULT NEEDED: on how many datasets the hybrid beats the best single base representation, and whether the preferred search settings differ between datasets and models]**.

5.1 Introduction

Chapter 4 showed that standard representations carry many features that do not vary, or do not matter, on a given dataset, and that a distance computed over all of those features inherits the problem. That chapter ended by calling for features chosen for the task from several representations, with the irrelevant ones removed first. Chapter 3 gave a practical reason to search over this choice instead of fixing it, since the best representation changed with both the dataset and the model. The

selection described here was first built to compute similarity and was later found to work as a model input.

In this chapter we build the adaptive hybrid representation, hereafter the hybrid, and report first results on when and why it helps. Each hybrid draws on base representations of the same molecules and is specific to one dataset and one downstream model.

This chapter measures prediction only, and distances computed in a hybrid, noise detection and learning with noisy labels are the subject of Chapter 6. The methods described here are final, and each dataset is split into 5 outer folds. The hybrid beats the best single base representation on **[RESULT NEEDED: number of the 16 datasets on which the hybrid beats the best single base representation]** of the 16 datasets. **[RESULT NEEDED: whether the preferred search settings vary between datasets, and which settings]**.

The rest of this introduction reviews ways of combining representations and the alternatives to combining them, then describes the hybrid, its blueprints and its use beyond small molecules, before the methods, results, discussion, limitations and conclusions.

5.1.1 Combining Representations and the Alternatives

Prior research in this field shows promise, with conjoint fingerprints achieving stronger performance on binding affinity prediction than their component fingerprints alone across a range of models [23]. Augmenting structure-based fingerprints like ECFP4 with bioactivity-based fingerprints has also been successful at improving both predictive performance and scaffold-hopping [147]. A very similar fusion strategy, in which features are selected from six non-hashed fingerprints and are combined by feature importance score within individual models, consistently improves predictive accuracy over the original fingerprints [148].

The majority of the work in this space deals with combining relatively standard fingerprints and linear representations like SMILES [23, 147]. However, graph-based representations capture important structural information that is often lost in fingerprints. We explore using GNN embeddings within such hybrid fusion methods. We further expand on the mechanisms used to compute feature importance and concatenate features, and we use these hybrids within similarity quantification and noise mitigation tasks. We also explore adding smaller sets of structural features to

a single base representation, referred to as augmentation.

The majority of the work in this space deals with combining relatively standard fingerprints and linear representations like SMILES [23, 147]. However, graph-based representations capture important structural information that is often lost in fingerprints, and we therefore include GNN embeddings in the hybrid. MHG-GNN and GROVER are base representations on every small-molecule property dataset except QM9, and GraphMVP is a base representation on QM9 and LogD (Table 5.11). We further expand on the mechanisms used to compute feature importance and to select and join features. We also use these hybrids for similarity quantification, noise detection and learning with noisy labels. Finally, we explore adding smaller sets of structural features to a single base representation, referred to as augmentation.

So far, the most common strategy for combining feature sets has been concatenation, which requires only a single downstream model [149]. Concatenation works best when the feature sets are homogeneous and degrades when they are not [149], and Deng et al. [106] called the practice "misleading" when the goal is to measure the contribution of individual representation types. Late fusion trains a separate model per source and aggregates the predictions [150], which avoids the dimensionality problem but loses interactions between features from different sources. Recovering those interactions requires intermediate fusion (MOGONET [151], cross-attention [152]) , at the cost of end-to-end training on all sources, so that adding a new source means retraining from scratch. Multiple kernel learning [153] operates entirely at the kernel level, so no interpretable feature subsets emerge from it. As Steyaert et al. noted, there remains no decisive evidence that any of these strategies is broadly superior [150].

So far, the most common strategy for combining feature sets has been concatenation, which requires only a single downstream model [149]. Concatenation works best when the feature sets are homogeneous and degrades when they are not [149]. It also obscures which representation is responsible for the score. Deng et al. found that appending RDKit2D descriptors to GROVER lowered RMSE on 9 of 24 datasets, yet they still called the practice "misleading" when the goal is to measure the contribution of individual representation types [106]. Late fusion trains a separate model per source and aggregates the predictions [150], which avoids the dimensionality problem but loses interactions between features from different sources. Recovering those interactions requires intermediate fusion (MOGONET [151], cross-attention), at the cost of end-to-end training on all sources, so that adding a new source means

retraining from scratch. Multiple kernel learning [153] operates entirely at the kernel level, so no interpretable feature subsets emerge from it. An example in drug bioactivity prediction is pairwiseMKL [154]. As Steyaert et al. noted, there remains no decisive evidence that any of these strategies is broadly superior [150].

In practice, most research groups default to one representation and ignore the rest. ECFP is common for small molecules, while deep learning groups may opt for embeddings like MoLFormer or ChemBERTa. ESM-2 embeddings are common for antibodies and can be computed per chain. Some research compares these representations, but little focuses on combining them beyond simple concatenation. Concatenating just three standard sources, however (ECFP4 at 2,048 dimensions, ESM-2 at 1,280, Mordred descriptors at ∼1,600), gives a ∼5,000-dimensional input for a dataset that may contain only a few hundred labeled samples. At these sample-to-feature ratios, more features mean worse predictions [155, 156]. Models sensitive to distance, such as k-nearest neighbors (kNN) and SVM, are affected most severely [157].

In practice, most research groups default to one representation and ignore the rest. ECFP is common for small molecules, while deep learning groups may opt for embeddings like MoLFormer or ChemBERTa. ESM-2 embeddings are common for antibodies and can be computed per chain. Some research compares these representations, but little focuses on combining them beyond simple concatenation. Concatenating just three standard sources, ECFP4 (2,048 features), ESM-2 (1,280) and Mordred descriptors (∼1,600), gives about 5,000 features, whereas a dataset may contain only a few hundred labeled samples. At these sample-to-feature ratios, more features mean worse predictions [155, 156]. Models that compute a distance over every feature, such as k-nearest neighbors (kNN) and SVM, are the most exposed, for the reasons given in Chapter 4. A concatenation of two representations scored between its two parts, rather than above the better one, on **[RESULT NEEDED: count of datasets]**. We propose that this happens because concatenation adds the irrelevant features of both parts along with the useful ones. A method that joins representations therefore has to combine and filter at the same time.

The pace of new model development shows no sign of slowing, and extends well beyond small molecules. New models may prefer different feature sets, which this method can provide. Beyond that, new models can be converted into frozen embeddings and used as further input to this framework.

The pace of new model development shows no sign of slowing, and extends well

beyond small molecules. A new downstream model may prefer a different feature set, and it therefore receives its own search. A new model can also be converted into frozen embeddings, which enter the method as one more base representation without any change to the method.

A comparison of 62,820 models in molecular property prediction benchmarks showed that classical RDKit2D descriptors matched or outperformed learned features from graph neural networks and pretrained language models on the majority of datasets [106]. Descriptor-based models occupied 73% of top-three positions on 11 benchmarks [158], and in 1,310 ChEMBL assays, feedforward networks on ECFP fingerprints outperformed graph convolutions and SMILES-based recurrent models [159]. The input thus often matters more than the model, yet no single representation wins and there is still no systematic way to exploit this. The same pattern holds outside small-molecule chemistry, where no single protein language model consistently achieves the highest performance on all tasks [160, 161]. For antibodies, general-purpose models sometimes outperform those built for the domain [160]. In computational pathology, no individual foundation model dominates on clinical endpoints [162], and in cancer genomics, integrating additional omics layers can reduce performance when the data types are not properly combined [150].

For small molecules alone, the landscape now includes extended-connectivity fingerprints, physicochemical descriptor sets exceeding 1,600 dimensions, graph neural network embeddings [12, 163], language models trained on billions of SMILES strings [40], image-based encodings (molecules rendered as 2D images) [164], and stereoelectronics-infused molecular graphs [165]. The same is true for proteins (ESM-2 [166], ProtTrans [167]), antibodies (AntiBERTy [168], AbLang [169]), histopathology [162], and multi-omics cancer profiles [150]. A recent review counted over 40 molecular foundation models published between 2020 and 2024 [170]. Increasingly, new models appear faster than we can compare them [171], so the choice is not getting simpler. Different sources presumably encode distinct aspects of the same biology. Single data modalities, noisy or incomplete on their own, give more sensitive and robust predictions when combined with independent signals from other modalities [150]. El Nahhas et al. found that an ensemble of frozen embeddings from the top four pathology foundation models achieved a mean area under the receiver operating characteristic curve (AUROC) 1.2% higher than the best individual model on 31 clinical tasks [162].

Representations now exist for small molecules, reactions, antibodies, nucleotide

sequences and tumor profiles, and those used in this thesis are listed in Tables 5.11, 5.12, 5.13, 5.14 and 5.15. A recent review counted over 40 molecular foundation models published between 2020 and 2024 [170]. Increasingly, new models appear faster than we can compare them [171], so the choice is not getting simpler. Different sources encode distinct aspects of the same biology. Single data modalities, noisy or incomplete on their own, give more sensitive and robust predictions when combined with independent signals from other modalities [150].

Concatenation, the most common method for combining feature sets, gives the full pool from which the hybrid starts, but on biological sample sizes it can degrade performance instead of helping [149, 156]. At the kernel level, multiple kernel learning [153] (e.g., pairwiseMKL for drug bioactivity [154]) can learn source weights, but no interpretable feature-level selections result. Consensus methods [172] combine predictions rather than features, and so cannot exploit cases where one source's noise is another's signal. The hybrid instead works at the feature level and selects features that span source boundaries. Perhaps the closest analogy in the optimization literature is the combined algorithm selection and hyperparameter optimization (CASH) problem [173] (the joint selection of algorithm and hyperparameters), extended here to include the input encoding as an additional search dimension. Tishby and Zaslavsky [174] showed why compression helps, since the information bottleneck principle states that optimal representations maximize mutual information with the target while minimizing mutual information with the input. In effect, the budget forces the model to keep only what matters, and the feature count controls the trade-off between informativeness and dimensionality.

The hybrid search starts from all base representations side by side but works at the feature level. It keeps original features drawn from several representations, and so selects features that span representation boundaries. Perhaps the closest analogy in the optimization literature is the combined algorithm selection and hyperparameter optimization (CASH) problem [173], the joint selection of algorithm and hyperparameters. The hybrid search adds a third choice, namely which features from which representations enter the model.

Ensembling combines representations after prediction rather than before it, by fitting one model per representation and pooling their predictions. Consensus QSAR follows this approach and combines the predictions of several models instead of their features [172], so it cannot exploit cases where one representation's noise is another's signal. In computational pathology, an ensemble built on the four best

foundation models reached a mean AUROC 1.2% above the best single model over 31 clinical tasks [162]. In an ensemble, however, no single model sees features from two representations together, and every representation and every model must be computed for each new molecule.

Feature selection keeps a subset of the original features and builds no new ones, which separates it from principal component analysis (PCA). Methods are usually grouped into three families [175], which Saeys et al. [176] reviewed for high-dimensional biological data. Filter methods score features against the labels before any model is fitted, as when minimum-redundancy maximum-relevance selection (mRMR) ranks features by their mutual information with the label minus their redundancy with features already chosen [177]. Wrapper methods search over subsets of features and score each subset by fitting the model [178], and in QSAR, descriptors were selected this way with a k-nearest-neighbor model [133]. Embedded methods select features while the model is fitted, as when the lasso's L1 penalty sets some coefficients to exactly zero [179]. Recursive feature elimination fits a model, drops the lowest-ranked features and repeats [180]. Two pitfalls of feature selection are well known. The selected set can change between resamples of the same data, which the Nogueira index measures across folds [181], and impurity importance in random forests favors some feature types over others [182].

The hybrid ranks features by the importance scores of a fitted model, as embedded methods do, and then keeps a fixed number of top-ranked features, as a filter does. Classical methods treat all features as one flat list and ignore the representation each feature came from. Their nearest relative to the hybrid is selection over predefined groups, as in the group lasso, which keeps or drops whole groups of variables together [183]. The hybrid, in contrast, does not keep or drop whole representations. Four of the five allocation methods in its search give each representation a share of the budget and take that many top-ranked features from it. The fifth, the global pool, gives each representation a share of a candidate pool twice the size of the budget, then ranks the pooled candidates together and keeps the top ones up to the budget. Because only original features are kept, every feature in the hybrid traces back to one representation, which is what makes blueprints possible.

Dimensionality reduction replaces the original features with fewer new features, each a combination of the originals, and in QSAR it turns many correlated descriptors into fewer uncorrelated ones [184]. PCA is the standard method for this purpose. Each component is a linear combination of the original features, and com-

ponents are ordered by the variance they explain [185]. Because components are usually functions of all the original features, they are hard to interpret [185]. PCA also depends on the units of the features, which z-scoring each feature first removes [185], and this matters when fingerprint bits, descriptors and embeddings are joined. PCA never sees the labels and keeps the directions of largest variance, which need not predict the property, while components with small variance can still predict it [186]. Partial least squares (PLS) is the supervised counterpart in chemometrics, with components built to covary with the property [187]. Both methods give mixed components, so a component cannot be traced to one substructure, descriptor or representation, and no blueprint can be read from it. Nonlinear methods include kernel PCA [184], t-distributed stochastic neighbor embedding (t-SNE) [188] and uniform manifold approximation and projection (UMAP) [189], although t-SNE and UMAP are used mainly for two- or three-dimensional maps rather than as model inputs.

Every alternative above except ensembling also reduces the number of features. Models may, however, want different numbers of features. Distance-based models such as kNN and SVM use every feature in every distance, whereas tree ensembles choose one feature at each split and can therefore partly ignore irrelevant features. Models may also want different features, and not only different numbers of them. Selection that uses the labels may overfit. Sort & Slice, which uses no labels, outperformed two label-based schemes for selecting substructure features [6], and Dablander suggests that using the labels may add a risk of overfitting [125, p. 138]. Table 5.1 compares these alternatives with the hybrid.

5.1.2 The Adaptive Hybrid Representation

Here, we present KIRBy (Knowledge Integration by Representation), a framework that takes a collection of feature matrices for a given data set and outputs a customized feature set optimized for a specific prediction task. The inputs can be any type of frozen feature vector, such as fingerprints, descriptor sets, foundation model embeddings, gene expression signatures, histopathology features. Within a given domain, these individual vectors typically contain its own type of information. It's a well-established phenomenon when it comes to machine learning, there's rarely ever a one-size-fits-all approach. Take the noise robustness study above as evidence, NGBoost's predictive performance on QM9 was only ever strong with given PDV input features. However, on Caco-2, NGBoost preferred Avalon, with PDV as a

Method	Uses the labels	Keeps original features	Several representations	Retrain to add one
Concatenation [149]	no	yes	yes	yes
Late fusion and ensembling [150, 172]	only if pooling weights are learned	yes	no	no, one more model
Intermediate fusion [151]	yes	no	yes	yes
Multiple kernel learning [153, 154]	yes	no, kernels only	yes	yes
PCA and PLS [185, 187]	PCA no, PLS yes	no	yes, on a concatenation	yes
Flat feature selection [175]	yes	yes	yes, on a concatenation	yes
Group lasso [183]	yes	yes, whole groups	yes	yes
Adaptive hybrid representation	yes	yes	yes	search rerun, method unchanged

Table 5.1: Methods for combining or reducing representations, compared by whether they use the labels, keep the original features, let one model see several representations, and must refit to add a representation.

close second. The model NGBoost itself produces better fits when given physicochemical properties, yet this preference can be overidden by a different prediction task. Avalon likely captures features that are pertinent to the log efflux ratio.

Here, we present the adaptive hybrid representation, which takes a collection of feature matrices for a given dataset and outputs a feature set customized for a specific prediction task. The inputs can be any type of frozen feature vector, such as fingerprints, descriptor sets, foundation model embeddings, gene expression signatures or histopathology features. Within a given domain, each of these vectors typically carries its own type of information. It is well established in machine learning that one approach rarely suits every problem, and the noise robustness study of Chapter 3 provides evidence of this for NGBoost on clean labels. On QM9, NGBoost's held-out R^2 was highest with PDV input features (0.865, median of 10 replicates), with MHG-GNN only 0.013 lower (0.852) and ECFP4 lowest (0.706). On LogD, PDV was again highest (0.673), followed by MHG-GNN (0.658). On hERG K_i, MHG-GNN was highest (0.511) and PDV second (0.456), the latter being the median of the 4 of 5 folds whose clean R^2 was at least 0.3. On Caco-2, NGBoost appeared to prefer Avalon (0.402), with ChemBERTa at 0.404. Both numbers are

medians of only 2 of 5 folds, because the other 3 fell below a clean R^2 of 0.3 and were excluded. Over all 5 folds, PDV is highest (0.370), ahead of Avalon (0.287) and ChemBERTa (0.256). NGBoost thus tends to produce better fits from physicochemical properties, yet a different prediction task can override this preference. Accuracy and robustness also favor different representations. On QM9 under Gaussian noise, NGBoost's highest noise robustness (AUC_{norm}, the normalized area under the curve of R^2 against noise level) was on Avalon, at 0.982 against 0.975 for PDV.

But as important as the selection of useful features is, it is also important to filter out the useless features. We could throw the kitchen sink at the models and pool together features across a wide array of representations. Not only would that pose computational challenges, but it also introduces a huge set of irrelevant features that function as noise in downstream applications (Chapter 4). With all of that in mind, I decided to develop a method that selects the best features for a given data set and model. In theory, it should effectively filter out noisy features and isolate the feature which carry the most relevant information.

Selecting useful features matters, but filtering out useless ones matters just as much. Pooling features from a wide array of representations would pose computational challenges and would also introduce a large set of irrelevant features that act as noise in downstream applications (Chapter 4). In an earlier exploratory study, the noise handling methods that used a distance between molecules performed worst (Section 4.4.1). That study served as motivation only, since it covered one dataset, the QM9 HOMO–LUMO gap, and one noise type, Gaussian noise. Chapter 6 addresses the label-noise question over more datasets and noise types. With this in mind, I decided to develop a method that selects the best features for a given dataset and model. In principle, such a method should filter out noisy features and isolate the features that carry the most relevant information.

The questions this chapter and Chapter 6 address are set out in Chapter 1.

Formally, an adaptive hybrid representation is a feature set built by selecting up to a fixed number of features, the budget, from several base representations of the same molecules. A search chooses it for one dataset and one downstream model, so a different dataset, split or model gives a different hybrid. Because the hybrid is an input rather than a model, any standard supervised model can be trained on it. It is intended for base representations that describe different aspects of the molecule, such as PDV descriptors, ECFP4 bits, graph embeddings and sequence embeddings, from all of which the features for the task are then picked.

The budget is split between the representations before any feature is picked. Representations differ in width by orders of magnitude, from tens to thousands of features, so in a single ranking over all their features a wide representation can crowd out a narrow one. Importance scores from different types of feature are also not on a common scale. For these reasons, each representation's features are ranked only against each other. The search also includes one pooled allocation method, the global pool, which ranks the features offered by every representation together, so that per-representation and pooled selection are compared instead of assumed. This design leaves six choices open. Three concern the features, namely how useful each feature is, how many features each representation receives and how many representations take part. The other three are how a candidate hybrid is judged during the search, the total budget, and whether the answer depends on the downstream model.

A hybrid can also be used to measure similarity in two ways. Distances can be computed in an ordinary hybrid, or the search can pick features by how well each molecule's activity is predicted from its five nearest neighbors. Both approaches are defined in the section "Similarity in a hybrid" of Chapter 6.

5.1.3 Blueprints and Their Chemical Interpretation

A finished search is saved as a blueprint, which records the settings needed to rebuild the hybrid. Every selected feature traces back to its representation and to its importance rank within that representation, so the search returns information about the problem as well as a better score. The search keeps the best configuration for each pair of downstream model and budget, and one dataset therefore gives one blueprint per downstream model and budget. Comparing these blueprints at one budget shows whether tree ensembles and neural networks draw on different features.

Some selected features can be read by a chemist. A descriptor from the PDV set carries its name, such as molecular weight or logP. A selected ECFP4 bit can be shown as the atom environment that sets it, and a MACCS key stands for a known substructure. Outside small molecules, the readable unit changes. For an antibody it is a sequence position and the residue at it, and for gene expression it is a named gene set, such as a pathway or a chromosome arm. A feature of a learned embedding, by contrast, indicates only which representation was useful and says nothing about the chemistry, for the reasons given in Chapter 4.

Readable features can be used in two directions. The first is discovery, in which substructures or descriptors that recur among the kept features point to parts of the molecule that may be involved in binding or activity. Such features are a starting point for a chemist rather than a finding. The second is confirmation, in which kept features that make chemical sense for the endpoint support the claim that the selection finds real signal instead of fitting noise. Explainability is a recognized need in drug discovery [190], and inputs that can be read directly are an alternative to explaining a black-box model after the fact [191]. Two caveats apply to this reading. First, a selected feature is predictive but not necessarily causal, and because correlated features can stand in for each other, an unselected feature may matter just as much. Second, a blueprint rebuilt on other molecules can select other features, so features should be read from the fit on the dataset in question.

A hybrid also defines a distance chosen for one task, which any method that uses nearby molecules as evidence could use in place of a standard fingerprint distance. Chapter 4 argues for several such uses, including noise detection, nearest-neighbor prediction, applicability domain, activity-cliff detection, clustering molecules for group-aware splits and weighting predictions from separate sites. One further use is not covered there. Generative design scores each proposed molecule with a predictor [192], and that predictor could take a hybrid as its input, which we leave to future work.

5.1.4 Beyond Small Molecules

The search needs only a supervised task and several vector representations of the same samples, each representation being one feature matrix with one row per sample. The default search objective scores prediction only and reads no chemical structure, so the method applies to tabular data in general, that is, to any task where several encoders or measurements exist for the same inputs. What does not carry over is specific to chemistry. Reading substructures from a blueprint requires named substructure features. An optional search objective, which scores how closely distances in the hybrid track the maximum common substructure shared by pairs of molecules, applies only to molecules. The nearest-neighbor objective of Chapter 6 needs only labels and a distance, so it does carry over and is searched on the antibody and other non-molecular datasets as well.

One-hot sequence encodings are very sparse, because at most positions the

residue never changes across a dataset, and those features therefore carry no importance. One-hot encodings can nevertheless still be useful. Among the four antibody datasets of this chapter, one-hot is a base representation on nanobody thermostability only, while the position-wise blocks substitution matrix (BLOSUM) encoding is a base representation on humanness and trastuzumab–HER2 binding. One-hot was among the selected representations on **[RESULT NEEDED: the antibody datasets where one-hot was among the selected representations, with the score in the dataset's metric]**. We recommend language-model sequence embeddings over raw one-hot encodings.

After small molecules, antibodies received the most attention in this work. This chapter uses four antibody datasets, Koenig expression, nanobody thermostability, humanness and trastuzumab–HER2 binding (Table 5.9). Their base representations are mostly embeddings from general protein language models and from antibody-specific language models, some of which cover only the complementarity-determining region (CDR) loops or come from early or middle layers of ESM-2 (Table 5.13). Chemical reactions serve as a proof of concept on two Buchwald–Hartwig yield datasets. Their base representations are computed on the reactants, on the products minus the reactants, or on the whole reaction, and product fingerprints enter only as augmentation (Table 5.12). Nucleotide sequences are a second proof of concept, on the H3K4me3 histone-mark and TATA promoter tasks (Table 5.14). In cancer genomics, on data from The Cancer Genome Atlas (TCGA), each representation is a different measurement of the patient, namely gene expression, methylation (lung task only), copy number, microRNA (miRNA), protein and slide images (Table 5.15). In the other domains, each representation is a different encoder of one input, and the search treats both cases in the same way.

Two conditions must hold on a dataset for the hybrid to help. The first is a sufficient number of samples. **[RESULT NEEDED: the number of samples below which the hybrid stops beating the best single representation; about 200 samples is expected]**. The second is base representations whose features carry importance, since where every representation already scores poorly, there are too few useful features to improve on.

5.2 Methods: The Hybrid Search

Figure 5.1 gives an overview of how a hybrid is built from several base representations of the same samples.

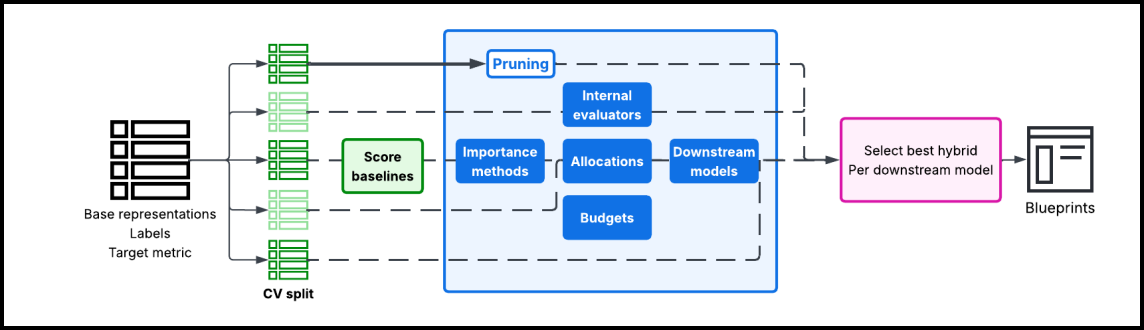


Figure 5.1: Schematic overview of the hybrid search. The inputs (left) are split into five cross-validation folds, and the single representations are scored as baselines. Each fold is searched over importance methods, allocation methods, budgets, internal evaluators and downstream models (center), with optional adaptive pruning on the first two folds. The best hybrid for each downstream model is stored as a blueprint (right).

The search is a grid in which every combination of setting values is built and scored inside the cross-validation folds. When adaptive pruning is switched on, it drops some combinations after the first two folds. A winner is kept for each pair of downstream model and budget, where the budget is a maximum number of features and a hybrid can have fewer. The comparison we report runs the whole search inside each outer training fold and scores the winner once on the untouched outer test fold.

The subsections below cover the steps from base representations to a hybrid, each searched setting and its rationale, augmentation with small or specialized feature sets, and two ways of making the search cheaper.

5.2.1 From Base Representations to a Hybrid

Given K frozen representation matrices $\mathbf{R}_1, \ldots, \mathbf{R}_K$ for the same n samples, a label vector $\mathbf{y}$, and optional group assignments for cross-validation, the search finds which features from which sources best predict for a specified metric and downstream model. The output is a hybrid feature matrix of at most b features, where b is the feature budget, plus any augmentation features added outside it. The search also returns a blueprint that records all parameters needed to reproduce the identical feature selection on new data. The search first scores each source independently, then scores feature importance on training data within each cross-validation fold, allocates the budget among sources according to baseline performance, and scores candidate

hybrid configurations via cross-validation. We retain the best configuration for each combination of downstream model and budget. Every selected feature traces back to its source and importance rank.

The search is given K frozen representation matrices $\mathbf{X}^{(k)} \in \mathbb{R}^{n \times d_k}$, $k = 1, \ldots, K$, for the same n samples, whose widths d_k range from tens to thousands of features. It is also given a label vector $\mathbf{y}$ and one group label per sample for cross-validation, for example its scaffold. It further needs the dataset's metric, the task type, one or more downstream models and a list of budgets. From these inputs, it finds which features from which representations best predict for that metric and downstream model. The output is a hybrid feature matrix of at most B features, where the feature budget B is a maximum rather than a target. Uncapped augmentation adds its extra features outside the budget, whereas capped augmentation stays within it. The output also includes a blueprint that records the settings needed to rebuild the hybrid. The search first scores each representation independently and then scores feature importance on training data within each cross-validation fold. It then allocates the budget among representations. Four of the five allocation methods do this according to each representation's standalone performance, while the global pool instead ranks the offered features from all representations together. Candidate hybrid configurations are scored by cross-validation, and we retain the best configuration for each combination of downstream model and budget. Every selected feature traces back to its representation and importance rank.

Inputs from different sources differ in scale, distribution, and missingness patterns, and naively pooling them before splitting would leak validation-set statistics into the training procedure, a well-documented source of optimistic bias in feature selection studies [193, 194]. All preprocessing therefore runs within each cross-validation fold. We impute missing values with per-feature medians computed on the training fold and drop features that are entirely missing. Every feature of the hybrid, binary fingerprint bits included, is standardized; only when a single representation is scored on its own are binary features left as 0/1. We standardize to zero mean and unit variance using training-fold statistics only.

Inputs from different representations differ in scale, distribution and missingness patterns. Pooling them naively before splitting would leak validation-set statistics into the training procedure, a well-documented source of optimistic bias in feature selection studies [193, 194]. All preprocessing therefore runs within each cross-validation fold. When a single representation is scored, we impute missing values

with per-feature medians computed on the training fold and drop features that are entirely missing. When features are selected for the hybrid, a missing value is instead set to 0 before standardization. Every feature of the hybrid, binary fingerprint bits included, is standardized to zero mean and unit variance using training-fold statistics only. Binary features are left as 0/1 in one case only, when the search scores a single representation to pick the comparator, whereas a single representation scored on the outer test fold is standardized like the hybrid. A representation wider than 5,000 features is first reduced, inside each fold, to the 500 features on which a ranking model splits most often. It keeps 200 features instead when the search is given fewer than 1,000 molecules. The ranking model is a LightGBM model of 50 trees of depth at most 5, fitted on the fold's training molecules. It ranks features by how many times its trees split on them rather than by gain. This reduction applies only when the search scores a single representation on its own. For such a representation, the comparator is therefore picked on its top 500 (or 200) features and then scored on the outer test fold at full width, while the hybrid search always sees the full width.

Each dataset is split into outer folds, 5 by default. The folds are group-aware, so that no group appears on both sides of a split, and classification folds are also stratified by class. A representation learned from the labels is retrained inside each outer training fold, on its training molecules only. The search then runs its own group-aware cross-validation, with 5 folds by default, on the molecules it is given. Inside each training fold of the search, one group-aware internal split holds out about 20% of the molecules as whole groups. This split is the first fold of a 5-fold grouped split and is therefore deterministic. When a training fold holds fewer than five groups, both splits use as many folds as there are groups. For example, Koenig expression has five groups, so each outer training fold holds four. The search inside that fold uses 4 folds of three groups each, and the internal split holds out one of the three. Importance and each representation's standalone score come from this training fold only, and the outer test molecules score only the finished hybrid.

For each representation k, the importance method gives every feature one score $s_j^{(k)} \geq 0$, and these scores rank features within that representation only. Importance is computed on the internal training part of the fold. The standalone score r_k comes from the internal evaluator, fitted on representation k alone and scored on the internal held-out part in the dataset's metric. We rescale r_k to $\tilde{r}_k$ so that 0 means "no better than a trivial predictor", which makes the budget split independent of the dataset's metric. For Spearman, Pearson, the Matthews correlation coefficient

(MCC) and R^2, $\tilde{r}_k = \max(0, r_k)$. For AUROC and the concordance index, chance is 0.5, so $\tilde{r}_k = \max(0, 2(r_k - 0.5))$. For RMSE, $\tilde{r}_k = \max(0, 1 - \mathrm{RMSE}_k/\mathrm{RMSE}_0)$, where RMSE_0 is the error of predicting the training mean, and the mean absolute error is rescaled in the same way with the training median as the trivial predictor. For accuracy and weighted F1, $\tilde{r}_k = \max(0, (r_k - r_0)/(1 - r_0))$, where r_0 is the score of always predicting the majority training class. The trivial predictor is scored on the same held-out molecules. A representation whose fit fails receives weight 0 and is recorded as a failure.

An optional representation count N keeps only the N representations with the highest standalone scores on this fold. Each remaining representation receives a weight $w_k = g(\tilde{r}_k)$, where the function g is set by the allocation method. The exact share of representation k is $b_k^* = B\, w_k / \sum_l w_l$ features, and we round the shares by largest remainder. Each representation first receives $\min(\lfloor b_k^* \rfloor, d_k)$ features, and the leftover features are then given out one at a time in order of largest fractional part, skipping representations already at full width. The result satisfies $\sum_k b_k = \min(B, \sum_k d_k)$ and $b_k \leq d_k$, and a representation with $w_k > 0$ keeps at least one feature. If every weight is 0, the budget is split equally and the run reports this, and if no representation can be scored at all, the configuration is refused. The global pool does not use this rounding. Instead, each representation offers candidate features, and one joint ranking keeps at most B of them.

Let S_k be the top b_k features of representation k by $s_j^{(k)}$, among those with $s_j^{(k)} > 0$. The same cut at 0 applies to the features offered to the global pool and to its final ranking. If no feature of a representation scores above 0, its full b_k is taken. The cut exists because tree importance gives exactly 0 to every feature on which no tree split, and filling the budget from those features would pick them by their position in the matrix. The hybrid is $\mathbf{H} = [\mathbf{X}^{(1)}_{:,S_1}, \ldots, \mathbf{X}^{(K)}_{:,S_K}]$, with $\sum_k |S_k| \leq B$, and it can therefore be narrower than its budget. The matrix $\mathbf{H}$ is then preprocessed inside the fold, as described above.

A configuration assigns one value to every searched setting. For each configuration, we fit the downstream model on the hybrid's training molecules in each fold of the search, score it on that fold's held-out molecules and average over the folds. The winner is picked separately for each pair of downstream model and budget and never across models. Because that score was used to choose among many configurations, it is optimistic [195, 196], and the comparison we report is therefore nested. The whole search runs inside each outer training fold, the winning settings are rebuilt

on that fold, and the result is scored once on the untouched outer test fold. Both the winner at each budget and each downstream model's best across budgets are scored in this way. The inner search score exceeds the outer test score by **[RESULT NEEDED: the gap between inner search score and outer test score, per dataset, in each dataset's own metric]**.

Chapter 6 uses two variants of this search. Under label noise, every quantity computed on the training molecules uses the noisy labels, and held-out scoring uses clean labels unless stated otherwise. Under the nearest-neighbor objective, each molecule's activity is predicted from its five nearest neighbors in the feature space. This score replaces both the standalone score and the downstream-model score, so no downstream model or internal evaluator is used.

Blueprints. To be able to better examine the selected features and to regenerate them on new datasets, we produce blueprints. After a hybrid feature selection search is completed, its results are stored as a blueprint containing the "recipe" required to recreate this exact hybrid representation on any data. These blueprints store the selected picks for the pre-specified budget, the full list of input representations searched and how many of them the recipe used, feature importance scoring method, allocation strategy, and internal evaluator. It contains other runtime information like the step size, number of folds, the seed, the metric being used to score, the task type (regression or classification), whether augmented features were being used, and the per-representation scores.

To examine the selected features more closely and to rebuild the hybrid on new datasets, we produce blueprints. A blueprint is the recipe for the winning hybrid, that is, the search settings needed to rebuild it. Every scored configuration has one, and the search returns the winner's blueprint with its result. On other molecules, a rebuild re-scores the representations and re-picks the features, so it need not recreate the same hybrid. A blueprint stores the budget, the full list of input representations searched and how many of them the recipe used. It also stores the feature importance method, allocation method, internal evaluator and objective. Further runtime information includes the number of folds, the seed, the metric used for scoring and its direction, and the task type (regression or classification). The augmentation settings are stored as well, covering how many extra features to take, from which candidates, and the rule for admitting one. Some information is recorded for later analysis only. This comprises each representation's best score across the downstream

models, the number of features each representation received in each fold, and the augmentation features kept in each fold. These records show which representations fed the hybrid and whether this holds across folds. The stored score differs from the standalone score that splits the budget, because it is measured on every molecule the search saw, and a rebuild therefore never reads it.

Listing 5.1 gives every field of a blueprint together with its type.

```
Blueprint
  # What the search chose. A rebuild reads all of these.
  reps                  : list[str]           # every representation searched, not only those used
  rep_count             : int or None         # how many of them may contribute; None = all
  budget                : int                 # most features the hybrid may have
  allocation_method     : str
  importance_method     : str
  internal_evaluator    : str
  objective             : str                 # 'predictive' or 'similarity'

  # How the search scored configurations.
  metric                : str
  lower_is_better       : bool
  task_type             : str                 # 'regression' or 'classification'
  n_folds               : int
  seed                  : int

  # Augmentation settings. A rebuild reads all of these.
  augmentation_strategy : str
  aug_per_source_budget : int, or {source: int}, or None
  augmentation_margin   : float
  augmentation_fraction : float or None
  candidate_ranking     : str
  augmentation_threshold: float or None
  augmentation_budget   : int
  aug_max_candidates    : int

  # Stored for inspection.
  rep_scores            : {representation: float}
                          # each representation's best score over every molecule the search saw;
                          # the rebuild uses it to choose which representations to keep only when
                          # rep_count is set and no per-fold scores are given
  allocation_per_fold   : {fold: {representation: int}}      # features each representation got
  kept_augmentations    : {fold: {source: list[int]}}         # extra features kept, by index

  # Left from a removed method.
  step_size             : int                 # used only by greedy forward selection
```

Listing 5.1: Fields of a blueprint and their types. Comment lines group the fields, and each field is given with its type and, where needed, a comment on its meaning.

The blueprint does not store selected feature indices, those can only be recovered when the blueprint is refit on the data, which re-runs the feature selection process deterministically from that recipe and seed.

The blueprint does not store the selected base-representation features, although it does store how many features each representation received in each fold and which augmentation features were kept. A blueprint can be reused in two ways. The first re-runs the recipe on new labeled data and recomputes importance and standalone scores, so that the features adapt to the new dataset. This costs a new selection and

can give a different feature set. The second takes the exact features of a labeled fit from the new molecules. This requires no labels and is therefore the only option for unlabeled molecules, but it assumes that the same features matter on the new data. Re-running the recipe without labels is refused, because every importance score would be 0 and the selection would fall back to feature order. Which of the two approaches transfers better to a smaller related dataset is **[RESULT NEEDED: re-run recipe vs. reused exact features, large to small related dataset]**. Importance and allocation can also be computed separately within k-means clusters of the molecules' ECFP fingerprints, to ask whether the budget split changes across chemical space.

It also provides a registry of selected features, stating where that feature came from. On some representations, we added an extension where features are mapped back to their function. For instance, ECFP4 and MACCS bits are mapped to SMILES arbitrary target specification (SMARTS) substructures. This blueprint will tell you which of those substructures were selected. This is not possible for learned embeddings for which the blueprint will return the bare representation name and dimension index. This mapping provides a layer of explainability. It can be used both to discover what features are useful for your task and to confirm that the features being selected are in fact lining up with the underlying known chemistry.

A separate explanation step, run on a finished hybrid, provides a registry of the selected features that states where each feature came from, since the blueprint itself does not store feature names. For some representations, we added an extension that maps features back to their function. An ECFP4 bit is named by the atom environment that sets it, written as a SMILES fragment rather than a SMILES arbitrary target specification (SMARTS) pattern. The fragment is taken from the first molecule, of at most 50 supplied, that has the bit set. MACCS keys are named by their SMARTS pattern and a short name, and the features of the descriptor vector carry the descriptor names. The explanation step thus reports which of these substructures and descriptors were selected. Outside small molecules, protein composition, descriptor and k-mer features are named too, as are antibody composition, one-hot and BLOSUM positions, structural and inverse-folding scores, gene-expression pathway sets and PCA components. MAP4 and DRFP features are returned as a bit index only. Avalon, Mordred, atom-pair and graph-kernel features, and every learned embedding, return only the bare representation name and dimension index. This mapping adds a layer of explainability. It can be used to discover

which features are useful for a task and to confirm that the selected features agree with known chemistry.

5.2.2 The Search Space

When I began to implement this idea, I expected a fairly straightforward way to select the most informative features, but several questions soon arose. How should the importance of a feature be measured? Given a handful of input representations, how many features should be taken from each? How should a candidate feature set be scored? As with the choice of representation, different prediction tasks preferred different answers to each of these questions. The search therefore treats each question as a setting and chooses its value per dataset. These settings are the importance method, the allocation method together with the budget and the representation count, and the internal evaluator.

Every combination of setting values in the grid is built and scored, and nothing is fixed in advance except the values each setting may take. The claim of this chapter is that no single setting suits every dataset, and a grid measures this claim instead of assuming it. The motivation is the familiar result that no learning method is best on every problem [101], although that result does not prove the claim for hybrids. We search the full product of values, instead of one setting at a time, because the settings interact. For example, the best budget depends on the downstream model, as the nanobody thermostability example below shows.

Six settings are searched, namely the downstream model, the budget, the representation count, the importance method, the allocation method and the internal evaluator. Their values at the full tier and at the fast tier are listed in Table 5.7. The defaults and the number of configurations for each downstream model are given in Additional files, Section S.3. The importance method is LightGBM gain, TreeSHAP or DeepLIFT. It is searched because importance depends on the model as well as on the data [197]. The three methods come from different model families and rank features differently, and a different ranking builds a different hybrid. The allocation method decides how the budget is split among the participating representations, and five allocation methods are searched. The internal evaluator scores each representation on its own. It is the matched evaluator (a model of the downstream model's own family), random forest, LightGBM or a deep neural network (DNN). Two further choices are not searched within a single search. The objective

is prediction by default, which Chapter 6 replaces with predicting each molecule's label from its nearest neighbors in the hybrid. Augmentation is absent, uncapped or capped. Each objective and each augmentation condition is therefore a separate search.

The downstream model is the model trained on the finished hybrid. Five downstream models are searched, namely random forest, XGBoost, LightGBM, SVM and DNN, and a Gaussian process is added on datasets under 1,000 molecules. Three of the 16 datasets are that small, but run time rules the Gaussian process out on two of them, so it runs on humanness only. The downstream model is searched because it determines what counts as a useful feature and how many features it can use. The best hybrid for one model therefore need not be the best for another.

The budget is the maximum number of features the hybrid may have, and eight budgets are searched (10, 25, 50, 100, 250, 500, 1,000 and 2,000 features). The budget is searched because too few features leave informative ones out, whereas too many add uninformative features that, on small datasets, add room to overfit. Budgets above the number of training molecules are kept. The best budget depends on the downstream model. In a preliminary run on nanobody thermostability with 5 folds, SVM performed best at 300 features, with a Spearman correlation of 0.6384. DNN, LightGBM and random forest performed best at 50 features, and XGBoost at 100. That run used an earlier list of budgets, which included 300 features. **[RESULT NEEDED: the best budget for each downstream model on nanobody thermostability, from the full runs]**

Each dataset has a fixed set of base representations, chosen before the search (Table 5.16). Fourteen of the 16 datasets have 10, H3K4me3 has 8 and TATA promoter has 9. The representation count, which takes the values 2, 3, 5 or all, sets how many of them may contribute features to the hybrid. On each fold, the representations with the highest standalone scores are retained. This is the only step that drops whole representations before allocation, and correlation between representations plays no part in it. The count is searched because datasets differ in how many representations are worth using, and pooled feature selection over all representations has no such setting. Every allocation method is paired with every count. In preliminary runs, some datasets performed best with 2 or 3 representations and others with all 10. **[RESULT NEEDED: which datasets do best with 2 or 3 representations and which with all of them, from the full runs]**

Feature Importance Scoring

Not all features from a given base representation are equally informative, and there are a variety of techniques that can assess each feature's relative importance. Tree-based feature importance measures favor features with high split gain, while neural attributions such as DeepLIFT measure how much each feature moves a neural network's output away from its output on a reference input [182, 198]. We tested whether, for the purposes of this task, there was a single feature importance method that worked across datasets and model architectures. Not only was there no clear winner, but there was evidence that certain networks preferred tree-based measures of importance while certain tree ensembles preferred gradient-based approaches. We landed on a set of three whose predictions did not correlate strongly with each other and whose selected features performed well as model input.

Not all features from a given base representation are equally informative, and a variety of techniques can assess each feature's relative importance. The search ranks the features of each representation before choosing any, using an importance method that gives every feature of one representation a single non-negative score. Each representation receives its own freshly fitted model on its own features only, and the downstream model is not used. Only the order within a representation is used, and because these scores are never compared across representations or across methods, no normalization is applied. Scores are computed inside each fold on the internal training part only, which consists of 80% of that fold's training molecules, split by group. Before fitting, missing values are set to 0 and each feature is standardized to mean 0 and variance 1 on the same molecules. Scores are computed once per fold and method and are then reused by every allocation method, budget, internal evaluator and downstream model. Split-based tree importance is biased toward features with many possible split points [182]. This bias matters here, because binary fingerprint bits sit beside continuous embeddings. Neural attributions such as DeepLIFT instead measure how much each feature moves a neural network's output away from its output on a reference input [198].

We tested whether a single feature importance method worked across datasets and downstream models for this task. We compared the methods' feature rankings on 7 datasets and the hybrids built from those rankings on one dataset with two downstream models. We also counted which method gave the best configuration on 5 datasets with 5 downstream models. There was no clear winner, and the preferred

method changed with the dataset. With a DNN downstream model, LightGBM gain was preferred on four of five datasets, whereas on LogD alone the tree ensembles XGBoost and LightGBM preferred DeepLIFT. We settled on a set of three methods whose rankings did not correlate strongly with each other and whose selected features performed well as model input.

We score features using LightGBM gain, DeepLIFT, and TreeSHAP. LightGBM gain is a tree based method which.... TreeSHAP is explanation-based, meaning it... DeepLIFT relies on neural attribution which...

We score features using LightGBM gain importance, SHapley Additive exPlanations (SHAP) values for tree ensembles (TreeSHAP) and DeepLIFT (Deep Learning Important FeaTures) [199], which come from three families, namely split gain, Shapley values and reference-based attribution. LightGBM gain measures how much a feature reduced the training loss where trees split on it [57]. Its model consists of 50 gradient-boosted trees of maximum depth 6, and a feature's score is the loss reduction of every split on it, summed over all trees, or 0 if no tree splits on it. TreeSHAP measures how much a feature moves each molecule's prediction, as an exact Shapley value for tree models [200]. Its model is a random forest of 50 trees with maximum depth 10 [44], and a feature's score is its mean absolute Shapley value over the training molecules. DeepLIFT measures how much a feature moves the output of a small network away from its output on a reference input [199, 201]. The network has two hidden layers of 64 and 32 ReLU units and is trained for 100 full-batch epochs of Adam at a learning rate of 0.001 [202]. A feature's score is its mean absolute attribution over the training molecules under DeepLIFT's Rescale rule. The reference input is all zeros, which after standardization places every feature at its training mean. For a fingerprint bit, this mean is the fraction of training molecules that have the bit set. On binary labels, TreeSHAP and DeepLIFT use the positive class, and on multiclass labels they average absolute values over the classes. For LightGBM gain and TreeSHAP, a representation wider than 2,000 features is pre-screened. Only its 2,000 highest-variance features, with variance measured before standardization, are fitted and the rest score 0, whereas DeepLIFT scores every feature. On Caco-2, the median share of features scoring above 0 was 30% for LightGBM gain, 70% for TreeSHAP and 90% for DeepLIFT. This preliminary median is taken over 10 representations and 3 folds. Table 5.2 summarizes the three importance methods.

When searching for the features to use in the hybrid, thse feature importance scoring methods are swept.

Method (family)	Model fitted per representation	Score of one feature	Classification labels	Representations over 2,000 features
LightGBM gain (split gain)	LightGBM, 50 trees, maximum depth 6	Loss reduction of every split on the feature, summed over all trees; 0 if never split on	One model on the class labels	Only the 2,000 highest-variance features are fitted; the rest score 0
TreeSHAP (Shapley values)	Random forest, 50 trees, maximum depth 10	Mean absolute Shapley value over the training molecules	Binary: positive class. Multiclass: mean absolute value over classes	As for LightGBM gain
DeepLIFT (reference-based attribution)	Small neural network, ReLU hidden layers of 64 and 32 units, 100 full-batch epochs of Adam, learning rate 0.001	Mean absolute attribution against an all-zero reference, Rescale rule	Binary: positive class. Multiclass: mean absolute value over classes	Every feature is scored

Table 5.2: Settings of the three importance methods. Each method is fitted separately on every representation inside each fold, and settings not listed are the library defaults.

All three feature importance methods are swept in the full search, whereas the fast tier uses LightGBM gain only (Table 5.7). Adaptive pruning is off unless a search requests it. When it is on, importance methods that rarely give the best configurations on the first two folds are dropped from the remaining folds, although at least one method is always kept. Pruning is described in the subsection "Fast tier and adaptive pruning".

Two comparisons were run, the first of how the methods rank features and the second of the hybrids those rankings build. The ranking comparison scored 8 importance methods on 7 datasets, namely LogD, hERG K_i, Buchwald–Hartwig yield, nanobody thermostability, antibody FcRn retention, breast invasive carcinoma (BRCA) PAM50 subtype and lung adenocarcinoma (LUAD) vs lung squamous cell carcinoma (LUSC). For each pair of methods, it measured the Spearman correlation between their scores over every feature of every representation. The three retained methods rank features differently. Across the 7 datasets, the correlation ranged from 0.20 to 0.64 between LightGBM gain and TreeSHAP, from 0.21 to 0.67 between LightGBM gain and DeepLIFT, and from 0.56 to 0.99 between TreeSHAP and DeepLIFT. These numbers are preliminary, because each method was fitted

once on the whole dataset rather than inside folds.

The hybrid comparison used Caco-2, with 2,158 molecules, 10 representations, 3 scaffold folds and a budget of 100 features. Hybrids were built from each method's ranking and scored with random forest and LightGBM downstream models. Three control rankings were added, namely feature variance, Laplacian score and a random order. The measure was the RMSE of the best single representation minus the RMSE of the hybrid, in $\log_{10}$ efflux ratio units, so that a positive value means the hybrid has the lower error. The preliminary results were $+0.0144$ for DeepLIFT, $+0.0100$ for LightGBM gain, $+0.0067$ for TreeSHAP and -0.0118 for the random ranking. The gap between DeepLIFT and LightGBM gain, 0.0044 RMSE, is smaller than the fold-to-fold spread. The methods also differ at the top of their rankings. Of each representation's 100 top-ranked features on Caco-2, the retained pairs of methods share a median of 28 to 54 features.

Three rules determined which methods were retained. First, a method must score every feature of every representation for regression, binary and multiclass labels, which all three retained methods do. Second, of two methods whose rankings agree above a Spearman correlation of 0.95, we keep the faster one. Under this rule, DeepLIFT replaced integrated gradients, which agreed with it above 0.95 on 4 of 6 datasets, and gradient SHAP, which agreed above 0.95 on 3 of 6. Third, we keep methods that rank features differently, because a different ranking builds a different hybrid. TreeSHAP came last of the three on Caco-2 and was kept under this third rule. Only two filters act on features before allocation, the cut at a score of 0 and the pre-screen on representations wider than 2,000 features. On a 2,048-bit fingerprint, the pre-screen drops the 48 lowest-variance bits. On hERG K_i all 48 are among the 105 constant ECFP4 bits, while on QM9, which has no constant ECFP4 bits, they are near-constant ones. Near-constant features otherwise remain candidates and are ranked by importance.

No single importance method suits every dataset and downstream model. A preliminary run used 5 datasets (nanobody thermostability, hERG K_i, humanness, LogD and BRCA PAM50 subtype), 5 folds, 10 representations, 5 downstream models and budgets of 25, 50, 100, 150 and 300 features. For each of the 25 pairs of downstream model and budget per dataset, we counted which importance method gave the best configuration, and Table 5.3 gives the counts. LightGBM gain won the most pairs on three datasets and tied with TreeSHAP on humanness. DeepLIFT won the most on one dataset, LogD, with 17 of its 25 pairs, and outside LogD it

gave the best configuration for 2 of 100 pairs. With the DNN downstream model, LightGBM gain won 17 of 20 budgets outside LogD, whereas on LogD the DNN took DeepLIFT at 3 of 5 budgets. On LogD, XGBoost and LightGBM each took DeepLIFT at 5 of 5 budgets, and random forest took LightGBM gain at 5 of 5. **[RESULT NEEDED: the same win counts, per dataset and importance method, from the full runs]**

		Wins (of 25)			Best score		
Dataset	Metric	LightGBM gain	TreeSHAP	DeepLIFT	LightGBM gain	TreeSHAP	DeepLIFT
Nanobody thermostability	Spearman	15	10	0	0.6696	0.6471	0.6440
hERG K_i	RMSE (pK_i)	14	11	0	0.5589	0.5605	0.5728
Humanness	AUROC	12	12	1	0.9723	0.9713	0.9689
LogD	RMSE (log units)	8	0	17	0.4250	0.4345	0.4170
BRCA PAM50 subtype	weighted F1	21	3	1	0.9097	0.9018	0.9038

Table 5.3: Importance method of the best configuration in a preliminary run on five datasets, with 5 folds, 10 representations, 5 downstream models and budgets of 25, 50, 100, 150 and 300 features. Wins count the pairs of downstream model and budget, out of 25 per dataset, at which a method gave the best configuration, and Best score is the best score reached with that method.

Feature Allocation Methods

The allocation strategy divides the total feature budget among sources, and we search five allocation strategies. Four divide the budget in fixed shares from transformed standalone scores (linear, squared, softmax, rank decay). The fifth, a global pool, lets features from all sources compete in one importance ranking. Once we have the allocation, we select the top features from each source by importance score and concatenate them to form the hybrid.

The allocation method divides the total feature budget among the representations. It decides how many of the budget's B features each representation contributes, while the importance method decides which ones. In total, we search five allocation methods. Four give each representation a fixed share of the budget, computed from its rescaled standalone score (linear, squared, softmax and rank-decay allocation). The fifth, the global pool, lets the candidate features offered by every representation compete in one importance ranking, in which only the offered candidates take part. Under the four fixed-share methods, we select the top features from each representation by importance score and join them to form the hybrid.

The four fixed-share methods are variants of performance-weighted allocation,

where the distribution of features from each base representation is proportional to that representation's standalone predictive performance on the training set. This works under the assumption that representations with stronger predictive performance contribute more valuable features. It is worth noting that this is not always the case, as high-quality, large representations may rely on a small subset of valuable features.

The four fixed-share methods set each representation's share from its standalone score. This score is measured on the internal validation part of the training fold rather than on the molecules on which the internal evaluator was fitted. Only linear allocation makes the share proportional to that score, while the other three transform it first. All four assume that representations with stronger predictive performance contribute more valuable features.

1. Fit the internal evaluator on each base representation independently, on the internal training part of the fold.

2. Score each representation on the internal validation part in the dataset's own metric, rescaled so 0 means no better than a trivial predictor.

3. Allocate features proportionally:

$$b_k^* = B \frac{w_k}{\sum_l w_l}, \quad w_k = g(\tilde{r}_k)$$

4. For each representation, rank features by the configuration's importance method and select the top b_k, after rounding b_k^* as in "From base representations to a hybrid".

5. Join the selected features to form the hybrid representation.

Each fixed-share method follows the steps described in "From base representations to a hybrid". It rescales each standalone score to $\tilde{r}_k$, sets b_k from $w_k = g(\tilde{r}_k)$ and takes the top b_k features of representation k. Allocation runs only over the representations retained by the representation count. The standalone scores cost one fit per representation, fold and internal evaluator, and every budget, allocation method and importance method reuses them.

This method cannot adapt allocation based on feature complementarity, and it relies on the assumption that many features in a well-performing representation are valuable.

Fixed shares rely on the assumption that many features in a well-performing representation are valuable. This is not always the case, since a strong representation may owe its score to a few features, so that a large share fills with weak ones. Fixed shares also cannot adapt the allocation to feature complementarity, and two representations carrying the same information both receive a share. Of the five methods, only the global pool ranks features from different representations against each other.

The four fixed-share methods differ only in the weight w_k computed from the rescaled score $\tilde{r}_k$ (Table 5.4), and their settings are fixed for every dataset rather than searched. Linear allocation sets $w_k = \tilde{r}_k$ and squared allocation sets $w_k = \tilde{r}_k^2$, so under both, a representation receives 0 features when its rescaled score is 0. Softmax allocation sets $w_k = \exp\left((\tilde{r}_k - \max_l \tilde{r}_l)/T\right)$, with temperature $T = 1$. Rank-decay allocation sets $w_k = e^{-0.3\rho_k}$, where ρ_k is the representation's rank by score, 0 for the best. Under these two methods, no representation receives 0 features because of its score. Since every rescaled score lies between 0 and 1, softmax allocation cannot give one representation most of the budget, and with 10 representations the best takes at most 23.2% of it. Under rank decay only the order counts, so with 10 representations the best receives 27.3% and the tenth 1.8%, whatever the gaps between their scores. Table 5.5 works through one example with a budget of 100 features and three representations with rescaled scores 0.6, 0.5 and 0.2.

Method	Weight w_k	Fixed setting	How features are chosen	When a representation receives 0 features
Linear	$\tilde{r}_k$	none	Top b_k of its own ranking	$\tilde{r}_k = 0$
Squared	$\tilde{r}_k^2$	power 2	Top b_k of its own ranking	$\tilde{r}_k = 0$
Softmax	$\exp\left((\tilde{r}_k - \max_l \tilde{r}_l)/T\right)$	$T = 1$	Top b_k of its own ranking	never from its score
Rank decay	$e^{-0.3\rho_k}$, ρ_k = rank by score, 0 for the best	decay 0.3	Top b_k of its own ranking	never from its score
Global pool	$\tilde{r}_k^2$, sizing its offer to a pool of $2B$ candidates	power 2; pool $2B$	Top B of one joint ranking over all offered candidates	$\tilde{r}_k = 0$, or none of its candidates in the joint top B

Table 5.4: The five allocation methods. w_k is the weight computed from representation k's rescaled standalone score $\tilde{r}_k$, B is the budget, b_k is the share of representation k, ρ_k is its rank by score and T is the softmax temperature. Under every method, a representation left out by the representation count, or whose standalone fit fails, receives no features.

Method	$\tilde{r}_1 = 0.6$	$\tilde{r}_2 = 0.5$	$\tilde{r}_3 = 0.2$
Linear	46	39	15
Squared	55	39	6
Softmax	39	35	26
Rank decay	44	32	24

Table 5.5: Worked example of the four fixed-share allocation methods for a budget of 100 features and three representations. Each entry is the number of features a representation receives, with shares rounded by largest remainder.

The global pool sizes each representation's offer with squared weights $\tilde{r}_k^2$, as shares of a pool of $2B$ candidate features. Each offer is capped at the representation's width in features, and only features scoring above 0 are offered. The offered features are joined into one matrix, on which the same importance method is fitted once, on the internal training part. The hybrid keeps the top B features of this joint ranking, again only those scoring above 0. Each representation's final count is therefore an outcome of the joint ranking rather than set in advance, and there is no minimum of one feature per representation. A representation receives 0 features when its score is 0 or when none of its candidates reach the joint top B. Under the global pool, a feature's rank depends on which features the other representations offered, whereas under fixed shares it does not. The cost is one extra importance fit per configuration, on the pooled candidates.

Under all five methods, a representation also receives no features when the representation count leaves it out or when its standalone fit fails.

We first tried greedy forward selection, which picks features for how well they work together, something fixed shares cannot do [175, 178]. In each round, it added the next block of features from whichever representation most improved the internal validation score. It stopped when the budget was spent or when the score had not improved for a set number of rounds. With blocks of s features and K representations, it needs up to $(B/s) \times K$ fits per configuration and fold. With blocks of 10 features, this amounts to 500 fits at 500 features and 10 representations, against 10 for the score-based methods. In preliminary runs, it reached a Spearman correlation of 0.3600 on Jain antibody FcRn retention, against 0.4526 for linear allocation, and placed 16th of 17 methods. On breast cancer PAM50 subtype, it reached an accuracy of 0.8551 against 0.8938 for linear allocation, again placing 16th of 17, and it was removed from the search.

The five methods were chosen in a preliminary comparison run before the main

experiments. It compared 8 candidate allocation methods, each at 4 representation counts, on 9 regression datasets. It used 3 downstream models (LightGBM, random forest and CatBoost) and budgets of 500 and 1,000 features, giving 54 combinations of dataset, downstream model and budget. A fixed-share method was the single best method in 38 of the 54 combinations and the global pool in 16. All five were kept because no single method was best in most combinations. This comparison predates the current score rescaling, and it sized the weights with an inner 3-fold group split instead of the 80/20 internal split used now.

Search Configuration and Internal Evaluation

Features that improve predictions under RF may not necessarily help a kNN model, and vice versa. This is because variable importance depends fundamentally on the model, as Dong and Rudin showed [197], and as we found in our feature importance scoring analysis. To address this, we use an internal evaluator which scores each base representation on its own with a model whose inductive bias best fits the downstream model. We use four evaluators, namely RF, DNN, LightGBM and the matched evaluator, a special case that matches the internal evaluator to the final downstream model. We fit each on the internal training part of the training fold and score it on the internal validation part.

Features that improve predictions under a random forest may not necessarily help an SVM, and vice versa, because variable importance depends fundamentally on the model, as Dong and Rudin showed [197]. To address this, we use an internal evaluator, a model fitted inside the search on one representation at a time and separate from the downstream model scored at the end. We search four evaluators, namely random forest, DNN, LightGBM and the matched evaluator, a special case that matches the internal evaluator to the downstream model. No evaluator is chosen in advance to suit the downstream model. Instead, all four are searched and the best configuration wins. Features are standardized on the internal training part of the training fold, where we fit each evaluator before scoring it on the internal validation part in the dataset's own metric.

This internal evaluator is used to score each representation, and those scores set how the budget is split and which representations are kept within the larger hybrid search. Thus, it is another setting we sweep while searching for the best set of features.

The internal evaluator scores each representation, and those scores set how the budget is split and which representations are kept within the larger hybrid search. It does not rank features inside a representation, which is the task of the importance method. Judging a set of features by the score of a model fitted on it is the wrapper idea of Kohavi and John [178]. The internal evaluator is thus another setting swept in the search for the best set of features.

The full search tests all combinations of budgets, allocations, importance methods, representation counts, downstream models, and internal evaluators under 5-fold cross-validation: 8,160 configurations per dataset when using 5 downstream models. We approach this problem combinatorially as it allows us to measure which factors contribute most to performance variance (model choice, budget, allocation, or their interactions) rather than assuming any single axis dominates.

The full search tests all combinations of budgets, allocation methods, importance methods, representation counts, downstream models and internal evaluators under 5-fold cross-validation. This gives 8,160 configurations per dataset where all five downstream models run, fewer where a dataset leaves a downstream model out, and 10,080 on humanness, where a Gaussian process is added. A search at the fast tier has 816 configurations. The breakdown by downstream model and the number of model fits are given in Additional file 14. We search every combination rather than assume that one setting dominates. This lets us measure which settings, or which interactions between them, account for most of the spread in scores.

The complete search result is cached to disk. The search runs once; everything after that is a lookup.

The complete search result is cached to disk. Importance scores are computed once per fold and importance method, and each representation's standalone score once per fold and internal evaluator. Every configuration reuses them, so the cost grows more slowly than the number of configurations. Because each configuration's score is stored by its settings, a fast search can later be extended to a full one. The chosen configuration for each pair of downstream model and budget is saved as a blueprint. The search thus runs once, and every later analysis is a lookup.

When I began to implement this idea, I expected a straightforward way to select the most informative features. Three questions arose instead, namely how to measure feature importance, how many features to take from each input representation, and how to score a candidate feature set. As with the choice of a single representation, no single answer to these questions suited every prediction task.

The search therefore treats each question as a setting (the importance method, the allocation method together with the budget and the number of representations, and the internal evaluator) and chooses the value of each setting separately for every dataset.

We ran a series of experiments to determine the best variables to include in the search space. When making a selection, we took into account how many potential hybrid representations would be gained by including that variable, as well as the compute time

Before the full runs, we ran a series of preliminary design experiments to determine the values each setting of the search may take. They used 1 to 9 datasets and 1 to 5 folds, and each dataset was scored in its own metric, with scores never averaged across datasets. When making a selection, we considered whether a value gave the best score for some dataset, downstream model and budget, as well as the compute time in hours. A value that won somewhere but cost many hours stayed in the full search and was left out of the fast tier. Each setting's values were then either kept, because each won for some dataset, model and budget, or fixed at one value. A setting was fixed when the spread between its values was smaller than the spread of one value across folds. As an example of a kept value, nine downstream models were fitted on each of 10 base representations over 5 folds. The datasets were Buchwald–Hartwig yield, Caco-2 efflux, Koenig expression, LogD and breast cancer PAM50 subtype. The DNN scored best on 4 of the 5 datasets and third on Caco-2 (0.2936 against 0.2851 RMSE, $\log_{10}$ efflux ratio), so it was added. As an example of a fixed rule, we tested five ways of rescaling each representation's score before the budget is split. On hERG K_i, with linear allocation at 300 features, the five rescalings gave RMSE values from 0.5870 to 0.5924 (pK_i). A single rescaling, however, ranged from 0.5475 to 0.6294 RMSE across the 5 folds, so rescaling is one fixed rule per metric. Every design experiment is listed in Additional file 15.

We search four values of the internal evaluator, namely the matched evaluator, random forest, LightGBM and DNN, and the fast tier searches the same four. The matched evaluator is an evaluator of the same model family as the downstream model. For a random forest, LightGBM or DNN downstream model, the matched evaluator is already in the list, so these three downstream models are searched with 3 evaluators. For an XGBoost or SVM downstream model, the matched evaluator adds an XGBoost or SVM evaluator, so these two are searched with 4. On humanness, where a Gaussian process is also a downstream model, the matched evaluator adds

a Gaussian process evaluator, so it is searched with 4 as well. The same family does not imply the same settings, since only the DNN evaluator shares the downstream DNN's architecture and training settings. Even for the DNN, one difference remains. The evaluator draws the 20% of molecules it uses for early stopping at random, whereas the downstream DNN splits its 20% by the same groups as the outer split. The tree evaluators are smaller than the downstream trees, with 50 trees against 100. Their depth is limited to 10 for the random forest and to 4 for LightGBM and XGBoost, which keeps scoring every representation inexpensive. The SVM and Gaussian process evaluators, whose cost grows with the square to the cube of the number of molecules, are fitted on at most 500 training molecules drawn with a fixed seed. The random forest evaluator tries all features at each split on regression datasets and the square root of the number of features on classification datasets. The downstream random forest tries the square root on both. If the DNN evaluator fails to fit on a fold, a random forest scores that fold and the run records a warning. Every evaluator setting is given in Additional file 12, alongside the settings of the same model used as a downstream model.

A user can also supply their own downstream model under any name, provided it can be fitted and can predict. The matched evaluator, however, knows only a fixed list of model families, so a model outside that list receives no matched evaluator, and no warning is given. That model is then scored only by the other evaluators, by default random forest, LightGBM and DNN. If the matched evaluator was the only one requested, a random forest scores the representations. A hand-written blueprint that specifies the matched evaluator and is rebuilt with a model the search does not know is scored by LightGBM instead. This is a limitation of the method, and a user with their own model should name an evaluator explicitly.

Variable importance depends on the model [197], so the representation that a random forest scores highest need not be the one an SVM uses best. Matching the evaluator to the downstream model is the obvious default. However, the evaluator only has to rank representations, and another model may rank them better for the downstream model. Because the matched evaluator is one of the four searched values, the search already covers this default, and a mismatched evaluator winning is informative rather than a negative result. We first counted wins in the search itself, in a preliminary run on 5 datasets (nanobody thermostability, hERG K_i, humanness, LogD and breast cancer PAM50 subtype). These runs used 10 representations, budgets of 25 to 300 features and 5 folds, and their scores are the search's

own fold scores rather than held-out scores. The matched evaluator gave the best score for its downstream model in 9 of 25 pairs of dataset and downstream model. Counted per budget, it was best in 43 of 125 combinations of dataset, downstream model and budget. With 3 or 4 evaluators per downstream model and no effect of matching, about 38 of 125 would be expected. On nanobody thermostability with a DNN downstream model, the matched evaluator reached a Spearman correlation of 0.6696 against 0.6506 for the LightGBM evaluator, a gap of 0.0190. Its 5 fold scores ranged from 0.5095 to 0.7920. On LogD with an SVM downstream model, the matched evaluator scored 0.4322 RMSE (log units) against 0.4217 for the LightGBM evaluator, where lower is better. This is a gap of 0.0105, the matched evaluator lost at all 5 budgets, and its 5 fold scores ranged from 0.4118 to 0.4634. We then scored the evaluators on held-out folds in a second preliminary run, with 3 outer folds, a budget of 250 features, 5 representations and LightGBM gain only. The downstream models were random forest and SVM. On nanobody thermostability with a random forest downstream model, the matched evaluator reached a Spearman correlation of 0.6098, the LightGBM evaluator 0.6220 and the SVM evaluator 0.6172. The matched evaluator's fold scores ranged from 0.5659 to 0.6734. With an SVM downstream model, the matched evaluator reached 0.6189, the Gaussian process evaluator 0.6265 and the LightGBM evaluator 0.6317, with fold scores for the matched evaluator from 0.5530 to 0.6703. On hERG K_i with a random forest downstream model, the matched evaluator scored 0.6046 RMSE ($\mathrm{p}K_i$), the SVM evaluator 0.6059 and the LightGBM evaluator 0.6075. Its fold scores ranged from 0.5957 to 0.6178, and it beat the SVM evaluator on 2 of 3 folds. With an SVM downstream model, the matched evaluator scored 0.5723 RMSE ($\mathrm{p}K_i$), the LightGBM evaluator 0.5724 and the Gaussian process evaluator 0.5765, with fold scores for the matched evaluator from 0.5605 to 0.5862. Every gap above, both held out and within the search, is smaller than the range of the matched evaluator's own fold scores. **[RESULT NEEDED: per dataset and downstream model, how often the winning configuration used the matched evaluator, and its gap to the best other evaluator in the dataset's metric]**

5.2.3 Augmentation

Some features are too low-dimensional or too specialized to take part in the budget allocation stage, for instance MACCS fingerprints (167 features) or log-likelihood

ratios (3 features). Excluding them from the allocation would lose potentially useful signal, whereas including them would distort the budget distribution because they cannot contribute many features. We therefore handle these features as augmentation. After the base hybrid is constructed, we pool the augmentation features and rank them by how strongly each correlates with the out-of-fold errors of a small random forest fitted on the base features. We keep the top 0, 5, 10, 20 or 40 of them, whichever scores best with the downstream model on the internal validation set.

Some features are too low-dimensional or too specialized to take part in the budget allocation stage, for instance MACCS fingerprints (167 features) or log-likelihood ratios (3 features). Others, such as mutation features that record differences from a parent antibody, only carry meaning alongside a full representation. Excluding such features from the allocation would lose potentially useful signal, whereas including them would distort the budget distribution because they cannot contribute many features. We therefore handle them as augmentation. We call the finished hybrid **H** the base, and we call the feature sets from which the extra features are drawn the donors. After the base is constructed, we pool the donor features and rank them by how strongly each correlates with the out-of-fold errors of a small random forest fitted on the base features. We keep the top 0, 5, 10, 20 or 40 of them, whichever scores best with the downstream model on the internal validation set. In the uncapped mode, the kept features are appended outside the budget. In the capped mode, they replace the weakest base features, so that the hybrid stays within the budget.

Augmentation features range from smaller fingerprints like MACCS or Coloumb matrices, sets of structural information like 3D pharmacophores, and features describing only the differences between the sames like mutation features for antibodies.

Donors are set per dataset rather than per domain and are listed in the Augmentation column of Table 5.16. Augmentation features range from smaller fingerprints such as MACCS to sets of structural information such as 3D pharmacophores and Coulomb matrix eigenvalues. They also include features that describe only the differences between variants, such as mutation features for antibodies. The Coulomb matrix donor is not a fingerprint but the 50 largest eigenvalues of a matrix built from atomic numbers and 3D distances between atoms. MACCS is usually a donor, serving as one on four of the six small-molecule datasets, and it is a base representation on MDR1–MDCK efflux. QM9 is the exception, since MACCS is not

offered there and the topological torsion fingerprint (2,048 features) is a donor, the only wide one. Two datasets, humanness and trastuzumab–HER2 binding, have no donors, and nanobody thermostability has one, mutation features. Each donor is small, at most 167 features apart from the torsions, or describes only one aspect such as shape, charge, the mutated positions or one tumor property. Each is therefore a plausible supplement but a weak representation on its own. Evidence from the literature suggests that 3D donors are worth offering. Bahia et al. [203] modeled six protein–ligand series (461 structures) with 2D, 3D and combined 2D and 3D descriptors, using k-nearest neighbors, random forest and lasso. Combining 2D and 3D descriptors gave many more significant models, which the authors attribute to the two descriptor types coding for complementary properties. Their 3D descriptors came from bioactive conformations in the Protein Data Bank, whereas our 3D donors use one generated conformer per molecule.

Donor features are ranked by what the base gets wrong. Inside each outer training fold, an 80/20 internal split is made with each scaffold or sequence group kept on one side, and the outer test fold is never used. A small random forest (50 trees, depth at most 10) is fitted on the base features, and its predictions for the internal-training molecules come from 3-fold group-aware cross-validation. Its out-of-fold residuals are $r_i = y_i - \hat{y}_i^{\text{oof}}$, where for binary tasks $\hat{y}$ is the predicted probability of the positive class. We use out-of-fold residuals because a forest's in-sample residuals shrink toward zero and mostly measure fit noise. All donor features are pooled, and each is scored by its absolute Pearson correlation with the residuals, $c_j = |\rho_{\text{P}}(\mathbf{x}_j, \mathbf{r})|$. The top $M = 50$ features proceed to the next step. Ranking against the labels $\mathbf{y}$ would favor features that repeat what the base already captures, whereas ranking against the residuals $\mathbf{r}$ favors features that explain its errors. The same idea underlies gradient boosting, which fits each new stage to the current residual [204], and least angle regression, which adds the variable most correlated with the residual [205]. The residual model is always this inexpensive forest and never the downstream model, so the ranking costs the same for every downstream model.

Block sizes of $k \in \{0, 5, 10, 20, 40\}$ features are tried, capped at the number of candidates, where block k consists of the top k ranked features. Each block is added to the base and scored with the downstream model on the internal-validation molecules. The chosen size k^* has the best score and must beat $k = 0$. The option of no extras, $k = 0$, is scored first and wins ties, so keeping no extras is the default outcome rather than a failure. Five whole-block choices replace about 50 one-by-

one decisions to accept or reject a feature. Choosing among many options on a small validation set overfits the choice [196], and five options leave less room for this. The chosen donors and feature positions are stored with the rest of the search settings and are re-extracted for the outer fold like the rest of the hybrid. **[RESULT NEEDED: per dataset and downstream model, how often $k^* = 0$ is chosen, and per-fold scores for no augmentation, uncapped and capped]**

There are two ways to fit the extras against the budget B. In uncapped mode, extras are appended outside the budget, so the final width is $|\mathbf{H}|+k^*$, at most $B+40$ features. In capped mode, extras must displace base features. The base features are ranked once by the importance of a random forest (50 trees, depth at most 10), and the k^* weakest are dropped, so the final width stays at most B. In capped mode, any unspent budget is filled first, and at least one base feature is always kept. Only in capped mode does an extra feature have to beat a base feature to earn its place. We run both modes because uncapped augmentation and no augmentation differ in width at the same nominal B. Their comparison therefore mixes better features with more features, and capped mode removes this difference in width.

Augmentation is a separate condition rather than a searched setting. For each dataset with donors, the whole search runs three times, with no augmentation, uncapped augmentation and capped augmentation. Every other setting (downstream model, budget, representation count, allocation method, importance method and internal evaluator) is searched within each condition. Fourteen of the 16 datasets run all three conditions, while humanness and trastuzumab–HER2 binding have no donors and so run without augmentation only. Without augmentation, the width is at most B and nothing is offered. Uncapped augmentation allows at most $B + 40$ features, and an extra feature must beat keeping no extras. Capped augmentation keeps the width at most B, and an extra feature must beat both keeping no extras and the weakest base feature it replaces. The three conditions are also run under the nearest-neighbor search objective of Chapter 6. Two datasets never run that objective. Breast cancer PAM50 subtype has 4 classes, whereas the objective is scored by AUROC, which needs 2. On trastuzumab–HER2 binding, a held-out block can contain only one group. Augmentation also has a second use outside the main search, in which a user keeps one strong representation at full width as the base and adds a few extras. In that case B is the width of that representation, so only uncapped mode makes sense. We test this use in a separate experiment that adds a few features to one strong representation, with decoy features planted to check that the rule admits

none of them. **[RESULT NEEDED: how many decoy features are admitted in the decoy check]** This use matters for adoption, because a chemist can keep the representation they already trust and learn which few outside features help, and how many. Augmentation also asks directly whether another representation holds signal the base lacks (Chapter 4). On Koenig expression, the mutation features and the ESM-2 log-likelihood ratios reach the model only through augmentation. **[RESULT NEEDED: which donors are kept most often, per domain]**

The residual ranking has four limitations. First, $|\rho_{\mathrm{P}}|$ captures only linear relations with the residual, one feature at a time. Second, it ignores redundancy among the extras, so two near-identical donor features can both rank high. Third, donor features are pooled with no limit per donor, so a 2,048-feature donor gets more chances in the ranking than a 49-feature one. Fourth, the margin over $k = 0$ is zero, so a near-tie on one internal split can admit extras.

Table 5.6 summarizes the three augmentation conditions.

Condition	Final width (features)	What an extra feature must beat	Datasets that run it
No augmentation	at most B	nothing is offered	all 16
Uncapped	$\lvert\mathbf{H}\rvert + k^*$, at most $B + 40$	keeping no extras	the 14 with donors
Capped	at most B	keeping no extras, and the weakest base feature it replaces	the 14 with donors

Table 5.6: The three augmentation conditions. B is the budget and $|\mathbf{H}|$ the width of the base hybrid, both in features, and k^* is the chosen number of extra features, which are drawn from a small or specialized feature set offered to the hybrid.

5.2.4 Fast Tier and Adaptive Pruning

The search can be made cheaper in two ways, which differ in when they decide. The fast tier uses shorter lists of values for some settings, which are fixed before the search and identical on every fast-tier dataset. Adaptive pruning, in contrast, decides during the search from the dataset's own fold scores. The fast tier knowingly accepts a possibly worse answer in exchange for a much smaller grid. Pruning instead aims to find the same best configuration while no longer scoring configurations that have already clearly lost. Without either, the full search does not finish on the largest datasets.

Six of the 16 datasets use the fast tier, namely QM9, LogD, Koenig expression, trastuzumab–HER2 binding, H3K4me3 and TATA promoter. They are marked in Table 5.16, and all other datasets use the full search. A dataset uses the fast tier when the full search would take longer than about 40 hours, the time that four consecutive 12-hour jobs can cover. It also uses the fast tier when it has more than 10,000 molecules or sequences. The estimated run times of the full search were about 103 hours for QM9, 64 for Koenig expression, 62 for LogD and 39 for TATA promoter, and H3K4me3 has 18,244 sequences. The values the fast tier drops are still searched on every other dataset. The fast tier is thus a compromise on compute and makes no claim that those values are useless. A fast run can later be extended to a full one without redoing work, because each configuration's score is stored by its settings rather than by the tier. A full search then scores only the configurations the fast one skipped.

The fast lists were chosen by one rule in a preliminary comparison. A value stays in the fast lists if it is inexpensive and improves the best score, whereas an expensive value that wins often stays in the full search only. Importance methods and downstream models were measured on five datasets, with 5 folds each, against the unpruned full search. The five were breast cancer PAM50 subtype, nanobody thermostability, humanness, hERG K_i and LogD. DeepLIFT took about half of the fast run's hours on all five datasets (49.9% to 51.6%), and dropping it changed only LogD's best score, from 0.4905 to 0.4930 RMSE (log units). Adding the DNN as a downstream model moved LogD from 0.4930 to 0.4273 RMSE, against a full-search best of 0.4170. Adding the SVM moved hERG K_i from 0.5776 to 0.5599 RMSE ($\mathrm{p}K_i$), while adding XGBoost changed no dataset's best score. All five downstream models are now searched at both tiers. The global pool won 60 of the 125 combinations of dataset, downstream model and budget on those five datasets. It is expensive, however, so it is in the full search only. The three fast allocation methods are linear, squared and rank-decay allocation. **[RESULT NEEDED: the share of run time the fast tier and adaptive pruning saved on the full runs]**

Search Tier Settings

Table 5.7 lists, for each search setting, the values tried at the full tier and at the fast tier.

Setting	Full tier	Fast tier
Downstream models	Random forest, XGBoost, LightGBM, SVM[a], DNN; Gaussian process[b]	Random forest, XGBoost, LightGBM, SVM[a], DNN
Internal evaluators	Matched evaluator, random forest, LightGBM, DNN	Same as full
Feature budgets (features)	10, 25, 50, 100, 250, 500, 1,000, 2,000	Same as full
Allocation methods	Linear, squared, softmax, rank decay, global pool	Linear, squared, rank decay
Importance methods	LightGBM gain, DeepLIFT, TreeSHAP	LightGBM gain
Representation counts	2, 3, 5, all representations	3, all representations

[a]SVM is not fitted above 10,000 molecules (5,000 when augmentation features are added outside the budget). [b]Added only on Humanness. Of the 16 datasets, it is the only one that is under 1,000 molecules, has its size recorded and is not excluded for run time.

Table 5.7: Values searched for each setting at the full and fast tiers. Configurations removed by adaptive pruning are scored on only the first two of the five folds. The matched evaluator belongs to the same model family as the downstream model.

Adaptive Pruning

In adaptive pruning, every configuration is first scored on the first two folds. At a budget of 100 features, within each group of configurations that share a downstream model, internal evaluator and representation count, pairs of allocation and importance method with no configuration in the better half are not searched at larger budgets. Each setting is then voted on within groups that fix every other setting. An allocation method is pruned if it is in the best 2 in fewer than 20% of groups, and a representation count if it is best in fewer than 10%. An importance method is pruned if it is in the best 2 in fewer than 20%, and an internal evaluator if it is in the best 3 in fewer than 10%. Survivors are scored on the remaining folds.

In adaptive pruning, every configuration is first scored on the first two folds, and the rules rank configurations by their mean score over those two folds, in the dataset's own metric. Budgets are scored from smallest to largest. Once every budget up to 100 features has been scored, a checkpoint runs within each group of configurations that share a downstream model, internal evaluator and representation count. Within each such group, pairs of allocation and importance method with no configuration in the better half are not searched at larger budgets, although at least one pair per group is kept. Each setting is then voted on within groups that

fix every other setting, including the budget. The best few values in each group receive a point, and a value is removed if it receives a point in fewer than a cut-off share of groups. An allocation method is pruned if it is among the best 2 in fewer than 20% of groups, and a representation count if it is the best in fewer than 10%. An importance method is pruned if it is among the best 2 in fewer than 20% of groups, and an internal evaluator if it is among the best 3 in fewer than 10%. A vote runs only if the setting has more values than receive a point, and it can never remove every value of a setting. A configuration without a score is ranked last in its group. The surviving configurations are scored on the remaining folds, and the best configuration is picked by its mean over all five folds, exactly as without pruning.

Some configurations are so far behind after two folds that the other three folds would only confirm a loss, and adaptive pruning stops scoring them. Pruning is judged on one criterion, namely that it must never remove a configuration that would have been the best, and the time saved is secondary. Pruning is off unless a search requests it. As submitted, the search stored once per dataset uses pruning on every dataset, at both the fast and the full tier. On a fast-tier dataset only one importance method is searched, so the importance vote does not run. Hyperparameter search uses a related idea, dropping configurations early on partial evaluations, as in successive halving [206] and Hyperband [207]. Here the partial evaluation is two of the five folds.

Table 5.8 lists the five pruning rules.

Because the votes are grouped by every other setting, values compete only with each other. A value that is best for a random forest at 25 features is therefore not removed for losing to a DNN at 500 features. A single fold is not enough for the votes. In the representation-count vote on breast cancer PAM50 subtype after fold 1 alone, 726 of 1,020 groups had every value at an identical score. Ties are then broken by list order, so a vote after one fold removes values by accident. After one fold, no cut-off of any vote was safe on the calibration datasets, whereas after two folds each vote had at least one safe cut-off.

The cut-offs were calibrated in a preliminary study on five datasets, LogD, hERG K_i, breast cancer PAM50 subtype, nanobody thermostability and humanness. Each dataset's grid was first scored without pruning on all five folds. Each rule was then replayed on the stored fold scores, so that no rule needed new fits. A cut-off counts as safe if the best five-fold score is unchanged for each of the 25 combinations of downstream model and budget, on every dataset. Where several safe cut-offs made

Rule	How groups are formed	Kept, or receives a point	Cut-off	Removed on the calibration datasets
Checkpoint at 100 features	configurations at budgets up to 100 features with the same downstream model, internal evaluator and representation count	(allocation, importance) pairs with a configuration in the better half of the group	better half of the configurations	configurations within groups only; never a whole allocation or importance method
Representation count	every other setting fixed, budget included	best 1	10% of groups	the count of 5 on LogD; the count of 2 on breast cancer PAM50 subtype
Allocation method	every other setting fixed, budget included	best 2	20% of groups	softmax on LogD
Importance method	every other setting fixed, budget included	best 2	20% of groups	DeepLIFT on nanobody thermostability and hERG K_i; TreeSHAP on LogD
Internal evaluator	every other setting fixed, budget included	best 3	10% of groups	the SVM evaluator on LogD

Table 5.8: The adaptive pruning rules. Every rule ranks configurations by their mean score over folds 1 and 2 in the dataset's own metric, and the last column gives what the rule removed on the five calibration datasets in a preliminary run.

the same cuts, we chose the one whose neighboring cut-off was also safe, and the saving in run time was not a criterion. The chosen rules removed the count of 5 representations on LogD and the count of 2 on PAM50 subtype. They also removed softmax allocation on LogD, DeepLIFT on nanobody thermostability and hERG K_i, TreeSHAP on LogD and the SVM evaluator on LogD. The checkpoint removed configurations within groups only and never a whole allocation or importance method. TreeSHAP accounts for 71% of LogD's fitting time and was never the best on LogD, so most of the saving comes from there. Estimated run times for eight budgets, without and with pruning, were 79 and 44 hours on LogD, 42 and 35 on hERG K_i, and 42 and 30 on PAM50 subtype. On nanobody thermostability they were 32 and 24 hours, and on humanness 15 and 13. The calibration applied the votes one after another, whereas the search applies all four at once to the same scores from folds 1 and 2. The cut-offs were therefore checked in a different order from the one the

search uses. Five datasets form a small calibration set, which is a limitation of the rules.

Pruning changes what the reported score means. With pruning on, folds 1 and 2 both select the survivors and score them, so the five-fold mean of the winner is slightly optimistic [196]. Single representations, by contrast, are never pruned. A fair comparison of the hybrid against a single representation therefore leaves both voting folds out of both scores, or re-scores without them.

5.3 Methods: Experimental Setup

This section sets out how the experiments in this chapter were run. It covers the 16 datasets of six domains and their folds, the base representations offered to the search, and the downstream models and metric used on each dataset. It also describes the comparators, controls and ablations that test the hybrid's gain, and three further studies on transfer between datasets, antibody mutational scans and data held at separate sites.

Every experiment follows the same protocol. Each of the 16 datasets is split into five outer folds, with no group split across folds. One analysis of the antibody expression scan uses three outer folds of unequal size instead, as stated where that result is reported. The hybrid search sees only the training samples of each outer fold, and the selected features are then rebuilt on those samples and applied unchanged to the held-out samples.

The hybrid is compared against the best single base representation used at its full number of features, which asks whether the hybrid beats what a user would otherwise run. This comparator is chosen from the search's own single-representation scores on the training samples of the outer fold, then refit on those samples and scored on the same held-out samples as the hybrid. Because it is chosen separately for each downstream model and each fold, it can be a different representation in each fold. Choosing it on the held-out samples would give it an advantage that the hybrid does not get. The hybrid is held to its budget of base features, to which any augmentation features are added. The comparator has no budget and is therefore often many times wider than the hybrid. Both widths are recorded for every fold, since a difference between the two scores is partly a difference in width as well as in method. The main quantity per fold is the hybrid's score minus that of the best single representation, signed so that a positive value always means the hybrid scored

better (for RMSE the sign is flipped). For each dataset and downstream model, we report the per-fold scores of the hybrid and of every single representation, and their spread across the five folds.

Chapter 6 reuses this protocol with noise added to the training labels of each outer fold, while the held-out labels stay clean.

5.3.1 Datasets and Splitting

The hybrid experiments use 16 datasets from six domains, summarized in Table 5.9. These are five small-molecule ADME and toxicity endpoints, one quantum-chemical property, two reaction-yield sets, four antibody sets, two nucleotide-sequence tasks and two cancer classification tasks from TCGA. Every score in this chapter is computed on held-out folds that keep each group whole, with the groups of each dataset chosen to match what an unseen sample means in its field. The curation of each dataset, with counts before and after each step, is given in Appendix A, and the representations each dataset uses are listed in Table 5.16.

Different biological domains have established different conventions for what constitutes a good prediction, and applying a single metric everywhere would misrepresent performance in at least some of them. We therefore assign each task a primary metric matched to its domain. These metrics are RMSE for molecular property prediction, Spearman rank correlation for antibody regression and developability, and AUROC for binary classification, including antibody classification and tumor subtype. Multiclass classification uses weighted F1, drug-target affinity uses the concordance index, and reaction yield uses R^2. Different biological domains have established different conventions for what constitutes a good prediction, and applying a single metric everywhere would misrepresent performance in at least some of them. The metric of each dataset is given in Table 5.9, and the reason for each choice is given in "Models and metrics".

No random splits are used in this chapter. Random splitting inflates performance estimates for molecular data by placing structurally similar compounds on both sides of the train/test boundary [163, 208]. In deep mutational scanning data, position-level leakage likewise allows models to memorize site-specific effects rather than learning generalizable sequence-fitness relationships [209, 210]. Splits therefore follow the conventions of each domain and use five-fold group k-fold cross-validation throughout. No split is random (Chapter 2, "Evaluation methodology"). Regression

datasets use five-fold group k-fold cross-validation, and classification datasets use five-fold stratified group k-fold cross-validation.

Dataset	Label (unit)	Task, metric	n	Folds grouped by (groups)
Small molecules: ADME and toxicity				
hERG K_i [110]	pK_i	regression, RMSE ↓	1,412	scaffold (769)
LogD [109]	LogD (log units)	regression, RMSE ↓	5,028	scaffold (2,413)
Caco-2 efflux [109]	$\log_{10}$ efflux ratio	regression, RMSE ↓	2,158	scaffold (1,086)
Rat liver microsome clearance [211, 212]	$\log_{10}$ CL_{int} (mL/min/kg)	regression, RMSE ↓	3,053	scaffold (2,625)
MDR1–MDCK efflux [211, 212]	$\log_{10}$ efflux ratio	regression, RMSE ↓	2,641	scaffold (2,318)
Quantum chemistry				
QM9 [107, 213]	HOMO–LUMO gap (eV)	regression, RMSE ↓	9,978	scaffold (3,452)
Chemical reactions				
Buchwald–Hartwig yield [214]	yield (%)	regression, RMSE ↓	3,955	additive (22)
Buchwald–Hartwig plates [214]	yield (%)	regression, RMSE ↓	4,312	additive (24)
Antibodies				
Koenig expression [215, 216]	expression enrichment ratio (no unit)	regression, Spearman ↑	4,275	mutated position modulo 5 (5)
Nanobody thermostability [216, 217]	melting temperature (°C)	regression, Spearman ↑	672	95% identity cluster (309)
Humanness [218]	human or not (198 : 366)	binary, AUROC ↑	564	95% identity cluster (461)
Trastuzumab–HER2 binding [219]	binder or not (1,639 : 3,361)	binary, AUROC ↑	5,000	mutations from wild type (10)
Nucleotide sequences				
H3K4me3 [220]	mark present or not (9,122 : 9,122)	binary, MCC ↑	18,244	chromosome (23)
TATA promoter [220]	promoter or not (2,637 : 2,637)	binary, MCC ↑	5,274	chromosome (23)
Cancer genomics (TCGA)				
Lung adenocarcinoma vs squamous [221–223]	tumor type (515 : 502)	binary, AUROC ↑	1,017	tissue source site (68)
Breast cancer PAM50 subtype [223–225]	subtype (421 : 192 : 141 : 67)	4 classes, weighted F1 ↑	821	tissue source site (19)

Table 5.9: The 16 datasets of the hybrid experiments after curation, grouped by domain. n is the number of samples, and the fold groups are given with the number of groups in parentheses. For classification tasks, the class counts are in parentheses in the label column. CL_{int} is the intrinsic clearance, and arrows show whether lower or higher is better.

Dataset	Label (unit)	Task, metric	n	Folds grouped by (groups)	Best single representation (score)
Small molecules: ADME and toxicity					
hERG K_i [110]	pK_i	regression, RMSE ↓	1,412	scaffold (769)	**[RESULT NEEDED: name, score]**
LogD [109]	LogD (log units)	regression, RMSE ↓	5,028	scaffold (2,413)	**[RESULT NEEDED: name, score]**
Caco-2 efflux [109]	$\log_{10}$ efflux ratio	regression, RMSE ↓	2,158	scaffold (1,086)	**[RESULT NEEDED: name, score]**
Rat liver microsome clearance [211, 212]	$\log_{10}$ CL_{int} (mL/min/kg)	regression, RMSE ↓	3,053	scaffold (2,625)	**[RESULT NEEDED: name, score]**
MDR1–MDCK efflux [211, 212]	$\log_{10}$ efflux ratio	regression, RMSE ↓	2,641	scaffold (2,318)	**[RESULT NEEDED: name, score]**
Quantum chemistry					
QM9 [107, 213]	HOMO–LUMO gap (eV)	regression, RMSE ↓	9,978	scaffold (3,452)	**[RESULT NEEDED: name, score]**
Chemical reactions					
Buchwald–Hartwig yield [214]	yield (%)	regression, RMSE ↓	3,955	additive (22)	**[RESULT NEEDED: name, score]**
Buchwald–Hartwig plates [214]	yield (%)	regression, RMSE ↓	4,312	additive (24)	**[RESULT NEEDED: name, score]**
Antibodies					
Koenig expression [215, 216]	expression enrichment ratio (no unit)	regression, Spearman ↑	4,275	mutated position modulo 5 (5)	**[RESULT NEEDED: name, score]**
Nanobody thermostability [216, 217]	melting temperature (°C)	regression, Spearman ↑	672	95% identity cluster (309)	**[RESULT NEEDED: name, score]**
Humanness [218]	human or not (198 : 366)	binary, AUROC ↑	564	95% identity cluster (461)	**[RESULT NEEDED: name, score]**
Trastuzumab–HER2 binding [219]	binder or not (1,639 : 3,361)	binary, AUROC ↑	5,000	mutations from wild type (10)	**[RESULT NEEDED: name, score]**
Nucleotide sequences					
H3K4me3 [220]	mark present or not (9,122 : 9,122)	binary, MCC ↑	18,244	chromosome (23)	**[RESULT NEEDED: name, score]**
TATA promoter [220]	promoter or not (2,637 : 2,637)	binary, MCC ↑	5,274	chromosome (23)	**[RESULT NEEDED: name, score]**
Cancer genomics (TCGA)					
Lung adenocarcinoma vs squamous [221–223]	tumor type (515 : 502)	binary, AUROC ↑	1,017	tissue source site (68)	**[RESULT NEEDED: name, score]**
Breast cancer PAM50 subtype [223–225]	subtype (421 : 192 : 141 : 67)	4 classes, weighted F1 ↑	821	tissue source site (19)	**[RESULT NEEDED: name, score]**

Table 5.10: The 16 datasets of the hybrid experiments after curation, grouped by domain, with the best single base representation of each and its mean score over the five outer folds. n is the number of samples, and the fold groups are given with the number of groups in parentheses. For classification tasks, the class counts are in parentheses in the label column. CL_{int} is the intrinsic clearance, and arrows show whether lower or higher is better.

Pre-processing. The pre-processing steps shared across datasets are listed below, and the per-dataset steps with the counts after each step are given in Appendix A.

1. Every molecule is parsed with RDKit [32], and molecules that fail to parse are dropped. On hERG K_i, LogD and Caco-2 the structures are also standardized by keeping the largest fragment, neutralizing charges, choosing the canonical tautomer and removing molecules that contain a metal.

2. Repeat measurements of one molecule are merged into their median on hERG K_i, LogD and Caco-2. On hERG K_i, molecules whose repeat measurements have a standard deviation above 1.0 pK_i units are removed, since K_i values combined from different sources are a known source of large label noise [16].

3. Duplicate molecules are removed by InChIKey on the six small-molecule datasets, keeping the first occurrence.

4. LogD and Caco-2 labels are clipped at the 1st and 99th percentiles of the dataset.

5. Two datasets are subsampled once with a fixed seed. From QM9, 10,000 of the 129,428 parsable molecules are kept, and from the trastuzumab set, 5,000 of the 524,346 variants are drawn in proportion to each combination of label and number of mutations.

Splitting rules. Four rules hold for every dataset. First, folds are formed as in Chapter 2, "Evaluation methodology", so no group is split, regression datasets use group k-fold and classification datasets use stratified group k-fold. Second, there are five outer folds, and the stratified folds use a fixed seed. Third, no split is ever random. The handling of a dataset with fewer groups than folds is stated in Chapter 2, "Evaluation methodology", but every dataset here has at least five groups, so all outer splits have five folds. Fourth, on a given dataset every representation, model and hybrid is scored on the same five folds.

The hybrid construction step uses a nested internal train/validation split within each training fold to score representations for feature allocation. This internal split is group-aware and takes the first fold of a grouped five-fold split with the same grouping variable as the outer cross-validation. This prevents the internal evaluator from seeing related samples on both sides of its split. Inside each outer training fold, the search scores representations on one group-aware internal split, which is described in "From base representations to a hybrid".

Groups by domain. The small-molecule and QM9 datasets are grouped by Bemis–Murcko scaffold, as defined in Chapter 2, "Evaluation methodology". In this chapter the scaffold keeps stereochemistry, and all acyclic molecules share one empty scaffold, which on QM9 forms the largest group and holds 1,041 of the 9,978 molecules. The two Biogen datasets (rat liver microsome clearance and MDR1–MDCK efflux) have 1.2 and 1.1 molecules per scaffold, respectively (the n and group columns of Table 5.9). Holding out a scaffold on these datasets is therefore close to holding out single molecules. Chapter 3 used a different scaffold key and different curation for its versions of LogD, Caco-2, hERG K_i and QM9, and Appendix A lists the differences.

The reaction datasets are Buchwald–Hartwig C–N couplings run on high-throughput plates, in which aryl halide, ligand, base and additive are varied. They are grouped by additive, following the out-of-sample tests of Ahneman et al. [214], so each held-out fold contains four or five additives the model never saw in training. The two sets are not independent, since they share 3,955 reactions (Appendix A).

On the Koenig deep mutational scan of the anti-vascular endothelial growth factor (VEGF) antibody G6, every variant carries one mutation. Variants are grouped by the position of that mutation modulo 5, as in ProteinGym [209], so that each fold is one residue class holding 836 to 874 variants. All mutations at one position therefore fall in the same fold, and a model cannot learn a per-position effect and look it up at test time [210]. The nanobody and humanness sets are grouped by single-linkage clusters at 95% sequence identity, after Kenlay et al. [160], and the identity measure, the cluster sizes and the humanness labels are given in Appendix A. The trastuzumab variants differ from the wild type only in the ten residues of CDR-H3 and are grouped by how many of those residues are mutated. A held-out fold therefore contains variants at mutation counts not seen in training.

The two nucleotide tasks come from the revised Nucleotide Transformer benchmark [220]. Both are grouped by chromosome, so each held-out fold contains between two and six of the 23 chromosome groups, none of which the model saw in training. Appendix A gives the task definitions and describes how the released files are regrouped.

The TCGA datasets use RNA sequencing (RNA-seq) expression and other measurements of primary tumors, one sample per patient, obtained through UCSC Xena [223]. Samples are grouped by tissue source site, the institution that supplied the sample, which is read from the TCGA barcode. A model therefore cannot score

well by recognizing the site a sample came from, and no patient appears in both the training and the test set.

Consequences of the grouping. Group definitions differ between domains by design, and the one rule they share is that no group spans training and test. Some groupings leave few or very unequal groups, and the folds inherit this imbalance. The trastuzumab held-out folds range from 13 to 3,035 variants, with binder shares from 15.5% to 92.3% against 32.8% overall, and the breast cancer folds range from 3 to 403 tumors. None of the 68 lung sites supplied both tumor types, so the lung folds are unbalanced, with one fold holding 20 adenocarcinomas and 113 squamous tumors. The Koenig folds are fixed by the five groups of that dataset. Per-fold scores on these datasets spread more widely, so results are read per dataset, in the dataset's own metric, and never pooled across datasets.

5.3.2 Representations by Domain

We combine a variety of component, or base, representations to create these hybrids.

Any representation that gives one fixed-length vector per sample can serve as a base representation, as set out in the description of how the hybrid is built earlier in this chapter. The first version of the method used three molecular representations: PDV, ECFP4 and MHG-GNN. These three are now one small-molecule example among many. Every base representation is frozen: it is computed once from the raw input (a SMILES string, a sequence, a reaction or a patient profile) and is never trained on the labels.

For every data set in this study, we pre-selected ten base representations that spanned across different domains of information and showed some positive results when used on their own for the given prediction task. For every dataset in this study, we pre-selected at most ten base representations that span different domains of information, each of which showed some positive results on its own for the given prediction task.

To choose them, every candidate representation was first scored on its own with a gradient-boosted tree model [57] under the dataset's grouped folds. Candidates were then compared by the Spearman correlation between their out-of-fold predictions, since two representations whose predictions correlate highly carry much the same signal. At most ten were then chosen by hand, looking at each candidate's own

score next to these correlations and favoring representations that encode the input in different ways. Two nucleotide datasets have fewer. H3K4me3 had 12 candidates, and four of them predicted almost the same as a better-scoring candidate, with a Spearman correlation of 0.95 or above between their out-of-fold predictions; eight remain. TATA promoter had 10 candidates, one was dropped by the same rule, and nine remain. Augmentation features do not count toward the ten. They are added after the hybrid is built (see the Augmentation subsection above), and the same representation can be a base representation on one dataset and augmentation on another. On each dataset, the two base representations whose out-of-fold predictions correlate most are **[RESULT NEEDED: most alike pair, with its Spearman correlation]**, and the two that correlate least are **[RESULT NEEDED: least alike pair, with its Spearman correlation]**.

If an input cannot be parsed or embedded, it gets an all-zero row, so that every representation of a dataset keeps the same rows. In computed descriptors, a missing or infinite value is set to 0, and values are stored unscaled. A pretrained model gives one vector per input, the mean of its last-layer token vectors with padding excluded, unless the tables say otherwise. Scaling and filling of missing values happen inside each fold, on the training rows only, as described in the construction of the hybrid.

Small molecules. The small-molecule representations (Table 5.11) are the fingerprint, descriptor, graph and language-model families described in Chapter 2. The only additions are four sets of 3D shape and pharmacophore features, used as augmentation only. All four are computed from a single conformer per molecule, embedded with a fixed seed and optimized with the Merck molecular force field. A single conformer may misdescribe flexible molecules.

Chemical reactions. Reaction datasets (Table 5.12) use three kinds of representation. The condensed graph of reaction (CGR) merges reactants and products into one graph in which the bonds that change are marked [226]; here it is hashed into a circular fingerprint after atom mapping [227]. Reaction language models read the reaction SMILES directly [228, 229]. Any molecular representation can also be applied to the reactants, to the product, or to their difference. A one-hot encoding of which aryl halide, ligand, base and additive was used carries no chemistry. It is kept because on plate data such identity features can predict yield about as well as chemical descriptors [230].

Antibodies. Antibody representations (Table 5.13) start from the heavy- and light-chain amino-acid sequences. Besides one-hot and BLOSUM62 encodings [231], most are frozen language models, either trained on all proteins or on antibody repertoires only. Most models embed each chain separately and join the two vectors, so their width doubles on datasets with both chains. Several representations read one model in different ways, for instance averaging over earlier layers or over the CDR residues only, with CDRs found by IMGT numbering [232] using ANARCI [233]. Datasets of point variants of one parent antibody also use features that describe each mutation.

Nucleotide sequences. A k-mer frequency vector counts how often each DNA word of length k occurs in a sequence, divided by the total number of words [234]; the canonical variant merges each word with its reverse complement [235]. DNA language models are networks pretrained on genomes to predict masked or next tokens [220, 236, 237]. Table 5.14 lists both kinds, along with the small sequence summaries used as augmentation.

Cancer patients. For the two TCGA datasets (Table 5.15), each representation is a different measurement of the same patient's tumor, not a different encoding of one input. The measurements are RNA-seq gene expression (read through foundation models or summarized per chromosome arm), DNA methylation on the HM450 array, gene-level copy number, microRNA abundance, protein levels from reverse-phase protein arrays, and embeddings of the diagnostic whole-slide image. Data were obtained from TCGA through UCSC Xena [223, 238]. The most-variable selections, the z-scoring and the principal components for methylation, copy number, microRNA and protein were fitted once on all patients in a cohort, held-out patients included. No labels are used in these steps. A patient with no measurement for one data type gets an all-zero row for that representation.

We list all of these representations alongside their associated datasets and prediction tasks below:

Table 5.11: Representations used for small molecules. In the Role column, B marks a base representation, A an augmentation and B/A a representation that is a base representation on some datasets and an augmentation on others.

Representation	What it encodes	Features	Source	Role
ECFP4	hashed circular substructures, radius 2	2048	[4]	B
Atom pairs	hashed atom-type pairs with bond distance	2048	[239]	B
Topological torsions	hashed paths of four bonded atoms	2048	[240]	B/A
Avalon	hashed paths, rings and atom environments	2048	[241]	B
MACCS keys	166 predefined substructure keys	167	[242]	B/A
WL graph kernel	Weisfeiler–Lehman label counts, 5 rounds, labels present in at least 5% of molecules	140–317	[243]	B
PDV	RDKit 2D physicochemical descriptors	200	[32]	B
Mordred	2D molecular descriptors	1613	[244]	B
GROVER	frozen pretrained graph transformer	4800	[245]	B
MHG-GNN	frozen pretrained graph encoder	1024	[8]	B
GraphMVP	frozen 2D graph encoder pretrained with 3D views	300	[246]	B
MoLFormer	frozen SMILES language model; public checkpoint trained on 10% of its corpus	768	[40]	B
SMI-TED	frozen SMILES encoder–decoder	768	[247]	B
ChemBERT	frozen SMILES language model; community checkpoint trained on ChEMBL following the cited method	256	[248]	B
ChemBERTa-2	frozen SMILES language model	384	[28]	B
SELFormer	frozen SELFIES language model	768	[249]	B
Mol2vec	summed substructure word vectors	300	[25]	B
3D pharmacophore	counts of 7 pharmacophore types, and mean and spread of distances for the 21 type pairs, one conformer	49	[32]	A
USR	shape moments from 4 reference points, one conformer	12	[250]	A

Representation	What it encodes	Features	Source	Role
USRCAT	USR per pharmacophore atom type, one conformer	60	[251]	A
Coulomb matrix	sorted eigenvalues of the Coulomb matrix, one conformer	50	[252]	A

Table 5.12: Representations used for chemical reactions. Role is read as in Table 5.11, and the suffix in parentheses states whether a molecular representation is computed on the reactants, on the product or as their difference.

Representation	What it encodes	Features	Source	Role
CGR fingerprint	circular fingerprint, radius 2, hashed from the condensed graph of reaction	2048	[226]	B
rxnfp	frozen reaction language model	256	[228]	B
ReactionT5	frozen reaction language model	768	[229]	B
Component one-hot	which aryl halide, ligand, base and additive	45	[214]	B
GROVER (reactants)	GROVER averaged over reactants	4800	[245]	B
Mordred (reactants)	Mordred averaged over reactants	1613	[244]	B
Mol2vec (reactants)	Mol2vec averaged over reactants	300	[25]	B
ECFP4 (reactants)	ECFP4, bitwise OR over reactants	2048	[4]	B
Avalon (difference)	product minus reactant Avalon bits	2048	[241]	B
MHG-GNN (difference)	product minus reactant embedding	1024	[8]	B
MACCS (difference)	product minus reactant MACCS keys	167	[242]	B
Mol2vec (difference)	product minus reactant Mol2vec	300	[25]	B
MACCS (reactants)	MACCS keys, bitwise OR over reactants	167	[242]	A
MACCS (product)	MACCS keys of the product	167	[242]	A
3D pharmacophore (reactants)	3D pharmacophore averaged over reactants	49	[32]	A
3D pharmacophore (product)	3D pharmacophore of the product	49	[32]	A

Table 5.13: Representations used for antibodies. Feature counts marked /chain double on the datasets with heavy and light chains, Koenig expression and trastuzumab–HER2 binding. A dash in the Source column marks a standard encoding or one built for this thesis, and Role is read as in Table 5.11.

Representation	What it encodes	Features	Source	Role
One-hot	one-hot residues, both chains padded to fixed length	5600	–	B
BLOSUM62	BLOSUM62 row per residue, both chains padded	5600	[231]	B
Amino-acid composition	fraction of each amino acid, per chain	40	–	A
ESM-2	protein LM, last layer, mean over residues	1280/chain	[166]	B
ESM-2 (joined chains)	ESM-2 on heavy and light chain joined into one string	1280	[166]	B
ESM-2 early layers	ESM-2, mean over layers 0–16	1280/chain	[166]	B
ESM-2 middle layers	ESM-2, mean over layers 20–33	1280/chain	[166]	B
ESM-2 CDR	ESM-2, mean over CDR residues only	1280/chain	[166]	B
ESM-2 per CDR	ESM-2, CDR1/2/3 pooled separately then joined	3840/chain	[166]	B
ESM-2 variable positions	ESM-2, mean over the 20 highest-entropy positions	1280/chain	[166]	B
ESM-2 shift from parent	ESM-2 variant minus parent embedding	1280/chain	[166]	B
ESM-2 log-likelihood ratio	masked mutant vs parent log-probability: sum, mean, min	3	[253]	A
ESM3	protein LM	1536/chain	[254]	B
ESM3 CDR	ESM3, mean over CDR residues	1536/chain	[254]	B
ESM C	protein LM	1152/chain	[255]	B
ProtT5	protein LM	1024/chain	[167]	B
ProtT5 CDR	ProtT5, mean over CDR residues	1024/chain	[167]	B
SaProt	structure-aware protein LM	1280/chain	[256]	B
AbLang-2	paired antibody LM, both chains read together	480	[257]	B
IgT5	paired antibody LM, both chains read together	1024	[258]	B
AntiBERTy	antibody LM	512/chain	[259]	B
AntiBERTy CDR	AntiBERTy, mean over CDR residues	512/chain	[259]	B
Ab-RoBERTa	antibody LM	768/chain	[260]	B

Representation	What it encodes	Features	Source	Role
Ab-RoBERTa CDR	Ab-RoBERTa, mean over CDR residues	768/chain	[260]	B
Mutation features	wild-type and mutant residue, substitution score, changes in hydrophobicity, weight and charge, position; averaged over mutations	48	–	B/A

Table 5.14: Representations used for nucleotide sequences. A dash in the Source column marks a simple summary built for this thesis, and Role is read as in Table 5.11.

Representation	What it encodes	Features	Source	Role
4-mer counts	normalized counts of every 4-letter word	256	[234]	B
5-mer counts	normalized counts of every 5-letter word	1024	[234]	B
6-mer counts	normalized counts of every 6-letter word	4096	[234]	B
Canonical TNF	4-mer counts merged with reverse complements	136	[235]	B
DNABERT-2	DNA LM	768	[236]	B
Nucleotide Transformer v2 250M	DNA LM	768	[220]	B
Nucleotide Transformer v2 500M	DNA LM	1024	[220]	B
HyenaDNA	long-context DNA LM	256	[237]	B
GROVER (DNA)	DNA LM (not the molecular GROVER)	768	[261]	B
Mistral-DNA	mixture-of-experts DNA LM	768	[262]	B
GC content	GC fraction, GC and AT skew, mean and spread of GC over 100-base windows	5	–	A
Dinucleotide frequency	frequency of each 2-letter word	16	[234]	A
Codon usage	relative usage of each of the 61 sense codons, read in frame from the first base	61	[263]	A
Sequence complexity	entropy of 1-, 2- and 3-letter words, linguistic complexity, fraction of alphabet used	5	–	A
DNA shape	minor groove width, propeller twist, helix twist and roll from a dinucleotide lookup table; mean and spread	8	–	A

Table 5.15: Representations used for cancer patients from TCGA. A dash in the Source column marks a representation computed for this thesis from the patient's own data, and Role is read as in Table 5.11.

Representation	What it encodes	Features	Source	Role
Geneformer (2,048 genes)	single-cell foundation model (V2-316M) on the 2,048 top-ranked genes, [CLS] vector	1152	[264, 265]	B
Geneformer (4,096 genes)	the same model on the 4,096 top-ranked genes, [CLS] vector	1152	[264, 265]	B
BulkRNABert	bulk RNA-seq foundation model	256	[266]	B
Top-variance methylation probes	beta values of the 5,000 most variable CpG probes	5000	[223]	B
Methylation PCA	50 principal components of the standardized CpG probes above median variance	50	[223]	B
Copy number PCA	50 principal components of gene-level copy number	50	[267]	B
miRNA	200 most variable miRNAs, z-scored	200	[223]	B
Protein (RPPA)	reverse-phase protein array, z-scored	258	[268]	B
Chromosome-arm expression	mean expression of the genes on each chromosome arm	42	–	B/A
TITAN	whole-slide image foundation model	768	[269]	B
Prov-GigaPath	whole-slide image foundation model	512	[270]	B
CTransPath	histopathology image encoder	512	[271]	B
Phikon	histopathology image encoder	512	[272]	B
H-optimus-0	histopathology image encoder	512	[273]	B
HRD score	homologous recombination deficiency: LOH, TAI, LST and their sum	4	[274]	A
Tumor mutation burden	non-silent, missense and truncating mutation counts, log count, rate per megabase	5	[275]	A
Tumor purity	purity, ploidy, genome doublings, subclonal genome fraction	4	[276]	A
Expression statistics	summaries of the patient's expression distribution	15	–	A
Immune signatures	10 CIBERSORT cell fractions and Th1 and Th2 signature scores	12	[277]	A

Representation	What it encodes	Features	Source	Role
CIBERSORT	immune cell fractions deconvolved from bulk expression	21	[277, 278]	A
Stemness index	stemness indices from expression and from epigenetically regulated expression	2	[279]	A
Hypermethylation summary	fraction of all methylation probes with beta above 0.3, and mean beta; a genome-wide stand-in for CIMP status [280]	2	–	A

Table 5.16: Base and augmentation representations of each dataset, grouped by domain. The hybrid search selects features from the base representations, and augmentation features are added after the hybrid is built. A dagger marks datasets searched at the fast tier.

Dataset	Task	Base representations	Augmentation
Small molecules			
Caco-2 efflux [109]	efflux ratio (log), regression	WL graph kernel, MHG-GNN, ECFP4, PDV, GROVER, Avalon, Topological torsions, SMI-TED, Atom pairs, MoLFormer	MACCS keys, 3D pharmacophore, USRCAT, Coulomb matrix, USR
LogD† [109]	LogD, regression	MHG-GNN, Mordred, GROVER, ECFP4, SMI-TED, MoLFormer, Avalon, Topological torsions, ChemBERT, GraphMVP	MACCS keys, 3D pharmacophore, USRCAT
hERG K_i [110]	hERG inhibition, pK_i, regression	MHG-GNN, Mordred, WL graph kernel, Avalon, GROVER, PDV, ECFP4, Atom pairs, ChemBERT, SMI-TED	MACCS keys, 3D pharmacophore, Coulomb matrix, USR, USRCAT
MDR1–MDCK efflux [211]	efflux ratio (log), regression	Mordred, PDV, MHG-GNN, Avalon, GROVER, SMI-TED, Atom pairs, WL graph kernel, MACCS keys, ECFP4	3D pharmacophore, Coulomb matrix
Rat liver microsome clearance [211]	intrinsic clearance (log), regression	PDV, MHG-GNN, SMI-TED, Mordred, MoLFormer, GROVER, WL graph kernel, Atom pairs, Avalon, ECFP4	3D pharmacophore, MACCS keys, USRCAT, Coulomb matrix, USR

Dataset	Task	Base representations	Augmentation
QM9† [107]	HOMO–LUMO gap, regression	Mordred, Avalon, SMI-TED, ECFP4, SELFormer, ChemBERTa-2, Mol2vec, ChemBERT, GraphMVP, MoLFormer	Topological torsions, 3D pharmacophore, USRCAT, Coulomb matrix
Chemical reactions			
Buchwald–Hartwig yield [214]	yield (%), regression	CGR fingerprint, GROVER (reactants), Avalon (difference), Mordred (reactants), MHG-GNN (difference), ReactionT5, Component one-hot, Mol2vec (reactants), rxnfp, MACCS (difference)	MACCS (product), MACCS (reactants), 3D pharmacophore (product), 3D pharmacophore (reactants)
Buchwald–Hartwig plates [214]	yield (%), regression	ECFP4 (reactants), GROVER (reactants), MHG-GNN (difference), CGR fingerprint, Component one-hot, Avalon (difference), ReactionT5, Mol2vec (difference), Mordred (reactants), rxnfp	MACCS (product), MACCS (reactants), 3D pharmacophore (product), 3D pharmacophore (reactants)
Antibodies			
Koenig expression† [215]	expression enrichment of variants, regression	ProtT5, ESM3 CDR, SaProt, IgT5, ESM-2 early layers, ESM C, AbLang-2, ESM-2 (joined chains), ESM-2, ESM-2 middle layers	Mutation features, ESM-2 log-likelihood ratio, Amino-acid composition
Nanobody thermostability [217]	melting temperature, regression	SaProt, ESM3, ESM C, ESM-2 variable positions, ESM-2 CDR, ESM-2 early layers, ESM-2 shift from parent, ProtT5 CDR, IgT5, One-hot	Mutation features
Humanness [218]	human vs non-human, binary	IgT5, AntiBERTy CDR, Ab-RoBERTa, AbLang-2, BLOSUM62, ESM C, ESM-2 per CDR, ESM-2 early layers, ProtT5, ESM-2 middle layers	none

Dataset	Task	Base representations	Augmentation
Trastuzumab–HER2 binding† [219]	binder vs non-binder, binary	BLOSUM62, ESM-2 per CDR, ESM-2 middle layers, ProtT5, ESM3, ESM C, Ab-RoBERTa CDR, AntiBERTy, Mutation features, AbLang-2	none
Nucleotide sequences			
H3K4me3† [220]	histone mark present vs absent, binary	GROVER (DNA), Mistral-DNA, Canonical TNF, Nucleotide Transformer v2 250M, Nucleotide Transformer v2 500M, HyenaDNA, 6-mer counts, DNABERT-2	Codon usage, Sequence complexity, Dinucleotide frequency, DNA shape, GC content
TATA promoter† [220]	promoter vs non-promoter, binary	Mistral-DNA, GROVER (DNA), HyenaDNA, 6-mer counts, 5-mer counts, Canonical TNF, 4-mer counts, Nucleotide Transformer v2 500M, Nucleotide Transformer v2 250M	Codon usage, Sequence complexity, Dinucleotide frequency, DNA shape, GC content
Cancer patients (TCGA)			
BRCA PAM50 subtype [224]	breast cancer subtype, 4 classes	Geneformer (4,096 genes), BulkRNABert, Protein (RPPA), Copy number PCA, TITAN, H-optimus-0, Prov-GigaPath, CTransPath, Phikon, miRNA	HRD score, Tumor mutation burden, Tumor purity, Expression statistics, Chromosome-arm expression, Immune signatures
LUAD vs LUSC [221]	lung cancer type, binary	Geneformer (2,048 genes), Top-variance methylation probes, BulkRNABert, TITAN, Methylation PCA, Chromosome-arm expression, Copy number PCA, miRNA, CTransPath, Protein (RPPA)	Immune signatures, Expression statistics, Stemness index, CIBERSORT, Hypermethylation summary, Tumor mutation burden, HRD score, Tumor purity

5.3.3 Models and Metrics

For a fair comparison, the pool of downstream models must cover a range of inductive biases, from methods that degrade in high dimensions (k-nearest neighbors, SVMs and Gaussian processes) to tree ensembles that tolerate irrelevant features. Eleven downstream models are available in total, spanning tree ensembles, kernel methods, nearest neighbors and neural networks. The tree ensembles are random forest, XGBoost, LightGBM, CatBoost and NGBoost, and the kernel methods are the SVM and the Gaussian process. The neural networks are a three-layer DNN, a Monte Carlo dropout BNN for uncertainty quantification and a multi-scale 1D convolutional neural network. Standard experiments use five of these models (random forest, XGBoost, LightGBM, SVM and DNN), with a Gaussian process added on datasets under 1,000 molecules. We use the full set for the model-matching and noise experiments, and on large datasets we skip the models with cubic or higher scaling, namely Gaussian processes above 1,000 samples and SVMs above 10,000. No per-dataset tuning is performed, and all models use fixed hyperparameters throughout, so that performance differences reflect the input rather than the model configuration. For a fair comparison, the downstream models must cover a range of inductive biases, from methods that degrade in high dimensions (SVMs and Gaussian processes) to tree ensembles that tolerate irrelevant features. Standard experiments use five downstream models (random forest, XGBoost, LightGBM, SVM and DNN), with a Gaussian process added on the humanness dataset. An experiment that uses a model outside these five names it in its own methods. On large datasets we skip the two models whose fitting cost grows fastest with the number of molecules, the Gaussian process above 1,000 samples and the SVM above 10,000. No per-dataset tuning is performed, and all models use fixed hyperparameters throughout, so that performance differences reflect the input rather than the model configuration.

The five downstream models cover bagged trees (random forest), boosted trees (XGBoost and LightGBM), a kernel method (SVM) and a neural network (DNN). Several models are used because the best set of features depends on the model that uses it. A hybrid that helped only one model family would reflect a property of that model rather than a better input. Tree ensembles are the baseline to beat, because a hybrid is a tabular feature matrix and tree-based models still lead on medium-sized tabular data of about 10,000 samples [281]. A Gaussian process [282] is added as a sixth model on datasets under 1,000 molecules. Three of the 16 datasets are that

small: humanness (564 sequences), nanobody thermostability (672 sequences) and breast cancer PAM50 subtype (821 tumors). In practice the Gaussian process runs on humanness only, and because humanness is a classification task, the Gaussian process there uses an RBF kernel. The regression version, with an RBF kernel plus a white-noise term, runs on none of the 16 datasets. PAM50 subtype is left out under every augmentation setting because of run time. In the search with augmentation features added beyond the budget, the Gaussian process took 44 of its 60 hours. Nanobody thermostability is left out under augmentation beyond the budget, where its searches did not finish within the time limit. Because a dataset that drops the Gaussian process under any setting receives it under none, nanobody thermostability is left out everywhere.

Every model uses the same settings on every dataset, so that differences between scores come from the input, and no per-dataset tuning is done by default. An optional per-fold random search tries 20 draws of a model's settings, each scored in the dataset's own metric on 3 group-aware inner folds of the outer training fold only. The DNN is never tuned in this way and relies on early stopping instead. **[RESULT NEEDED: whether tuning pays on base representations, tuned against untuned models, in each dataset's metric]**. Two models are skipped above a cap on the number of molecules. The Gaussian process is skipped above 1,000 molecules, because it builds and factorizes the full molecule-by-molecule kernel matrix. The SVM is skipped above 10,000 molecules, since beyond that size its kernel cache no longer holds the support vectors and a fit no longer finishes in useful time. Under augmentation beyond the budget, the SVM is skipped above 5,000 molecules. The caps count the molecules the search is given, which is the whole dataset for the search run once per dataset. A search inside one outer fold is given the training molecules of that fold, about 80% of the dataset. On two datasets the two counts give different downstream models, in both cases under augmentation beyond the budget. LogD has 5,028 molecules and TATA promoter 5,274, so the SVM is dropped when the whole dataset is counted and kept when the roughly 4,000 training molecules of a fold are counted. Some default models are also not run on named datasets where a single fit was too slow, and these are listed in Additional file 13. The settings, library, molecule cap and citation of every downstream model are given in Table S.12 (Additional files).

The kNN model is not a default downstream model. Where it is used, it takes $k = 5$ neighbors by default, lowered to the square root of the number of training

molecules when a fold has fewer than 25. It is the model used to compare importance-weighted with unweighted distances, a comparison that uses 5, 10 and 20 neighbors. Its settings are stated here because Chapter 7, "Simple is often better", relies on that result.

Each dataset is scored in the metric its field normally uses, so that its numbers can be compared with published work on the same dataset. Small-molecule, quantum-chemistry and reaction-yield regression use RMSE, which is in the units of the label. Antibody regression uses Spearman ρ, because these assays are used to rank candidates and their scales differ between laboratories. Binary classification uses AUROC, which does not depend on a decision threshold. The two nucleotide datasets are the exception and use MCC, which is high only when all four entries of the confusion matrix are good [283]. The four PAM50 subtypes, whose class sizes range from 67 to 421 tumors, are scored with support-weighted F1. Table 5.9 gives the metric of each dataset, and the definitions are given in Chapter 2, "Evaluation methodology". Metrics are never averaged across datasets or domains.

Every score is computed on an outer test fold that the search never saw, and we report the per-fold scores and their spread. Each hybrid is paired by fold with the best single representation on the same fold. Each comparison is reported as its per-fold differences and their spread across the 5 folds, with no threshold and no test. We use no significance tests and no confidence intervals, within or across datasets, because with 5 folds a p-value or an interval would rest on 5 correlated values. The fold scores are correlated because the folds share training molecules. No unbiased estimator of the variance of k-fold cross-validation exists, and estimates that ignore the overlap underestimate it [284].

A full search takes **[RESULT NEEDED: hours per dataset for the full search]**, and the searches ran on **[RESULT NEEDED: hardware, as CPU cores, memory, and a GPU where the DNN or the embeddings used one]**.

5.3.4 Comparators, Controls and Ablations

The hybrid is the best of many configurations, each a combination of allocation method, importance method, internal evaluator and budget, so its gain may be no more than the reward for a large search. The experiments below address this and related questions, one question each. They test whether the search overfits and

whether a larger search helps, compare fusing features with fusing predictions and with standard feature reduction, and ask whether the internal evaluator must match the downstream model. Further experiments measure the stability of the selected features across folds, the change in gain with dataset size and the effect of using importance during training. The last three add a few features to one representation, test robustness to label noise and join two whole representations. All of them use the group folds of "Datasets and splitting".

The hybrid's score is the best of many configurations, whereas each single representation gives one score, and the best of many cross-validated scores is biased upward [195]. Several experiments therefore use leave-one-fold-out reselection, in which, for held-out fold f, we pick the hybrid configuration with the best mean score on the other folds,

$$c^*_{-f} = \arg\max_c \frac{1}{F-1} \sum_{g \neq f} s_{c,g},$$

where $s_{c,g}$ is the score of configuration c on fold g and F is the number of folds (arg min for RMSE). The best single representation, k^*_{-f}, is picked in the same way, and we then report

$$\Delta_f = s_{c^*_{-f},f} - s_{k^*_{-f},f},$$

signed so that $\Delta_f > 0$ means the hybrid is better (for RMSE the sign is flipped). Because this is arithmetic on stored per-fold scores, it needs no extra model fits. Importance scores and the feature selection are always recomputed on the training molecules of each fold only. Some experiments, however, choose the configuration by a search over all molecules and redo only the feature selection per fold. Their scores are therefore not fully held out, and we say so where this applies.

The first experiment uses nested cross-validation to ask whether the search overfits as the budget grows. For every budget B, the winning configuration at that budget is rebuilt on the outer training molecules and scored on the outer held-out fold. The selection bias at a budget is the score reported by the inner search minus the held-out score, signed so that a positive value means the search promised more than the held-out fold gave. The budgets are the standard list from 10 to 2,000 features, plus 4,000 and 8,000 features. To these we add budgets at 0.5, 0.75, 1, 1.25, 1.5 and 2 times the smallest outer-fold training-set size in molecules, which place a budget where the number of features equals the number of molecules. For the five

downstream models, the budget does not set the number of fitted parameters. The DNN and the random forest have more fitted parameters than training molecules at every budget. The SVM is fitted with at most one coefficient per training molecule, so it never can, and the capacity of the boosted trees is set by the number and depth of their trees. For these five models, the curves therefore show whether the search's choice of budget overfits. The experiment adds one model outside the five, a linear model with almost no penalty, either ridge regression with penalty 10^{-6}, or logistic regression with $C = 10^{6}$ for classification. This model has one weight per selected feature, so its parameter count equals the budget. We report two numbers per model, the budget picked by the search of each outer fold and the budget with the best held-out score. A pick far above the best held-out budget indicates that the search overfits. **[RESULT NEEDED: held-out score and selection bias against budget, per model and dataset]**.

The size of a search is the number of configurations it scores, that is, the product of the numbers of budgets, allocation methods, importance methods, internal evaluators and representation counts. At the full tier this gives 480 configurations for each pair of downstream model and internal evaluator, and at the fast tier 48 (Additional file 14). The question is whether the hybrid's lead over each comparator grows with the number of configurations. The comparators, namely the ensemble, the best single representation, PCA and feature reduction at the same width, receive no search. Part of any gap that grows with search size therefore reflects the hybrid taking the best of more draws. Leave-one-fold-out reselection charges the hybrid for this, and we report the reselected numbers. In the full runs each dataset is searched at one tier only, full or fast, whereas a search-size comparison needs the same dataset searched at two sizes. **[RESULT NEEDED: hybrid minus each comparator, per dataset, at each search size]**.

The ensemble fits one model per representation and pools their predictions, whereas the hybrid fits one model on one fused feature vector. Joining features before a single model is early fusion, and pooling the predictions of one model per feature set is late fusion [149]. The hybrid runs its own search on the training molecules of each outer fold. Every choice for the ensemble is made on an inner 3-fold group split of the same training molecules, so the held-out fold is used once. The ensemble either fits the same model on every representation or lets each representation use its own best model. Its members are either the top 3 representations or every representation with at least 50 features. The predictions are pooled in one of

seven ways. Three are the mean, the median and a 20% trimmed mean, and two are means after removing outliers found by interquartile range or by median absolute deviation. The last two are a mean weighted by the inner score of each member and the mean of the top 3 members. Classification ensembles pool predicted probabilities, and where the metric reads class labels, a majority vote is also offered. Three controls are included in the comparison. The first is the best single representation, and the second joins all representations and reduces them by PCA to the width of the hybrid. The third is the best pair of model and representation read off the held-out fold, which serves as an upper bound rather than as a fair comparator. Beside each score we report the cost as the number of representations computed at prediction time, the number of models deployed, and the fit and prediction time in seconds. The hybrid saves cost only if it draws on fewer representations than the ensemble. Because an ensemble is the obvious simpler alternative, a result in which it matches the hybrid at lower cost belongs in Chapter 7, "Simple is often better". In QSAR, Turkina et al. [148] combined six non-hashed fingerprints by low-, mid- and high-level fusion on six regression and classification datasets. Mid-level fusion, which keeps fingerprint bits by their importance in the per-fingerprint models, gave the best accuracy. **[RESULT NEEDED: held-out score and cost per method]**.

The next experiment asks whether the gain comes from drawing on several representations or only from having fewer features. Every method receives the same number of features, B, drawn from all base representations and fitted on the training molecules of the fold, and the experiment runs on regression datasets only. The methods are the hybrid, PCA, mRMR [177], recursive feature elimination [180] and B features drawn at random. The hybrid uses its stored configuration at budget B, with the features re-selected on each fold. For PCA, all base representations are joined and each feature is z-scored on the training molecules of the fold. PCA is then fitted on those molecules with B components, capped at the total number of features and at one fewer than the number of training molecules. Held-out molecules are projected with the fitted PCA, and the PCA control of the ensemble comparison is built in the same way. The mRMR ranking stops at 200 features for run time. When the representations hold more than 2,000 features in total, mRMR first keeps the 2,000 with the highest mutual information with the label. Recursive feature elimination uses a random forest of 50 trees of depth at most 5. A single downstream model, a random forest of 100 trees, scores every method, so only the features differ. This comparison is not fully held out, because the configuration of

the hybrid is chosen using all molecules and only its feature selection is redone per fold. **[RESULT NEEDED: held-out score against B (features) for each method]**.

The search scores each representation with an internal evaluator to split the budget, and the winning hybrid then goes to a downstream model. This experiment asks whether the two must match. As in the overfitting experiment, it uses nested cross-validation, with one search per outer fold for each evaluator setting, and each setting is compared with the matched evaluator. The matched evaluator belongs to the same model family as the downstream model and uses the lighter evaluator settings of Table S.12. The near-family setting scores the random forest with LightGBM and the reverse, and XGBoost with LightGBM. It also scores the SVM with a Gaussian process, and the DNN with a narrower DNN of 64 and 32 units. The far-family setting scores the three tree models with an SVM, the SVM with LightGBM, and the DNN with k-nearest neighbors. A fourth setting scores the DNN with a wider DNN of 512, 256 and 128 units. The last setting uses a fixed set of three evaluators (random forest, LightGBM and XGBoost) and keeps the best. We report each setting minus the matched evaluator, paired per outer fold, at the same number of features. Evaluators with the exact settings of the downstream random forest and LightGBM are implemented, but no search uses them, so the full runs do not report them. The answer determines whether one cheap evaluator is enough or whether the search must be rerun for each model. **[RESULT NEEDED: each setting minus matched, per downstream model]**.

Selection on each of the M folds gives a binary vector $\mathbf{z}_m \in \{0,1\}^d$, where d is the number of features in all base representations joined. We measure the stability of the selection with the index of Nogueira et al. [181],

$$\hat{\Phi} = 1 - \frac{\frac{1}{d}\sum_{f=1}^{d} s_f^2}{\frac{\bar{k}}{d}\left(1 - \frac{\bar{k}}{d}\right)}, \qquad s_f^2 = \frac{M}{M-1}\hat{p}_f(1-\hat{p}_f).$$

Here $\hat{p}_f$ is the fraction of folds that selected feature f, and $\bar{k}$ is the mean number of features selected. A value of $\hat{\Phi} = 1$ means that the same features were chosen on every fold, and $\hat{\Phi} \approx 0$ is what random selection gives. The width d must be the full number of features in all base representations joined, rather than the size of the union of the selected features. Unstable feature sets would weaken any chemical reading of the blueprints. **[RESULT NEEDED: $\hat{\Phi}$ per dataset and model]**.

The learning curves draw subsamples of 10, 25, 50, 75 and 100% of the molecules as whole groups, never with fewer than 50 molecules. Above 5,000 molecules, absolute sizes of 250, 500, 1,000 and 2,000 molecules are used instead. Each size is drawn 3 times, except the 100% point, which is drawn once because every draw is the same. A full search runs on each subsample, with budgets above the number of training molecules dropped. The gain at each size is the hybrid minus the best single representation, with leave-one-fold-out reselection. Because small subsamples search fewer budgets, part of any rise in gain with size may come from searching more configurations. **[RESULT NEEDED: gain per dataset, model and size]**.

This experiment asks whether importance also helps during training, in addition to its use in selection. It uses the hybrid at 100 features with random forest, LightGBM and XGBoost, the three models that accept per-molecule weights. Importance is computed on the training molecules with the importance method the search picked for that hybrid. The experiment compares the following five conditions:

1. No weighting, in which the hybrid is used as selected.

2. Feature weighting, in which features are multiplied by the square root of their importance after standardizing, with the weights normalized to average 1.

3. Molecule weighting, in which molecules are weighted by how well their label agrees with their 10 nearest neighbors in the rescaled space.

4. Combined weighting, which applies feature weighting and molecule weighting together.

5. Above-median selection, which keeps only the features strictly above the median importance.

The weight of molecule i is $1 - |y_i - \bar{y}_{N(i)}|/(\max y - \min y)$, clipped to lie between 0.1 and 1. Here $\bar{y}_{N(i)}$ is the mean label of its 10 nearest neighbors by Manhattan distance. This comparison is not fully held out, because the configuration comes from a search over all molecules and only the feature selection is redone per fold. Molecule weighting is a noise-mitigation method that down-weights suspect labels. Tree ensembles are unchanged when a feature is multiplied by a positive constant, because their splits depend only on the order of values [285]. Feature weighting therefore differs from no weighting in one respect only. Features with zero importance receive weight zero, become constant and drop out, while every other feature is only

rescaled, which the three tree models ignore. The rescaling does matter for molecule weighting, because it changes which molecules are nearest neighbors. Checking a label against its nearest neighbors goes back to the editing of a training set by its nearest neighbors [103], although here the molecule is down-weighted rather than removed. We found no QSAR study that down-weights training molecules whose labels disagree with their nearest neighbors. The nearest prior work lies outside QSAR, where nearest-neighbor noise filters adapted to regression labels remove such samples instead of down-weighting them [146]. Learned per-sample training weights for noisy labels come from general machine learning [286]. **[RESULT NEEDED: one result line]**.

This experiment asks whether a few extra features help one strong single representation. Two kinds of decoy are planted among the candidate features. The first is a block of pure Gaussian noise as wide as the real candidate features combined. The second consists of near-copies of 64 base features with added noise at 5% of the standard deviation of each feature. We count how many decoy features are taken on each fold, and a working rule takes none of them. The same extra features are also offered to the hybrid built from several representations, to ask whether extra features do more for one representation or for the hybrid. **[RESULT NEEDED: one result line, with the decoy features taken per fold]**.

This experiment asks whether hybrids are more robust to label noise than their base representations. Robustness is measured by $\mathrm{AUC}_{\mathrm{norm}}$, the normalized area under the retention curve of Chapter 3 (Equation 2.7). The setup differs from that of Chapter 3 in two ways. First, the retention curve uses each dataset's own metric instead of R^2, with the ratio inverted for an error metric so that 1 still means no loss. Second, there are seven noise levels, 0, 0.2, 0.3, 0.5, 0.75, 1.0 and 1.5, each a fraction of the spread of the clean training labels of the fold. Censoring has its own axis, the fraction of labels clipped, with values of 0, 0.10, 0.20, 0.25, 0.30 and 0.40. We compare the hybrid at each budget with each single representation and with the best single representation, re-picked in each fold. One downstream model, a random forest, is used under each regression noise type. The settings of the hybrid are searched again at every noise type and level on the noisy training labels, while the held-out labels stay clean. Because the search uses the scores of the first fold to drop configurations, that fold is left out of both sides and four folds are reported. In Chapter 3 most of the variance in $\mathrm{AUC}_{\mathrm{norm}}$was explained by the model rather than by the representation, whereas here the model is fixed, so only the representation

changes.

The last experiment asks whether joining two whole representations scores above the better of the two or between them. For each dataset we take its top three single representations, form each of the three pairs, and score each representation alone and each concatenation with the same model and folds. A concatenation that lands between its two parts supports the case for combining and filtering at once. **[RESULT NEEDED: for each pair, the score of each representation alone and of their concatenation]**.

5.3.5 Transfer, Mutational Scans and Separate Sites

Three further studies go beyond the single-dataset setting by reusing search settings on a second dataset, building hybrids on antibody mutational scans, and selecting features with data held at separate sites. Each needs several numeric representations of the same objects, one supervised target, and a grouped split that keeps near-duplicates on one side. For each study we give a short background and describe how the setup was built and scored, and the outcomes are reported in the results sections.

The first study asks whether search settings found on a large dataset can be reused on a small one instead of running a new search. These settings, namely the importance method, the allocation method, the budget B and the internal evaluator, are decisions about the search rather than a set of features. The source dataset is LogD (lipophilicity), with 5,028 molecules, and the target dataset is hERG binding affinity ($\mathrm{p}K_i$), with 1,412 molecules. The candidate representations are those listed for both datasets, kept only where the width (number of features) matches, so that both searches range over the same features. The current lists share seven representations, namely Avalon, ECFP4, Mordred and four pretrained embeddings (GROVER, MHG-GNN, SMI-TED and ChemBERT), although the width check can still drop some of them. **[RESULT NEEDED: the shared representation set the full run uses]**. Because the search is the expensive step, settings that transfer would make it a one-off cost, paid on the largest dataset. The closest prior work is Meta-QSAR [287], which chose the learning method and molecular representation for a new QSAR dataset from results on over 2,700 other QSAR datasets. It beat random forest on a fingerprint by up to 13% on average and, like the source search settings here, reuses decisions rather than a set of selected features. We have not

found a paper that applies feature indices selected on one QSAR dataset unchanged to another, although our search of the literature was not complete.

All four comparisons run inside every held-out fold of the target. The first is the best single representation, re-chosen on the training molecules of that fold, and the second is a hybrid searched on the same training molecules, which serves as the upper reference. The third applies the source search settings with the features re-selected on the training molecules of the target. The fourth applies the source's own selected feature indices unchanged, so that no target label is used to pick features. The last two separate two questions, whether the search decisions transfer and whether the chosen features transfer. We use five scaffold-group folds, RMSE in pK_i units and the five downstream models (random forest, LightGBM, XGBoost, SVM and DNN). **[RESULT NEEDED: hybrid minus best single representation for each transfer comparison, per downstream model]**.

A deep mutational scan measures a property, here expression or binding, for many single- or few-point mutants of one antibody, and is used to map which positions tolerate change. We use four scans with a single wild type, taken from FLAb [216]. Two are the expression and binding scans of the anti-VEGF antibody G6 [215], of which the expression scan is one of the 16 datasets of this chapter. The other two are the expression and binding scans of the anti-fluorescein antibody 4-4-20. The candidate representations are the protein language-model embeddings and the cheaper sequence features of "Representations by domain", to which we add two representations of the change from the wild type. The first has 5 features, the changes in hydrophobicity, charge, isoelectric point, molecular weight and aromaticity summed over the mutations, with each property first z-scored over the 20 amino acids. The second places the same five features at the mutated position, so it has five features for every sequence position. Under nested cross-validation, the search sees the outer training variants only, and the hybrid and every single representation are scored on the untouched outer fold. The continuous scans are scored by Spearman correlation. The 4-4-20 binding scan is binarized, with a variant counted as a binder if its pK_d is above 5.0, and scored by AUROC. We report the hybrid minus the best single representation, per scan, on the outer folds. The two wild-type-difference representations are also scored as single representations, and each run reports their rank among all representations. The number of features each representation received is stored per fold with every result, but this comparison does not report it. Without that count, the difference between the hybrid and the best single representation says nothing

about the wild-type-difference representations. **[RESULT NEEDED: features the hybrid took from each of the two wild-type-difference representations, per scan, per fold]**. **[RESULT NEEDED: hybrid minus best single representation per scan, per downstream model]**.

Federated learning trains one model across data held by several parties without moving the data [288], and it has been run across pharmaceutical companies on shared property-prediction targets [289]. In this field the motivation is commercial, because structures and assay results are the asset and pooling them is therefore not possible [18, 19]. Our collaborator, Lhasa Limited, uses the teacher-student process described in Chapter 2, "Noise in molecular data" [139]. We use the collaborator's code for the fingerprints, the site models and the site clustering, so that the numbers stay comparable to theirs. The step that combines the predictions of the site models is re-implemented from that code.

The predictions of the site models are combined in one of two ways. Under distance weighting, the vote of each site model on a molecule is weighted by how close that molecule is to the site's own training molecules. A site therefore counts most on chemistry it has seen. Closeness is the mean fingerprint distance from the query molecule to its 8 nearest neighbors among a random 20% of the site's training molecules. The distance is set per run and is Manhattan by default, whereas the collaborator's code uses Euclidean distance. For each site, these distances are rescaled to the range 0 to 1 over the query molecules. Each is turned into a similarity as $(1 - \text{distance})^2$, and the similarities are normalized to sum to 1 over the sites. Under score weighting, each site instead receives a single weight from its own cross-validated MCC. Only predictions on shared molecules cross a site boundary. In the published process, a student model is then trained on the most confident combined labels, with the same number from each class. In the hERG comparison below, no student is trained, and the combined predictions score the held-out molecules directly.

The addition made in this thesis is that feature selection, as well as model fitting, takes place at the sites. Each site runs the hybrid search on its own molecules, choosing features on 80% of them and scoring them on the other 20%, split by scaffold group. Each site reports three things, namely feature indices, importance values and one score, and no molecule or feature value leaves the site. A coordinator splits the total budget B across sites in proportion to their scores, rounding each share b_s down with at least one feature per site, so that $\sum_s b_s \approx B$. If every site

scores 0, the budget is split equally. Only the proportional split is run, so without an equal-split comparison we cannot say what the site scores buy. A feature that another site has already taken is skipped, and the site takes its next-ranked feature instead, so the features in the hybrid are all distinct. Indices and importances are smaller, and harder to invert, than model weights or gradients. Splitting the budget between sites is also the same question that the allocation method answers between representations. In related work, each client runs a mutual-information feature selection on its own data and a server aggregates the results, with no raw data exchanged [290].

hERG channel blockade is a cardiotoxicity liability screened early in drug discovery, and its public data come from many laboratories [291]. Three real sources act as sites, namely the public ChEMBL hERG K_i set used elsewhere in this thesis and the collaborator's training set (7,772 molecules) and test set (4,383 molecules). The two collaborator sets were labeled by different curation rules, so the sites differ in curation as well as in chemistry. This source heterogeneity exists in practice, whereas the process is usually simulated with sites made by clustering one dataset. Molecules held by more than one source are deduplicated on InChIKey, keeping the public copy. The target of the public set is binary, with a molecule counted as a blocker if $\mathrm{p}K_i \geq 6$, that is, $K_i \leq 1\,\mu\mathrm{M}$. We use five scaffold-group folds over the pooled molecules, and the search is scored by AUROC. The reported number is MCC, which does not reward predicting the majority class.

The comparison crosses two choices, whether molecules are pooled or kept at their sites, and whether all features or a selected set are used. Pooled molecules with all features form the upper reference, which is not possible in practice. Pooled molecules with selected features measure what the selection is worth on its own, and molecules at their sites with all features correspond to federated training as usually done. Molecules at their sites with selected features form the only setting in which nothing but indices, importances, one score and predictions moves between sites. The difference between selected and all features at fixed site structure is what selection buys, and the difference between pooled and site-held molecules at fixed features is what pooling buys. We add four context comparisons, namely the single best source picked on training molecules, both ways of weighting the site models, the same search on one fingerprint alone, and one site model per representation. The budget B is swept over 50, 100, 250, 500 and 1,000 features, so that no comparison rests on one width. Both all-features settings fit one 2,048-bit Morgan fingerprint of

radius 2 with pharmacophoric atom types. Both selected settings instead fit features drawn from the candidate representations of the hERG K_i dataset. Selected minus all features therefore differs in two ways at once, in width and in source. The budget sweep varies only the width of the selected set and cannot separate the source difference. The closest control for the source difference is the single-fingerprint search, which is the same search run on ECFP4 only. Two curators disagreeing on one molecule constitute label noise with a known source (Chapter 6).

Four further separate-site experiments are run. In the first, the sites carry different amounts of label noise. Selection at the sites is compared, at several noise levels, with the ensembles of site models and with one model trained on the pooled molecules of all sites. In one version of this experiment, each site first removes the molecules it flags as noisy. In the second, every site holds the same molecules as a different representation, and fusing features is compared with pooling the predictions of the site models. In the third, the sites hold different families of representations (substructure, physicochemical and pretrained) of shared molecules. In the fourth, the sites are three antibody sources, namely a binding-energy-change set, a deep mutational scan of binding and a thermostability set. Sub-sites within a site are made by clustering language-model embeddings instead of fingerprints. The antibody version skips the two ensemble comparisons. Each site's features are reduced separately, so one site's model cannot predict on another site's features, and the labels are in incompatible units (kcal/mol, a normalized score and degrees Celsius). **[RESULT NEEDED: one line on the outcome of each of the four]**.

The separate-site work has four limitations. One source is proprietary and cannot be redistributed, so the comparison cannot be reproduced from public data alone (Appendix B). The comparison also depends on the collaborator's implementation of the process. The published process labels a large separate public set and trains a student on it. Where a student is trained here, the training molecules play that role as a compromise, and held-out molecules are excluded from it. Finally, the sites differ in size and in how their labels were derived, so a site's score reflects its curation as well as its chemistry.

Three external benchmark tasks that are not among the current datasets are possible later additions and are not run here. The first is activity-cliff prediction on the dopamine D2 receptor, factor Xa and SARS-CoV-2 main protease, following a published comparison of representations and models [73]. The second is antibody developability, on the biophysical properties of a public competition set, using that

competition's cluster splits [292]. The third is antibody–antigen binding free-energy change, asking how close sequence representations get to a structure-based graph network [293].

5.4 Results: Comparison with Single Representations and Ensembles

5.5 Results: Sources of the Gain

5.6 Results: Blueprints on Specific Datasets

5.7 Results: Transfer across Datasets and Antibody Mutational Scans

5.8 Results: Data Held at Separate Sites

5.9 Discussion

Ambroise and McLachlan [193] showed the extreme case of overfitting through feature selection. An SVM achieved near-zero cross-validated error on random labels when genes were selected on the full dataset before cross-validation. When selection was nested inside each fold, the median error was 11 out of 40, close to the 50% expected by chance. Vabalas et al. [194] showed that the bias introduced by preselection can exceed the bias from hyperparameter tuning. Cawley and Talbot [196] argued that feature selection is itself a form of model selection whose complexity cost must be accounted for. The hybrid search follows the standard remedy of computing all importance scores within each cross-validation fold, on training data only. The problem of search degrees of freedom [294] nevertheless remains, since searching over many possible feature subsets adds complexity even when each individual selection is unbiased. The budget acts as a hard capacity constraint. Selecting 100 features from a pool of 5,000 limits the space of possible selections far more tightly than L_1 regularization over the same pool. The reason is that the feasible set grows polynomially in the budget while the continuous parameter space of a regularized

model is unbounded [295]. The features being selected are, moreover, frozen embeddings rather than parameters being fitted. We consider choosing which pre-existing knowledge to trust a less expressive operation than learning new features. In support of this view, Kumar et al. [296] showed that frozen embeddings fed through a linear probe can outperform full fine-tuning by up to 10% in out-of-distribution accuracy, because fine-tuning distorts pretrained structure. Finally, stability can be measured directly with the Nogueira index [181], which quantifies the consistency of selected feature sets between folds. If selection were overfit, the selected feature sets would differ widely from fold to fold.

5.10 Limitations

The adaptive hybrid representation has several limitations. For small datasets, below about 200 samples, the hybrid search cannot reliably outperform the best single source. We suspect that this is because the internal validation set within each fold shrinks to roughly 20 samples, at which size the evaluator cannot distinguish genuine improvements from noise, and tuning cannot fix this. At the other extreme, frozen embeddings have a ceiling, since no signal can be added beyond what the inputs already contain. This ceiling binds on tasks where the true signal requires end-to-end learning from raw inputs, for example certain 3D quantum-mechanical properties, where SchNet outperforms any frozen embedding by a wide margin. The method naturally accommodates embeddings from sequence-trained models such as ESM-2 and ProtBERT. Encoding raw sequences directly as one-hot or k-mer features, however, disperses the positional and compositional information that makes sequences useful in the first place. Integrating 20 to 50 foundation models into a single pipeline, each with its own dependencies, GPU requirements and version constraints, is also difficult engineering. Embeddings need to be computed only once per dataset and are cached, but the initial integration burden is real and scales with the number of models in the pool. Finally, on endpoints with high experimental noise, even a well-constructed hybrid may not improve on simpler baselines. Kramer et al. [15] documented that the experimental uncertainty in heterogeneous public K_i data is approximately 0.54 log units, which sets a natural accuracy ceiling of $R^2 \approx 0.81$ regardless of model sophistication. The budget constraint plausibly guards against overshoot, but it cannot push past this noise floor.

5.11 Conclusions

Chapter 6

Hybrid Representations for Similarity and Label Noise

The hybrid representation was first conceived as a way to repair molecular similarity, and this chapter tests whether features chosen for the task give better neighbors than the single representations from which they are drawn. We also ask whether better neighbors help to find noisy labels, to estimate the size of their errors and to train on them. These are distinct tasks, and a method can succeed at the first while failing at the third. Because the type of noise determines what can be corrected, we treat censoring, Gaussian noise and real labels separately.

6.1 Introduction

Chapter 4 showed where the similarity assumption fails on standard representations. For example, ECFP4 bits compared by Tanimoto similarity mix a few features relevant to the task with many that are constant or irrelevant on the dataset. Any method that trusts a molecule's neighbors inherits this error, including methods that find noisy labels by comparing each label with the labels of its neighbors. Chapter 5 introduced the adaptive hybrid representation, a budget of features selected from several base representations for one task and one downstream model. The hybrid is therefore an input to a model and leaves the model itself unchanged.

The original idea behind the hybrid was to select the features relevant to the task first and then to compute distances on those features alone. Hybrids later proved useful as ordinary model inputs (Chapter 5), and this chapter returns to the original motivation.

We test three methods that rely on neighbors or on held-out predictions to handle noisy labels, namely nearest-neighbor editing [103], filtering by classifiers trained on other folds [144] and confident learning [52]. Each of these methods depends on the feature space in which it operates, so we ask whether a hybrid changes how well they work. The chapter addresses three research questions, each answered in one results section:

1. Is predicting each molecule's activity from its five nearest neighbors more accurate in a hybrid than in the single representations from which the hybrid is built?

2. Are noisy labels easier to find, or the size of their errors easier to estimate, in a hybrid?

3. Does a hybrid change how well noise mitigation, through label correction and cleaning, and noise-robust training perform?

This chapter is intended as the basis of a second publication on hybrid representations. Its methods are final, whereas its results are still preliminary. **[RESULT NEEDED: one-sentence preview of the pattern found for each of the three research questions]**

6.2 Methods

The methods cover the computation of similarity in a hybrid, the noise conditions and the three experiments on learning with noisy labels.

The three experiments share their datasets, scaffold-group-aware folds and metrics with Chapter 5, and noise is added to the training labels only. Each downstream model is compared with its own top two single representations, ranked by that model's own score, rather than with a fixed list of representations or a single chosen fingerprint.

In all three experiments, the features of the hybrid are rebuilt within each fold from that fold's training labels, which are noisy, clipped or uncleaned depending on the experiment. The search settings, namely the representations, the budget and the allocation method, are chosen differently in each experiment. In the survey of twelve methods for learning with noisy labels, the search is repeated in each fold and at each noise level on the noisy training labels. Candidates are scored by the same

neural network that the methods use. This search covers budgets of 50, 100 and 250 features with 2 to 3 inner folds, whereas Chapter 5 searches eight budgets between 10 and 2000 features. If the search returns no score in a fold, that fold uses the settings found for the neural network by the search on clean labels, and the results record which folds did so. In the experiment with labels clipped at a detection limit, the settings are taken from the search on the clean, unclipped labels. Only the feature ranking and the budget split within each fold see the clipped labels. In the experiment that cleans real labels, no clean labels exist, so the settings come from a search on the same real labels used for training. Settings searched on clean labels give the hybrid information that a real user would not have, an advantage that the survey avoids and the clipped-label experiment does not.

We distinguish three tasks that are often treated as one, namely finding noisy labels, estimating the size of the noise and improving the held-out score of the model. A method can perform the first task well and still fail at the third. **[FIGURE NEEDED: one-panel flow. Left: dataset with injected noise, or none. Middle: the hybrid built per fold on the training labels, beside the top two single representations. Right: similarity score, and the held-out score after each method for learning with noisy labels.]**

6.2.1 Similarity in a Hybrid

We use hybrids for similarity in two ways. The first is an ordinary hybrid, in which the search of Chapter 5 selects features that help a downstream model and distances are then computed on those features. The second is a similarity hybrid, which is produced by the same search but ranks each candidate by how accurately each molecule's activity is predicted from its five nearest neighbors in the hybrid. No downstream model is fitted during the search for a similarity hybrid. The two searches share their importance methods, allocation methods, budgets and folds, and differ only in the score that ranks candidates. The neighbor score also sets each representation's share of the budget, so no internal evaluator is used. For the budget split, this score is computed on a validation set drawn from each fold's training molecules, whereas the score that ranks whole candidates uses the fold's held-out molecules. Every representation in the dataset's set is a candidate, and the number of representations (2, 3, 5 or all) is not searched. Chapter 4 showed that the representation determines which molecules count as neighbors. The similarity

hybrid tests whether selecting features for the quality of the neighbors addresses this problem more directly than selecting them for prediction. **[FIGURE NEEDED: D6.1, two pipelines sharing a first half (base representations, importance, allocation, hybrid). Top: hybrid, downstream model, dataset metric. Bottom: hybrid, 5 nearest neighbors from other scaffold groups, distance-weighted mean activity, Spearman or rescaled AUROC. Caption: same search, different score.]**

As in Chapter 5, each feature is z-scored with the mean and standard deviation of the training molecules only. The distance is the Manhattan distance on the z-scored features, as defined in Chapter 4. Because every z-scored feature contributes one term of similar size to this distance, the number of features that a representation contributes to a hybrid acts as its weight. The influence of a representation on the distance is therefore roughly its number of features divided by the budget, so the allocation method also weights the representations within the distance. The neighbor score used by the search applies no importance weighting to the z-scored features. **[RESULT NEEDED: confirm Manhattan distance and one-over-distance neighbor weighting once the design experiment crossing distance, neighbor weighting and number of neighbors has run in full]**

To rank whole candidates, the score is computed on the held-out molecules of each outer fold, with other held-out molecules rather than training molecules serving as neighbors. The scaffold groups of the held-out molecules are split into up to 5 parts, or fewer when there are fewer groups. The 5 nearest neighbors of each molecule are drawn from the other parts and hence from other scaffold groups, because a neighbor with the same scaffold is an easy match that would inflate the score. Each neighbor is weighted by the inverse of its distance:

$$\hat{y}_i = \frac{\sum_{j \in N_5(i)} y_j / d_{ij}}{\sum_{j \in N_5(i)} 1 / d_{ij}}. \tag{6.1}$$

A neighbor at distance 0 receives all of the weight. On a regression dataset, the score is the Spearman correlation $\rho(\hat{\mathbf{y}}, \mathbf{y})$. On a binary classification dataset, $\hat{y}_i$ is the distance-weighted fraction of neighbors in class 1, and the score is $2(\text{AUROC} - 0.5)$, where AUROC is the area under the receiver operating characteristic curve. This rescaling places chance at 0 under both scores, which matters because the budget split divides by the score. The modelability index of Golbraikh et al. [118] (Chapter 4) is a related published measure, but it uses a single nearest neighbor and

a hard class match, whereas our score retains the graded fraction of neighbors. The score thus measures the extent to which the similar property principle (Chapter 2) holds in the hybrid.

A fold is not scored when it has fewer than 10 held-out molecules, fewer than 2 scaffold groups, or constant labels or predictions, and such a fold is omitted from the mean rather than counted as 0. Because AUROC is defined for two classes, multiclass labels are not scored either, which excludes the PAM50 breast cancer subtypes (4 classes) and the trastuzumab HER2 multiclass dataset (3 classes). The folds of the trastuzumab HER2 binary dataset are cut along edit-distance shells around the wild-type CDR-H3. A held-out block can therefore contain a single shell, in which case no molecule has a neighbor from another group. The similarity hybrid is not searched on these three datasets, but it is searched on every other dataset in the thesis, including the antibody and other non-molecule datasets.

Feature importance weighting. Beyond simply using the feature importance scores for allocation, we investigate whether feature weighting by importance during distance computation improves noise detection. For a feature importance vector w, weighted distances are computed by scaling the standardized features: $x' = x \odot w$ under Manhattan distance ($x \odot \sqrt{w}$ under Euclidean), with negative importances set to 0 and w rescaled to average 1.

6.2.2 Noise Conditions

The noise conditions are defined in Section 2.5.1 (Figure 2.3). Three of them, namely censoring, Gaussian noise and real labels with nothing injected, drive the three experiments on learning with noisy labels. Outlier and grouped-wider noise are used to grade detection, whereas grouped-shifted noise grades estimation only, because every training label moves under it. Noise is added only to the training labels of each outer fold, and the held-out labels are never changed. The size of the noise, the censoring limit and the scaffold families that grouped-wider noise widens are all computed from the clean training labels of that fold. The simulation of several sources per molecule with correlated errors is an exception to this rule. It sizes its noise from the standard deviation of the labels of all molecules, held-out molecules included, although its held-out labels remain clean.

Under censoring, the level is the fraction of training labels clipped at a detection

limit, with levels of 0, 0.10, 0.20, 0.25, 0.30 and 0.40. The survey and the detection results use the levels 0, 0.10, 0.25 and 0.40, with level 0 serving as the negative control. Because the clip involves no random draw, we run one draw per level. The resulting error is one-sided, a bias rather than a scatter, and both its direction and its location, the limit itself, are known. Censoring represents the range limits of assays, for example a value of "$> 10\,\mu$M" recorded as $10\,\mu$M [81]. It applies to regression only, because a class label has no value to clip.

Under Gaussian noise, the levels are multiples of the standard deviation of the clean training labels of the fold, namely 0, 0.2, 0.3, 0.5, 0.75, 1.0 and 1.5, with three noise draws per level. On a classification dataset, the level is instead the fraction of training labels flipped to the wrong class, namely 0, 0.05, 0.10, 0.20, 0.30 and 0.40. Gaussian noise represents the random error of a repeated measurement and is also the standard assumption in studies of noise robustness. It is the noise condition used in the survey of twelve methods for learning with noisy labels.

Outlier, grouped-wider and grouped-shifted noise are used only for detection and estimation, and their parameters are given in Section 2.5.1. Outlier noise represents annotation and transcription errors [14, 98]. We apply it with 1%, 5% and 10% of training molecules affected, which extends the single level of 10% used in Chapter 3. Grouped-wider and grouped-shifted noise represent differences between laboratories or sources [80]. Both grouped conditions require a scaffold for each molecule, so they are applied only to datasets with SMILES strings.

In the final condition, labels are used as they come and no noise is injected. Since no list of wrong labels exists, success can only be measured as a better held-out score after the training labels are cleaned. Such an improvement shows that cleaning helped the model, but it does not show that errors were found, because the cleaning may have removed difficult molecules instead. QM9 serves as a control, since its labels come from a quantum-chemical calculation with no experimental error [107]. A cleaning method that improves results on QM9 is therefore not removing measurement error. Table 6.1 lists, for each condition, what it represents, its levels, the experiments that use it and the datasets it covers. **[RESULT NEEDED: the datasets each condition covers in the full runs, for Table 6.1]**

6.2.3 Noise Mitigation Methods

In addition to evaluating hybrid representation quality for distance-based reasoning, our investigation evolved to explore alternative noise handling strategies. Initial experiments with sample removal methods revealed limitations when applied to Gaussian noise affecting all samples. This motivated a pivot toward sample weighting and label smoothing approaches, which do not require accurate binary classification of noisy versus clean samples but instead leverage continuous noise magnitude estimates.

Sample Removal Methods

Distance-based approaches rely on the assumption that similar molecules have similar properties [31]. Presumably, noisy data would break this pattern, making it a method for detecting anomalous data. k-Nearest neighbor filtering examines the variance of a sample's neighborhood. For sample x_i with k nearest neighbors $\mathcal{N}_k(x_i)$, we compute the standard deviation of neighbor labels:

$$\sigma_i = \text{std}(\{y_j : x_j \in \mathcal{N}_k(x_i)\}) \tag{6.2}$$

Samples with σ_i exceeding the 90th percentile are marked as potentially noisy. We test $k \in \{5, 10, 20\}$.

Local Outlier Factor (LOF) identifies anomalous samples by detecting local density estimates between a sample and its neighbors [297]. These density estimates are based on the distance to its k nearest neighbors. If a point's density is lower than that of its neighbors, its LOF is high. We flag the 10% of training molecules with the highest LOF, using 20 neighbors.

We also tested the following noise removal methods: distance-weighted prediction filtering, neighborhood consensus filtering, AC detection, Mahalanobis outlier detection, cross-validation disagreement, bootstrap ensemble filtering, co-teaching [141], and DivideMix [298].

Noise Magnitude Estimation

The central part of the noise mitigation work is the estimation of noise magnitude, which allows noisy labels to be handled in proportion to their error instead of being removed outright. We used three families of estimators, which are listed in Ta-

ble 6.1. Neighbor-based estimators compare the label of a molecule with the labels of its k nearest training neighbors, and model-based estimators use the disagreement between predictions made without the molecule. Probabilistic estimators read the noise from the predicted uncertainty of a model.

Estimator	Family	Score of molecule i
Neighbor residual	Neighbors	$\lvert y_i - \bar{y}_{\mathcal{N}} \rvert / \sigma_y$
Local variance ratio	Neighbors	$\lvert y_i - \bar{y}_{\mathcal{N}} \rvert / \max(s_{\mathcal{N}}, c)$, with $c = \max(0.01\,\sigma_y, 10^{-3})$ so that a neighborhood sharing one label gives a finite score
AC-aware	Neighbors	$\frac{\lvert y_i - \bar{y}_{\mathcal{N}} \rvert}{\sigma_y} \cdot \frac{1}{1 + s_{\mathcal{N}}/\sigma_y}$: a large deviation among neighbors that agree suggests noise; among neighbors that disagree, an activity cliff [73]
Cross-validation residual	Model	Absolute residual of the out-of-fold prediction
Ensemble disagreement	Model	Standard deviation of several models' out-of-fold predictions, plus the absolute residual of their mean
Gaussian process	Probabilistic	Posterior variance, with a learned noise level [66]
Heteroscedastic network	Probabilistic	Predicted variance $\sigma^2(x)$, trained with the β-NLL loss of Equation 6.3 [299]
Conformal prediction	Probabilistic	Width of the split-conformal prediction interval [300]
Deep ensemble	Probabilistic	M networks: variance of their means (epistemic) plus mean of their variances (aleatoric) [69]
Bayesian random forest	Probabilistic	Standard deviation of the trees' predictions, plus half the absolute out-of-fold residual [301]
Quantile regression	Probabilistic	Width between the fitted quantiles
NGBoost	Probabilistic	Predicted standard deviation σ [58]

Table 6.1: Noise magnitude estimators by family. The last column gives the noise score of molecule i, where $\bar{y}_{\mathcal{N}}$ and $s_{\mathcal{N}}$ are the mean and standard deviation of the labels of its k nearest training neighbors, σ_y is the standard deviation of all training labels and M is the number of networks in the deep ensemble. A higher score marks a label as more likely to be noisy.

The heteroscedastic network predicts a mean $\mu(x)$ and a variance $\sigma^2(x)$ and is trained with

$$\mathcal{L} = \frac{1}{2}\left(\frac{(y-\mu)^2}{\sigma^2} + \log \sigma^2\right) \cdot \sigma^{2\beta}, \tag{6.3}$$

where $\beta = 0.5$ prevents the variance from hiding the mean errors.

NGBoost in particular was selected due to its promising uncertainty values with respect to the correlation between noise and prediction error, as discussed in Section 3.7.

Label Smoothing and Sample Weighting

Because all labels contain some noise, we focused on label smoothing in preference to removal. In addition to modifying labels, we gave more weight to trustworthy samples during training. Both approaches use the noise score after min-max normalization to $[0, 1]$, written $\tilde{s}_i$, and the rules are listed in Table 6.2.

Rule	Changes	New label or weight of molecule i
Proportional	Label	$y_i + \alpha\,\tilde{s}_i\,(\text{target} - y_i)$
Threshold	Label	Only above the $100(1-\alpha)$th percentile of $\tilde{s}$: $(1-\alpha)\,y_i + \alpha \cdot \text{target}$, so α sets both how many labels move and how far
Soft threshold	Label	Correction strength $\alpha \cdot \text{sigmoid}\big(5(\tilde{s}_i - 0.5)\big)$
Confidence	Label	$(1-c_i)\,y_i + c_i\,\mu_i$, with $c_i = \alpha\sigma_i/(1+\alpha\sigma_i)$; for estimators that output σ, such as NGBoost
Inverse	Weight	$1/(1+\alpha\,\tilde{s}_i)$
Exponential	Weight	$\exp(-\alpha\,\tilde{s}_i)$
Soft	Weight	$1 - \alpha\,\tilde{s}_i^2$, clipped to $[0.1, 1]$

Table 6.2: Label smoothing and sample weighting rules. $\tilde{s}_i$ is the noise score of molecule i scaled to $[0, 1]$, α sets the strength, the target is the mean label of the molecule's nearest neighbors, and μ_i and σ_i are the mean and standard deviation predicted by a probabilistic model. Weights are rescaled to a mean of 1.

Recovery Rate

For noise detection accuracy, we compute precision, recall, and F1 score, treating noise injection as ground truth.

Each downstream model m is trained and tested three times, once on the clean training labels, once on the noisy labels and once on the labels after noise estimation and correction. These runs give the scores s^m_{clean}, s^m_{noisy} and s^m_{denoised}, respectively, in the dataset's own metric and averaged over folds. The recovery rate is the share of the lost performance that correction recovers,

$$\text{Recovery}_m = 100 \times \frac{s^m_{\text{denoised}} - s^m_{\text{noisy}}}{s^m_{\text{clean}} - s^m_{\text{noisy}}}\ \%, \tag{6.4}$$

which reads the same way whether a higher or a lower value of the metric is better. Each model is normalized by its own gap between the clean and the noisy score. When that gap is no more than 2% of the clean score, the noise did too little damage to be recovered, and the recovery rate is left undefined. A recovery rate of 100% indicates complete restoration of clean performance, and negative values indicate

that the corrected labels gave a worse score than the noisy labels. The exploratory study reused in Chapter 4 computed the same ratio in R^2, with one random forest on ECFP4 as the only downstream model.

6.3 Results: Similarity

6.4 Results: Noise Detection and Estimation

6.5 Results: Noise Mitigation by Noise Type

6.5.1 Censoring: Correction of Clipped Labels

6.5.2 Gaussian Noise: Methods for Learning with Noisy Labels

6.5.3 Real Labels: Cleaning without Injected Noise

6.5.4 Comparison across Noise Types

6.6 Conclusions

Chapter 7

Discussion and Conclusions

Real assay labels are not noisy in the way most studies assume. Measurements pooled across laboratories carry an offset shared by all values from one laboratory [80]. Labels near the detection limit of an assay are clipped to that limit [81], as described in "Noise in molecular data" in Chapter 2. Gaussian noise, the usual assumption, is also the hardest case to correct, because every label is off by an unknown amount in an unknown direction ("Comparison across noise types", Chapter 6). Cleaning such labels can even leave a model worse than the uncorrected labels would. Before choosing a method, we therefore ask whether the labels were pooled from many laboratories, measured in one controlled experiment or taken from an unknown source ("Comparison across noise types", Chapter 6).

This chapter summarizes each chapter, argues that no single method fits every dataset and that simple methods are often better, and closes with limitations, future directions and conclusions.

7.1 Summary

QSAR models are trained on measured labels that carry experimental noise, and most work on label noise in QSAR assumes that this noise is Gaussian. Chapter 2 defined the representations, models and noise types used throughout, the NoiseInject package that injects the noise, and the procedure by which models were evaluated.

Chapter 3 asked how the representation, the model and the type of label noise shape the performance of a QSAR model, and whether the predicted uncertainty of the model identifies the noisy labels. For accuracy, the model and the representation explained comparable shares of the variance in R^2. On the QM9 HOMO–LUMO

gap, with Gaussian noise as wide as the clean training labels, the model explained 29.1% of this variance and the representation 26.6%. For robustness, measured as $\mathrm{AUC}_{\mathrm{norm}}$, the model dominated, explaining 49.5% of the variance under Gaussian noise against 9.2% for the representation. Changing the shape of the noise barely changed performance, although grouped-shifted and censored noise cost the most. Censoring was run on five pairings of model and representation only, and over these five pairings the median $\mathrm{AUC}_{\mathrm{norm}}$ was 0.813 under censoring and 0.951 under Gaussian noise. Mean predicted uncertainty rose with the amount of noise, but the per-molecule uncertainty of no model identified the corrupted labels. Under six of the seven noise conditions, the Spearman correlation between the predicted uncertainty of a molecule and its injected noise stayed between -0.16 and 0.16. Under censoring, the models were most confident on the clipped labels.

Chapter 4 asked whether standard representations, and the distances computed over them, fail on a given dataset. On any one dataset, many features of a general-purpose fingerprint do not vary or do not relate to the property, yet a distance sums over all of them, and some representations also give different molecules identical feature vectors. In the exploratory study of learning with noisy labels, the configurations that used a distance between molecules did worst, and judging a label by the labels of its neighbors usually made the model worse. A representation that avoids these flaws would consist of a small number of features chosen for the task, drawn from several representations and traceable to their source.

Chapter 5 asked how to select the best set of features for a given prediction task. The adaptive hybrid representation chooses features from several representations for one task under a budget in features, and every choice is made inside each fold from the training labels of that fold only. The hybrid beat the best single representation on **[RESULT NEEDED: which datasets, with the score pair in each dataset's own metric]**, and did not on **[RESULT NEEDED: which datasets, with the score pair]**. Across datasets, the preferred search settings **[RESULT NEEDED: whether different datasets preferred different search settings]**. **[RESULT NEEDED: transfer result, one line.]** **[RESULT NEEDED: antibody result, one line.]** **[RESULT NEEDED: separate-site result, one line.]** Importance weighting **[RESULT NEEDED: whether it helped during hybrid construction and during model training, in each dataset's own metric]**. The limits of the method are set out in the Limitations section of Chapter 5.

Chapter 6 asked whether a representation chosen for the task repairs

nearest-neighbor prediction, noise detection and learning with noisy labels. For nearest-neighbor prediction, **[RESULT NEEDED: answer]**. For noise detection, **[RESULT NEEDED: answer]**. For learning with noisy labels, **[RESULT NEEDED: answer, and under which noise types any LNL method helped]**.

Chapter 3 tells a reader which model to choose when labels are noisy, and Chapters 4 to 6 tell them which features to give that model. What works depends on the dataset, as the next section argues. The contributions of this thesis are the NoiseInject package, the adaptive hybrid representation and its blueprints, and evidence that distance-based noise handling on standard representations usually made models worse. The resulting papers and software are listed in the Summary of Research Contributions.

7.2 Dataset Dependence of the Best Method

The settings of the hybrid search are the downstream model, the budget, the number of base representations, the importance method, the allocation method and the internal evaluator. No single value of any of these settings was best on every dataset **[RESULT NEEDED: confirm on the full runs]**. The same holds for label noise, since which LNL method helps, if any, depends on the data **[RESULT NEEDED: confirm on the full runs]**. Within the LNL methods, noise mitigation, the subset that changes the labels themselves, is the harder part (Chapter 6). We therefore report no overall winner averaged across datasets.

The best choice depends on at least five factors. The first two are the type of label noise ("Comparison across noise types", Chapter 6) and its amount. The third is the number of molecules, because a small dataset leaves few molecules inside a fold on which to score each setting, which makes the chosen setting less stable. The size threshold is given in the Limitations section of Chapter 5. The fourth is whether any feature describes the property at all, since the search cannot add signal that its inputs do not carry. The fifth is the downstream model. The best hybrid for a tree ensemble need not be the best for a neural network, so the internal evaluator is searched separately for each downstream model.

The evidence is spread across the thesis. In Chapter 3, the representation with the highest $\mathrm{AUC}_{\mathrm{norm}}$ for a model changed between Gaussian and grouped-shifted noise for five of the thirteen base models on QM9. Model ranks also moved with the

amount of noise. On ECFP4 under Gaussian noise, for instance, random forest went from mid-ranking on clean labels to the top at higher noise levels. Tree and kernel models kept a similar AUC_{norm} on every representation, whereas the AUC_{norm} of the plain and Bayesian networks and of NGBoost depended on the representation. On the three assay datasets, XGBoost and LightGBM ranked lower for robustness than on QM9. In Chapter 5, the winning allocation method, number of base representations, importance method and budget were **[RESULT NEEDED: per dataset]**, and the winning internal evaluator was **[RESULT NEEDED: per dataset and downstream model]**. A fixed share of features per representation won on **[RESULT NEEDED: which datasets]**, and one pooled ranking across all representations won on **[RESULT NEEDED: which datasets]**. In Chapter 6, under censoring, **[RESULT NEEDED: whether raising the clipped labels helped, per dataset and downstream model]**. Under Gaussian noise, the number of the twelve LNL methods that beat the plain network **[RESULT NEEDED: whether it changed with the amount of noise and with the input]**. The LNL methods that helped, including the noise-mitigation ones, were **[RESULT NEEDED: per noise type]**.

The thesis therefore offers neither a fixed recipe nor a new architecture tuned to one problem. Instead, it offers a set of methods that might work and a procedure that gives each supervised task its best chance by choosing the features for that task. For a practitioner, there is consequently no default to trust, and the useful question is which settings to try and how to compare them fairly.

7.2.1 Simple Models and Small Feature Sets

On QM9 under Gaussian noise, the forests, boosted trees, SVM and the GP were both accurate on clean labels and robust to noise. The four plain and Bayesian networks were among the most accurate models on clean labels but also the four least robust, with AUC_{norm} values from 0.881 to 0.897 on ECFP4. Their AUC_{norm} varied more across the six representations than that of any other model. Under label noise, the simpler choice is therefore tree ensembles and SVM rather than plain neural networks, while the VBLL networks sat in the middle, at AUC_{norm} values of 0.924 and 0.926. Robustness is nevertheless not the same as accuracy. NGBoost had the highest AUC_{norm} on all six representations under Gaussian noise on QM9, yet the lowest clean R^2 on four of the six. Rankings also did not carry over unchanged

to the assay datasets. NGBoost ranked first on QM9 on every representation but sixth to ninth on hERG K_i, whereas random forest ranked between first and seventh on every assay dataset and representation and rose in rank as noise grew.

The hybrid search sweeps the budget in features and so makes no assumption that more features are better. The best hybrid used fewer features than the largest budget in **[RESULT NEEDED: how often, per dataset and downstream model]**. An ensemble of single-representation models **[RESULT NEEDED: whether it matched the hybrid at lower cost, per dataset]**. We also predicted the activity of each molecule from its five nearest neighbors, comparing a small hybrid chosen for the task with the top two single representations of each downstream model. The hybrid scored **[RESULT NEEDED: per dataset, the score pair in the dataset's own metric]**. In this comparison the model stays simple, and any gain comes from the input.

Several kinds of LNL method did not help. In the exploratory study, the methods that used a distance between molecules did worst (Section 4.4.1), and neighbor-based label cleaning usually made the model worse (Section 4.4.2). Chapter 6 compared twelve LNL methods with one plain neural network under added Gaussian noise ("Gaussian noise: methods for learning with noisy labels"). At noise levels of 0.5 and 1.5 times the standard deviation of the labels, **[RESULT NEEDED: how many of the twelve beat the plain network, per dataset, on each model's top two single representations and on the hybrid]**. On real labels with no injected noise, **[RESULT NEEDED: per dataset, whether any cleaning method scored better than leaving the labels as they came]** ("Real labels: cleaning without injected noise"). Labels clipped at a detection limit are a separate case, in which correcting the labels helped **[RESULT NEEDED: confirm on the full runs]**. For data held at separate sites with different noise levels, removing flagged molecules **[RESULT NEEDED: whether it improved on pooling the sites]**.

LNL methods from other fields carry over poorly to assay data, for several reasons. Most were built and tested on large image datasets with flipped class labels [100]. Assay datasets, by contrast, are small, with 1,415 to 5,039 molecules in each of the three used in Chapter 3, and their labels are continuous, with some error in every label. Many LNL methods rely on the neighbors of a sample, or on how easily the sample is learned. Both require a space in which close molecules have similar activity, and Chapter 4 shows that standard representations do not provide one. On medium-sized tabular data, tree models still outperform deep networks,

partly because uninformative features hurt networks more [281]. Financial price data offer a useful contrast. Ma et al. [302] treat labels built from price movements as noisy throughout, because it cannot be known whether a price rose from signal or from noise. They replace those labels with labels from a denoising autoencoder and report better downstream classification, although their labels remain classes that record whether the price went up or down.

A method tuned closely to one dataset or task loses generality. The tasks in this thesis span binding affinity, ADME, quantum-chemical properties and antibodies, and this range is the reason learned models are used at all. Simple methods do not generalize automatically either, since feature selection needs tuning for each dataset, which is why the search sweeps its settings. The search itself can also overfit (Discussion section of Chapter 5). The balance is to fix the simple, robust parts, such as a tree ensemble or nearest neighbors on a small feature set, and to search only the choices that vary by dataset.

7.3 Limitations

The limits of the hybrid search itself, including its run time, repeating the search and reproducibility, are set out in the Limitations section of Chapter 5. This thesis set out to mitigate label noise, and our conclusion is that correcting label noise without ground truth is a hard problem with no clear general solution. Every experimental label carries an unknown amount of noise, so there is no clean copy against which to check a correction, and finding a noisy label is easier than correcting it ("Comparison across noise types", Chapter 6). What remains possible is outlier detection and the detection of systematic errors with a known structure, such as a shift shared by all labels from one source or clipping at a detection limit. In the assay experiments, "clean" means that no noise was added rather than that the labels are true, and only QM9 is close to noise-free. The injected noise conditions are also only models of noise. Real data mix them in unknown amounts, so a method that works under one condition may fail on real data. **[RESULT NEEDED: per noise condition, how many LNL methods, and how many noise-mitigation methods, improved on the uncorrected model, and where recovery went negative.]**

Noise mitigation estimates the error of each label and writes the correction into the data before training, so a wrong estimate stays in the data for every later model. LNL methods that change training instead adapt throughout training and use all

the data [100]. Noise mitigation is cheaper, since it is done once and its output can be reused by any model. Correcting labels directly is justified in two cases. The first is noise with a known direction, as in censoring, where both the limit and the direction are known. The second is a clearly separate subset of labels, such as gross outliers or an offset tied to one source. Otherwise, we recommend LNL methods that change training. Either approach can be paired with uncertainty estimation where that estimation is reliable. The overall level of predicted uncertainty rises with the amount of noise (Chapter 3), but per-molecule uncertainty does not identify which labels are noisy. Uncertainty should therefore be used to judge how noisy a dataset is rather than which of its labels are wrong.

Other limitations concern the scope of the experiments. The label-noise experiments are almost all regression tasks, and only the survey of twelve LNL methods flips class labels, uniformly at random, so molecule-dependent flips are the first next step. The controlled label-noise experiments use QM9 and three small assay datasets, LogD, Caco-2 and hERG K_i. No dataset here has repeated measurements with a known true value for every molecule, which a direct test of label correction would require. The available compute limited how many noise conditions, noise levels, folds and datasets could be crossed. Results are reported as per-fold scores and their spread over 3 to 5 folds, with no significance tests, so some patterns indicate a direction only. **[RESULT NEEDED: final number of folds and number of datasets per chapter.]**

7.4 Future Directions

The first next step is classification. The label-noise experiments in this thesis are almost all regression tasks, in which a noisy label is a shifted value. The exception is the survey of twelve LNL methods in Chapter 6, which flips a fraction of class labels uniformly at random on its classification datasets. In classification, a noisy label is a flipped class. Many QSAR class labels are obtained by cutting a continuous value at a threshold, so flips concentrate near the cut-off and the noise depends on the molecule. NoiseInject already provides six flipping strategies (Section 2.6.1), and the hybrid search already handles classification. However, the molecule-dependent flipping strategies have not yet been studied at the scale of the regression study. Most LNL methods that change training were designed for flipped labels, so whether the regression findings carry over remains an open question. One restriction applies to

nearest-neighbor prediction, which is scored by AUROC. This metric is defined for two classes, so it gives no score with more than two classes or on a held-out block that contains only one class.

Several kinds of input and importance method are not yet part of the search. The missing inputs are transformed features, such as continuous features that have been normalized or binarized, and features derived from distances in the molecular graph. The missing importance methods are those that treat the model as a black box. The software already computes KernelSHAP and permutation importance, but the search sweeps neither, and importance measured by dropping one feature and refitting is not implemented. The search objective could also reward robustness to label noise in addition to accuracy, although no such objective has been built.

Chapter 6 compares LNL methods under censoring, Gaussian noise and real labels. Outliers and grouped offsets, in which all labels from one laboratory or source document are shifted together, are used there only to grade noise detection. Two questions therefore remain open for these noise types. The first is whether cleaning labels using the nearest neighbors of each molecule, which fails under Gaussian noise, does better on outliers, and the second is whether label smoothing and sample weighting still perform best.

Hybrid features could also serve as inputs to generative models, for example as the input of the model that scores molecules for a molecule generator. It is not clear what the search should optimize for that task, since held-out accuracy on known molecules may not reward the right properties in new ones. This use also has a cost, because the hybrid features of every generated molecule must be computed, including a pass through any embedding model in the hybrid.

In its current form, the full search is too computationally expensive for routine use. Two ways of reducing its cost already exist, namely the fast search tier, which sweeps shorter lists of values (Table 5.7), and adaptive pruning, which drops configurations early. The next step is a warm start, which begins from the blueprint of a similar dataset and searches near it. Chapter 5 already reuses the search settings of one dataset on a second dataset without any search, whereas a warm start would search near those settings. A faster search must still be able to reach every configuration and should only visit fewer of them. **[RESULT NEEDED: hours and number of model fits for the full search against the fast tier, per dataset.]**

Three further, smaller directions remain open. The first is whether the hybrid improves a weak new embedding as well as absorbing a strong one. This would

be tested by scoring the hybrid that includes the embedding against the embedding alone with its best downstream model. The same hybrid would also be scored against a hybrid in which a random matrix of the same width replaces the embedding. The second is the transfer of blueprints across domains. Because the representations differ between domains, individual features cannot transfer, so such transfer would have to work at the level of representation families. The third concerns data held at separate sites, with which Chapter 5 used the hybrid only indirectly. The next step there is a fuller federated-learning deployment with Lhasa, in which real sites run the feature selection on their own data [288].

7.5 Conclusions

This thesis set out to predict molecular properties from labels that carry experimental noise. Its first half asked what makes a QSAR model robust to that noise. Model architecture matters more than representation for how much performance a model keeps as noise rises, whereas the shape of the noise distribution matters little. Noise shared within a scaffold family and labels clipped at an assay limit do the most damage. Predicted uncertainty rises with the amount of noise but does not identify which labels are corrupted.

The second half began from a negative result, since methods that judge a label by its neighbors usually made models worse on standard representations. On a given dataset, most features of a general-purpose representation do not vary or do not relate to the property, yet a distance counts them all (Chapter 4). The adaptive hybrid representation instead selects features for the task from several representations under a budget in features. **[RESULT NEEDED: one sentence, what Chapter 5 found.] [RESULT NEEDED: one sentence, what Chapter 6 found.]**

Both halves lead to the same advice. Before choosing a model, a representation or an LNL method, a practitioner should ask where the labels came from and what shape their noise is likely to have. They should then choose the features for the task rather than take one representation whole. Because no single choice was best on every dataset, the choices that vary by dataset should be searched and compared fairly.

This matters in practice because representations keep changing, with a new foundation model appearing every few months in molecular, protein and antibody mod-

eling. With the hybrid search, a researcher need not decide whether to replace their current representation or retrain their pipeline. Instead, they compute the embeddings of the new model, add them to the representations searched and re-run the search, which then decides whether the new embedding contributes features and, if so, how many and which. The choice of representation is often set by familiarity or laboratory convention. The search replaces that choice with one measured on each task, and the measurement can be repeated as new representations appear.

Summary of Research Contributions

Research Papers

In preparation: Punt, A., Morris, G., Werner, S., & Hanser, T. (2026). Improving noise prediction robustness and mitigation with feature-importance hybrid representations in multi-modal contexts
I am finishing a draft and will extend the experiments in Chapters 5 and 6 within the next few months.

Conference Presentations

Punt, A., Morris, G., Werner, S., & Hanser, T. (2024). Investigating the impact and mitigation of noise in QSAR models. Posted presented at the Autumn UK-QSAR 2024 Meeting.

Blogposts

I have contributed to the *Blopig* blog of the OPIG:

- Are you addicted to dopamine?
- Demystifying Git and merge conflicts
- AI in Academic Writing: Ethical Considerations and Best Practices
- Memory-mapped files for efficient data processing
- Exploring multilingual programming

Appendix A

Datasets and curation

This appendix records where each dataset came from, what its label measures, and every curation step applied to it, in the order the steps run. It covers the four datasets of Chapter 3, the 16 datasets of Chapter 5, and the further sets used only for the separate-site experiments of Chapter 5 and in Chapter 6. The label, metric and fold grouping of each of the 16 datasets are summarized in Table 5.9, and the splitting rules are given in the Datasets and splitting subsection of Chapter 5; neither is repeated here. Table A.1 gives the number of samples in each release and after curation. All counts were obtained by running the curation code on the downloaded files.

A.1 Small Molecules

Three steps are shared by several small-molecule datasets. The first is structure standardization with RDKit [32], which applies a general cleanup, keeps the largest fragment to remove salts and solvents, neutralizes charges and selects the canonical tautomer, in that order. Any molecule that then contains a metal atom is rejected, and the result is written as canonical SMILES. Standardization is applied to hERG K_i, LogD and Caco-2. The Biogen sets, QM9 and the hERG blocker sets are only checked to parse in RDKit.

The second shared step removes duplicate molecules by InChIKey [303], keeping the first occurrence. Two different SMILES strings can describe the same molecule, and without this step one molecule could sit in both a training fold and a test fold. The step is applied to hERG K_i, LogD, Caco-2, the two Biogen sets and QM9. The hERG blocker sets are deduplicated by InChIKey only against the other sites, as

Dataset	Released	Used	Steps that remove samples
Small molecules			
hERG K_i	1,815	1,412	no pChEMBL value (3); not a binding assay (75); repeats merged into one median per compound (322); InChIKey duplicates (3)
LogD	5,326	5,028	no value (287); repeats merged after standardization (11)
Caco-2 efflux	5,326	2,158	no value (3,165); repeats merged after standardization (3)
Rat liver microsome clearance	3,521	3,053	no value (467); InChIKey duplicates (1)
MDR1–MDCK efflux	3,521	2,641	no value (879); InChIKey duplicates (1)
QM9	130,831	9,978	not parsable by RDKit (1,403); random draw of 10,000; InChIKey duplicates (22)
hERG blockers, training set	7,772	7,606	also in the hERG K_i set (166)
hERG blockers, test set	4,383	4,213	also in the hERG K_i set (170)
hERG IC_{50}	12,010	12,008	missing or non-positive value (2)
Chemical reactions			
Buchwald–Hartwig yield	3,955	3,955	none
Buchwald–Hartwig plates	4,599	4,312	no aryl halide (287)
Antibodies			
Koenig expression	4,275	4,275	none
Nanobody thermostability	672	672	none
Humanness	564	564	none
Trastuzumab–HER2 binding	524,346	5,000	stratified random draw of 5,000
Nucleotide sequences			
H3K4me3	18,244	18,244	none (training and test files merged)
TATA promoter	5,274	5,274	none (training and test files merged)
Cancer genomics (TCGA)			
Lung adenocarcinoma vs squamous	1,129	1,017	not a primary tumor (112)
Breast cancer PAM50 subtype	1,097	821	no PAM50 call, or called normal-like (276); count is of primary tumors

Table A.1: Sample counts of each dataset before and after curation, grouped by domain. Released is the number of rows in the downloaded files, or of primary tumors with expression data for breast cancer, and Used is the number left after every curation step. The last column lists the steps that remove samples, with the number removed where it applies.

described below. The step is intentionally not applied to hERG IC_{50}, which keeps one row per measurement. The third shared step is the fold grouping by Bemis–Murcko scaffold [304], whose form in the hybrid experiments is given in the Datasets and splitting subsection of Chapter 5.

hERG K_i. Records for the hERG potassium channel (ChEMBL target CHEMBL240) were retrieved from ChEMBL [110] through its web interface. The query keeps records with standard type K_i, a pChEMBL value present, an exact relation ("="), and no curator data-validity comment. Records reported as "<" or ">" a value are therefore never included. Only binding assays (ChEMBL assay type B) are kept. The label is the pChEMBL value, $\mathrm{p}K_i = -\log_{10} K_i$ with K_i in molar units. Repeat measurements of one compound are merged into their median, and compounds whose repeats have a standard deviation above 1.0 $\mathrm{p}K_i$ unit are removed, following Landrum and Riniker [16]. The structures are then standardized and deduplicated by InChIKey. Because repeats are merged on ChEMBL's own SMILES before standardization, two ChEMBL entries that standardize to the same molecule are not merged by median; the InChIKey step keeps the first of them. In a query run in September 2026, 1,815 K_i records reduce to 1,812 with a pChEMBL value and 1,737 from binding assays. These give 1,415 compounds, 226 of them measured more than once and none removed by the standard-deviation rule. The InChIKey step removes 3 more, leaving 1,412 compounds with $\mathrm{p}K_i$ from 4.07 to 9.41. The file read by the experiments was pulled on 1 February 2026; the ChEMBL release was not recorded at the time. Re-running the same query in September 2026 against ChEMBL release 37 gives a byte-identical file of 1,415 compounds.

LogD and Caco-2 efflux. Both endpoints come from the training split of the OpenADMET–ExpansionRx challenge data, released on Hugging Face under a CC-BY-4.0 license [109]. The file holds 5,326 molecules; 5,039 have a LogD value and 2,161 a Caco-2 efflux ratio. For each endpoint, rows without a value are dropped and the structures are standardized. Rows that standardize to the same SMILES are merged into their median label. This merges 11 LogD pairs and 3 Caco-2 pairs. The efflux ratio, which has no unit, is then transformed to $\log_{10}$. Labels are clipped at the 1st and 99th percentiles of each dataset, which are -1.0 and 4.5 for LogD and -0.319 and 1.679 for the $\log_{10}$ efflux ratio. Clipping sets 116 LogD labels (54 low, 62 high) and 45 Caco-2 labels (23 low, 22 high) to a limit. The extreme labels

of both sets are therefore flattened before any noise is added in Chapter 6. The InChIKey step removes nothing on either set, leaving 5,028 and 2,158 molecules.

Rat liver microsome clearance and MDR1–MDCK efflux. These are two endpoints of the Biogen ADME set of 3,521 compounds [211], downloaded from Polaris as `biogen/adme-fang-v1` [212]. Rat liver microsome clearance is the intrinsic clearance CL_{int} in mL/min/kg, and the MDR1–MDCK endpoint is an efflux ratio. Both are released already as $\log_{10}$ values and are used as released. Rows without a value, or whose SMILES RDKit cannot parse, are dropped, and duplicates are removed by InChIKey. There is no standardization, no median merge and no clipping. Clearance has 3,054 values and efflux 2,642; the InChIKey step removes one molecule from each, leaving 3,053 and 2,641.

QM9. QM9 [107, 213] was read through PyTorch Geometric [112], which distributes 130,831 molecules. Of these, 129,428 parse in RDKit; the other 1,403 carry an over-valent carbon atom in the released structure file. This list of parsable molecules is identical to the one used in Chapter 3. From it, 10,000 molecules were drawn once at random with seed 42, so the subsample is a random draw of molecules, not a random split. The InChIKey step removes 22 duplicates, leaving 9,978. The label is the HOMO–LUMO gap in eV, from 1.88 to 11.37 eV in this sample.

hERG blockers held at separate sites. The separate-site experiments of Chapter 5 use two further hERG sets from Lhasa Limited, a training set of 7,772 molecules (3,630 blockers, 46.7%) and a test set of 4,383 molecules (1,084 blockers). Both were provided through the Effiris and FLuID collaboration [18, 19]. Each label is Lhasa's own binary blocker call, and the two sets were labeled under different rules. The molecules are stored as RDKit molecule objects and converted to SMILES. Entries that give no valid SMILES are dropped, which removes none, and neither set is standardized. The sets are proprietary and cannot be redistributed (Appendix B). In the same experiments the hERG K_i set acts as a third site. Its labels are made binary, with a compound counted as a blocker when $pK_i \geq 6.0$ ($K_i \leq 1\,\mu M$), which gives 467 blockers and 945 non-blockers. Molecules are then matched by InChIKey across the three sites, taken in the order hERG K_i, training set, test set, and only the first copy is kept. This removes 166 molecules from the training set and 170 from the test set, all of them also in the hERG K_i set, leaving 7,606 and 4,213. The two

Lhasa sets share no molecule. For 56 of the 166 and 57 of the 170 shared molecules, the Lhasa blocker call disagrees with the call from pK_i. The training and test sets fall into 4,002 and 2,298 scaffold groups before this step, and 3,903 and 2,222 after it.

hERG IC_{50}. This set is used only in Chapter 6, where the disagreement between repeat measurements and the curator flags serve as real evidence of label error. It keeps every measurement as its own row. The query takes ChEMBL records for CHEMBL240 with standard type IC_{50} in nM and an exact relation [110]. Records with a missing or non-positive value are dropped, and the label is $pIC_{50} = 9 - \log_{10}(IC_{50}/\text{nM})$. No filter on assay type is applied. Of the 12,010 records, 10,219 come from binding assays, 1,543 from toxicity assays, 217 from functional assays and 31 from ADME assays, as ChEMBL classifies them. Records with a curator data-validity comment are kept and marked; there are 301 of them, on 215 molecules. There is no standardization, no median merge and no InChIKey step. The result is 12,008 measurements of 9,802 molecules. Folds are grouped by scaffold, and every measurement of a molecule shares that molecule's scaffold, so no molecule is split between training and test.

A.2 The Chapter 3 versions of LogD, Caco-2, hERG K_i and QM9

The datasets used in this study are publicly available. The QM9 dataset is available on Figshare [213] and is also redistributed through the OpenADMET benchmark suite [107, 109]. The LogD and Caco-2 datasets are available in the OpenADMET repository [109]. The hERG K_i dataset was derived from ChEMBL [110] following the curation protocol described in Section 2, and can be regenerated from ChEMBL using that protocol.

Chapter 3 used the same sources as Chapter 5 for LogD, Caco-2 and hERG K_i, but a lighter curation. The differences are listed in Table A.2. They explain why the sample counts differ between the two chapters. The QM9 experiments of Chapter 3 draw ten replicates of 10,000 molecules from the same 129,428 parsable molecules. Each replicate has its own seed and its own 80/10/10 scaffold split into 8,000, 1,000 and 1,000 molecules. Chapter 3 also merges hERG K_i repeats a second time, on its

own standardized SMILES; on this file that merges nothing, so its set keeps 1,415 compounds.

	Chapter 3	Chapter 5
Structure standardization	largest fragment and canonical SMILES only	cleanup, largest fragment, neutralized charges, canonical tautomer, metals rejected
Label clipping (LogD, Caco-2)	none	1st and 99th percentile
InChIKey deduplication	none	first occurrence kept
Scaffold key	stereochemistry removed; each acyclic molecule its own group	stereochemistry kept; all acyclic molecules one group
Molecules: LogD, Caco-2, hERG K_i	5,039; 2,161; 1,415	5,028; 2,158; 1,412
Scaffold groups: LogD, Caco-2, hERG K_i	2,245; 1,038; 749	2,413; 1,086; 769

Table A.2: Curation of LogD, Caco-2 and hERG K_i in Chapters 3 and 5. The last two rows give the numbers of molecules and of Bemis–Murcko scaffold groups, in the order LogD, Caco-2, hERG K_i.

A.3 Chemical Reactions

Both reaction datasets come from the high-throughput Buchwald–Hartwig C–N coupling experiments of Ahneman et al. [214], in which aryl halide, palladium ligand, base and additive were varied on reaction plates. The label is the reaction yield in %, from 0 to 100. Here one sample is a whole reaction, written as its reactants and product, so the representations and the grouping differ from those of the single-molecule sets.

Buchwald–Hartwig yield. This is the version released with Schwaller et al. [305], taken from the first cross-validation sheet of the input workbook in the public repository of that paper. All 16 sheets of the workbook hold the same 3,955 reactions. Each reaction is written as aryl halide, 4-methylaniline, ligand, base and additive going to a product. The product is generated by applying a C–N coupling template to the aryl halide. A reaction would be dropped if the template failed, which it never does, so all 3,955 reactions are kept. They cover 15 aryl halides, 4 ligands, 3

bases and 22 additives, and fall into 22 additive groups of 178 to 180 reactions.

Buchwald–Hartwig plates. This is the data table of the public code repository of Ahneman et al. [214], which holds 4,599 rows over three plates. Its product SMILES are taken from the file. The 287 rows without an aryl halide are dropped. The 180 remaining rows without an additive are kept and form their own group, so the set has 24 additive groups of 177 to 180 reactions. The 4,312 reactions kept are not independent of the first set. Every one of the 3,955 reactions in the first set appears here, with the same aryl halide, ligand, base, additive and yield. The plates set adds 357 reactions, namely the 180 without an additive and 177 with 5-phenyl-1,2,4-oxadiazole as the additive.

A.4 Antibodies

The Koenig and nanobody sets were downloaded as CSV files from the FLAb benchmark repository [216]. In both, rows with a missing heavy chain, a heavy chain shorter than 11 residues, or a missing or non-numeric label are dropped; no row is removed by this rule. The humanness set was downloaded from the BioPhi publication repository and the trastuzumab set from its own repository. The FLAb and BioPhi files are taken from each repository's main branch, with no commit pinned. Each file has been changed only once. The Koenig and nanobody files were changed in FLAb commits `c41e56e` and `005e6c1`, both on 6 August 2025, and the two humanness files in BioPhi commit `c0797f2` on 24 August 2021. The local copies of the Koenig and humanness files are identical to those commits.

The nanobody and humanness sets are grouped by sequence identity. The identity of two sequences a and b is

$$\mathrm{id}(a, b) = 1 - \frac{\mathrm{lev}(a, b)}{\max(|a|, |b|)}, \tag{A.1}$$

where lev is the Levenshtein (edit) distance and $|a|$ is the length in residues. Sequences are clustered by single linkage at id ≥ 0.95, so that two sequences share a cluster if a chain of pairs, each at identity 0.95 or more, joins them. For a set with paired chains, heavy and light chain would be joined into one string before clustering, as in Kenlay et al. [160]; both clustered sets here carry one chain per antibody. A cluster can therefore hold more than one antibody. The 672 nanobodies fall into

309 clusters, of which 185 hold one nanobody and the largest holds 32. The 564 humanness antibodies fall into 461 clusters, of which 404 hold one antibody and the largest holds 13.

Koenig expression. This is a deep mutational scan of the anti-VEGF antibody G6 [215], taken from FLAb. The label is the expression enrichment ratio, which has no unit and runs from 0.00098 to 3.178. All 4,275 variants carry both chains, with heavy chains of 120 residues and light chains of 108. Each variant carries exactly one mutation, 2,261 of them in the heavy chain and 2,014 in the light chain, at 225 distinct positions. No two variants have the same pair of sequences.

Nanobody thermostability. This is the NbThermo set [217], taken from FLAb. The label is the melting temperature T_m in °C, from 39.0 to 98.2 °C, for 672 single-domain antibodies with no light chain. The melting temperatures were measured with four main methods and pooled into one label. The methods are circular dichroism (307), differential scanning fluorimetry with SYPRO dye (254), nanoDSF (76), differential scanning calorimetry (30), and other methods (5). The domains come from llama (395), camel (189), alpaca (36), shark (30), human (2) and unrecorded species (20). The 672 rows hold 556 distinct sequences. Each of 116 sequences appears twice, and 106 of those pairs carry two different melting temperatures, differing by up to 8.8 °C. These repeats are kept, so this set carries disagreement between measurements in its own labels before any noise is added.

Humanness. This is the list of therapeutic antibodies from IMGT/mAb-DB released with BioPhi [218], 564 antibodies with one sequence each. The sequence used is the first one listed for each antibody in the released sequence file. An antibody is labeled human (1) only when its species field is exactly "Human", which gives 198 human antibodies. The other 366 are labeled non-human (0), comprising 229 humanized, 63 chimeric, 41 humanized/chimeric, 13 mouse, 7 human/humanized, 6 caninized, 4 human/chimeric and 3 felinized. The 564 antibodies hold 506 distinct sequences, mostly because an antibody and its drug conjugate share a sequence. One sequence is shared by three antibodies with different labels, two humanized and one human. All three sit in one identity cluster, so they never fall on both sides of a split.

Trastuzumab–HER2 binding. This is the library of variants of the ten-residue CDR-H3 (the third complementarity-determining region of the heavy chain) of trastuzumab, screened for binding to HER2 by Chinery et al. [219]. The released training, validation and test files are joined and re-split. Variants whose CDR-H3 is not exactly 10 residues would be dropped, but there are none, so all 524,346 variants are kept, of which 172,149 are binders (32.8%) and 352,197 non-binders. From these, 5,000 variants were drawn once with seed 42. The draw is stratified on each combination of label and number of mutated CDR-H3 residues, so each combination keeps its share of the full set. The sample holds 1,639 binders (32.8%). Each full heavy chain of 120 residues is rebuilt by inserting the variant CDR-H3 into the trastuzumab framework. The light chain is the fixed trastuzumab light chain, the same on every row. The variants carry 7.3 of the 10 CDR-H3 residues mutated on average.

A.5 Nucleotide Sequences

Both nucleotide tasks come from the revised downstream tasks of the Nucleotide Transformer [220], downloaded from Hugging Face (`InstaDeepAI/nucleotide_transformer_downstream_tasks_revised`). The download is not pinned to a revision; the files used are those of revision `851f994`, dated 30 June 2025, the latest at the time of writing. The released training and test files are merged. Each sequence's chromosome is read from its name, and the folds are grouped by chromosome. Nothing else is filtered, and no sequence contains a letter other than A, C, G or T. Both tasks cover chromosomes 1 to 21, X and Y; chromosome 22 is absent.

H3K4me3. Each sample is a 1,000-base sequence, labeled by whether it carries the H3K4me3 histone mark. The released files hold 17,468 training and 776 test sequences, 18,244 in all, with 9,122 in each class. The dataset card lists 30,000 training sequences; the count here is that of the files.

TATA promoter. Each sample is a 300-base sequence, labeled by whether it is a promoter with a TATA box. The released files hold 5,062 training and 212 test sequences, 5,274 in all, with 2,637 in each class.

A.6 Cancer Genomics (TCGA)

The two cancer datasets use data from TCGA [306], downloaded through UCSC Xena [223]. Expression is read from each cohort's Illumina HiSeq RNA-seq matrix on the Xena TCGA hub (version 2017-10-13). It holds 20,530 genes as $\log_2(x + 1)$ of the RSEM-normalized count [307] from the UNC RNA-seq pipeline. The same 20,530 genes appear in every cohort, and no gene filter is applied. Only primary tumors are kept (sample-type code 01 in the TCGA barcode). Barcodes are cut to the sample level, which leaves one sample per patient. Missing expression values are set to 0. No batch correction is applied, because every fold holds out whole tissue source sites. A correction fitted on the training sites would have no parameter for a site it never saw, and a correction fitted on all sites would already have seen the test samples.

Lung adenocarcinoma against lung squamous cell carcinoma. The adenocarcinoma [221] and squamous cell carcinoma [222] cohorts are stacked, and the cohort is the label. The two matrices hold 576 and 553 samples. Dropping 110 normal-tissue and 2 recurrent-tumor samples leaves 515 adenocarcinomas and 502 squamous tumors, 1,017 in all, from 68 tissue source sites.

Breast cancer PAM50 subtype. The label is the PAM50 subtype call [225] from the breast cancer clinical matrix on Xena [224]. Four classes are kept, namely luminal A, luminal B, basal-like and HER2-enriched. Tumors called normal-like, and tumors without a call, are dropped. Of 1,097 primary tumors with expression data, 821 remain, comprising 421 luminal A, 192 luminal B, 141 basal-like and 67 HER2-enriched, from 19 tissue source sites.

Appendix B

Data and code availability

Three pieces of software lie behind this thesis. NoiseInject is the label-noise package released with Chapter 3. A separate pipeline ran the QM9 experiments of that chapter. The KIRBy repository implements the adaptive hybrid representation and holds the experiments of Chapters 4, 5 and 6, together with the assay-dataset runs of Chapter 3. The three are separate, so each can be reused on its own; the hybrid search, for example, runs on feature matrices a reader has computed elsewhere, without the Rust part of the QM9 pipeline. The source and curation of each dataset are given in Appendix A.

The NoiseInject benchmarking framework, implementing all six regression and six classification noise injection strategies together with the robustness metrics (noise degradation slope and retention) and uncertainty-calibration metrics (ECE, coverage at $1\sigma/2\sigma$, mean interval width, and uncertainty–error / uncertainty–noise correlation) described in this work, is available as an open-source Python package under an MIT license [104]. It also contains optional reference wrappers (a Gaussian process, split-conformal prediction, and Monte Carlo dropout) that produce per-sample uncertainty for use with these metrics. NoiseInject is model-agnostic, operates on label arrays (NumPy arrays or pandas Series), and is designed to integrate into existing cheminformatics or other data pipelines. Worked examples covering molecular-descriptor, fingerprint, and GNN-embedding workflows are provided in the repository.

Its noise conditions and metrics are described in Section 2.5.1.

- Project name: NoiseInject
- Project home page: `https://github.com/adpunt/noise_inject`

- Archived version: https://doi.org/10.5281/zenodo.20532710
- Version: 1.0.0
- Operating system(s): Platform independent
- Programming language: Python 3.7 or later
- Other requirements: NumPy, pandas, SciPy and scikit-learn; see the repository's `requirements.txt`
- License: MIT
- Any restrictions to use by non-academics: None

The QM9 results of Chapter 3 were produced by a separate repository, written in Python and Rust, which also holds the hyperparameter tuning, the variance decomposition and the figure scripts for that chapter. It is described in Chapter 3, under "QM9 Data Pipeline". The three assay datasets of that chapter were run from the KIRBy repository, which calls NoiseInject directly.

- Project name: qsar_qm_models
- Project home page: https://github.com/adpunt/qsar_qm_models
- Archived version: not yet archived
- Operating system(s): Linux and macOS
- Programming language: Python and Rust (2021 edition)
- Other requirements: RDKit installed through conda, to which the Rust program links; PyTorch and PyTorch Geometric
- License: GNU GPL v3.0
- Any restrictions to use by non-academics: those of the GNU GPL v3.0

The adaptive hybrid representation is implemented in the KIRBy repository. It is written in Python and works on feature matrices computed beforehand, so the hybrid search can be applied to representations from any domain. The repository contains the hybrid search itself, which comprises feature importance scoring, the allocation of a feature budget across representations, internal evaluation and a store of every

configuration evaluated for each dataset. It also contains loaders for the datasets in Appendix A and generators for the molecular, antibody, nucleotide, reaction and gene-expression representations. The neighbor-based similarity analyses, the learning-with-noisy-labels methods of Chapter 6 and every experiment script are included as well. Label noise is added through NoiseInject, so the noise definitions are the same as in Chapter 3. A GPU is not needed for the hybrid search, but it speeds up the pretrained encoders.

- Project name: KIRBy
- Project home page: `https://github.com/adpunt/KIRBy`
- Archived version: [Zenodo DOI]
- Operating system(s): Linux and macOS
- Programming language: Python 3.8 or later; several pretrained encoders need Python 3.10 or later
- Other requirements: NumPy, pandas, SciPy, scikit-learn, and RDKit installed through conda; each application area has an optional install
- License: [license]
- Any restrictions to use by non-academics: [depends on the license]

All datasets but one are publicly available. QM9 [107] is deposited on Figshare [213]. The LogD and Caco-2 efflux data come from the OpenADMET–ExpansionRx challenge release, under a CC-BY-4.0 license [109]. The hERG K_i data were extracted from ChEMBL [110], whose data are released under CC BY-SA 3.0. They can be regenerated with the curation protocol in Appendix A. The rat liver microsome clearance and MDR1–MDCK efflux data are the Biogen ADME set [211], released on Polaris [212] under CC-BY-4.0. The reaction-yield, antibody, nucleotide and tumor gene-expression datasets were downloaded from their public releases, which are cited in Appendix A. Pretrained model weights were downloaded from their original sources and are not redistributed.

The exception is the hERG blocker data used for feature selection with data held at separate sites (Chapter 5). These are Lhasa Limited's curated hERG training set of 7,772 molecules and test set of 4,383 molecules [139]. They are proprietary

and are not redistributed. FLuID's separate public set of molecules is not used; the training molecules take its place, as stated in Chapter 5. The comparison with FLuID also calls Lhasa's FLuID code and cannot be run without it. Every other result in this thesis can be reproduced from public data and the code above.

The adaptive hybrid representation, the KIRBy repository, and the experiments of Chapters 4, 5 and 6 were designed, implemented, run and written up by A.P. alone. Lhasa Limited supplied the curated hERG data and the FLuID code [139]. The use of generative AI is described in the declaration in the front matter.

Bibliography

[1] Markus A. Lill. Multi-dimensional QSAR in drug discovery. *Drug Discovery Today*, 12(23):1013–1017, 2007. ISSN 1359-6446.

[2] Bruno J. Neves, Rodolpho C. Braga, and Cleber C. Melo-Filho et al. QSAR-based virtual screening: Advances and applications in drug discovery. *Frontiers in Pharmacology*, 9:1275, 2018. ISSN 1663-9812.

[3] María Virginia Sabando, Ignacio Ponzoni, Evangelos E Milios, and Axel J Soto. Using molecular embeddings in QSAR modeling: Does it make a difference? *Briefings in Bioinformatics*, 23(1):bbab365, 2022. ISSN 1477-4054.

[4] David Rogers and Mathew Hahn. Extended-connectivity fingerprints. *Journal of Chemical Information and Modeling*, 50(5):742–754, 2010. doi: 10.1021/ci100050t. PMID: 20426451.

[5] Luís David, Abhijit Thakkar, Raul Mercado, et al. Molecular representations in AI-driven drug discovery: a review and practical guide. *Journal of Cheminformatics*, 12(1):56, 2020.

[6] Markus Dablander, Thierry Hanser, Renaud Lambiotte, and Garrett M. Morris. Sort & slice: a simple and superior alternative to hash-based folding for extended-connectivity fingerprints. *Journal of Cheminformatics*, 16(1), December 2024. ISSN 1758-2946. doi: 10.1186/s13321-024-00932-y.

[7] Peter Gedeck, Bernhard Rohde, and Christian Bartels. QSAR — how good is it in practice? Comparison of descriptor sets on an unbiased cross section of corporate data sets. *Journal of Chemical Information and Modeling*, 46(5): 1924–1936, 2006. doi: 10.1021/ci050413p.

[8] Akihiro Kishimoto, Hiroshi Kajino, Masataka Hirose, Junta Fuchiwaki, Indra Priyadarsini, Lisa Hamada, Hajime Shinohara, Daiju Nakano, and Seiji

Takeda. Mhg-gnn: Combination of molecular hypergraph grammar with graph neural network, 2023.

[9] Scott S. Kolmar and Christopher M. Grulke. The effect of noise on the predictive limit of QSAR models. *Journal of Cheminformatics*, 13(1):92, 2021.

[10] Kjell Jorner, Tore Brinck, Per-Ola Norrby, and David Buttar. Machine learning meets mechanistic modelling for accurate prediction of experimental activation energies. *Chem. Sci.*, 12:1163–1175, 2021. doi: 10.1039/D0SC04896H.

[11] Denis Fourches, Eugene Muratov, and Alexander Tropsha. Trust, but verify: On the importance of chemical structure curation in cheminformatics and qsar modeling research. *Journal of Chemical Information and Modeling*, 50 (7):1189–1204, 2010. doi: 10.1021/ci100176x. PMID: 20572635.

[12] Xiuwen Fang and Mang Ye. Robust federated learning with noisy and heterogeneous clients. In *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10062–10071, 2022.

[13] W. Patrick Walters. We need better benchmarks for machine learning in drug discovery. Practical Cheminformatics Blog, 2023. URL http://practicalcheminformatics.blogspot.com/2023/08/we-need-better-benchmarks-for-machine.html. Accessed: 2026-04-12.

[14] Tuomo Kalliokoski, Christian Kramer, Anna Vulpetti, and Peter Gedeck. Comparability of mixed IC50 data – a statistical analysis. *PLoS ONE*, 8 (4):e61007, 2013. doi: 10.1371/journal.pone.0061007.

[15] Christian Kramer, Tuomo Kalliokoski, Peter Gedeck, and Anna Vulpetti. The experimental uncertainty of heterogeneous public Ki data. *Journal of Medicinal Chemistry*, 55(11):5165–5173, 2012. doi: 10.1021/jm300131x.

[16] Gregory A. Landrum and Sereina Riniker. Combining IC50 or Ki values from different sources is a source of significant noise. *Journal of Chemical Information and Modeling*, 64(5):1560–1567, 2024. doi: 10.1021/acs.jcim.4c00049. PMID: 38394344.

[17] Isidro Cortes-Ciriano, Andreas Bender, and Thérèse E. Malliavin. Comparing the influence of simulated experimental errors on 12 machine learning algorithms in bioactivity modeling using 12 diverse data sets. *Journal of Chemical Information and Modeling*, 55(7):1413–1425, 2015. doi: 10.1021/acs.jcim.5b00101. PMID: 26038978.

[18] Davide Bassani, Alessandro Brigo, and Andrea Andrews-Morger. Federated learning in computational toxicology: An industrial perspective on the effiris hackathon. *Chemical Research in Toxicology*, 36(9):1503–1517, 2023.

[19] Thierry Hanser. Federated learning for molecular discovery. *Current Opinion in Structural Biology*, 79:102545, 2023. ISSN 0959-440X.

[20] David Weininger. SMILES, a chemical language and information system. 1. Introduction to methodology and encoding rules. *Journal of Chemical Information and Computer Sciences*, 28(1):31–36, 1988.

[21] Michael Backenköhler, Paula Linh Kramer, and Joschka et al. Groß. TeachOpenCADD goes Deep Learning: Open-source Teaching Platform Exploring Molecular DL Applications. *ChemRxiv preprint*, 2023. doi: 10.26434/chemrxiv-2023-kz1pb.

[22] Alexandre V. Fassio, Laura Shub, and Luca Ponzoni et al. Prioritizing virtual screening with interpretable interaction fingerprints. *Journal of Chemical Information and Modeling*, 62(18):4300–4318, 2022.

[23] Liangxu Xie, Lei Xu, Ren Kong, Shan Chang, and Xiaojun Xu. Improvement of prediction performance with conjoint molecular fingerprint in deep learning. *Frontiers in Pharmacology*, 11, 2020. doi: 10.3389/fphar.2020.606668.

[24] Isidro Cortés-Ciriano, Ctibor Škuta, Andreas Bender, and Daniel Svozil. QSAR-derived affinity fingerprints (part 2): Modeling performance for potency prediction. *Journal of Cheminformatics*, 12(1):41, 2020.

[25] Sabrina Jaeger, Simone Fulle, and Samo Turk. Mol2vec: Unsupervised machine learning approach with chemical intuition. *Journal of Chemical Information and Modeling*, 58(1):27–35, 2018. doi: 10.1021/acs.jcim.7b00616.

[26] Seyone Chithrananda, Gabriel Grand, and Bharath Ramsundar. Chemberta: Large-scale self-supervised pretraining for molecular property prediction. *arXiv preprint arXiv:2010.09885*, 2020.

[27] Benedek Fabian, Thomas Edlich, and Héléna et al. Gaspar. Molecular representation learning with language models and domain-relevant auxiliary tasks. *arXiv preprint arXiv:2011.13230*, 2020.

[28] Walid Ahmad, Elana Simon, Seyone Chithrananda, Gabriel Grand, and Bharath Ramsundar. Chemberta-2: Towards chemical foundation models. *arXiv preprint arXiv:2209.01712*, 2022.

[29] Roberto Todeschini and Viviana Consonni. *Molecular Descriptors for Chemoinformatics*. Wiley-VCH, Weinheim, 2nd edition, 2009. doi: 10.1002/9783527628766.

[30] Andrea Mauri, Viviana Consonni, Manuela Pavan, and Roberto Todeschini. DRAGON software: An easy approach to molecular descriptor calculations. *MATCH Communications in Mathematical and in Computer Chemistry*, 56 (2):237–248, 2006.

[31] Artem Cherkasov, Eugene N. Muratov, and Denis et al. Fourches. QSAR modeling: Where have you been? where are you going to? *Journal of Medicinal Chemistry*, 57(12):4977–5010, 2014. doi: 10.1021/jm4004285.

[32] Greg Landrum. Rdkit: Open-source cheminformatics. https://www.rdkit.org. Accessed: 2026-01-09.

[33] Zhichun Guo, Kehan Guo, and Bozhao Nan et al. Graph-based molecular representation learning, 2023. Accessed: 2024-11-30.

[34] Stefan M.N. Hönig, Carsten Lemmen, and Matthias Rarey. Small molecule superposition: A comprehensive overview on pose scoring of the latest methods. *WIREs Computational Molecular Science*, 13(2):e1640, 2023.

[35] Ge Yan, Huaijin Wu, and Junchi Yan. Quantum 3D graph learning with applications to molecule embedding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 39126–39137. PMLR, 2023.

[36] Ziduo Yang, Weihe Zhong, Qiujie Lv, Tiejun Dong, and Calvin Yu-Chian Chen. Geometric interaction graph neural network for predicting protein–ligand binding affinities from 3D structures (GIGN). *The Journal of Physical Chemistry Letters*, 14(8):2020–2033, 2023. doi: 10.1021/acs.jpclett.2c03906.

[37] Wenjie Du, Xiaoting Yang, and Di et al. Wu. Fusing 2D and 3D molecular graphs as unambiguous molecular descriptors for conformational and chiral stereoisomers. *Briefings in Bioinformatics*, 24(1):bbac560, 2022. ISSN 1477-4054.

[38] Denis Kuzminykh, Daniil Polykovskiy, and Artur Kadurin et al. 3D molecular representations based on the wave transform for convolutional neural networks. *Molecular Pharmaceutics*, 15(10):4378–4385, 2018.

[39] Giorgos Bouritsas, Fabrizio Frasca, Stefanos Zafeiriou, and Michael M. Bronstein. Improving graph neural network expressivity via subgraph isomorphism counting. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(1):657–668, 2023.

[40] Jerret Ross, Brian Belgodere, Vijil Chenthamarakshan, Inkit Padhi, Youssef Mroueh, and Payel Das. Large-scale chemical language representations capture molecular structure and properties, 2022.

[41] Esther Heid, Charles J. McGill, Florence H. Vermeire, and William H. Green. Characterizing uncertainty in machine learning for chemistry. *Journal of Chemical Information and Modeling*, 63(13):4012–4029, 2023. doi: 10.1021/acs.jcim.3c00373. PMID: 37338239.

[42] Leo Breiman, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. *Classification and Regression Trees*. Wadsworth & Brooks/Cole, Monterey, CA, 1984. ISBN 978-0-412-04841-8.

[43] Vladimir Svetnik, Andy Liaw, Christopher Tong, John Culberson, Robert Sheridan, and Bradley Feuston. Random forest: A classification and regression tool for compound classification and qsar modeling. *Journal of chemical information and computer sciences*, 43:1947–58, 11 2003. doi: 10.1021/ci034160g.

[44] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. doi: 10.1023/A:1010933404324.

[45] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016. doi: 10.1145/2939672.2939785.

[46] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989. doi: 10.1016/0893-6080(89)90020-8.

[47] Junshui Ma, Robert P Sheridan, Andy Liaw, George E Dahl, and Vladimir Svetnik. Deep neural nets as a method for quantitative structure-activity relationships. *Journal of Chemical Information and Modeling*, 55(2):263–274, 2015. doi: 10.1021/ci500747n.

[48] Zhenqin Wu, Bharath Ramsundar, Evan N. Feinberg, Joseph Gomes, Caleb Geniesse, Aneesh S. Pappu, Karl Leswing, and Vijay Pande. Moleculenet: A benchmark for molecular machine learning, 2018.

[49] Igor I. Baskin, Vladimir A. Palyulin, and Nikolai S. Zefirov. Neural networks in building QSAR models. In *Methods in Molecular Biology*, volume 458, pages 137–158. Humana Press, 2008. doi: 10.1007/978-1-60327-101-1_8.

[50] Alexios Koutsoukas, Keith J. Monaghan, Xiaoli Li, and Jun Huan. Deep-learning: investigating deep neural networks hyper-parameters and comparison of performance to shallow methods for modeling bioactivity data. *Journal of Cheminformatics*, 9:42, 2017. doi: 10.1186/s13321-017-0226-y.

[51] Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. Understanding deep learning requires rethinking generalization. In *International Conference on Learning Representations (ICLR)*, 2017.

[52] Curtis G Northcutt, Lu Jiang, and Isaac L Chuang. Confident learning: Estimating uncertainty in dataset labels. *Journal of Artificial Intelligence Research*, 70:1373–1411, 2021. doi: 10.1613/jair.1.12125.

[53] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks?, 2019.

[54] Nicolai Meinshausen. Quantile regression forests. *Journal of Machine Learning Research*, 7:983–999, 06 2006.

[55] Ismail Babajide Mustapha and Faisal Saeed. Bioactive molecule prediction using extreme gradient boosting. *Molecules*, 21(8), 2016. ISSN 1420-3049. doi: 10.3390/molecules21080983.

[56] Hao Tian, Rajas Ketkar, and Peng Tao. Accurate admet prediction with xgboost, 2022.

[57] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 3146–3154, 2017.

[58] Tony Duan, Anand Avati, Daisy Yi Ding, Khanh K. Thai, Sanjay Basu, Andrew Y. Ng, and Alejandro Schuler. Ngboost: Natural gradient boosting for probabilistic prediction, 2020.

[59] Vladimir N. Vapnik. *The Nature of Statistical Learning Theory*. Springer, New York, 1995. doi: 10.1007/978-1-4757-2440-0.

[60] Alex J. Smola and Bernhard Schölkopf. A tutorial on support vector regression. *Statistics and Computing*, 14(3):199–222, 2004. doi: 10.1023/B:STCO.0000035301.49549.88.

[61] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20(3):273–297, 1995. doi: 10.1007/BF00994018.

[62] Chih-Chung Chang and Chih-Jen Lin. LIBSVM: A library for support vector machines. *ACM Transactions on Intelligent Systems and Technology*, 2(3): 1–27, 2011. doi: 10.1145/1961189.1961199.

[63] Olga Obrezanova, Joelle M.R. Gola, and Matthew D. Segall. Gaussian processes: A method for automatic qsar and adme modelling. Presentation at ACS Boston, 22 August 2007, 2007.

[64] Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. The MIT Press, 2005. ISBN 9780262256834.

[65] Joaquin Quiñonero-Candela and Carl Edward Rasmussen. A unifying view of sparse approximate gaussian process regression. *Journal of Machine Learning Research*, 6(65):1939–1959, 2005.

[66] Ryan-Rhys Griffiths and Leo Klarner et al. Gauche: A library for gaussian processes in chemistry, 2022. Accessed: 2024-08-21.

[67] L. Ralaivola, S. Joshua Swamidass, Hiroto Saigo, and Pierre Baldi. Graph kernels for chemical informatics. *Neural networks : the official journal of the International Neural Network Society*, 18:1093–110, 11 2005. doi: 10.1016/j.neunet.2005.07.009.

[68] Henry Moss and Ryan-Rhys Griffiths. Gaussian process molecule property prediction with flowmo, 10 2020.

[69] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles, 2017.

[70] Glenn Palmer, Siqi Du, Alexander Politowicz, Joshua Emory, Xiyu Yang, Anupraas Gautam, Grishma Gupta, Zhelong Li, Ryan Jacobs, and Dane Morgan. Calibration after bootstrap for accurate uncertainty quantification in regression models. *npj Computational Materials*, 8:115, 05 2022. doi: 10.1038/s41524-022-00794-8.

[71] Dávid Bajusz, Anita Rácz, and Károly Héberger. Why is tanimoto index an appropriate choice for fingerprint-based similarity calculations? *Journal of Cheminformatics*, 7(1), May 2015. ISSN 1758-2946. doi: 10.1186/s13321-015-0069-3.

[72] Dagmar Stumpfe and Jürgen Bajorath. Exploring activity cliffs in medicinal chemistry. *Journal of Medicinal Chemistry*, 55(7):2932–2942, 2012. doi: 10.1021/jm201706b.

[73] Markus Dablander, Thierry Hanser, Renaud Lambiotte, and Garrett M. Morris. Exploring QSAR models for activity-cliff prediction. *Journal of Cheminformatics*, 15(1):47, 2023. doi: 10.1186/s13321-023-00708-w.

[74] Mark A. Johnson and Gerald M. Maggiora, editors. *Concepts and Applications of Molecular Similarity*. Wiley, New York, 1990.

[75] Derek van Tilborg, Alisa Alenicheva, and Francesca Grisoni. Exposing the limitations of molecular machine learning with activity cliffs. *Journal of Chemical Information and Modeling*, 62(23):5938–5951, 2022. doi: 10.1021/acs.jcim.2c01073.

[76] Gerald Maggiora, Martin Vogt, Dagmar Stumpfe, and Jürgen Bajorath. Molecular similarity in medicinal chemistry. *Journal of Medicinal Chemistry*, 57(8):3186–3204, 2014. doi: 10.1021/jm401411z.

[77] Gerald M. Maggiora. On outliers and activity cliffs—why QSAR often disappoints. *Journal of Chemical Information and Modeling*, 46(4):1535, 2006. doi: 10.1021/ci060117s.

[78] Charu C. Aggarwal, Alexander Hinneburg, and Daniel A. Keim. On the surprising behavior of distance metrics in high dimensional space. In *Database Theory – ICDT 2001*, volume 1973 of *Lecture Notes in Computer Science*, pages 420–434, Berlin, Heidelberg, 2001. Springer. doi: 10.1007/3-540-44503-X_27.

[79] Felix A. Krüger and John P. Overington. Global analysis of small molecule binding to related protein targets. *PLoS Computational Biology*, 8(1): e1002333, 2012. doi: 10.1371/journal.pcbi.1002333.

[80] Joe Bentz, Michael P. O'Connor, Dallas Bednarczyk, JoAnn Coleman, Caroline Lee, Johan Palm, Y. Anne Pak, Elke S. Perloff, Eric Reyner, Praveen Balimane, et al. Variability in P-glycoprotein inhibitory potency (IC50) using various in vitro experimental systems: Implications for universal digoxin drug-drug interaction risk assessment decision criteria. *Drug Metabolism and Disposition*, 41(7):1347–1366, 2013. doi: 10.1124/dmd.112.050500.

[81] Emma Svensson, Hannah Rosa Friesacher, Susanne Winiwarter, Lewis Mervin, Adam Arany, and Ola Engkvist. Enhancing uncertainty quantification in drug discovery with censored regression labels. *Artificial Intelligence in the Life Sciences*, 7:100128, 2025. doi: 10.1016/j.ailsci.2025.100128.

[82] Ricardo Macarron, Martyn N. Banks, and Dejan et al. Bojanic. Impact of high-throughput screening in biomedical research. *Nature Reviews Drug Discovery*, 10(3):188–195, 2011. doi: 10.1038/nrd3368.

[83] James Inglese, Douglas S. Auld, and Ajit et a.. Jadhav. Quantitative high-throughput screening: A titration-based approach that efficiently identifies biological activities in large chemical libraries. *Proceedings of the National Academy of Sciences*, 103(31):11473–11478, 2006. doi: 10.1073/pnas.0604348103.

[84] Christine Brideau, Bert Gunter, Bill Pikounis, and Andy Liaw. Improved statistical methods for hit selection in high-throughput screening. *Journal of Biomolecular Screening*, 8(6):634–647, 2003. doi: 10.1177/1087057103258285.

[85] Ji-Hu Zhang, Thomas D. Y. Chung, and Kevin R. Oldenburg. A simple statistical parameter for use in evaluation and validation of high throughput screening assays. *Journal of Biomolecular Screening*, 4(2):67–73, 1999. doi: 10.1177/108705719900400206.

[86] Anton Simeonov, Ajit Jadhav, and Craig J. et al. Thomas. Fluorescence spectroscopic profiling of compound libraries. *Journal of Medicinal Chemistry*, 51(8):2363–2371, 2008. doi: 10.1021/jm701301m.

[87] Christos Hatzis, Philippe L. Bedard, Nicolai J. Birkbak, Andrew H. Beck, Hugo J.W.L. Aerts, David F. Stern, Leming Shi, Robert Clarke, John Quackenbush, and Benjamin Haibe-Kains. Enhancing reproducibility in cancer drug screening: How do we move forward? *Cancer Research*, 74(15):4016–4023, July 2014. ISSN 1538-7445. doi: 10.1158/0008-5472.can-14-0725.

[88] Elena Postnikova, Yu Cong, Lisa Evans DeWald, and Julie et al. Dyall. Testing therapeutics in cell-based assays: Factors that influence the apparent potency of drugs. *PLOS ONE*, 13(3):e0194880, March 2018. ISSN 1932-6203. doi: 10.1371/journal.pone.0194880.

[89] Anna Gaulton, Anne Hersey, and Michał et al. Nowotka. The ChEMBL database in 2017. *Nucleic Acids Research*, 45(D1):D945–D954, 2017. doi: 10.1093/nar/gkw1074.

[90] Michael K Gilson, Tiqing Liu, Michael Baitaluk, and et al. BindingDB in 2015: A public database for medicinal chemistry, computational chemistry and systems pharmacology. *Nucleic Acids Research*, 44(D1):D1045–D1053, 2016. doi: 10.1093/nar/gkv1072.

[91] Denis Fourches, Eugene Muratov, and Alexander Tropsha. Trust, but verify II: A practical guide to chemogenomics data curation. *Journal of Chemical Information and Modeling*, 56(7):1243–1252, 2016. doi: 10.1021/acs.jcim.6b00129.

[92] Antony J. Williams, Christopher M. Grulke, and Jeff et al. Edwards. The comptox chemistry dashboard: a community data resource for environmental chemistry. *Journal of Cheminformatics*, 9:61, 2017. doi: 10.1186/s13321-017-0247-6.

[93] Douglas Young, Todd Martin, Raghuraman Venkatapathy, and Phil Harten. Are the chemical structures in your QSAR correct? *QSAR & Combinatorial Science*, 27(11-12):1337–1345, 2008. doi: 10.1002/qsar.200810084.

[94] A Patrícia Bento, Anne Hersey, Eloy Félix, and Greg et al. Landrum. An open source chemical structure curation pipeline using RDKit. *Journal of Cheminformatics*, 12(1):51, 2020. doi: 10.1186/s13321-020-00456-1.

[95] Insha Ullah and A. H. Welsh. On the effect of noise on fitting linear regression models, 2024.

[96] Alexander Camuto, Matthew Willetts, Umut Şimşekli, Stephen Roberts, and Chris Holmes. Explicit regularisation in gaussian noise injections, 2021.

[97] Peter J. Huber. Robust estimation of a location parameter. *The Annals of Mathematical Statistics*, 35(1):73–101, 1964. doi: 10.1214/aoms/1177703732.

[98] Frank R. Hampel. Robust statistics: a brief introduction and overview. Research Report 94, Seminar für Statistik, Eidgenössische Technische Hochschule (ETH) Zürich, 2001.

[99] Alex Avdeef. Multi-lab intrinsic solubility measurement reproducibility in CheqSol and shake-flask methods. *ADMET and DMPK*, 7(3):210–219, 2019. doi: 10.5599/admet.698.

[100] Hwanjun Song, Minseok Kim, Dongmin Park, Yooju Shin, and Jae-Gil Lee. Learning from noisy labels with deep neural networks: A survey. *IEEE Transactions on Neural Networks and Learning Systems*, 34(11):8135–8153, 2023. doi: 10.1109/TNNLS.2022.3152527.

[101] David H. Wolpert. The lack of a priori distinctions between learning algorithms. *Neural Computation*, 8(7):1341–1390, 1996. doi: 10.1162/neco.1996.8.7.1341.

[102] D.H. Wolpert and W.G. Macready. No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation*, 1(1):67–82, 1997. doi: 10.1109/4235.585893.

[103] Dennis L. Wilson. Asymptotic properties of nearest neighbor rules using edited data. *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-2(3):408–421, 1972. doi: 10.1109/TSMC.1972.4309137.

[104] Adelaide Punt. NoiseInject: A Python package for label-noise robustness benchmarking in QSAR models. https://github.com/adpunt/noise_inject, 2026. Accessed: 2026-06-01.

[105] Yarin Gal and Zoubin Ghahramani. Dropout as a bayesian approximation: Representing model uncertainty in deep learning, 2016.

[106] Jianyuan Deng, Zhibo Yang, and Hehe Wang et al. Unraveling key elements underlying molecular property prediction: A systematic study, 2023.

[107] Raghunathan Ramakrishnan, Pavlo O. Dral, Matthias Rupp, et al. Quantum chemistry structures and properties of 134 kilo molecules. *Scientific Data*, 1:140022, 2014.

[108] Artem Fediai et al. Accurate GW frontier orbital energies of 134 kilo molecules. *Scientific Data*, 10:581, 2023. doi: 10.1038/s41597-023-02486-4.

[109] Open ADMET Consortium. OpenADMET-ExpansionRx challenge dataset, 2026. URL https://huggingface.co/datasets/openadmet/openadmet-expansionrx-challenge-data. CC-BY-4.0. ADMET data from drug discovery campaigns by Expansion Therapeutics targeting RNA-mediated diseases. Challenge closed January 19, 2026.

[110] Barbara Zdrazil, Eloy Felix, Fiona Hunter, et al. The ChEMBL database in 2023: a drug discovery platform spanning multiple bioactivity data types and time periods. *Nucleic Acids Research*, 52(D1):D1180–D1192, 11 2023. ISSN 0305-1048. doi: 10.1093/nar/gkad1004.

[111] Jacob R. Gardner, Geoff Pleiss, Kilian Q. Weinberger, David Bindel, and Andrew Gordon Wilson. GPyTorch: Blackbox matrix-matrix Gaussian process inference with GPU acceleration. In *Advances in Neural Information Processing Systems*, volume 31, 2018.

[112] Matthias Fey and Jan E. Lenssen. Fast graph representation learning with pytorch geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

[113] Alex Kendall and Yarin Gal. What uncertainties do we need in bayesian deep learning for computer vision? *CoRR*, abs/1703.04977, 2017.

[114] James Harrison, John Willes, and Jasper Snoek. Variational bayesian last layers, 2024.

[115] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, pages 2623–2631, 2019.

[116] Markus Orsi and Jean-Louis Reymond. One chiral fingerprint to find them all. *Journal of Cheminformatics*, 16:53, 2024. doi: 10.1186/s13321-024-00849-6.

[117] Lien Ai Nguyen, Hua He, and Chuong Pham-Huy. Chiral drugs: an overview. *International Journal of Biomedical Science*, 2(2):85–100, 2006. PMID: 23674971; PMCID: PMC3614593.

[118] Alexander Golbraikh, Eugene Muratov, Denis Fourches, and Alexander Tropsha. Data set modelability by QSAR. *Journal of Chemical Information and Modeling*, 54(1):1–4, 2014. doi: 10.1021/ci400572x.

[119] Robert P. Sheridan, Prabha Karnachi, Matthew Tudor, Yuting Xu, Andy Liaw, Falgun Shah, Alan C. Cheng, Elizabeth Joshi, Meir Glick, and Juan Alvarez. Experimental error, kurtosis, activity cliffs, and methodology: What limits the predictivity of quantitative structure–activity relationship models? *Journal of Chemical Information and Modeling*, 60(4):1969–1982, 2020. doi: 10.1021/acs.jcim.9b01067.

[120] José L. Medina-Franco. Activity cliffs: Facts or artifacts? *Chemical Biology & Drug Design*, 81(5):553–556, 2013. doi: 10.1111/cbdd.12115.

[121] Sereina Riniker and Gregory A. Landrum. Open-source platform to benchmark fingerprints for ligand-based virtual screening. *Journal of Cheminformatics*, 5:26, 2013. doi: 10.1186/1758-2946-5-26.

[122] Mateusz Praski, Jakub Adamczyk, and Wojciech Czech. Benchmarking pre-trained molecular embedding models for molecular representation learning, 2025.

[123] Florian Huber and Julian Pollmann. Count your bits: fingerprint benchmarking to assess broad chemical space representation. *Journal of Cheminformatics*, 18:83, 2026. doi: 10.1186/s13321-026-01245-y.

[124] Pawel Dabrowski-Tumanski, Bartosz Topolski, Dariusz Plewczynski, and Tomasz Jetka. The geometry of activity cliffs: Representation dependence and multi-scale characterization of activity landscapes, 2026.

[125] Michael F. Dablander. *Investigating Graph Neural Networks and Classical Feature-Extraction Techniques in Activity-Cliff and Molecular Property Prediction.* PhD thesis, University of Oxford, 2023.

[126] Rajarshi Guha and John H. Van Drie. Structure–activity landscape index: Identifying and quantifying activity cliffs. *Journal of Chemical Information and Modeling*, 48(3):646–658, 2008. doi: 10.1021/ci7004093.

[127] Kenneth López-Pérez, Juan F. Avellaneda-Tamayo, Lexin Chen, Edgar López-López, K. Eurídice Juárez-Mercado, José L. Medina-Franco, and Ramón Alain Miranda-Quintana. Molecular similarity: Theory, applications, and perspectives. *Artificial Intelligence Chemistry*, 2(2):100077, 2024. doi: 10.1016/j.aichem.2024.100077.

[128] Alice Capecchi, Daniel Probst, and Jean-Louis Reymond. One molecular fingerprint to rule them all: drugs, biomolecules, and the metabolome. *Journal of Cheminformatics*, 12(1):43, 2020. doi: 10.1186/s13321-020-00445-4.

[129] Noel M. O'Boyle and Roger A. Sayle. Comparing structural fingerprints using a literature-based similarity benchmark. *Journal of Cheminformatics*, 8:36, 2016. doi: 10.1186/s13321-016-0148-0.

[130] Miloš Radovanović, Alexandros Nanopoulos, and Mirjana Ivanović. Hubs in space: Popular nearest neighbors in high-dimensional data. *Journal of*

Machine Learning Research, 11:2487–2531, 2010. URL https://jmlr.org/papers/v11/radovanovic10a.html.

[131] David E. Patterson, Richard D. Cramer, Allen M. Ferguson, Robert D. Clark, and Laurence E. Weinberger. Neighborhood behavior: A useful concept for validation of "molecular diversity" descriptors. *Journal of Medicinal Chemistry*, 39(16):3049–3059, 1996. doi: 10.1021/jm960290n.

[132] David E. Coupry and Péter Pogány. Application of deep metric learning to molecular graph similarity. *Journal of Cheminformatics*, 14(1):11, 2022.

[133] Weifan Zheng and Alexander Tropsha. Novel variable selection quantitative structure–property relationship approach based on the k-nearest-neighbor principle. *Journal of Chemical Information and Computer Sciences*, 40(1): 185–194, 2000. doi: 10.1021/ci980033m.

[134] Peter Itskowitz and Alexander Tropsha. k nearest neighbors QSAR modeling as a variational problem: Theory and applications. *Journal of Chemical Information and Modeling*, 45(3):777–785, 2005. doi: 10.1021/ci049628+.

[135] J. Roger Bray and J. T. Curtis. An ordination of the upland forest communities of southern Wisconsin. *Ecological Monographs*, 27(4):325–349, 1957. doi: 10.2307/1942268.

[136] Prasanta Chandra Mahalanobis. On the generalized distance in statistics. *Proceedings of the National Institute of Sciences of India*, 2(1):49–55, 1936.

[137] Alexander Tropsha. Best practices for QSAR model development, validation, and exploitation. *Molecular Informatics*, 29(6-7):476–488, 2010. doi: 10.1002/minf.201000061.

[138] Darko Butina. Unsupervised data base clustering based on daylight's fingerprint and tanimoto similarity: A fast and automated way to cluster small and large data sets. *Journal of Chemical Information and Computer Sciences*, 39 (4):747–750, 1999. doi: 10.1021/ci9803381.

[139] Thierry Hanser, Ernst Ahlberg, Alexander Amberg, Lennart T. Anger, Chris Barber, Richard J. Brennan, Alessandro Brigo, Annie Delaunois, Susanne Glowienke, Nigel Greene, Laura Johnston, Daniel Kuhn, Lara Kuhnke, Jean-François Marchaland, Wolfgang Muster, Jeffrey Plante, Friedrich Rippmann,

Yogesh Sabnis, Friedemann Schmidt, Ruud van Deursen, Stéphane Werner, Angela White, Joerg Wichard, and Tomoya Yukawa. Data-driven federated learning in drug discovery with knowledge distillation. *Nature Machine Intelligence*, 7(3):423–436, 2025. doi: 10.1038/s42256-025-00991-2.

[140] Junnan Li, Richard Socher, and Steven C.H. Hoi. Dividemix: Learning with noisy labels as semi-supervised learning. In *International Conference on Learning Representations*, 2020.

[141] Bo Han, Quanming Yao, and Xingrui Yu et al. Co-teaching: Robust training of deep neural networks with extremely noisy labels. 2018. Accessed: 2024-09-01.

[142] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning*, ICML '09, pages 41–48. ACM, 2009.

[143] Haitao Liu, Yew-Soon Ong, and Jianfei Cai. Large-scale heteroscedastic regression via gaussian process, 2020.

[144] Carla E. Brodley and Mark A. Friedl. Identifying mislabeled training data. *Journal of Artificial Intelligence Research*, 11:131–167, 1999. doi: 10.1613/jair.606.

[145] Benoît Frénay and Michel Verleysen. Classification in the presence of label noise: A survey. *IEEE Transactions on Neural Networks and Learning Systems*, 25(5):845–869, 2014. doi: 10.1109/TNNLS.2013.2292894.

[146] Juan Martín, José A. Sáez, and Emilio Corchado. On the regressand noise problem: Model robustness and synergy with regression-adapted noise filters. *IEEE Access*, 9:145800–145816, 2021. doi: 10.1109/ACCESS.2021.3123151.

[147] Oliver Laufkötter, Noé Sturm, Jürgen Bajorath, Hongming Chen, and Ola Engkvist. Combining structural and bioactivity-based fingerprints improves prediction performance and scaffold hopping capability. *Journal of Cheminformatics*, 11(1), August 2019. ISSN 1758-2946. doi: 10.1186/s13321-019-0376-1.

[148] Viktoriia Turkina, Melanie R.W. Messih, Etienne Kant, and Jelle T. et al. Gringhuis. Enhancing predictive modeling with molecular fingerprint fusion strategies. July 2025. doi: 10.26434/chemrxiv-2025-b3lfq.

[149] Sören Richard Stahlschmidt, Benjamin Ulfenborg, and Jane Synnergren. Multimodal deep learning for biomedical data fusion: a review. *Briefings in Bioinformatics*, 23(2):bbab569, 2022. doi: 10.1093/bib/bbab569.

[150] Sandra Steyaert, Marija Pizurica, Divya Nagaraj, Priya Khandelwal, Tina Hernandez-Boussard, Andrew J. Gentles, and Olivier Gevaert. Multimodal data fusion for cancer biomarker discovery with deep learning. *Nature Machine Intelligence*, 5(4):351–362, 2023. doi: 10.1038/s42256-023-00633-5.

[151] Tongxin Wang, Wei Shao, Zhi Huang, Haixu Tang, Jie Zhang, Zhengming Ding, and Kun Huang. MOGONET integrates multi-omics data using graph convolutional networks allowing patient classification and biomarker identification. *Nature Communications*, 12(1):3445, 2021. doi: 10.1038/s41467-021-23774-w.

[152] Aurélien Beaude, Franck Augé, Farida Zehraoui, and Blaise Hanczar. CrossAttOmics: multiomics data integration with cross-attention. *Bioinformatics*, 41 (6):btaf302, 2025. doi: 10.1093/bioinformatics/btaf302.

[153] Mehmet Gönen and Ethem Alpaydın. Multiple kernel learning algorithms. *Journal of Machine Learning Research*, 12:2211–2268, 2011. URL https://www.jmlr.org/papers/v12/gonen11a.html.

[154] Anna Cichonska, Tapio Pahikkala, Sandor Szedmak, Heli Julkunen, Antti Airola, Markus Heinonen, Tero Aittokallio, and Juho Rousu. Learning with multiple pairwise kernels for drug bioactivity prediction. *Bioinformatics*, 34 (13):i509–i518, 2018. doi: 10.1093/bioinformatics/bty277.

[155] Gordon F. Hughes. On the mean accuracy of statistical pattern recognizers. *IEEE Transactions on Information Theory*, 14(1):55–63, 1968. doi: 10.1109/TIT.1968.1054102.

[156] Richard E. Bellman. *Adaptive Control Processes: A Guided Tour*. Princeton University Press, Princeton, NJ, 1961. doi: 10.1515/9781400874668.

[157] Manuel Fernández-Delgado, Eva Cernadas, Senén Barro, and Dinani Amorim. Do we need hundreds of classifiers to solve real world classification problems? *Journal of Machine Learning Research*, 15(90):3133–3181, 2014. URL https://jmlr.org/papers/v15/delgado14a.html.

[158] Dejun Jiang, Zhenxing Wu, Chang-Yu Hsieh, Guangyong Chen, Ben Liao, Zhe Wang, Chao Shen, Dongsheng Cao, Jian Wu, and Tingjun Hou. Could graph neural networks learn better molecular representation for drug discovery? A comparison study of descriptor-based and graph-based models. *Journal of Cheminformatics*, 13(1):12, 2021. doi: 10.1186/s13321-020-00479-8.

[159] Andreas Mayr, Günter Klambauer, Thomas Unterthiner, Marvin Steijaert, Jörg K. Wegner, Hugo Ceulemans, Djork-Arné Clevert, and Sepp Hochreiter. Large-scale comparison of machine learning methods for drug target prediction on ChEMBL. *Chemical Science*, 9(24):5441–5451, 2018. doi: 10.1039/C8SC00148K.

[160] Henry Kenlay, Frédéric A. Dreyer, Aleksandr Kovaltsuk, Dom Miketa, Douglas Pires, and Charlotte M. Deane. Large scale paired antibody language models. *PLOS Computational Biology*, 20(12):e1012646, 2024. doi: 10.1371/journal.pcbi.1012646.

[161] Robert Schmirler, Michael Heinzinger, and Burkhard Rost. Fine-tuning protein language models boosts predictions across diverse tasks. *Nature Communications*, 15(1):7407, 2024. doi: 10.1038/s41467-024-51844-2.

[162] Peter Neidlinger, Omar S. M. El Nahhas, Hannah Sophie Muti, Tim Lenz, Michael Hoffmeister, Hermann Brenner, Marko van Treeck, Rupert Langer, Bastian Dislich, Hans Michael Behrens, Christoph Röcken, Sebastian Foersch, Daniel Truhn, Antonio Marra, Oliver Lester Saldanha, and Jakob Nikolas Kather. Benchmarking foundation models as feature extractors for weakly supervised computational pathology. *Nature Biomedical Engineering*, 10(6):1113–1123, 2025. doi: 10.1038/s41551-025-01516-3.

[163] Kevin Yang, Kyle Swanson, Wengong Jin, Connor Coley, Philipp Eiden, Hua Gao, Angel Guzman-Perez, Timothy Hopper, Brian Kelley, Miriam Mathea, Andrew Palmer, Volker Settels, et al. Analyzing learned molecular representations for property prediction. *Journal of Chemical Information and Modeling*, 59(8):3370–3388, 2019. doi: 10.1021/acs.jcim.9b00237.

[164] Xiangxiang Zeng, Hongxin Xiang, Linhui Yu, Jianmin Wang, Kenli Li, Ruth Nussinov, and Feixiong Cheng. Accurate prediction of molecular properties and drug targets using a self-supervised image representation learn-

ing framework. *Nature Machine Intelligence*, 4(11):1004–1016, 2022. doi: 10.1038/s42256-022-00557-6.

[165] Daniil A. Boiko, Thiago Reschützegger, Benjamin Sanchez-Lengeling, Samuel M. Blau, and Gabe Gomes. Advancing molecular machine learning representations with stereoelectronics-infused molecular graphs. *Nature Machine Intelligence*, 7(5):771–781, 2025. doi: 10.1038/s42256-025-01031-9.

[166] Zeming Lin, Halil Akin, Roshan Rao, Brian Hie, Zhongkai Zhu, Wenting Lu, Nikita Smetanin, Robert Verkuil, Ori Kabeli, Yaniv Shmueli, Allan dos Santos Costa, Maryam Fazel-Zarandi, Tom Sercu, Salvatore Candido, and Alexander Rives. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637):1123–1130, 2023. doi: 10.1126/science.ade2574.

[167] Ahmed Elnaggar, Michael Heinzinger, Christian Dallago, Ghalia Rehawi, Yu Wang, Llion Jones, Tom Gibbs, Tamas Feher, Christoph Angerer, Martin Steinegger, Debsindhu Bhowmik, and Burkhard Rost. Prottrans: Toward understanding the language of life through self-supervised learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(10):7112–7127, 2022. doi: 10.1109/TPAMI.2021.3095381.

[168] Jeffrey A. Ruffolo, Jeffrey J. Gray, and Jeremias Sulam. Deciphering antibody affinity maturation with language models and weakly supervised learning, 2021. URL https://arxiv.org/abs/2112.07782. Machine Learning for Structural Biology Workshop, NeurIPS 2021.

[169] Tobias H. Olsen, Iain H. Moal, and Charlotte M. Deane. AbLang: an antibody language model for completing antibody sequences. *Bioinformatics Advances*, 2(1):vbac046, 2022. doi: 10.1093/bioadv/vbac046.

[170] Bosheng Song, Jiayi Zhang, Ying Liu, Yuansheng Liu, Jing Jiang, Sisi Yuan, Xia Zhen, and Yiping Liu. A systematic review of molecular representation learning foundation models. *Briefings in Bioinformatics*, 27(1):bbaf703, 2026. doi: 10.1093/bib/bbaf703.

[171] Rishi Bommasani, Drew A. Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S. Bernstein, Jeannette Bohg, Antoine Bosselut,

Emma Brunskill, Erik Brynjolfsson, Shyamal Buch, Dallas Card, Rodrigo Castellon, Niladri Chatterji, Annie Chen, Kathleen Creel, Jared Quincy Davis, Dora Demszky, Chris Donahue, Moussa Doumbouya, Esin Durmus, Stefano Ermon, John Etchemendy, Kawin Ethayarajh, Li Fei-Fei, Chelsea Finn, Trevor Gale, Lauren Gillespie, Karan Goel, Noah Goodman, Shelby Grossman, Neel Guha, Tatsunori Hashimoto, Peter Henderson, John Hewitt, Daniel E. Ho, Jenny Hong, Kyle Hsu, Jing Huang, Thomas Icard, Saahil Jain, Dan Jurafsky, Pratyusha Kalluri, Siddharth Karamcheti, Geoff Keeling, Fereshte Khani, Omar Khattab, Pang Wei Koh, Mark Krass, Ranjay Krishna, Rohith Kuditipudi, Ananya Kumar, Faisal Ladhak, Mina Lee, Tony Lee, Jure Leskovec, Isabelle Levent, Xiang Lisa Li, Xuechen Li, Tengyu Ma, Ali Malik, Christopher D. Manning, Suvir Mirchandani, Eric Mitchell, Zanele Munyikwa, Suraj Nair, Avanika Narayan, Deepak Narayanan, Ben Newman, Allen Nie, Juan Carlos Niebles, Hamed Nilforoshan, Julian Nyarko, Giray Ogut, Laurel Orr, Isabel Papadimitriou, Joon Sung Park, Chris Piech, Eva Portelance, Christopher Potts, Aditi Raghunathan, Rob Reich, Hongyu Ren, Frieda Rong, Yusuf Roohani, Camilo Ruiz, Jack Ryan, Christopher Ré, Dorsa Sadigh, Shiori Sagawa, Keshav Santhanam, Andy Shih, Krishnan Srinivasan, Alex Tamkin, Rohan Taori, Armin W. Thomas, Florian Tramèr, Rose E. Wang, William Wang, Bohan Wu, Jiajun Wu, Yuhuai Wu, Sang Michael Xie, Michihiro Yasunaga, Jiaxuan You, Matei Zaharia, Michael Zhang, Tianyi Zhang, Xikun Zhang, Yuhui Zhang, Lucia Zheng, Kaitlyn Zhou, and Percy Liang. On the opportunities and risks of foundation models, 2021.

[172] Cecile Valsecchi, Francesca Grisoni, Viviana Consonni, and Davide Ballabio. Consensus versus individual QSARs in classification: Comparison on a large-scale case study. *Journal of Chemical Information and Modeling*, 60(3):1215–1223, 2020. doi: 10.1021/acs.jcim.9b01057.

[173] Chris Thornton, Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. Auto-WEKA: Combined selection and hyperparameter optimization of classification algorithms. In *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 847–855. ACM, 2013. doi: 10.1145/2487575.2487629.

[174] Naftali Tishby and Noga Zaslavsky. Deep learning and the information bot-

tleneck principle. In *2015 IEEE Information Theory Workshop (ITW)*, pages 1–5. IEEE, 2015. doi: 10.1109/ITW.2015.7133169.

[175] Isabelle Guyon and André Elisseeff. An introduction to variable and feature selection. *Journal of Machine Learning Research*, 3:1157–1182, 2003. URL https://www.jmlr.org/papers/v3/guyon03a.html.

[176] Yvan Saeys, Iñaki Inza, and Pedro Larrañaga. A review of feature selection techniques in bioinformatics. *Bioinformatics*, 23(19):2507–2517, 2007. doi: 10.1093/bioinformatics/btm344.

[177] Hanchuan Peng, Fuhui Long, and Chris Ding. Feature selection based on mutual information criteria of max-dependency, max-relevance, and min-redundancy. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 27(8):1226–1238, 2005. doi: 10.1109/TPAMI.2005.159.

[178] Ron Kohavi and George H. John. Wrappers for feature subset selection. *Artificial Intelligence*, 97(1–2):273–324, 1997. doi: 10.1016/S0004-3702(97)00043-X.

[179] Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B (Methodological)*, 58(1):267–288, 1996. doi: 10.1111/j.2517-6161.1996.tb02080.x.

[180] Isabelle Guyon, Jason Weston, Stephen Barnhill, and Vladimir Vapnik. Gene selection for cancer classification using support vector machines. *Machine Learning*, 46:389–422, 2002. doi: 10.1023/A:1012487302797.

[181] Sarah Nogueira, Konstantinos Sechidis, and Gavin Brown. On the stability of feature selection algorithms. *Journal of Machine Learning Research*, 18(174): 1–54, 2018. URL https://jmlr.org/papers/v18/17-514.html.

[182] Carolin Strobl, Anne-Laure Boulesteix, Achim Zeileis, and Torsten Hothorn. Bias in random forest variable importance measures: Illustrations, sources and a solution. *BMC Bioinformatics*, 8:25, 2007. doi: 10.1186/1471-2105-8-25.

[183] Ming Yuan and Yi Lin. Model selection and estimation in regression with grouped variables. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 68(1):49–67, 2006. doi: 10.1111/j.1467-9868.2005.00532.x.

[184] Sarfaraz K. Niazi and Zamara Mariam. Recent advances in machine-learning-based chemoinformatics: A comprehensive review. *International Journal of Molecular Sciences*, 24(14):11488, 2023. doi: 10.3390/ijms241411488.

[185] Ian T. Jolliffe and Jorge Cadima. Principal component analysis: a review and recent developments. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 374(2065):20150202, 2016. doi: 10.1098/rsta.2015.0202.

[186] Ian T. Jolliffe. A note on the use of principal components in regression. *Journal of the Royal Statistical Society Series C: Applied Statistics*, 31(3):300–303, 1982. doi: 10.2307/2348005.

[187] Svante Wold, Michael Sjöström, and Lennart Eriksson. PLS-regression: a basic tool of chemometrics. *Chemometrics and Intelligent Laboratory Systems*, 58 (2):109–130, 2001. doi: 10.1016/S0169-7439(01)00155-1.

[188] Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9(86):2579–2605, 2008.

[189] Leland McInnes, John Healy, and James Melville. UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018. doi: 10.48550/arXiv.1802.03426.

[190] José Jiménez-Luna, Francesca Grisoni, and Gisbert Schneider. Drug discovery with explainable artificial intelligence. *Nature Machine Intelligence*, 2(10): 573–584, 2020. doi: 10.1038/s42256-020-00236-4.

[191] Cynthia Rudin. Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence*, 1(5):206–215, 2019. doi: 10.1038/s42256-019-0048-x.

[192] Marcus Olivecrona, Thomas Blaschke, Ola Engkvist, and Hongming Chen. Molecular de-novo design through deep reinforcement learning. *Journal of Cheminformatics*, 9(1):48, 2017. doi: 10.1186/s13321-017-0235-x.

[193] Christophe Ambroise and Geoffrey J. McLachlan. Selection bias in gene extraction on the basis of microarray gene-expression data. *Proceedings of the National Academy of Sciences*, 99(10):6562–6566, 2002. doi: 10.1073/pnas.102102699.

[194] Andrius Vabalas, Emma Gowen, Ellen Poliakoff, and Alexander J. Casson. Machine learning algorithm validation with a limited sample size. *PLOS ONE*, 14(11):e0224365, 2019. doi: 10.1371/journal.pone.0224365.

[195] Sudhir Varma and Richard Simon. Bias in error estimation when using cross-validation for model selection. *BMC Bioinformatics*, 7:91, 2006. doi: 10.1186/1471-2105-7-91.

[196] Gavin C. Cawley and Nicola L. C. Talbot. On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research*, 11(70):2079–2107, 2010. URL https://www.jmlr.org/papers/v11/cawley10a.html.

[197] Jiayun Dong and Cynthia Rudin. Exploring the cloud of variable importance for the set of all good models. *Nature Machine Intelligence*, 2(12):810–824, 2020. doi: 10.1038/s42256-020-00264-0.

[198] Marco Ancona, Enea Ceolini, Cengiz Öztireli, and Markus Gross. Towards better understanding of gradient-based attribution methods for deep neural networks. In *International Conference on Learning Representations (ICLR)*, 2018. URL https://arxiv.org/abs/1711.06104.

[199] Avanti Shrikumar, Peyton Greenside, and Anshul Kundaje. Learning important features through propagating activation differences. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 3145–3153, 2017. URL https://proceedings.mlr.press/v70/shrikumar17a.html.

[200] Scott M. Lundberg, Gabriel Erion, Hugh Chen, Alex DeGrave, Jordan M. Prutkin, Bala Nair, Ronit Katz, Jonathan Himmelfarb, Nisha Bansal, and Su-In Lee. From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence*, 2(1):56–67, 2020. doi: 10.1038/s42256-019-0138-9.

[201] Narine Kokhlikyan, Vivek Miglani, Miguel Martin, Edward Wang, Bilal Alsallakh, Jonathan Reynolds, Alexander Melnikov, Natalia Kliushkina, Carlos Araya, Siqi Yan, and Orion Reblitz-Richardson. Captum: A unified and generic model interpretability library for PyTorch. *arXiv preprint arXiv:2009.07896*, 2020. URL https://arxiv.org/abs/2009.07896.

[202] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations (ICLR)*, 2015. URL https://arxiv.org/abs/1412.6980.

[203] Malkeet Singh Bahia, Omer Kaspi, Meir Touitou, Idan Binayev, Seema Dhail, Jacob Spiegel, Netaly Khazanov, Abraham Yosipof, and Hanoch Senderowitz. A comparison between 2D and 3D descriptors in QSAR modeling based on bio-active conformations. *Molecular Informatics*, 42(4):e2200186, 2023. doi: 10.1002/minf.202200186.

[204] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001. doi: 10.1214/aos/1013203451.

[205] Bradley Efron, Trevor Hastie, Iain Johnstone, and Robert Tibshirani. Least angle regression. *The Annals of Statistics*, 32(2):407–499, 2004. doi: 10.1214/009053604000000067.

[206] Kevin Jamieson and Ameet Talwalkar. Non-stochastic best arm identification and hyperparameter optimization. In *Proceedings of the 19th International Conference on Artificial Intelligence and Statistics*, volume 51 of *Proceedings of Machine Learning Research*, pages 240–248, 2016. URL https://proceedings.mlr.press/v51/jamieson16.html.

[207] Lisha Li, Kevin Jamieson, Giulia DeSalvo, Afshin Rostamizadeh, and Ameet Talwalkar. Hyperband: A novel bandit-based approach to hyperparameter optimization. *Journal of Machine Learning Research*, 18(185):1–52, 2018. URL https://jmlr.org/papers/v18/16-558.html.

[208] Izhar Wallach and Abraham Heifets. Most ligand-based classification benchmarks reward memorization rather than generalization. *Journal of Chemical Information and Modeling*, 58(5):916–932, 2018. doi: 10.1021/acs.jcim.7b00403.

[209] Pascal Notin, Aaron W. Kollasch, Daniel Ritter, Lood van Niekerk, Steffanie Paul, Hansen Spinner, Nathan Rollins, Ada Shaw, Ruben Weitzman, Jonathan Frazer, Mafalda Dias, Dinko Franceschi, et al. ProteinGym: Large-scale benchmarks for protein design and fitness prediction. *bioRxiv*, 2023. doi: 10.1101/2023.12.07.570727.

[210] Luiz C. Vieira, Sophia Lin, and Claus O. Wilke. Intrinsic dataset features drive mutational effect prediction by protein language models. *bioRxiv*, 2026. doi: 10.64898/2026.03.08.710389. Preprint.

[211] Cheng Fang, Ye Wang, and Richard et al. Grater. Prospective validation of machine learning algorithms for absorption, distribution, metabolism, and excretion prediction: An industrial perspective. *Journal of Chemical Information and Modeling*, 63(11):3263–3274, 2023. doi: 10.1021/acs.jcim.3c00160. PMID: 37216672.

[212] Cas Wognum et al. Polaris: A benchmarking platform for drug discovery, 2024. URL https://doi.org/10.5281/zenodo.13652587.

[213] Raghunathan Ramakrishnan, Pavlo O. Dral, Matthias Rupp, and O. Anatole von Lilienfeld. Quantum chemistry structures and properties of 134 kilo molecules (QM9 dataset). https://doi.org/10.6084/m9.figshare.978904, 2014. Accessed: 2026-06-02.

[214] Derek T. Ahneman, Jesús G. Estrada, Shishi Lin, Spencer D. Dreher, and Abigail G. Doyle. Predicting reaction performance in C–N cross-coupling using machine learning. *Science*, 360(6385):186–190, 2018. doi: 10.1126/science.aar5169.

[215] Patrick Koenig, Chingwei V. Lee, Benjamin T. Walters, Vasantharajan Janakiraman, Jeremy Stinson, Thomas W. Patapoff, and Germaine Fuh. Mutational landscape of antibody variable domains reveals a switch modulating the interdomain conformational dynamics and antigen binding. *Proceedings of the National Academy of Sciences*, 114(4):E486–E495, 2017. doi: 10.1073/pnas.1613231114.

[216] Michael Chungyoun, Jeffrey Ruffolo, and Jeffrey Gray. FLAb: Benchmarking deep learning methods for antibody fitness prediction. *bioRxiv*, 2024. doi: 10.1101/2024.01.13.575504.

[217] Mario S. Valdés-Tresanco, Mario E. Valdés-Tresanco, Esteban Molina-Abad, and Ernesto Moreno. NbThermo: a new thermostability database for nanobodies. *Database*, 2023:baad021, 2023. doi: 10.1093/database/baad021.

[218] David Prihoda, Jad Maamary, Andrew Waight, Veronica Juan, Laurence Fayadat-Dilman, Daniel Svozil, and Danny A. Bitton. BioPhi: A platform for antibody design, humanization, and humanness evaluation based on natural antibody repertoires and deep learning. *mAbs*, 14(1):2020203, 2022. doi: 10.1080/19420862.2021.2020203.

[219] Lewis Chinery, Alissa M. Hummer, Brij Bhushan Mehta, Rahmad Akbar, Puneet Rawat, Andrei Slabodkin, Khang Le Quy, Fridtjof Lund-Johansen, Victor Greiff, Jeliazko R. Jeliazkov, and Charlotte M. Deane. Baselining the buzz. Trastuzumab-HER2 affinity, and beyond. *bioRxiv*, 2024. doi: 10.1101/2024.03.26.586756. Version 1; later versions retitled "Simple computational methods can outperform deep learning in designing diverse, binder-enriched antibody libraries".

[220] Hugo Dalla-Torre, Liam Gonzalez, Javier Mendoza-Revilla, Nicolas Lopez Carranza, Adam Henryk Grzywaczewski, Francesco Oteri, Christian Dallago, Evan Trop, Bernardo P. de Almeida, Hassan Sirelkhatim, Guillaume Richard, Marcin Skwark, Karim Beguir, Marie Lopez, and Thomas Pierrot. Nucleotide transformer: building and evaluating robust foundation models for human genomics. *Nature Methods*, 22(2):287–297, 2025. doi: 10.1038/s41592-024-02523-z.

[221] The Cancer Genome Atlas Research Network. Comprehensive molecular profiling of lung adenocarcinoma. *Nature*, 511(7511):543–550, 2014. doi: 10.1038/nature13385.

[222] The Cancer Genome Atlas Research Network. Comprehensive genomic characterization of squamous cell lung cancers. *Nature*, 489(7417):519–525, 2012. doi: 10.1038/nature11404.

[223] Mary J. Goldman, Brian Craft, Mim Hastie, Kristupas Repečka, Fran McDade, Akhil Kamath, Ayan Banerjee, Yunhai Luo, Dave Rogers, Angela N. Brooks, Jingchun Zhu, and David Haussler. Visualizing and interpreting cancer genomics data via the Xena platform. *Nature Biotechnology*, 38(6):675–678, 2020. doi: 10.1038/s41587-020-0546-8.

[224] The Cancer Genome Atlas Network. Comprehensive molecular portraits

of human breast tumours. *Nature*, 490(7418):61–70, 2012. doi: 10.1038/nature11412.

[225] Joel S. Parker, Michael Mullins, Maggie C. U. Cheang, Samuel Leung, David Voduc, Tammi Vickery, Sherri Davies, Christiane Fauron, Xiaping He, Zhiyuan Hu, John F. Quackenbush, Inge J. Stijleman, Juan Palazzo, J. S. Marron, Andrew B. Nobel, Elaine Mardis, Torsten O. Nielsen, Matthew J. Ellis, Charles M. Perou, and Philip S. Bernard. Supervised risk predictor of breast cancer based on intrinsic subtypes. *Journal of Clinical Oncology*, 27(8):1160–1167, 2009. doi: 10.1200/JCO.2008.18.1370.

[226] Frank Hoonakker, Nicolas Lachiche, Alexandre Varnek, and Alain Wagner. A representation to apply usual data mining techniques to chemical reactions – illustration on the rate constant of SN2 reactions in water. *International Journal on Artificial Intelligence Tools*, 20(2):253–270, 2011. doi: 10.1142/S0218213011000140.

[227] Philippe Schwaller, Benjamin Hoover, Jean-Louis Reymond, Hendrik Strobelt, and Teodoro Laino. Extraction of organic chemistry grammar from unsupervised learning of chemical reactions. *Science Advances*, 7(15):eabe4166, 2021. doi: 10.1126/sciadv.abe4166.

[228] Philippe Schwaller, Daniel Probst, Alain C. Vaucher, Vishnu H. Nair, David Kreutter, Teodoro Laino, and Jean-Louis Reymond. Mapping the space of chemical reactions using attention-based neural networks. *Nature Machine Intelligence*, 3(2):144–152, 2021. doi: 10.1038/s42256-020-00284-w.

[229] Tatsuya Sagawa and Ryosuke Kojima. Reactiont5: a large-scale pre-trained model towards application of limited reaction data, 2023. URL https://arxiv.org/abs/2311.06708.

[230] Kangway V. Chuang and Michael J. Keiser. Comment on "predicting reaction performance in C–N cross-coupling using machine learning". *Science*, 362(6416):eaat8603, 2018. doi: 10.1126/science.aat8603.

[231] S. Henikoff and J. G. Henikoff. Amino acid substitution matrices from protein blocks. *Proceedings of the National Academy of Sciences*, 89(22):10915–10919, 1992. doi: 10.1073/pnas.89.22.10915.

[232] Marie-Paule Lefranc, Christelle Pommié, Manuel Ruiz, Véronique Giudicelli, Elodie Foulquier, Lisa Truong, Valérie Thouvenin-Contet, and Gérard Lefranc. Imgt unique numbering for immunoglobulin and t cell receptor variable domains and ig superfamily v-like domains. *Developmental & Comparative Immunology*, 27(1):55–77, 2003. doi: 10.1016/S0145-305X(02)00039-3.

[233] James Dunbar and Charlotte M. Deane. Anarci: antigen receptor numbering and receptor classification. *Bioinformatics*, 32(2):298–300, 2016. doi: 10.1093/bioinformatics/btv552.

[234] Susana Vinga and Jonas Almeida. Alignment-free sequence comparison—a review. *Bioinformatics*, 19(4):513–523, 2003. doi: 10.1093/bioinformatics/btg005.

[235] Hanno Teeling, Anke Meyerdierks, Margarete Bauer, Rudolf Amann, and Frank Oliver Glöckner. Application of tetranucleotide frequencies for the assignment of genomic fragments. *Environmental Microbiology*, 6(9):938–947, 2004. doi: 10.1111/j.1462-2920.2004.00624.x.

[236] Zhihan Zhou, Yanrong Ji, Weijian Li, Pratik Dutta, Ramana Davuluri, and Han Liu. Dnabert-2: Efficient foundation model and benchmark for multi-species genome, 2023. URL https://arxiv.org/abs/2306.15006.

[237] Eric Nguyen, Michael Poli, Marjan Faizi, Armin Thomas, Callum Birch-Sykes, Michael Wornow, Aman Patel, Clayton Rabideau, Stefano Massaroli, Yoshua Bengio, Stefano Ermon, Stephen A. Baccus, and Chris Ré. Hyenadna: Long-range genomic sequence modeling at single nucleotide resolution, 2023. URL https://arxiv.org/abs/2306.15794.

[238] The Cancer Genome Atlas Research Network, John N Weinstein, Eric A Collisson, Gordon B Mills, Kenna R Mills Shaw, Brad A Ozenberger, Kyle Ellrott, Ilya Shmulevich, Chris Sander, and Joshua M Stuart. The cancer genome atlas pan-cancer analysis project. *Nature Genetics*, 45(10):1113–1120, 2013. doi: 10.1038/ng.2764.

[239] Raymond E. Carhart, Dennis H. Smith, and R. Venkataraghavan. Atom pairs as molecular features in structure-activity studies: definition and applications. *Journal of Chemical Information and Computer Sciences*, 25(2):64–73, 1985. doi: 10.1021/ci00046a002.

[240] Ramaswamy Nilakantan, Norman Bauman, J. Scott Dixon, and R. Venkataraghavan. Topological torsion: a new molecular descriptor for sar applications. comparison with other descriptors. *Journal of Chemical Information and Computer Sciences*, 27(2):82–85, 1987. doi: 10.1021/ci00054a008.

[241] Peter Gedeck, Bernhard Rohde, and Christian Bartels. Qsar – how good is it in practice? comparison of descriptor sets on an unbiased cross section of corporate data sets. *Journal of Chemical Information and Modeling*, 46(5): 1924–1936, 2006. doi: 10.1021/ci050413p.

[242] Joseph L. Durant, Burton A. Leland, Douglas R. Henry, and James G. Nourse. Reoptimization of MDL keys for use in drug discovery. *Journal of Chemical Information and Computer Sciences*, 42(6):1273–1280, 2002. doi: 10.1021/ci010132r.

[243] Nino Shervashidze, Pascal Schweitzer, Erik Jan van Leeuwen, Kurt Mehlhorn, and Karsten M. Borgwardt. Weisfeiler-lehman graph kernels. *Journal of Machine Learning Research*, 12(77):2539–2561, 2011. URL https://www.jmlr.org/papers/v12/shervashidze11a.html.

[244] Hirotomo Moriwaki, Yu-Shi Tian, Norihito Kawashita, and Tatsuya Takagi. Mordred: a molecular descriptor calculator. *Journal of Cheminformatics*, 10 (1):4, 2018. doi: 10.1186/s13321-018-0258-y.

[245] Yu Rong, Yatao Bian, Tingyang Xu, Weiyang Xie, Ying Wei, Wenbing Huang, and Junzhou Huang. Self-supervised graph transformer on large-scale molecular data, 2020. URL https://arxiv.org/abs/2007.02835.

[246] Shengchao Liu, Hanchen Wang, Weiyang Liu, Joan Lasenby, Hongyu Guo, and Jian Tang. Pre-training molecular graph representation with 3d geometry, 2022. URL https://arxiv.org/abs/2110.07728.

[247] Eduardo Soares, Emilio Vital Brazil, Victor Shirasuna, Dmitry Zubarev, Renato Cerqueira, and Kristin Schmidt. An open-source family of large encoder-decoder foundation models for chemistry. *Communications Chemistry*, 8, 2025. doi: 10.1038/s42004-025-01585-0.

[248] Xiao-Chen Zhang, Cheng-Kun Wu, Jia-Cai Yi, Xiang-Xiang Zeng, Can-Qun Yang, Ai-Ping Lu, Ting-Jun Hou, and Dong-Sheng Cao. Pushing the boundaries of molecular property prediction for drug discovery with multitask learning bert enhanced by smiles enumeration. *Research*, 2022, 2022. doi: 10.34133/research.0004.

[249] Atakan Yüksel, Erva Ulusoy, Atabey Ünlü, and Tunca Doğan. Selformer: molecular representation learning via selfies language models. *Machine Learning: Science and Technology*, 4(2):025035, 2023. doi: 10.1088/2632-2153/acdb30.

[250] Pedro J. Ballester and W. Graham Richards. Ultrafast shape recognition to search compound databases for similar molecular shapes. *Journal of Computational Chemistry*, 28(10):1711–1723, 2007. doi: 10.1002/jcc.20681.

[251] Adrian M. Schreyer and Tom Blundell. Usrcat: real-time ultrafast shape recognition with pharmacophoric constraints. *Journal of Cheminformatics*, 4 (1):27, 2012. doi: 10.1186/1758-2946-4-27.

[252] Matthias Rupp, Alexandre Tkatchenko, Klaus-Robert Müller, and O. Anatole von Lilienfeld. Fast and accurate modeling of molecular atomization energies with machine learning. *Physical Review Letters*, 108(5):058301, 2012. doi: 10.1103/PhysRevLett.108.058301.

[253] Joshua Meier, Roshan Rao, Robert Verkuil, Jason Liu, Tom Sercu, and Alexander Rives. Language models enable zero-shot prediction of the effects of mutations on protein function. *bioRxiv*, 2021. doi: 10.1101/2021.07.09.450648.

[254] Thomas Hayes, Roshan Rao, Halil Akin, Nicholas J. Sofroniew, Deniz Oktay, Zeming Lin, Robert Verkuil, Vincent Q. Tran, Jonathan Deaton, Marius Wiggert, Rohil Badkundri, Irhum Shafkat, Jun Gong, Alexander Derry, Raul S. Molina, Neil Thomas, Yousuf A. Khan, Chetan Mishra, Carolyn Kim, Liam J. Bartie, Matthew Nemeth, Patrick D. Hsu, Tom Sercu, Salvatore Candido, and Alexander Rives. Simulating 500 million years of evolution with a language model. *Science*, 387(6736):850–858, 2025. doi: 10.1126/science.ads0018.

[255] Salvatore Candido, Thomas Hayes, Alexander Derry, Roshan Rao, Zeming Lin, Robert Verkuil, Bryan Z. Wu, Jin Sub Lee, Elise S. Bruguera, Jehan A.

Keval, Mykhailo Kopylov, John E. Pak, Wesley Wu, Neil Thomas, Samson Mataraso, Alvin Hsu, Ashton C. Trotman-Grant, Kilian Fatras, Allan dos Santos Costa, Rohil Badkundri, Halil Akın, Deniz Oktay, Jonathan Deaton, Elizabeth Montabana, Hrishita Sitwala, Yue Yu, Marius Wiggert, Dylan Alexander Carlin, Anthony W. Goering, Tomasz Blazejewski, McCullen Sandora, Michael Hla, Tina Z. Jia, Leon H. Kloker, Nicholas J. Sofroniew, Masatoshi Uehara, Jassi Pannu, Sharrol Bachas, Daniel S. Liu, Tom Sercu, and Alexander Rives. Language modeling materializes a world model of protein biology. *bioRxiv*, 2026. doi: 10.64898/2026.06.03.729735. Preprint, posted 4 June 2026.

[256] Jin Su, Chenchen Han, Yuyang Zhou, Junjie Shan, Xibin Zhou, and Fajie Yuan. Saprot: Protein language modeling with structure-aware vocabulary. *bioRxiv*, 2023. doi: 10.1101/2023.10.01.560349.

[257] Tobias H. Olsen, Iain H. Moal, and Charlotte M. Deane. Addressing the antibody germline bias and its effect on language models for improved antibody design. *Bioinformatics*, 40(11), 2024. doi: 10.1093/bioinformatics/btae618.

[258] Henry Kenlay, Frédéric A. Dreyer, Aleksandr Kovaltsuk, Dom Miketa, Douglas Pires, and Charlotte M. Deane. Large scale paired antibody language models. *PLOS Computational Biology*, 20(12):e1012646, 2024. doi: 10.1371/journal.pcbi.1012646.

[259] Jeffrey A. Ruffolo, Jeffrey J. Gray, and Jeremias Sulam. Deciphering antibody affinity maturation with language models and weakly supervised learning, 2021. URL https://arxiv.org/abs/2112.07782.

[260] Eunna Huh, Hyeonsu Lee, and Hyunjin Shin. Antibody foundational model: Ab-RoBERTa, 2025.

[261] Melissa Sanabria, Jonas Hirsch, Pierre M. Joubert, and Anna R. Poetsch. Dna language model grover learns sequence context in the human genome. *Nature Machine Intelligence*, 6(8):911–923, 2024. doi: 10.1038/s42256-024-00872-0.

[262] Raphael Mourad. Mistral-DNA: Large language model Mistral for DNA. https://github.com/raphaelmourad/Mistral-DNA, 2024. Code and model release, no paper; model RaphaelMourad/Mistral-DNA-v1-422M-hg38 on Hugging Face.

[263] Paul M. Sharp, Therese M. F. Tuohy, and Krzysztof R. Mosurski. Codon usage in yeast: cluster analysis clearly differentiates highly and lowly expressed genes. *Nucleic Acids Research*, 14(13):5125–5143, 1986. doi: 10.1093/nar/14.13.5125.

[264] Christina V. Theodoris, Ling Xiao, Anant Chopra, Mark D. Chaffin, Zeina R. Al Sayed, Matthew C. Hill, Helene Mantineo, Elizabeth M. Brydon, Zexian Zeng, X. Shirley Liu, and Patrick T. Ellinor. Transfer learning enables predictions in network biology. *Nature*, 618(7965):616–624, 2023. doi: 10.1038/s41586-023-06139-9.

[265] Christina V. Theodoris. Geneformer model repository: Geneformer-V2-316M. https://huggingface.co/ctheodoris/Geneformer, 2024. Model card: V2 trained December 2024 on about 104 million human single-cell transcriptomes, input size 4096; default model in the main directory is Geneformer-V2-316M (18 layers, 1152 hidden units).

[266] Maxence Gélard, Guillaume Richard, Thomas Pierrot, and Paul-Henry Cournède. Bulkrnabert: Cancer prognosis from bulk rna-seq based language models. *bioRxiv*, 2024. doi: 10.1101/2024.06.18.599483.

[267] Craig H Mermel, Steven E Schumacher, Barbara Hill, Matthew L Meyerson, Rameen Beroukhim, and Gad Getz. Gistic2.0 facilitates sensitive and confident localization of the targets of focal somatic copy-number alteration in human cancers. *Genome Biology*, 12(4):R41, 2011. doi: 10.1186/gb-2011-12-4-r41.

[268] Jun Li, Yiling Lu, Rehan Akbani, Zhenlin Ju, Paul L Roebuck, Wenbin Liu, Ji-Yeon Yang, Bradley M Broom, Roeland G W Verhaak, David W Kane, Chris Wakefield, John N Weinstein, Gordon B Mills, and Han Liang. Tcpa: a resource for cancer functional proteomics data. *Nature Methods*, 10(11): 1046–1047, 2013. doi: 10.1038/nmeth.2650.

[269] Tong Ding, Sophia J. Wagner, Andrew H. Song, Richard J. Chen, Ming Y. Lu, Andrew Zhang, Anurag J. Vaidya, Guillaume Jaume, Muhammad Shaban, Ahrong Kim, Drew F. K. Williamson, Bowen Chen, Cristina Almagro-Perez, Paul Doucet, Sharifa Sahai, Chengkuan Chen, Daisuke Komura, Akihiro Kawabe, Shumpei Ishikawa, Georg Gerber, Tingying Peng, Long Phi Le,

and Faisal Mahmood. Multimodal whole slide foundation model for pathology, 2024. URL https://arxiv.org/abs/2411.19666.

[270] Hanwen Xu, Naoto Usuyama, Jaspreet Bagga, Sheng Zhang, Rajesh Rao, Tristan Naumann, Cliff Wong, Zelalem Gero, Javier González, Yu Gu, Yanbo Xu, Mu Wei, Wenhui Wang, Shuming Ma, Furu Wei, Jianwei Yang, Chunyuan Li, Jianfeng Gao, Jaylen Rosemon, Tucker Bower, Soohee Lee, Roshanthi Weerasinghe, Bill J. Wright, Ari Robicsek, Brian Piening, Carlo Bifulco, Sheng Wang, and Hoifung Poon. A whole-slide foundation model for digital pathology from real-world data. *Nature*, 630(8015):181–188, 2024. doi: 10.1038/s41586-024-07441-w.

[271] Xiyue Wang, Sen Yang, Jun Zhang, Minghui Wang, Jing Zhang, Wei Yang, Junzhou Huang, and Xiao Han. Transformer-based unsupervised contrastive learning for histopathological image classification. *Medical Image Analysis*, 81: 102559, 2022. doi: 10.1016/j.media.2022.102559.

[272] Alexandre Filiot, Ridouane Ghermi, Antoine Olivier, Paul Jacob, Lucas Fidon, Alice Mac Kain, Charlie Saillard, and Jean-Baptiste Schiratti. Scaling self-supervised learning for histopathology with masked image modeling. *medRxiv*, 2023. doi: 10.1101/2023.07.21.23292757.

[273] Charlie Saillard, Rodolphe Jenatton, Felipe Llinares-López, Zelda Mariet, David Cahané, Eric Durand, and Jean-Philippe Vert. H-optimus-0, 2024. URL https://github.com/bioptimus/releases/tree/main/models/h-optimus/v0.

[274] Theo A. Knijnenburg, Linghua Wang, Michael T. Zimmermann, Nyasha Chambwe, Galen F. Gao, Andrew D. Cherniack, et al. Genomic and molecular landscape of DNA damage repair deficiency across The Cancer Genome Atlas. *Cell Reports*, 23(1):239–254.e6, 2018. doi: 10.1016/j.celrep.2018.03.076.

[275] Kyle Ellrott, Matthew H. Bailey, Gordon Saksena, Kyle R. Covington, Cyriac Kandoth, Chip Stewart, et al. Scalable open science approach for mutation calling of tumor exomes using multiple genomic pipelines. *Cell Systems*, 6(3): 271–281.e7, 2018. doi: 10.1016/j.cels.2018.03.002.

[276] Scott L. Carter, Kristian Cibulskis, Elena Helman, Aaron McKenna, Hui Shen, Travis Zack, Peter W. Laird, Robert C. Onofrio, Wendy Winckler, Barbara A.

Weir, Rameen Beroukhim, David Pellman, Douglas A. Levine, Eric S. Lander, Matthew Meyerson, and Gad Getz. Absolute quantification of somatic DNA alterations in human cancer. *Nature Biotechnology*, 30(5):413–421, 2012. doi: 10.1038/nbt.2203.

[277] Vésteinn Thorsson, David L. Gibbs, Scott D. Brown, Denise Wolf, Dante S. Bortone, Tai-Hsien Ou Yang, et al. The immune landscape of cancer. *Immunity*, 48(4):812–830.e14, 2018. doi: 10.1016/j.immuni.2018.03.023.

[278] Aaron M. Newman, Chih Long Liu, Michael R. Green, Andrew J. Gentles, Weiguo Feng, Yue Xu, Chuong D. Hoang, Maximilian Diehn, and Ash A. Alizadeh. Robust enumeration of cell subsets from tissue expression profiles. *Nature Methods*, 12(5):453–457, 2015. doi: 10.1038/nmeth.3337.

[279] Tathiane M. Malta, Artem Sokolov, Andrew J. Gentles, Tomasz Burzykowski, Laila Poisson, John N. Weinstein, et al. Machine learning identifies stemness features associated with oncogenic dedifferentiation. *Cell*, 173(2):338–354.e15, 2018. doi: 10.1016/j.cell.2018.03.034.

[280] Minoru Toyota, Nita Ahuja, Mutsumi Ohe-Toyota, James G. Herman, Stephen B. Baylin, and Jean-Pierre J. Issa. CpG island methylator phenotype in colorectal cancer. *Proceedings of the National Academy of Sciences*, 96(15):8681–8686, 1999. doi: 10.1073/pnas.96.15.8681.

[281] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022), Datasets and Benchmarks Track*, 2022. URL https://arxiv.org/abs/2207.08815.

[282] Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, Cambridge, MA, 2006. ISBN 0-262-18253-X.

[283] Davide Chicco and Giuseppe Jurman. The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation. *BMC Genomics*, 21(1):6, 2020. doi: 10.1186/s12864-019-6413-7.

[284] Yoshua Bengio and Yves Grandvalet. No unbiased estimator of the variance of

K-fold cross-validation. *Journal of Machine Learning Research*, 5:1089–1105, 2004. URL https://www.jmlr.org/papers/v5/grandvalet04a.html.

[285] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer Series in Statistics. Springer, New York, 2 edition, 2009. doi: 10.1007/978-0-387-84858-7.

[286] Jun Shu, Qi Xie, Lixuan Yi, Qian Zhao, Sanping Zhou, Zongben Xu, and Deyu Meng. Meta-weight-net: Learning an explicit mapping for sample weighting. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[287] Ivan Olier, Noureddin Sadawi, G. Richard Bickerton, Joaquin Vanschoren, Crina Grosan, Larisa Soldatova, and Ross D. King. Meta-QSAR: a large-scale application of meta-learning to drug design and discovery. *Machine Learning*, 107(1):285–311, 2018. doi: 10.1007/s10994-017-5685-x.

[288] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, volume 54 of *Proceedings of Machine Learning Research*, pages 1273–1282, 2017. URL https://proceedings.mlr.press/v54/mcmahan17a.html.

[289] Wouter Heyndrickx, Lewis Mervin, Tobias Morawietz, Noé Sturm, Lukas Friedrich, Adam Zalewski, Anastasia Pentina, Lina Humbeck, Martijn Oldenhof, Ritsuya Niwayama, Peter Schmidtke, Nikolas Fechner, Jaak Simm, Adam Arany, Nicolas Drizard, Rama Jabal, Arina Afanasyeva, Regis Loeb, Shlok Verma, Simon Harnqvist, Matthew Holmes, Balazs Pejo, Maria Telenczuk, Nicholas Holway, Arne Dieckmann, Nicola Rieke, Friederike Zumsande, Djork-Arné Clevert, Michael Krug, Christopher Luscombe, Darren Green, Peter Ertl, Peter Antal, David Marcus, Nicolas Do Huu, Hideyoshi Fuji, Stephen Pickett, Gergely Acs, Eric Boniface, Bernd Beck, Yax Sun, Arnaud Gohier, Friedrich Rippmann, Ola Engkvist, Andreas H. Göller, Yves Moreau, Mathieu N. Galtier, Ansgar Schuffenhauer, and Hugo Ceulemans. MELLODDY: Cross-pharma federated learning at unprecedented scale unlocks benefits in QSAR without compromising proprietary information. *Journal of Chemical Information and Modeling*, 64(7):2331–2344, 2024. doi: 10.1021/acs.jcim.3c00799.

[290] Pietro Cassarà, Alberto Gotta, and Lorenzo Valerio. Federated feature selection for cyber-physical systems of systems. *IEEE Transactions on Vehicular Technology*, 71(9):9937–9950, 2022. doi: 10.1109/TVT.2022.3178612.

[291] Paul Czodrowski. herg me out. *Journal of Chemical Information and Modeling*, 53(9):2240–2251, 2013.

[292] Lood van Niekerk, Joshua Moller, Seth Ritter, Porfirio Quintero-Cadena, Ariel Cohen, Alexander Channing, Michael Chungyoun, Ulfert Rand, Ian Smith, Nikhil Bhatt, Emmanuel Pierre, Casey Harris, Yi Ao, Vincent Grippo, Jasmin Schwenk, Amos Rosenbaum, Lucas Allen, Naif Asi, Jiaqi Zhu, Ritika Singh, Sonia Sammi, Vikram Jadhav, Jan Dušek, Sarath Chandra, Andreea Badea, Nels Thorsteinson, Marissa Blalock, Jared Kim, Oliver Turnbull, Shreyas Kulkarni, Vivek Kohar, Abiy Gebremedhin, Charlotte M. Deane, Peter M. Tessier, and Ammar Arsiwala. 2025 Ginkgo Datapoints antibody developability competition outcomes: limited model performance and a call for data standardization. *mAbs*, 18(1):2634216, 2026. doi: 10.1080/19420862.2026.2634216.

[293] Alissa M. Hummer, Constantin Schneider, Lewis Chinery, and Charlotte M. Deane. Investigating the volume and diversity of data needed for generalizable antibody–antigen $\delta\delta g$ prediction. *Nature Computational Science*, 5(8):635–647, 2025. doi: 10.1038/s43588-025-00823-8.

[294] Ryan J. Tibshirani. Degrees of freedom and model search. *Statistica Sinica*, 25(3):1265–1296, 2015. doi: 10.5705/ss.2014.147.

[295] Andrew Y. Ng. Feature selection, L1 vs. L2 regularization, and rotational invariance. In *Proceedings of the Twenty-First International Conference on Machine Learning (ICML '04)*, page 78, 2004. doi: 10.1145/1015330.1015435.

[296] Ananya Kumar, Aditi Raghunathan, Robbie Matthew Jones, Tengyu Ma, and Percy Liang. Fine-tuning can distort pretrained features and underperform out-of-distribution. In *International Conference on Learning Representations (ICLR)*, 2022. URL https://openreview.net/forum?id=UYneFzXSJWh.

[297] Markus M. Breunig, Hans-Peter Kriegel, Raymond T. Ng, and Jörg Sander. Lof: identifying density-based local outliers. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, SIGMOD '00,

page 93–104, New York, NY, USA, 2000. Association for Computing Machinery. ISBN 1581132174. doi: 10.1145/342009.335388.

[298] Junnan Li, Richard Socher, and Steven C. H. Hoi. Dividemix: Learning with noisy labels as semi-supervised learning, 2020.

[299] Maximilian Seitzer, Arash Tavakoli, Dimitrije Antic, and Georg Martius. On the pitfalls of heteroscedastic uncertainty estimation with probabilistic neural networks, 2022.

[300] Glenn Shafer and Vladimir Vovk. A tutorial on conformal prediction. *Journal of Machine Learning Research*, 9(12):371–421, 2008.

[301] Matt Taddy, Chun-Sheng Chen, Jun Yu, and Mitch Wyle. Bayesian and empirical bayesian forests, 2015.

[302] Yanqing Ma, Carmine Ventre, and Maria Polukarov. Denoised labels for financial time series data via self-supervised learning. In *Proceedings of the Third ACM International Conference on AI in Finance*, pages 471–479. ACM, 2022. doi: 10.1145/3533271.3561687.

[303] Stephen R. Heller, Alan McNaught, Igor Pletnev, Stephen Stein, and Dmitrii Tchekhovskoi. InChI, the IUPAC international chemical identifier. *Journal of Cheminformatics*, 7(1):23, 2015. doi: 10.1186/s13321-015-0068-4.

[304] Guy W. Bemis and Mark A. Murcko. The properties of known drugs. 1. molecular frameworks. *Journal of Medicinal Chemistry*, 39(15):2887–2893, 1996. doi: 10.1021/jm9602928.

[305] Philippe Schwaller, Alain C. Vaucher, Teodoro Laino, and Jean-Louis Reymond. Prediction of chemical reaction yields using deep learning. *Machine Learning: Science and Technology*, 2(1):015016, 2021. doi: 10.1088/2632-2153/abc81d.

[306] The Cancer Genome Atlas Research Network, John N. Weinstein, Eric A. Collisson, Gordon B. Mills, Kenna R. Mills Shaw, Brad A. Ozenberger, Kyle Ellrott, Ilya Shmulevich, Chris Sander, and Joshua M. Stuart. The Cancer Genome Atlas pan-cancer analysis project. *Nature Genetics*, 45(10):1113–1120, 2013. doi: 10.1038/ng.2764.

[307] Bo Li and Colin N. Dewey. RSEM: accurate transcript quantification from RNA-Seq data with or without a reference genome. *BMC Bioinformatics*, 12: 323, 2011. doi: 10.1186/1471-2105-12-323.